STRATO v1.0.0: Full Mathematical Derivation

Spherical TRISO Analysis and Transport Operations

2026-11-06

Contents

How to read this document **3**
- Audience . . . 3
- How the parts connect . . . 3
- From principle to code . . . 4
- Labels used throughout . . . 4
- Other conventions . . . 4
- What v1.0.0 implements . . . 5

Notation **5**

Part I: Physical foundation **13**
- 1.1 The TRISO particle . . . 13
- 1.2 Concentration and amount . . . 17
- 1.3 Flux and Fick's law . . . 17
- 1.4 Sphere area and shell volume . . . 18
- 1.5 Inventory . . . 20
- 1.6 Assumptions of v1.0.0 . . . 21
- 1.7 Summary and classification . . . 22

Part II: Governing conservation equation **23**
- 2.0 The quantities that enter the balance . . . 23
- 2.1 Conservation in words . . . 25
- 2.2 Balance on a thin shell . . . 26
- 2.3 Thin-shell limit and the $1/r^2$ factor . . . 28
- 2.4 Divergence . . . 30
- 2.5 Spherical diffusion equation . . . 33
- 2.6 Layer equation of v1.0.0 . . . 35
- 2.7 Units check of every term . . . 36
- 2.8 Size of diffusion times . . . 36
- 2.9 Classification of Part II and link to the code . . . 38
- 2.10 Verification . . . 39

Part III: Five-layer model and conditions **40**
- 3.0 Scope: the five-layer domain . . . 40
- 3.1 Centre: symmetry and regularity . . . 42
- 3.2 Interfaces . . . 44
- 3.3 Outer surface: a perfect sink . . . 47
- 3.4 Initial state . . . 48
- 3.5 Inventory and release . . . 49

3.6 Case B reference: the Robin surface . . . 57
3.7 Summary and verification . . . 58

Part IV: Analytical solution **59**
4.1 Separation of variables . . . 60
4.2 Radial transformation . . . 62
4.3 Solution in one layer and the transfer matrix . . . 63
4.4 Interface matching . . . 65
4.5 Centre start and characteristic function . . . 67
4.6 Eigenvalue concepts and the Sturm count . . . 68
4.7 Root isolation . . . 72
4.8 Orthogonality . . . 74
4.9 Expansion of the initial state . . . 76
4.10 Release fraction and weights . . . 77
4.11 Evaluating the integrals . . . 79
4.12 Truncation and validity time . . . 81
4.13 Round-off: flux ratio and accuracy floor . . . 82
4.14 Homogeneous checks . . . 83
4.15 Robin sphere root equation (Case B reference) . . . 84
4.16 Relation to Gelbard (2003) . . . 85
4.17 Summary and verification . . . 86

Part V: Discrete finite-volume solution **87**
5.1 Control volumes and cell averages . . . 88
5.2 The mesh . . . 89
5.3 Cell volume, centre and face areas . . . 90
5.4 Exact balance of one cell . . . 93
5.5 Face flux and the series conductance . . . 94
5.6 Centre and outer boundary . . . 97
5.7 The semi-discrete system . . . 98
5.8 Implicit time stepping (Backward Euler) . . . 99
5.9 The time step actually used . . . 101
5.10 The tridiagonal matrix and its solution . . . 101
5.11 Inventory, release, balance and release fraction . . . 105
5.12 Summary and verification . . . 108

Part VI: Numerical properties and stability **109**
6.1 Discrete conservation . . . 110
6.2 M-matrix, positivity and the maximum principle . . . 112
6.3 Positivity in floating point and the tridiagonal solver . . . 114
6.4 Stability of Backward Euler and the explicit bound . . . 116
6.5 Consistency and order . . . 121
6.6 Richardson time extrapolation . . . 124
6.7 Observed order of accuracy and convergence . . . 125
6.8 Round-off in the elimination and in the balance . . . 127
6.9 Cost of the refinement study . . . 129
6.10 Convergence of the semi-discrete scheme: an energy argument . . . 129
6.11 Accuracy of the analytical series (Code C) . . . 131
6.12 Summary of numerical properties . . . 133

Part VII: Connection between theory and implementation **135**
7.1 From the derivation to Code B . . . 136
7.2 From the derivation to Code C . . . 139

7.3 From the derivation to Code A: comparison, extrapolation and observed orders 142
7.4 Final equations used in the implementation 145
7.5 Known inconsistencies and open issues 152

References **163**

Appendix A: Verification scripts **165**

Purpose. This document derives, one step at a time, every equation that the STRATO v1.0.0 code uses to compute the release of a fission product from a TRISO fuel particle, and it states which lines of the Rust code compute which equation. It also derives the equations that the code does not compute, so that the limits of v1.0.0 are visible. The companion document STRATO_Final_Deliverable.md is the short form: for every operation it gives the final equation, the pseudocode and the actual Rust in one table row, with the same logic in all three.

Version. 1.0.0, repository tag `v1.0.0` of https://github.com/weewzr/STRATO. **Date.** 2026-11-06.

Contents. Notation (Tables 0.1 to 0.4); Part I, physical foundation; Part II, governing conservation equation; Part III, five-layer model and conditions; Part IV, analytical solution (Code C); Part V, discrete finite-volume solution (Code B); Part VI, numerical properties and stability; Part VII, connection between theory and implementation; References; Appendix A, verification scripts.

How to read this document

Audience

The text is written for a reader who has met algebra and the basic ideas of derivatives and integrals at school, and who is willing to follow a long argument. Every transformation is shown as its own step together with the reason for it. Every symbol is defined where it first appears and again in the notation tables, with its SI unit. Where a result needs a theorem that is not proved here (for example the spectral theorem for symmetric matrices), the text says so and labels the result as established textbook result.

How the parts connect

Part I describes the particle and defines concentration, flux, area, volume and inventory. Part II turns the principle that substance is neither created nor destroyed into a differential equation. Part III adds the conditions that single out one solution for the five-layer particle: the centre, the interfaces, the outer surface and the starting state. It also defines the release rate and the release fraction, the quantities the code reports. Parts IV and V solve the problem in two independent ways. Part IV builds an analytical series (Code C, `src/analytical/sturm_liouville.rs`) and Part V builds a finite-volume time-stepping scheme (Code B, `src/numerical/finite_volume.rs`). Part VI states what the numerical method guarantees and what is only observed. Part VII connects every equation to the lines of Rust that compute it, compares the two methods, lists the verified results and records where the earlier Phase 1 documents differ from the code. The driver, the input reader, the comparison and the refinement study are Code A (`src/main.rs`, `src/config.rs`, `src/postprocessing/comparison.rs`, `src/postprocessing/plots.rs`); they are described where they are used.

From principle to code

Each result follows the same progression: physical principle, governing equation, derivation, final formulation, pseudocode, Rust. Parts I and II cover the first two stages, Parts III to V the derivation and the final formulation, Part VII and the Final Deliverable the last two. In the Final Deliverable the final equation, the pseudocode and the Rust for one operation sit in one row, and the three describe one and the same computation.

Labels used throughout

Every tagged equation and every operation carries one classification:

- **IMPLEMENTED**: a line of Rust computes exactly this expression. The text gives a link pinned to tag `v1.0.0` and the line range.
- **DERIVED NOT IMPLEMENTED**: the expression follows from the mathematics, or describes what the code does, but no line of code evaluates it.
- **FUTURE IMPROVEMENT**: the item is absent from v1.0.0.

Every numerical property and every statement about the behaviour of a solution carries one status:

- **PROVED**: the proof is written out here, under the assumptions stated with it, and any algebra in it is also checked by a script.
- **DEMONSTRATED NUMERICALLY**: no proof is given. A test in the repository, the output of `examples/verify.rs` or a script of Appendix A shows the property for specific cases, usually the synthetic five-layer input.
- **CLAIMED ONLY**: stated in a code comment, a document or a source, and neither proved here nor shown beyond a single case.

External results also carry a source label: **established textbook result**, **adapted from published literature** or **derived for STRATO**.

Other conventions

- **Equations and tables.** Equations are numbered by part, for example Eq. (4.12) is the twelfth tagged equation of Part IV. Tables and figures follow the same rule. Eq. (7.1) and Eq. (7.2) belong to Part VII.
- **Numbers.** Every number computed for the synthetic five-layer input is labelled calculated. The synthetic input (`inputs/synthetic/synthetic_five_layer.toml`) is a numerical benchmark chosen to test the code. It is not literature data for any TRISO particle or fission product.
- **Code links.** Links of the form src/numerical/finite_volume.rs, lines 50 to 99 point into the tagged source, so that they stay valid when the repository moves on.
- **Checks.** Quoted PASS lines, and the output blocks that start with `PASS`, are the verbatim output of the scripts listed in Appendix A. They keep the working symbol names of the scripts, which can differ from the notation of this document. Run a script from the repository root with `.venv/Scripts/python.exe -I -X utf8 <script>`.
- **Open points.** Statements that rest on a source audit that this document did not repeat, and the differences between the Phase 1 documents and the code, are collected in Part VII, Tables 7.3 to 7.5.

What v1.0.0 implements

The code models one intact, perfectly spherical particle of five concentric layers (kernel, buffer, inner pyrolytic carbon, silicon carbide, outer pyrolytic carbon). Each layer has one constant diffusivity. The layers touch perfectly, so the concentration and the flow are continuous across every interface. Substance reaches the outer surface and disappears there. At time zero the kernel holds a uniform concentration and the coatings are empty. There is no source and no radioactive decay. The temperature is read and never used. The code computes the fractional release in two independent ways, with Code B (finite volumes and Backward Euler) and with Code C (a Sturm-Liouville series), and Code A compares them and runs a refinement study. Assumptions A1 to A8 of Section 1.6 state the model in full, and Table 1.2 lists what is excluded.

Notation

Tables 0.1 to 0.4 collect every symbol of Parts I to VI, grouped by topic. A symbol that occurs only inside one proof is defined there and listed here only when it needs a unit or a code name. The Final Deliverable reproduces the rows it uses.

Conventions.

- Index letters have fixed roles. The letter ℓ numbers a layer (1 to 5), j a cell (1 to N_c), n a mode when it is a subscript and a time level when it is a superscript, m a second mode index and k a refinement level. A subscript such as the 1 in w_1 is read with the role of its own symbol family: cell width in Part V and modal weight only as w_n.
- Bold type marks vectors and matrices ($\mathbf{J}$, $\mathbf{n}$, $\mathbf{e}_r$, $\mathbf{T}_\ell$, $\mathbf{Q}_\ell$, $\mathbf{A}$, $\mathbf{B}$, $\mathbf{V}$, $\mathbf{S}$, $\mathbf{1}$). The italic letters S (source), T (temperature) and A (area) never denote the matrices of the same letter.
- Calligraphic letters mark counts, constants and proof functionals ($\mathcal{N}$, $\mathcal{B}_{mn}$, $\mathcal{E}$, $\mathcal{W}$, $\mathcal{Z}$, $\mathcal{K}$, $\mathcal{C}$, $\mathcal{R}$).
- Units are SI throughout. The word arbitrary marks the scale fixed by the shooting start $q(0) = 1$ (Section 4.5); every result built from such quantities (w_n, F) is independent of that scale.
- Dummy letters of a single proof (x, y, f, g, a, b, i and similar) are defined where they appear and carry no meaning in other sections. The last column of each table gives the section where the symbol is introduced.
- The temperature T in K is the argument of the diffusivity $D(T)$ in assumption A8. It is read and validated by the code and used in no formula of v1.0.0.

Table 0.1 Notation: Indices, counts, operators, constants and geometry and physics.

Symbol	Meaning	Unit	Rust name	Section
ℓ	layer index, 1 Kernel, 2 Buffer, 3 IPyC, 4 SiC, 5 OPyC	1	`mat` ($= \ell - 1$)	1.1.1
j	cell index $j = 1, \dots, N_c$ (the face between cells j and $j+1$ is "$j + 1/2$")	1	`i` ($= j - 1$)	5.1 (used earlier in 1.5 and 3.5.1)
n	subscript: mode index (Parts III, IV); superscript: time level (Parts V, VI); never both in one symbol	1	loop counters	4.1 (mode); 5.8.2 (level)

Symbol	Meaning	Unit	Rust name	Section
m	dummy integer: second mode index (4.8), Gauss-Legendre node index (4.11), geometric midpoint of a bracket (4.7), centroid midpoint (5.3.2), maximum (6.3.3), time level in a sum (6.8.4)	1	`m` (bracket midpoint)	4.8
i	dummy index and, in Part VI, the imaginary unit of $\zeta = x + iy$ (6.4.1)	1	(none)	2.0.3
n_ℓ	number of cells in layer ℓ (n_1 kernel cells)	1	`cells`	5.2
ι	offset $0, 1, \dots, n_\ell - 1$ of a cell inside its layer	1	`k` (loop variable)	5.2
n_{out}	output interval in steps	1	`output_every_steps`	5.11.6
N_c	total number of cells	1	`mesh.cells.len()`	5.2 (forward use in 3.3)
N_t	number of time steps	1	`steps`	5.9
N	number of equal shells in the sum of Section 1.5 (local to Part I)	1	(none)	1.5
$\mathcal{N}(\Lambda)$	Sturm count, number of eigenvalues $\leq \Lambda$	1	`eigenvalue_count`	4.6
π, e, d, ∂	circle constant, Euler number, differential, partial derivative	1	`PI`	1.2
Δr, Δt	small radial increment (Parts I, II); time step actually used, $\Delta t = t_{\text{end}}/N_t$ (Parts V, VI)	m; s	`dt`	2.2; 5.9
δV, δA	small volume and surface element (Part II)	m^3; m^2	(none)	2.0.1, 2.0.2
δ	small length: half-width of the interface neighbourhood (Section 3.2.2)	m	(none)	3.2.2
∇, $[X]$, $\lfloor\cdot\rfloor$, $\lceil\cdot\rceil$, max, min, diag (also typeset diag), Re, T	gradient and divergence; units of X (dimension check); floor and ceiling; maximum, minimum, diagonal matrix, real part, transpose	(none)	`floor`, `ceil`, `max`, `min`	2.0.3; 5.9
$O(\cdot)$	order symbol	1	(none)	2.4.4
r	radial distance from the particle centre; r' is the dummy radius of the thin-shell integral	m	`r` (closures), `radius_m` (CSV)	1.1.1
t	time	s	`t`, `time_s`	1.1.3
R_ℓ	outer radius of layer ℓ; $R_0 = 0$; R^- means "just inside R"	m	`outer_radius_m`, `Layer::outer_radius`	1.1.1
R	particle radius, $R = R_5$	m	`Mesh::outer_radius`	1.1.1
L_ℓ	thickness of layer ℓ, $R_\ell - R_{\ell-1}$; L^2 also names the square-integrable function space in Part VI (no clash in use)	m	`length`, `l.outer_radius - left`	1.1.1
D_ℓ	diffusivity of layer ℓ (constant in v1.0.0), $D_1, \dots, D_5$; D inside one layer; D_i, D_j, D_{j+1} cell values	$\text{m}^2\ \text{s}^{-1}$	`diffusivity_m2_s`, `Cell::diffusivity`, `Layer::diffusivity`, `d`	1.3

Symbol	Meaning	Unit	Rust name	Section
$D_{\min}$	smallest of $D_1, \dots, D_5$	$\mathrm{m^2\ s^{-1}}$	(none)	3.5.5
D_h	harmonic mean of D_j and D_{j+1} (Part V remark)	$\mathrm{m^2\ s^{-1}}$	(none)	5.5
D_L, D_R	diffusivity inside and outside an interface (Part VI truncation analysis)	$\mathrm{m^2\ s^{-1}}$	(none)	6.5.2
$c(r,t)$, c_ℓ	concentration; c_ℓ its restriction to layer ℓ ($c_1, \dots, c_5$, $c_{\ell+1}$); c', c'' radial derivatives	$\mathrm{mol\ m^{-3}}$	`c`	1.2
c_0	initial kernel concentration	$\mathrm{mol\ m^{-3}}$	`kernel_concentration_mol_m3`	3.4 (used in 1.3)
c^-	negative part $\min(c, 0)$ (local proof symbol)	$\mathrm{mol\ m^{-3}}$	(none)	3.5.5
c_{SiC}, c_{PyC}, c_{IPyC}	concentrations quoted from the Gelbard Section V example	$\mathrm{mol\ m^{-3}}$	(none)	3.2.1
c_∞	ambient concentration of the Robin condition (Case B reference)	$\mathrm{mol\ m^{-3}}$	(none)	3.6
J_r	radial diffusive flux, positive outward; J_x, J_y, J_z Cartesian components; $J_{j\pm1/2}$ at a face	$\mathrm{mol\ m^{-2}\ s^{-1}}$	(not stored)	1.3
$\mathbf{J}$, $\mathbf{n}$, $\mathbf{e}_r$	flux vector, outward unit normal, radial unit vector ($\mathbf{e}_x$ Cartesian)	$\mathrm{mol\ m^{-2}\ s^{-1}}$; 1; 1	(none)	2.0.2
S, S_j, S_i	volumetric source (zero in v1.0.0; FUTURE); S_j the value in cell j; S_i the value in layer i (notation of the Phase 1 document, quoted in Section 2.9)	$\mathrm{mol\ m^{-3}\ s^{-1}}$	(none)	2.1
$A(r)$	area of the sphere of radius r, $4\pi r^2$	$\mathrm{m^2}$	`area`, `outer_area`	1.4
$V(a,b)$	volume of the shell $a \le r \le b$; V also names a control volume (Part II)	$\mathrm{m^3}$	`shell_volume(a, b)`	1.4
$M(t)$	inventory (amount inside the particle); M^n at level n	mol	`inventory`	1.2 (dM), 1.5
M^0	initial inventory	mol	`initial`	1.5 (formula 3.5.1)
$M_{\mathrm{rel}}(t)$	cumulative amount released; M^n_{rel}; $M^0_{\mathrm{rel}} = 0$	mol	`released`, `cumulative_release`	3.5.4 (used in 3.5.3)
$F(t)$	fractional release $M_{\mathrm{rel}}/M^0 = 1 - M/M^0$; F^n at level n; F_0 the time-converged value on one mesh (6.6); $F(\Delta t)$	1	`release_fraction`	3.5.4
$\Pi_\ell(r,t)$	inward-counted flow $4\pi r^2 D_\ell \partial_r c_\ell = -4\pi r^2 J_r$	$\mathrm{mol\ s^{-1}}$	(none)	3.5.3
$\Phi(r,t)$, Φ_R, $\Phi_{j+1/2}$	outward flow through the sphere of radius r, $\Phi = A(r)J_r$; release rate $\Phi_R = \Phi(R,t)$; outward flow through face $j+1/2$ in Eq. (6.1)	$\mathrm{mol\ s^{-1}}$	`release_rate` (`release_rate_mol_s`)	2.3 (release rate: 3.5.2)

Symbol	Meaning	Unit	Rust name	Section
T	temperature, read and validated, used in no formula in v1.0.0 (argument of $D(T)$)	K	`temperature_k`	1.6 (assumption A8)
h_{m}, Bi	Robin mass-transfer coefficient and Biot number $h_{\mathrm{m}}R/D$ (Case B reference only)	m s^{-1}; 1	`bi`	3.6
z, z_n	dimensionless Robin eigenvalue and its roots of $z\cot z = 1 - \mathrm{Bi}$	1	`z`	3.6 (4.15)
$t_{\mathrm{D},\ell}$	diffusion time L_ℓ^2/D_ℓ	s	(none)	2.8
α, β	exponents in the dimensional-analysis example (local to Section 2.8)	1	(none)	2.8
$\mathcal{K}'$, $\mathcal{K}$	positive bound constants of two local arguments	see text	(none)	2.5.1; 3.1.2
γ_ℓ	Gelbard's partition factor, $= 1$ in v1.0.0 (FUTURE); γ_{SiC}, γ_{others} in his example	1	(none)	1.6 (A4); 3.2.1
$K_{\ell,\ell+1}$	ratio $c_{\ell+1}/c_\ell$ in a possible future convention (Section 3.2.1)	1	(none)	3.2.1
ζ_ℓ, ξ_ℓ; $\zeta_0, \zeta_1, \zeta_2$	coefficients of trial profiles: steady shell $\zeta_\ell + \xi_\ell/r$ and shape $(\zeta_\ell \sin kr + \xi_\ell \cos kr)/r$; centroid tests $\zeta_0 + \zeta_1 r$ and $\zeta_2 r^2$	as the profile	(none)	3.2.2; 5.3.2

Table 0.2 Notation: Analytical solution (Code C).

Symbol	Meaning	Unit	Rust name	Section
Λ, Λ_n	decay constant (eigenvalue) and n-th eigenvalue in ascending order (Λ_1, Λ_2, Λ_m, $\Lambda_{\ell+1}$)	s^{-1}	`lambda`, `lam`, `eigenvalues`	4.1 (used earlier in 2.5.1)
$\vartheta(t)$	time factor in $c = \phi\vartheta$	1	(none)	4.1
$\phi(r)$, ϕ_n	eigenfunction (concentration shape)	arbitrary	(none; only u is stored)	4.1 (used earlier in 1.3)
$u = r\phi$	transformed eigenfunction; u_n; u_{in}, u_{out} at the layer ends; $\widehat{u}_n = u_n/\sqrt{N_n}$ the unit-norm shape drawn in Figure 4.1	arbitrary $\times$ m	`u`	4.2 (used earlier in 1.3)
$q = D\,du/dr$	transformed flux-like state; q_{in}, q_{out}	arbitrary $\times$ m^2 s^{-1}	`q`, `du_flux`	4.3 (used earlier in 3.1.4)
q^-, q^+	q just inside and just outside an interface	as q	`q_inner`, return of `interface_state`	4.4
k_ℓ	wavenumber in layer ℓ, $\sqrt{\Lambda/D_\ell}$; k inside one layer; $k_n = n\pi/R$ in the homogeneous check	m^{-1}	`k`	4.2 (used in 2.6)
s	local coordinate in a layer, $r - R_{\ell-1}$	m	`s`	4.3
a_ℓ, b_ℓ	layer coefficients in $u = a_\ell \cos k_\ell s + b_\ell \sin k_\ell s$	as u	`a`, `b`	4.3

Symbol	Meaning	Unit	Rust name	Section
$\mathbf{T}_\ell$	2 by 2 layer transfer matrix, $\mathbf{T}_1, \dots, \mathbf{T}_5$; maps the state (u, q) at the layer entrance (subscript in) to the exit (subscript out)	mixed	`transfer_layer`	4.3
$\mathbf{Q}_\ell$	2 by 2 interface matrix (compact form of Eq. 4.8, not coded)	mixed	(none)	4.4.1
$g(\Lambda)$	characteristic function $u(R; \Lambda)$	as u	first value of `shoot`	4.5
H_ℓ, θ_0	amplitude and phase in $u = H_\ell \sin(k_\ell s + \theta_0)$; $\theta_{0,\ell}$ per layer	as u; rad	`theta0`	4.6
ν	integer counting multiples of π (local)	1	(none)	4.6
$Y(r)$	$r^2 D \, d\phi/dr$, continuous through the particle	arbitrary $\times$ m^3 s^{-1}	(none)	4.6
$\psi(r)$, $\Xi(r)$	Pruefer angle and amplitude, $\phi = \Xi \sin\psi$, $Y/Y_0 = \Xi \cos\psi$, with the reference flow $Y_0 = 1$ m^3 s^{-1}	rad; arbitrary	(none)	4.6
$t_{\min}$	smallest positive time requested	s	`t_min`	4.12
$\Lambda_{\max}$	requested upper eigenvalue, $45/t_{\min}$	s^{-1}	`lambda_max`	4.12
$K_{\max}$	mode cap, 20 000	1	`MAX_MODES`, `max_modes`	4.7
Λ_{cov}	largest eigenvalue bound actually covered	s^{-1}	`lambda_covered`	4.7
K	number of modes kept (Parts IV, VI); no other meaning after	1	`modes`, `eigenvalues.len()`	4.7
Λ_{lo}	lower bracket with $\mathcal{N} = 0$	s^{-1}	`lo`	4.7
a, b, n_a, n_b	ends of a bracket on the bisection stack and their counts; the code names `lo` and `hi`; m is the geometric midpoint	s^{-1}; 1	`lo`, `hi`, `m`	4.7
$\varpi(r)$	coefficient $r^2 D(r)$ of the Sturm-Liouville form	m$^2 \cdot$ m^2 s^{-1}	(none)	4.8
$\mathcal{B}_{mn}(r)$	boundary bracket $r^2 D(\phi_n \phi'_m - \phi_m \phi'_n)$ in the orthogonality proof	mixed	(none)	4.8
N_n	norm $\int_0^R u_n^2 \, dr$	arbitrary$^2 \times$ m^3	`norm`	4.9 (used in 3.5.1)
$I_{1,n}$, I_n	kernel and whole-particle moments $\int r \, u_n \, dr$	arbitrary $\times$ m^3	`i1`, `moment`	4.9 (formulas in 3.4 and 3.5.1)
A_n	expansion coefficient $c_0 I_{1,n}/N_n$ (DERIVED NOT IMPLEMENTED)	mol m^{-3} / arbitrary	(none)	4.9
σ	arbitrary nonzero scale factor of an eigenfunction (local)	1	(none)	4.5
w_n	modal weight $3 I_{1,n} I_n / (R_1^3 N_n)$; index n marks a mode, index ℓ a layer width	1	`weights`	4.10 (formula in 3.5.1)
$N_{\text{p},\ell}$	Gauss-Legendre panels in layer ℓ	1	`panels`	4.11

Symbol	Meaning	Unit	Rust name	Section
P_k, x_m, ω_m, ξ	Legendre polynomial; 16-point Gauss-Legendre nodes and weights on $[-1, 1]$; Newton variable ξ	1	`x`, `w` in `gauss_legendre_16`	4.11
h	width of one panel $L_\ell / N_{\mathrm{p},\ell}$ (Section 4.11); the same letter is a generic step elsewhere: derivative step (Section 3.1.2), cell width $b - a$ (Section 5.3.2), BE step (Section 6.5.3), step in the order model (Section 6.7); the Robin h is h_{m}	see text	(local)	4.11
mid_i	midpoint of panel i	m	`mid`	4.11
t_{valid}	earliest time the series is used, $40/\Lambda_{\mathrm{cov}}$	s	`min_valid_time()`, `min_valid_time_s`	4.12
ρ, $\rho_{\min}$	flux-cancellation ratio and its minimum	1	`min_flux_ratio`	4.13
ε_F	accuracy floor of the series on F	1	`accuracy_floor()`	4.13
ϵ_{m}	double-precision unit roundoff $2^{-52} \approx 2.2 \times 10^{-16}$ (the code uses the literal)	1	literal `2.2e-16`	4.13 (front matter of Part VI)
β_n, Ψ, $A_{j,n}$, $B_{j,n}$	Gelbard (2003) symbols quoted in Section 4.16 only: separation constant $-\beta_n^2$ with $\beta_n = \sqrt{\Lambda_n}$, layer shape Ψ, and coefficients of his Eq. (IV.13)	as quoted	(none)	4.16

Table 0.3 Notation: Finite-volume solution (Code B).

Symbol	Meaning	Unit	Rust name	Section
w_ℓ	uniform cell width in layer ℓ, L_ℓ / n_ℓ; w a generic cell width; w_L, w_R widths inside and outside an interface (Section 6.5.2)	m	`width`	5.2
$r_{j-1/2}$, $r_{j+1/2}$	inner and outer face of cell j; $r_{1/2} = 0$, $r_{N_c+1/2} = R$	m	`left`, `right`	5.2
r_j	volume centroid of cell j (r_1, r_2, r_{N_c})	m	`centroid`	5.3.2
r_P, r_E	inner and outer node radii of a shell (Patankar remark, local)	m	(none)	5.5
r_f	radius of a generic face (Section 6.5); r_k (Section 1.1.2) and r_i (Section 2.5.1) are local	m	(none)	6.5
s_j	centroid shift of cell j from its mid-radius (Section 5.5 remark)	m	(none)	5.5
V_j	volume of cell j (V_1, V_{N_c})	m^3	`volume`	5.3.1 (used in 1.5)
$A_{j+1/2}$	area of face $j + 1/2$ ($A_{1/2}$, $A_{j-1/2}$; A_f for a generic face)	m^2	`area`	5.3.3

Symbol	Meaning	Unit	Rust name	Section
$\bar{c}_j$, C_j, C_j^n	$\bar{c}_j$: exact cell average; C_j: discrete cell average and unknown of the semi-discrete system; C_j^n at level n; C_j^0 initial; C_{N_c} last cell; $C_{N_c+1} = 0$ ghost value	mol m^{-3}	`c`, `old`, `next`, `x`	5.1
$\delta^L_{j+1/2}$, $\delta^R_{j+1/2}$	distances from r_j and r_{j+1} to the face	m	`dl`, `dr`	5.5
$C_{j+1/2}$	concentration at the face (eliminated)	mol m^{-3}	(none)	5.5
$G_{j+1/2}$	internal face conductance ($G_{j-1/2}$; G_f generic)	m^3 s^{-1}	`conductance[j-1]`	5.5
G_R	outer-boundary conductance	m^3 s^{-1}	`outer_conductance`	5.6 (used in 3.3)
g_j^W, g_j^E	inner (west) and outer (east) conductance of cell j	m^3 s^{-1}	`gw[i]`, `ge[i]`	5.6, 5.7
$\mathbf{A}$	conductance matrix (symmetric, tridiagonal), entries A_{jj}, $A_{j,j\pm1}$	m^3 s^{-1}	(assembled implicitly)	5.8.2
$\mathbf{V}$	diagonal matrix of cell volumes	m^3	(none)	5.8.2
$\mathbf{B}$	one-step system matrix $\mathbf{V}/\Delta t + \mathbf{A}$, entries B_{jj}, $B_{j,j\pm1}$, B_{jk}	m^3 s^{-1}	(none)	5.8.2 (used in Part VI)
v_j	$V_j/\Delta t$	m^3 s^{-1}	`v`	5.8.2
rhs_j	right-hand side $v_j C_j^n$	mol s^{-1}	`rhs`	5.8.2
Δt_{in}, t_{end}	requested step and end time	s	`dt_s`, `end_time_s`	5.9 (and 2.8)
t_n, Δt_k	output time at level n (Section 5.11.6); time step of refinement level k, with $\Delta t_k = 2\Delta t_{k+1}$ (Section 6.7)	s	`time_s`; `dt_s` scaled by `m`	5.11.6
e_j, d_j, y_j	subtraction-free reduced diagonal, pivot and forward-sweep value	m^3 s^{-1}; m^3 s^{-1}; mol m^{-3}	`e`, `den`, `dp`	5.10.2 (pivot), 5.10.3 (reduced form)
υ, Σ, κ	Kahan: corrected increment and new sum, compensation term κ	mol	`y`, `t` (locals in `run`), `release_compensation`	5.11.4
ε^n	balance residual $M^n + M^n_{\mathrm{rel}} - M^0$	mol	`balance_residual`	5.11.5 (used in 3.5.3)
Fo	cell Fourier number $D\Delta t/w^2$	1	(comment only)	5.10.3 (6.4.5)
Δt_{FE}	forward Euler positivity bound $\min_j V_j/(g_j^W + g_j^E)$ (the scalar analogue $1/\mu$ appears first in Section 5.8.1)	s	`explicit_dt_bound`	6.4.4 (first mentioned in 5.8.1)
$y(t)$, y^n	scalar test equation $dy/dt = -\mu y$ and its BE sequence y^n; the abbreviation $x = \mu\Delta t$ used in V 5.8.1 and VI 6.4.5 is local	1	(none)	5.8.1

Table 0.4 Notation: Numerical properties, comparison and refinement (Part VI and Code A).

Symbol	Meaning	Unit	Rust name	Section
μ, μ_k, $\mu_{\min}$, $\mu_{\max}$	decay rate of the scalar test equation (6.4.1); eigenvalues of **S**; extremes (DERIVED NOT IMPLEMENTED)	s^{-1}	(none)	6.4.1 (eigenvalues in 6.4.2)
ζ, $\mathcal{R}(\zeta)$	BE amplification variable $-\mu\Delta t$ and factor $1/(1-\zeta)$	1	(none)	6.4.1
S	scaled matrix $\mathbf{V}^{-1/2}\mathbf{A}\mathbf{V}^{-1/2}$ (bold; the source is italic S)	s^{-1}	(none)	6.4.2 (named at the head of Part VI)
$\hat{\mathbf{C}}$, $\|\cdot\|_V$	scaled state $\mathbf{V}^{1/2}\mathbf{C}$ and the volume-weighted norm	$\mathrm{mol\ m}^{-3/2}$	(none)	6.4.2, 6.4.3
1, $\mathbf{e}_i$, **x**, **y**, **b**, **a**, **I**	ones vector, unit vector, generic vectors (**y** is an eigenvector of **S** in 6.4.4), identity matrix in Part VI proofs	1	(none)	6.1, 6.2, 6.3
j^*	index of the extreme entry in a maximum-principle proof (local)	1	(none)	6.3.3
θ, $\mathrm{fl}(\cdot)$	relative rounding error, $\|\theta\| \le \epsilon_{\mathrm{m}}/2$, and the floating-point result $\mathrm{fl}(a \circ b) = (a \circ b)(1+\theta)$	1	(none)	front matter of Part VI
$\circ$	one of $+, -, \times, /$ in the rounding model	1	(none)	front matter of Part VI
err_k, $\mathrm{err}(t)$	error at refinement level k and absolute difference $\|F^{\mathrm{FV}} - F^{\mathrm{ex}}\|$ (the plan's $e(t)$)	1	`abs_error`, `err`, `max_abs_error`, `final_abs_error`	6.7
p_k	observed order at level k; p the exponent in h^p	1	`orders`	6.7
$\mathcal{C}$	constant in the model error $\mathrm{err} = \mathcal{C}h^p$	see text	(none)	6.7
e_{sp}	spatial error contaminating the temporal study	1	(none)	6.7
$\varsigma_4, \varsigma_2, \varsigma_1$; χ_1, χ_2, χ_3	Richardson weights of the runs with $\Delta t/4$, $\Delta t/2$, Δt; coefficients of the error expansion	1; s^{-1}, s^{-2}, s^{-3}	(none)	6.6
F_{RE}	Richardson time-extrapolated release fraction	1	`time_extrapolated_release`	6.6
F^{FV}, F^{ex}	finite-volume and series release fraction	1	`fv`, `exact`, `exact_final`	6.4.5 (4.12 for F^{ex})
λ_{x}, λ_{t}	spatial and temporal refinement factors in Eq. (6.14)	1	`factor`, `m`	6.9
W, W_{study}, W_{lim}	cell-step cost of one run, of the study, limit 4×10^9	1	`run_cost`, `study_cost`, `STUDY_COST_LIMIT`	6.9
$\mathcal{E}(t)$, $\mathcal{W}(t)$, $\mathcal{Z}(t)$	proof functionals of Section 3.5.5: $\int c^2\, 4\pi r^2 dr$, $\int 4\pi r^2 D(\partial_r c)^2 dr$, $\int (c^-)^2\, 4\pi r^2 dr$	$\mathrm{mol}^2\ \mathrm{m}^{-3}$, per second for $\mathcal{W}$	(none)	3.5.5
$X(t)$	the factor $e^{-D_\ell k^2 t}$ of the trial shape	1	(none)	2.6

Symbol	Meaning	Unit	Rust name	Section
$\Omega(r)$, λ, a_i, b_i	local symbols of the positivity proofs in Section 3.5.5: test function, Cauchy-Schwarz parameter and numbers; the decay rate $a = 12D_{\min}/R^2$ (not a layer coefficient), the centre slope b of Section 3.1.1 and the interval ends a, b of Section 5.3.1 are further local uses of those letters	see text	(none)	3.1.1, 3.5.5
σ^m, $\tilde{\mathbf{C}}^m$	step residual vector and computed solution at level m in the balance-roundoff argument	mol s^{-1}; mol m^{-3}	(none)	6.8.4
η_j, η	discretisation error $C_j - \bar{c}_j$	mol m^{-3}	(none)	6.10
τ, $\tau_{j+1/2}$, τ_R, $\tau_{1/2}$, τ_f	truncation error of a face flow	mol s^{-1}	(none)	6.5.1
$\Delta\eta_f$	difference of η across face f	mol m^{-3}	(none)	6.10
x, y	Cartesian coordinates in the gradient formula of Section 2.0.3 (the third one is called z there, a local use of the letter of the Robin eigenvalue); other dummy uses: $x < y$ integers (Section 4.6), integration variable x (Section 4.11)	m	(none)	2.0.3
f, g (dummy)	generic functions in mean-value, integration and quadrature statements	as used	(none)	2.3 (3.5.3; 4.11)
ε (small)	see δ	m	(none)	2.2
β_k	coefficients of the leading truncation error of the face flow at the faces $r_f = kw$ near the centre, $k = 1, 2, 3$ (Section 6.5.2)	1	(none)	6.5.2

Part I: Physical foundation

This part describes the object that STRATO models, a single TRISO fuel particle, and sets up the few quantities every later part relies on: radius, concentration, flux, area, volume and inventory. It ends with the list of assumptions of v1.0.0, a table of what the code includes and leaves out, and a classification of every equation of the part. Equations are numbered (1.1) to (1.7). The checks quoted as P1-1 to P1-10 are the PASS lines of `scripts/derivation_checks/v1_0_0_p1_geometry.py` (17 PASS lines, ending in `ALL PASS`).

1.1 The TRISO particle

1.1.1 Five nested layers

TRISO stands for tristructural isotropic. A TRISO particle is a small sphere of nuclear fuel, the kernel, wrapped in four concentric coatings: a low-density carbon buffer, a dense inner pyrolytic carbon layer (IPyC), a silicon carbide layer (SiC) and a dense outer pyrolytic carbon layer (OPyC) (Morris et al., 2016, p. 24). Thousands of such

particles are mixed into a graphite matrix to make a fuel compact; an AGR-1 compact holds 4145 particles (Morris et al., 2016, Table 2). A good mental picture is an onion cut through its middle: each layer is a spherical shell sitting tightly on the one inside it. Figure 1.1 draws the cross-section.

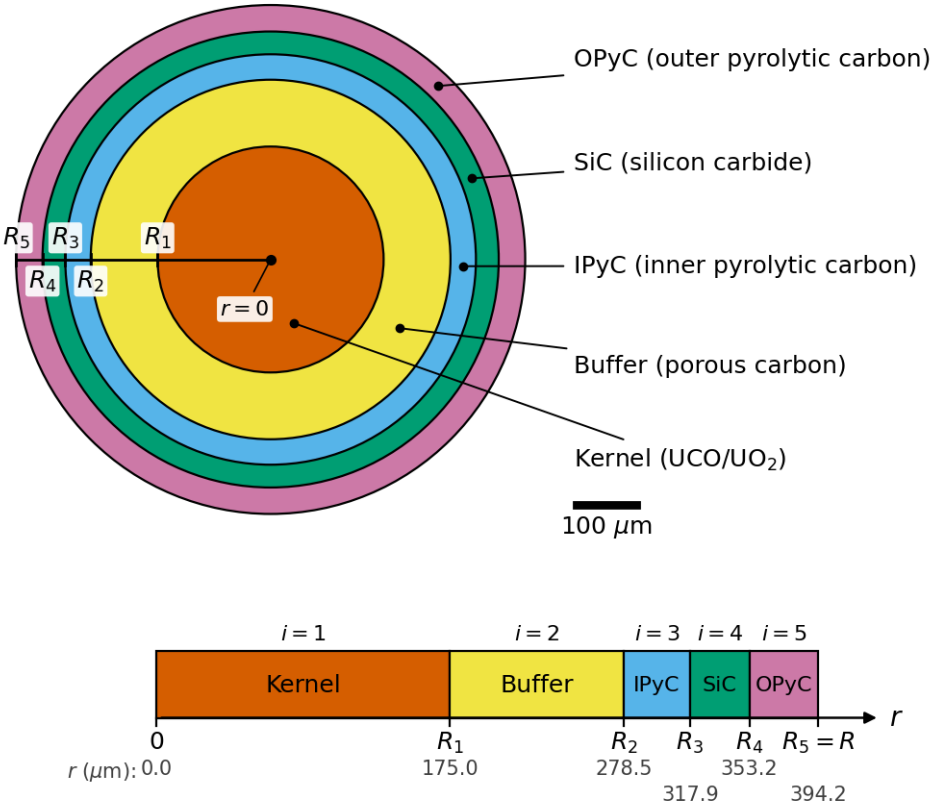


Figure 1.1: Cross-section of a TRISO particle showing the kernel and the four coatings (buffer, IPyC, SiC, OPyC) with their outer radii, drawn to scale for the AGR-1 baseline radii of Table 1.1 (Morris et al., 2016, Table 2); the figure numbers the layers 1 to 5 with the index i, which the text calls ell.

Each layer has a job. Table 1.1 lists the main functions given by the IAEA for a UO_2 TRISO particle (International Atomic Energy Agency [IAEA], 2012, Table 3.2, p. 51), together with the AGR-1 baseline dimensions measured by Morris et al. (2016, Table 2, p. 27). The AGR-1 kernel is UCO, a mixture of uranium oxide and uranium carbide (Morris et al., 2016, p. 24; Collin et al., 2016, p. 379). In UCO kernels the formation of carbon monoxide gas is suppressed and no kernel migration occurs (IAEA, 1997, Section 3.2.1, p. 46).

Table 1.1 Layers of a TRISO particle. Functions: IAEA (2012), Table 3.2 (written for the HTR-10 UO_2 particle). AGR-1 sizes: measured baseline averages, Morris et al. (2016), Table 2; Collin et al. (2016, p. 380) give the rounded nominal values 350, 100, 40, 35 and 40 μm. AGR-1 radii: calculated here by adding thicknesses to the kernel radius (P1-5). Last column: the bundled synthetic input `inputs/synthetic/synthetic_five_layer.toml`, which is a numerical benchmark and not literature data.

ℓ	Layer	Main functions (IAEA, 2012, Table 3.2)	AGR-1 size (μm)	AGR-1 R_ℓ (μm)	v1.0.0 input R_ℓ (μm)
1	Kernel (UO_2 or UCO)	source of fission; retains fission products; substrate for the buffer	diameter 350	175.0	200

ℓ	Layer	Main functions (IAEA, 2012, Table 3.2)	AGR-1 size (μm)	AGR-1 R_ℓ (μm)	v1.0.0 input R_ℓ (μm)
2	Buffer (porous carbon)	free volume for gaseous fission products; absorbs kernel swelling; shields the IPyC from fission-fragment recoil	103.5	278.5	300
3	IPyC (dense pyrolytic carbon)	protects the SiC from reactions with the fuel and rare-earth fission products; prevents reaction between the UO_2 kernel and chlorine-containing materials released during SiC deposition; carries part of the gas pressure	39.4	317.9	350
4	SiC (silicon carbide)	pressure vessel for CO, CO_2 and fission gases; barrier against release of gaseous and solid fission products	35.3	353.2	400
5	OPyC (dense pyrolytic carbon)	protects the SiC from mechanical damage during manufacture; extra barrier to fission gases if the SiC fails	41.0	394.2	450

Literature values against the bundled input. The AGR-1 numbers in Table 1.1 are published measurements (adapted from published literature). The only input file that v1.0.0 can run is synthetic: its header reads "SYNTHETIC NUMERICAL BENCHMARK - NOT LITERATURE TRISO DATA", its species is `"synthetic"`, and its five diffusivities were chosen to test the solvers. The literature template `inputs/literature/literature_inputs.template.toml` contains only `INSERT_VALUE_HERE` placeholders, and src/config.rs, lines 62 to 73 refuses to run it until they are filled. The synthetic radii happen to give a kernel that fills a similar share of the particle volume as in AGR-1: $(R_1/R)^3 = 0.08779$ for the synthetic input and 0.08749 for AGR-1 (P1-5c, P1-6c).

Radii and thicknesses. Number the layers from the centre outward with the layer index $\ell = 1, \dots, 5$ (1 Kernel, 2 Buffer, 3 IPyC, 4 SiC, 5 OPyC). Let r (m) be the distance from the centre of the particle and R_ℓ (m) the outer radius of layer ℓ. The centre is $R_0 = 0$ and the particle surface is $R = R_5$. Layer ℓ occupies $R_{\ell-1} \le r \le R_\ell$. Every layer must have a positive thickness, so the radii increase strictly (IMPLEMENTED: the input check in src/config.rs, lines 89 to 99 rejects any radius that is not larger than the previous one, and src/analytical/sturm_liouville.rs, lines 112 to 127 repeats the check for Code C):

$$0 = R_0 < R_1 < R_2 < R_3 < R_4 < R_5 = R \tag{1.1}$$

The thickness L_ℓ (m) of layer ℓ is the difference of its outer and inner radii (IMPLEMENTED: src/numerical/finite_volume.rs, line 55 divides it by the number of cells; src/analytical/sturm_liouville.rs, line 180 uses it as the layer length):

$$L_\ell = R_\ell - R_{\ell-1}, \qquad \ell = 1, \dots, 5 \tag{1.2}$$

For the kernel, $L_1 = R_1$ is simply the kernel radius. For the synthetic input, $L = 200, 100, 50, 50, 50$ μm (P1-6b). The radius axis below shows both sets of radii.

```
r = 0          R1          R2          R3          R4          R5 = R
  |  Kernel    |  Buffer    |  IPyC      |  SiC       |  OPyC      |
  0           200         300         350         400         450     v1.0.0 synthetic input (um)
  0         175.0       278.5       317.9       353.2       394.2   AGR-1 baseline, Morris et al. (2016)
↪  (um)
```

1.1.2 Fission products: from birth to release

Generation. When a uranium-235 nucleus in the kernel absorbs a neutron, it can split (fission) into two lighter nuclei, the fission products, plus a few neutrons and energy. The fission products are new chemical elements, for example silver, caesium, strontium and krypton, many of them radioactive.

Recoil. The two fragments fly apart at high speed, like the halves of a firecracker, and stop after a short distance. Fragments born close to the kernel surface can leave the kernel this way. In its discussion of gas release, the IAEA reports that 2 to 3 % of the recoiled fission-product atoms are ejected from the kernel and come to rest mainly in the buffer (IAEA, 1997, Section 3.3.1, p. 50). Protecting the IPyC from this recoil damage is one of the buffer functions in Table 1.1.

Diffusion. Once at rest inside a solid, an atom moves by occasional random jumps between sites. Each jump is equally likely in every direction, yet where atoms are crowded more of them jump out of a region than into it, so the population drifts from crowded places toward empty ones. A drop of ink spreading in still water behaves the same way. This net drift is diffusion, and Section 1.3 turns it into a formula.

Release. An atom that diffuses through all four coatings and crosses the outer surface $r = R$ has left the particle and entered the surrounding graphite matrix. STRATO calls this release and tracks the released fraction of the starting amount.

Species of interest. Collin et al. (2016, p. 378) modelled the release of silver, caesium, strontium and krypton in the AGR-1 safety tests at 1600 to 1800 °C, where Morris et al. (2016, p. 24) measured $^{110\mathrm{m}}$Ag, ^{134}Cs, ^{137}Cs, ^{154}Eu, ^{155}Eu, ^{90}Sr and ^{85}Kr. Two findings set the scene for a diffusion model. Caesium was very well retained by intact SiC, with a fractional release below 5×10^{-6} after 300 h at 1600 °C, but was released quickly from particles whose SiC had failed. The release of silver, europium and strontium in the heating tests appears to be dominated by inventory that had already passed through intact SiC during irradiation and was held in the compact (Morris et al., 2016, p. 24). The IAEA likewise names SiC as the coating that gives the highest degree of fission-product retention (IAEA, 1997, Section 4, p. 125). For orientation, the evaluated half-lives are 249.83 d for $^{110\mathrm{m}}$Ag, 2.0652 y for ^{134}Cs, 30.08 y for ^{137}Cs and 28.91 y for ^{90}Sr (ENSDF adopted values, retrieved from the LiveChart of Nuclides on 2026-11-05; International Atomic Energy Agency, Nuclear Data Section, n.d.). Version 1.0.0 contains none of these data: it follows one generic, non-decaying species whose properties come from the input file.

1.1.3 Why one radius is enough

A quantity has spherical symmetry when its value depends only on the distance from the centre and is the same in every direction at that distance. For a perfectly round particle with concentric layers, uniform properties within each layer and the same conditions all around its surface, nothing distinguishes one direction from another, so the concentration is a function of r and t alone. This reduces a three-dimensional problem to a one-dimensional one. Manufacturing aims at exactly this shape: the IAEA

stresses reproducible processes “achieving spherical kernels and spherical particles” (IAEA, 1997, Section 1), and diffusion analyses of TRISO particles “typically” assume perfect spherical geometry and use a one-dimensional mesh (Hales et al., 2021, Section 2.1.1).

The symmetry breaks when the particle stops looking the same in all directions. Two documented mechanisms do this:

- **Kernel migration (amoeba effect).** A temperature gradient across the particle, or an asymmetric kernel from production, drives carbon transport from the hot side to the cold side of the IPyC and moves the kernel toward the hot side (IAEA, 1997, Section 3.2.1, p. 46). The kernel is then off centre.
- **Failed or cracked layers.** A crack or local failure of the SiC opens a fast path on one side only. Morris et al. (2016, p. 24) found that the caesium release of a compact was dominated by the fraction of particles with defective or failed SiC.

For failure analysis, Hales et al. (2021, Section 2.1.1) note that two- and three-dimensional meshes are needed. Version 1.0.0 models one intact, perfectly spherical particle and includes neither mechanism.

1.2 Concentration and amount

Amounts of substance are counted in moles (mol); one mole is $6.02214076 \times 10^{23}$ atoms, much as a dozen is 12 eggs. The concentration $c(r, t)$ is the amount per unit volume at radius r and time t, in mol m^{-3}. It is a density of atoms: a high c means crowded, $c = 0$ means empty. Inside layer ℓ we write $c_\ell(r, t)$.

To find how much substance is present, cut the particle into thin spherical shells, like the skins of an onion. A shell between radius r and $r + dr$ has a tiny thickness dr. Its volume is its surface area times its thickness, $4\pi r^2\, dr$ (the area $4\pi r^2$ of a sphere and this thin-shell volume are justified in Section 1.4). Across so thin a shell the concentration is practically constant, so the amount in the shell is concentration times volume:

$$dM = c(r, t)\, 4\pi r^2\, dr \tag{1.3}$$

Here dM (mol) is the amount in the shell. Units check: mol m^{-3} $\times$ m^2 $\times$ m $=$ mol. Eq. (1.3) is the building block of every inventory in STRATO. The continuous form is DERIVED NOT IMPLEMENTED (no code evaluates it at a point); its cell-integrated counterpart, cell volume times cell-average concentration, is IMPLEMENTED in src/numerical/finite_volume.rs, line 118 (Part V, Eq. (5.21)).

1.3 Flux and Fick’s law

Flux. The radial flux $J_r(r, t)$ is the amount of substance that crosses one square metre of the sphere of radius r in one second, in mol m^{-2} s^{-1}. It is positive when the net flow points outward (toward larger r) and negative when it points inward. Think of people leaving a crowded hall through a wide doorway: the flux is how many pass through each metre of the doorway per second.

Rate of change in one direction. The concentration depends on two variables, r and t. The partial derivative $\partial c/\partial r$ is the slope of c against r with t held fixed: how fast the concentration changes as one walks outward at one instant. It is a

change of concentration (mol m^{-3}) divided by a change of distance (m), so its unit is mol $\mathrm{m}^{-3}/\mathrm{m}$ = mol m^{-4}. Likewise $\partial c/\partial t$ is the rate of change in time at a fixed radius.

Fick's first law. The random-jump picture of Section 1.1.2 says that the net flow runs from high to low concentration, and that it is larger when the concentration drops more steeply. The simplest law with both properties makes the flux proportional to the slope, with a minus sign so that the flow runs downhill (established textbook result; Crank, 1975):

$$J_r = -D\,\frac{\partial c}{\partial r} \tag{1.4}$$

- D is the diffusivity (diffusion coefficient) in $\mathrm{m}^2\ \mathrm{s}^{-1}$. A larger D means faster spreading. Its units are area per time, so calling it a speed (as the Phase 1 document did) is wrong; the distance over which a diffusing population spreads grows like $\sqrt{Dt}$ rather than in proportion to t. Part II, Section 2.8, shows why with units alone: the only time that can be built from a length L (in m) and D (in $\mathrm{m}^2\ \mathrm{s}^{-1}$) is L^2/D, so after a time t the only length available is $\sqrt{Dt}$ (in m). In v1.0.0 each layer has one constant value $D_\ell > 0$.
- Units: $\mathrm{m}^2\ \mathrm{s}^{-1} \times \mathrm{mol\ m}^{-4} = \mathrm{mol\ m}^{-2}\ \mathrm{s}^{-1}$, the flux unit (P1-7).
- Sign: if c falls outward, $\partial c/\partial r < 0$ and $J_r > 0$, an outward flow. For example, take $c = c_0(1 - r/R)$, which falls from c_0 at the centre to 0 at the surface. Its slope is $\partial c/\partial r = -c_0/R$, so the law gives $J_r = -D \times (-c_0/R) = Dc_0/R > 0$ (P1-8).

Assumptions behind Eq. (1.4): the material is isotropic (the same D in every direction) and D does not depend on c. The law also takes the flux to respond instantly to the local slope. In real TRISO layers D depends strongly on temperature and is an "effective" value that lumps several transport paths together (Collin et al., 2016, p. 381). Note that Collin et al. (2016, Eq. (2), p. 380) print the law as $J = D\nabla C$ without the minus sign; the minus sign is required for downhill flow, and Hales et al. (2021, Eq. (4)) print it with the minus sign, $J = -D\nabla C$.

Status in the code: IMPLEMENTED in discrete form. The flux J_r itself is never stored. Code B applies Eq. (1.4) across each pair of neighbouring cells through the face conductance in src/numerical/finite_volume.rs, lines 78 to 87 (Part V, Eqs. (5.8) and (5.9)), and between the last cell and the outer surface in src/numerical/finite_volume.rs, lines 88 to 91. Code C never forms J_r either. It enforces equal flux on both sides of each interface through `interface_state` in src/analytical/sturm_liouville.rs, lines 94 to 100, which keeps $D(u' - u/r)$ continuous (Part IV, Eqs. (4.7) and (4.8)). Here $\phi(r)$ is the radial shape of one term (mode) of the solution, $u = r\phi$ and the prime means d/dr. Since $u' = \phi + r\phi'$ and $u/r = \phi$, the kept quantity is $D(u' - u/r) = rD\phi'$. At a fixed interface radius r, keeping $rD\phi'$ continuous keeps $D\phi'$ continuous, which is the flux of Eq. (1.4) for that mode up to its minus sign.

1.4 Sphere area and shell volume

Area of a sphere. The surface of a sphere of radius r has area (established textbook result of geometry; IMPLEMENTED for every cell face in src/numerical/finite_volume.rs, line 83 and for the outer surface in line 90)

$$A(r) = 4\pi r^2 \tag{1.5}$$

in m^2. One way to see it: a ball of radius r has volume $\frac{4}{3}\pi r^3$. Painting a skin of thickness dr onto the ball adds volume equal to area times thickness, so the area is the rate at which the volume grows with radius. Differentiating the power r^3 brings down the exponent 3 and lowers the power by one:

$$\frac{d}{dr}\left(\frac{4}{3}\pi r^3\right) = \frac{4}{3}\pi \cdot 3r^2 = 4\pi r^2$$

(P1-1 confirms this and the direct surface integral). The key feature is that the area grows like r^2. For AGR-1 the particle surface is $(394.2/175.0)^2 = 5.07$ times as large as the kernel surface (P1-5d), so the same flux per square metre carries about five times as much substance per second across the particle surface as across the kernel boundary. This growth of area with radius is the origin of the $1/r^2$ factor in the spherical diffusion equation (Part II, Section 2.3).

Volume of a shell. The shell $a \le r \le b$ is a stack of thin skins, each of volume $A(r)\,dr$. Adding them is an integral.

Step 1. Volume as the sum of thin skins:

$$V(a,b) = \int_a^b A(r)\,dr = \int_a^b 4\pi r^2\,dr$$

Step 2. An antiderivative of $4\pi r^2$ is $\frac{4\pi}{3}r^3$, because its derivative is $\frac{4\pi}{3}\cdot 3r^2 = 4\pi r^2$. Evaluate it at the two limits and subtract (IMPLEMENTED as `shell_volume` in src/numerical/finite_volume.rs, lines 38 to 40; established textbook result, checked by P1-2):

$$V(a,b) = \frac{4\pi}{3}\left(b^3 - a^3\right) \tag{1.6}$$

in m^3. This is the volume of the ball of radius b minus the ball of radius a, which is what a shell is. Layer ℓ has volume $V(R_{\ell-1}, R_\ell)$, and the five layer volumes add up to the whole particle. Writing the sum out, each $-R_\ell^3$ of one layer cancels the $+R_\ell^3$ of the layer inside it, and only the first and last terms survive (P1-4):

$$\begin{aligned}\sum_{\ell=1}^{5} V(R_{\ell-1}, R_\ell) &= \frac{4\pi}{3}\left[(R_1^3 - R_0^3) + (R_2^3 - R_1^3) + (R_3^3 - R_2^3)\right.\\ &\qquad \left. + (R_4^3 - R_3^3) + (R_5^3 - R_4^3)\right]\\ &= \frac{4\pi}{3}\left(R_5^3 - R_0^3\right) = \frac{4\pi}{3}R^3\end{aligned}$$

The last step uses $R_0 = 0$ and $R_5 = R$ from Eq. (1.1).

Thin-shell limit. Eq. (1.3) used the volume $4\pi r^2\,dr$ for a thin shell. To see where it comes from, set $a = r$ and $b = r + \Delta r$ in Eq. (1.6) and expand the cube $(r+\Delta r)^3 = r^3 + 3r^2\Delta r + 3r\Delta r^2 + \Delta r^3$:

$$\begin{aligned} V(r, r+\Delta r) &= \frac{4\pi}{3}\left(r^3 + 3r^2\Delta r + 3r\,\Delta r^2 + \Delta r^3 - r^3\right) \\ &= 4\pi r^2\,\Delta r + 4\pi r\,\Delta r^2 + \frac{4\pi}{3}\Delta r^3 \end{aligned}$$

Divide by Δr: $V/\Delta r = 4\pi r^2 + 4\pi r\,\Delta r + \frac{4\pi}{3}\Delta r^2$. As $\Delta r \to 0$ the last two terms vanish, leaving $4\pi r^2$ (P1-3, P1-3b). So for a thin shell the volume is area times thickness. The size of the error follows from dividing the two neglected terms by the kept term $4\pi r^2\Delta r$: the relative error is $\Delta r/r + \Delta r^2/(3r^2)$, which is small when $\Delta r \ll r$. Part V uses Eq. (1.6) exactly for every cell, so the finite-volume code never relies on the thin-shell approximation.

1.5 Inventory

The inventory $M(t)$ (mol) is the total amount of substance inside the particle. Add the amounts of Eq. (1.3) over all thin shells from the centre to the surface. With N shells of thickness $\Delta r = R/N$ and midpoints r_k, the sum is $\sum_k c(r_k, t)\,4\pi r_k^2\,\Delta r$. As N grows the sum approaches a definite limit, which is the integral (definition of the integral as a limit of sums; P1-9 shows the sum converging to the integral with error proportional to Δr^2, observed orders 1.9998, 2.0000, 2.0000):

$$M(t) = \int_0^R c(r, t)\,4\pi r^2\,dr \tag{1.7}$$

Because the layers do not overlap, the integral splits into one integral per layer, $M = \sum_{\ell=1}^{5} \int_{R_{\ell-1}}^{R_\ell} c_\ell\,4\pi r^2\,dr$. The factor $4\pi r^2$ gives outer shells more weight than inner ones: per unit of thickness, the same concentration holds far more substance near the surface than near the centre.

Example. In the v1.0.0 starting state the kernel has the uniform concentration c_0 and the coatings are empty (Part III, Eq. (3.5)). Only the kernel term survives. Because c_0 is a constant it moves outside the integral, and the remaining integral is Eq. (1.6) with $a = 0$, $b = R_1$:

$$M = c_0 \int_0^{R_1} 4\pi r^2\,dr = c_0\,\frac{4\pi}{3}\left(R_1^3 - 0^3\right) = \frac{4\pi}{3} R_1^3 c_0$$

For the synthetic input ($R_1 = 2.0 \times 10^{-4}$ m, $c_0 = 1.0$ mol m^{-3}) the numbers are $R_1^3 = 8.0 \times 10^{-12}$ m^3 and $\frac{4\pi}{3} = 4.18879\ldots$, so $M = 4.18879\ldots \times 8.0 \times 10^{-12}$ m^3 $\times$ 1.0 mol m^{-3} $= 3.351032163829113 \times 10^{-11}$ mol. This is identical in every printed digit to the initial inventory the code writes in the first row of its regression output `tests/golden/v1.0/timeseries.csv` (P1-10; calculated, synthetic input; Part III, Eq. (3.6)).

Status in the code: the continuous integral Eq. (1.7) is DERIVED NOT IMPLEMENTED. Its cell form is IMPLEMENTED: `inventory` in src/numerical/finite_volume.rs, lines 114 to 120 computes $\sum_j V_j C_j$, which equals Eq. (1.7) exactly when C_j is the cell average of c (Part V, Eqs. (5.1) and (5.21)). Code C never forms $M(t)$ in moles. It evaluates the remaining fraction $M(t)/M^0$ from modal weights built from moment integrals in `remaining_fraction`, src/analytical/sturm_liouville.rs, lines 382 to 388 (Part IV, Eqs. (4.21) and (4.22)).

1.6 Assumptions of v1.0.0

The model of Parts II to VII rests on eight assumptions. The module documentation states the core of them in src/lib.rs, lines 11 to 12.

- **A1. Spherical symmetry.** $c = c(r, t)$ only (Section 1.1.3).
- **A2. Fickian diffusion with constant diffusivity per layer.** Eq. (1.4) holds in each layer with one constant, isotropic $D_\ell > 0$ that does not change with time or temperature. It does not depend on c either.
- **A3. Perfect contact at interfaces.** The concentration is continuous and all substance leaving one layer enters the next (Part III, Eqs. (3.2) and (3.3)). This is Gelbard's interface condition with partition factor $\gamma_\ell = 1$ (Gelbard, 2003, Eq. (IV.3)). Measured partition coefficients range from about 0.3 to 3 (IAEA, 1997, p. 436), so A3 is a simplification.
- **A4. No source and no decay.** No fission products are created after $t = 0$ and none decay.
- **A5. Absorbing outer surface.** Anything that reaches $r = R$ is removed at once: $c(R, t) = 0$ (Part III, Eq. (3.4)).
- **A6. Kernel-only start.** At $t = 0$ the kernel holds a uniform concentration c_0 and the four coatings are empty (Part III, Eq. (3.5)). Recoil deposits in the buffer are ignored.
- **A7. Fixed, intact geometry.** Radii do not change; no layer cracks or fails; the kernel stays centred.
- **A8. Temperature read but unused.** `temperature_k` is read and checked to be positive and finite in src/config.rs, lines 86 to 88, but no formula in v1.0.0 uses it. The diffusivities D_ℓ are given directly in the input.

Table 1.2 sets these assumptions against the physics of Section 1.1. The "Planned version" column follows the development table of the repository README at tag v1.0.0; "not scheduled" means that no version is assigned to the item there.

Table 1.2 Scope of STRATO v1.0.0. Planned versions as stated in `README.md` at tag v1.0.0 (development table; the AGR-1 comparison is assigned to v2.0 in the verification section, and literature inputs to "later versions" in the objective).

Topic	In scope in v1.0.0	Excluded from v1.0.0	Planned version
Geometry	one intact, perfectly spherical particle of five concentric layers	kernel migration, cracked or failed layers, aspherical particles	not scheduled
Particles and surroundings	single particle with an absorbing surface	particle populations, matrix transport	v3.0
Transport law	Fickian diffusion, constant D_ℓ per layer	Arrhenius $D(T)$, temperature histories	v2.0
Creation of fission products	initial inventory in the kernel only	fission-product generation during the run	v2.0

Topic	In scope in v1.0.0	Excluded from v1.0.0	Planned version
Recoil	none	recoil deposition in the buffer	not scheduled
Radioactive decay	none	radioactive decay	v2.0
Interfaces	perfect contact, $\gamma_\ell = 1$	partition coefficients	v3.0
Outer boundary	perfect sink, $c(R, t) = 0$	finite surface mass transfer (Robin condition) for the layered particle; only the homogeneous-sphere root equation exists (Part IV, Section 4.15)	not scheduled
Chemical effects	none	palladium penetration into SiC (IAEA, 1997, Section 3.2.2, p. 47), trapping, sorption	not scheduled
Input data	synthetic benchmark, species `"synthetic"`	literature diffusivities and AGR-1 geometry; comparison with measured AGR-1 release	v2.0 for the AGR-1 comparison; literature inputs “in later versions”
Parameter studies	refinement study of the numerical error only	inverse modelling, uncertainty quantification	v3.0

1.7 Summary and classification

Table 1.3 gives every equation of Part I its classification, its source label and its status.

Table 1.3 Classification of the equations of Part I. Line numbers refer to tag v1.0.0; the pinned links are given where each equation is introduced.

Eq.	Content	Classification	Source label	Status
(1.1)	radii increase strictly from $R_0 = 0$	IMPLEMENTED as an input check: `src/config.rs` 89 to 99; `src/analytical/sturm_liouville.rs` 112 to 127	derived for STRATO (definition)	definition, enforced by the code
(1.2)	layer thickness $L_\ell = R_\ell - R_{\ell-1}$	IMPLEMENTED: `src/numerical/finite_volume.rs` 55; `src/analytical/sturm_liouville.rs` 180	derived for STRATO (definition)	definition
(1.3)	amount in a thin shell $dM = c\,4\pi r^2\,dr$	DERIVED NOT IMPLEMENTED (cell form IMPLEMENTED, `src/numerical/finite_volume.rs` 118)	derived for STRATO	PROVED from Eq. (1.6) by the thin-shell limit; DEMONSTRATED NUMERICALLY by P1-3, P1-3b
(1.4)	Fick’s first law $J_r = -D\,\partial c/\partial r$	IMPLEMENTED in discrete form: `src/numerical/finite_volume.rs` 78 to 91	established textbook result (Crank, 1975)	CLAIMED ONLY (constitutive law, assumption A2); units and sign DEMONSTRATED NUMERICALLY by P1-7, P1-8
(1.5)	sphere area $4\pi r^2$	IMPLEMENTED: `src/numerical/finite_volume.rs` 83 and 90	established textbook result	PROVED from the ball volume $\frac{4}{3}\pi r^3$, which is taken as known; DEMONSTRATED NUMERICALLY by P1-1
(1.6)	shell volume $\frac{4\pi}{3}(b^3 - a^3)$	IMPLEMENTED: `shell_volume`, `src/numerical/finite_volume.rs` 38 to 40	established textbook result	PROVED; DEMONSTRATED NUMERICALLY by P1-2, P1-4

Eq.	Content	Classification	Source label	Status
(1.7)	inventory $M = \int_0^R c\, 4\pi r^2\, dr$	DERIVED NOT IMPLEMENTED (cell form $\sum_j V_j C_j$ IMPLEMENTED, `src/numerical/finite_volume.rs` 114 to 120)	derived for STRATO	PROVED as the limit of the shell sum; DEMONSTRATED NUMERICALLY by P1-9, P1-10

Part II: Governing conservation equation

Part I described the particle and defined the concentration $c(r, t)$ and the radial flux J_r. This part turns one physical rule, "amount of substance is neither created nor destroyed unless a source makes it", into the partial differential equation that both STRATO solvers work with. The equation is derived twice. The first route uses nothing but a thin spherical shell and the definition of a derivative. The second route uses the divergence of a vector field and the divergence theorem. Both routes give the same result, and seeing both explains where the factor $1/r^2$ in the spherical equation comes from.

A summary of the outcome. No equation of this part is evaluated directly by the code. The code works with consequences of these equations: Code B with the cell balance of Part V, which is Eq. (2.2) applied to one cell and then approximated, and Code C with the separated form of Eq. (2.7) in Part IV. Section 2.9 lists every equation with its classification and the code that depends on it.

2.0 The quantities that enter the balance

This section recalls the quantities of Part I that the balance needs and adds what Part II uses beyond them: the flux as a vector and the units of D.

2.0.1 Concentration

Concentration $c(r, t)$ is the amount of substance per unit volume at radius r and time t, in mol m^{-3}. Picture the particle filled with a fine fog of atoms. Cut out a tiny cube of volume δV around a point and count the atoms inside. Divide the count by the Avogadro constant to get moles, then divide by δV. That ratio, in the limit of a cube small compared with the particle but still holding many atoms, is c.

Why moles and not kilograms. The rule that drives this whole part is a counting rule: atoms that leave one region must appear in another. Moles count atoms (one mole is a fixed number of entities), so a balance written in moles is a balance of atoms. A mass balance would also be correct for a single species, since mass is moles times a constant molar mass, but moles keep the bookkeeping identical for every species and match how fission products are produced (by number of fissions). The code uses moles throughout: the input field is `kernel_concentration_mol_m3` (src/config.rs, lines 15 to 18), and the reported release fraction F is a ratio of two amounts, so its value does not depend on the unit chosen for amount.

The amount in a region is found by adding concentration times volume over the region. For a thin spherical shell of radius r and thickness dr this is Eq. (1.3) of Part I, $dM = c\, 4\pi r^2\, dr$. For a general region V we write

$$\text{amount in } V = \int_V c\, dV,$$

where dV is a small piece of volume (in m^3) and the integral adds $c\, dV$ (in mol) over all pieces.

2.0.2 Flux as a vector

Flux describes motion of substance. Hold a small flat frame of area δA at a point and count the moles that pass through it in one second. Divide by δA. The result depends on how the frame is turned: facing the flow it catches the most, turned edge-on it catches nothing. The flux vector $\mathbf{J}$ (in mol m^{-2} s^{-1}) packages this. Its direction is the direction of net motion and its length is the amount per second per square metre through a frame that faces the motion squarely. Through a frame whose unit normal (a vector of length one standing perpendicular to the frame) is $\mathbf{n}$, the amount per second is

$$\mathbf{J} \cdot \mathbf{n}\, \delta A.$$

The dot product $\mathbf{J} \cdot \mathbf{n}$ keeps only the part of $\mathbf{J}$ that points through the frame. The part that slides along the frame does not cross it.

In a spherically symmetric particle nothing distinguishes one direction from another except “in” and “out”. The flux must therefore point along the radius:

$$\mathbf{J} = J_r(r, t)\, \mathbf{e}_r,$$

where $\mathbf{e}_r$ is the unit vector pointing radially outward and J_r is the radial component in mol m^{-2} s^{-1}. $J_r > 0$ means net outward motion.

2.0.3 Fick’s first law

Diffusion moves substance from where it is crowded to where it is sparse, and the steeper the drop in concentration the faster the flow. Fick’s first law (established textbook result; Crank, 1975) states this as a proportionality:

$$\mathbf{J} = -D\, \nabla c.$$

The gradient ∇c is the vector of the three slopes of c along the three axes, $\nabla c = (\partial c/\partial x,\ \partial c/\partial y,\ \partial c/\partial z)$, in mol m^{-4}. It points in the direction in which c increases fastest (uphill) and its length is the rate of increase per metre in that direction. The minus sign turns the flux downhill. The proportionality factor D is the diffusivity.

In spherical symmetry c depends on position only through $r = \sqrt{x^2 + y^2 + z^2}$. First find how r changes with x: differentiate $r^2 = x^2 + y^2 + z^2$ with respect to x to get $2r\, \partial r/\partial x = 2x$, so $\partial r/\partial x = x/r$. The chain rule (a change in x changes r, which changes c) then gives $\partial c/\partial x = (\partial c/\partial r)(\partial r/\partial x) = (\partial c/\partial r)(x/r)$, and similarly for y and z. The vector $(x, y, z)/r$ has length $\sqrt{x^2 + y^2 + z^2}/r = 1$ and points away from the centre, so it is $\mathbf{e}_r$. Hence

$$\nabla c = \frac{\partial c}{\partial r}\,\frac{(x,y,z)}{r} = \frac{\partial c}{\partial r}\,\mathbf{e}_r.$$

Comparing with $\mathbf{J} = J_r \mathbf{e}_r$ gives the radial form used throughout STRATO, which is Eq. (1.4) of Part I:

$$J_r = -D\,\frac{\partial c}{\partial r}.$$

Units of D. Solve for D and insert units:

$$[D] = \frac{[J_r]}{[\partial c/\partial r]} = \frac{\text{mol m}^{-2}\ \text{s}^{-1}}{\text{mol m}^{-3}\,/\,\text{m}} = \frac{\text{mol m}^{-2}\ \text{s}^{-1}}{\text{mol m}^{-4}} = \text{m}^2\ \text{s}^{-1}.$$

A diffusivity has units of area per time, whereas a speed has units m s^{-1}. The delivered Phase 1 derivation calls D_ℓ a "constant speed" (Table 7.3, item I18), which this document corrects. Area per time reflects the way diffusion works: the distance spread grows with the square root of time, so the quantity that grows in proportion to time is the square of the distance. Section 2.8 uses this to estimate how long diffusion takes across each layer.

In the code, D_ℓ is read per layer as `diffusivity_m2_s` (src/config.rs, lines 8 to 13) and must be positive and finite (src/config.rs, lines 91 to 96).

2.1 Conservation in words

Physical problem. Substance diffuses through the particle. We want an equation that links how fast c changes at a point to how J varies near that point.

Governing principle. Pick a region of space, called a control volume. It is a fixed, imaginary closed surface, like a net bag hung in place: substance passes through its surface freely, and the bag itself never moves or changes shape. For any control volume, over any short time,

change of the amount inside = amount that flowed in − amount that flowed out + amount created inside.

Per unit time, each item becomes a rate:

- Amount inside: $\int_V c\,dV$ (Section 2.0). Its rate of change is $\frac{d}{dt}\int_V c\,dV$, in mol s^{-1}.
- Net outflow: split the closed surface ∂V into small patches dA, each with outward unit normal $\mathbf{n}$. The amount leaving per second through one patch is $\mathbf{J}\cdot\mathbf{n}\,dA$ (negative where substance enters). Adding over the closed surface gives $\oint_{\partial V} \mathbf{J}\cdot\mathbf{n}\,dA$, in mol s^{-1}. The circle on the integral sign marks a closed surface.
- Creation: a volumetric source S (in mol m^{-3} s^{-1}) creates $S\,dV$ moles per second in each piece of volume, $\int_V S\,dV$ in total.

Writing the word statement with these symbols gives the integral conservation law:

$$\frac{d}{dt}\int_V c\,dV = -\oint_{\partial V} \mathbf{J}\cdot\mathbf{n}\,dA + \int_V S\,dV \tag{2.1}$$

Table 2.1 Symbols of Eq. (2.1).

Symbol	Meaning	SI unit
V, ∂V	fixed control volume and its closed boundary surface	m^3, m^2
c	concentration	mol m^{-3}
$\mathbf{J}$	flux vector	mol m^{-2} s^{-1}
$\mathbf{n}$	outward unit normal of ∂V	1
S	volumetric source	mol m^{-3} s^{-1}

Assumptions. (i) The control volume does not move, so its boundary does not sweep substance in or out. (ii) Substance moves only by the flux $\mathbf{J}$; nothing is created or destroyed on surfaces, which includes layer interfaces (perfect contact, assumption list of Part I, Section 1.6). Beyond these, the fields only need to be integrable and the amount inside V must change smoothly in time. Eq. (2.1) needs no smoothness in space: the profile c may have a kink inside V, with the slope $\partial c/\partial r$ jumping, as happens at a layer interface where D changes. This is why Part III derives the interface conditions from Eq. (2.1) and not from the local equations below.

Meaning. The minus sign in front of the surface integral is there because $\mathbf{n}$ points outward: a positive $\mathbf{J}\cdot\mathbf{n}$ is a loss.

Classification. Eq. (2.1) is the conservation of amount of substance (established textbook result; Crank, 1975). DERIVED NOT IMPLEMENTED as a separate computation: Code B applies it to each cell (Part V, Eq. (5.6)), approximates the face flows (Part V, Eqs. (5.8) and (5.9)) writes the resulting semi-discrete system (Part V, Eq. (5.12)) and steps it in time by backward Euler (Part V, Eqs. (5.13) and (5.14)); the resulting cell equations are assembled in `backward_euler_step` (src/numerical/finite_volume.rs, lines 162 to 182). The source term is carried here for generality only. In v1.0.0, $S = 0$: the right-hand side of each cell is `rhs[i] = v[i] * old[i]` with no source contribution (src/numerical/finite_volume.rs, line 179). A nonzero S (fission production, radioactive decay as a negative source) is a FUTURE IMPROVEMENT (Table 7.3, item I21).

2.2 Balance on a thin shell

What we want. Apply Eq. (2.1) to the simplest region that respects the symmetry of the particle: a thin spherical shell. This is route (a) to the local equation.

Setup. Take the shell $r \le r' \le r + \Delta r$, with $r > 0$ and thickness $\Delta r > 0$ (in m). Here r' is only a dummy integration variable for radius inside the shell; it has nothing to do with the flux ratio r' of Part IV. The shell's boundary consists of two spheres:

- the outer wall, radius $r + \Delta r$, area $A(r + \Delta r) = 4\pi(r + \Delta r)^2$ by Eq. (1.5);
- the inner wall, radius r, area $A(r) = 4\pi r^2$.

Step 1: the outward normal on each wall. "Outward" means out of the shell. On the outer wall that is away from the centre, $\mathbf{n} = +\mathbf{e}_r$. On the inner wall that is towards the centre, $\mathbf{n} = -\mathbf{e}_r$.

Step 2: $\mathbf{J}\cdot\mathbf{n}$ on each wall. With $\mathbf{J} = J_r\mathbf{e}_r$ and $\mathbf{e}_r\cdot\mathbf{e}_r = 1$:

$$\mathbf{J}\cdot\mathbf{n} = \begin{cases} +J_r(r+\Delta r,t) & \text{on the outer wall,} \\ -J_r(r,t) & \text{on the inner wall.} \end{cases}$$

Step 3: integrate over each wall. By symmetry J_r has the same value at every point of a sphere of given radius, so the surface integral over that sphere is the value times the area:

$$\begin{aligned} \oint_{\partial V} \mathbf{J}\cdot\mathbf{n}\, dA &= \int_{\text{outer}} J_r(r+\Delta r,t)\, dA + \int_{\text{inner}} \left(-J_r(r,t)\right) dA \\ &= A(r+\Delta r)\, J_r(r+\Delta r,t) - A(r)\, J_r(r,t). \end{aligned}$$

The first term is the amount per second leaving through the outer wall; the second is the amount per second entering through the inner wall.

Step 4: amount and source inside the shell. Adding thin sub-shells of thickness dr', each holding $c\,4\pi r'^2\,dr'$ (Eq. (1.3)), gives the amount in the shell; the source is added the same way:

$$\int_V c\, dV = \int_r^{r+\Delta r} c(r',t)\, 4\pi r'^2\, dr', \qquad \int_V S\, dV = \int_r^{r+\Delta r} S(r',t)\, 4\pi r'^2\, dr'.$$

Step 5: insert into Eq. (2.1). The amount in the shell depends on t and on the chosen r; holding r and Δr fixed, the time derivative is a partial derivative. The minus sign of Eq. (2.1) distributes over the two wall terms of Step 3:

$$\frac{\partial}{\partial t}\int_r^{r+\Delta r} c\, 4\pi r'^2\, dr' = A(r)\, J_r(r) - A(r+\Delta r)\, J_r(r+\Delta r) + \int_r^{r+\Delta r} S\, 4\pi r'^2\, dr' \quad (2.2)$$

(the argument t is dropped for brevity).

Meaning. Rate of increase in the shell = inflow through the inner wall − outflow through the outer wall + creation. Each term is in mol s^{-1}. No approximation has been made: Eq. (2.2) is exact for any thickness, including a shell that contains a layer interface. That exactness is what a finite-volume method exploits; Part V, Eq. (5.6), is Eq. (2.2) written for one computational cell with $S = 0$.

Classification. Derived for STRATO from Eq. (2.1). DERIVED NOT IMPLEMENTED in this continuous form. Its cell form (5.6), once the face flows are approximated by Eqs. (5.8) and (5.9) and time is discretised by backward Euler, is IMPLEMENTED through the cell coefficients of src/numerical/finite_volume.rs, lines 162 to 182. The wall areas $4\pi r^2$ appear in the code as `area` for the faces between cells (src/numerical/finite_volume.rs, line 83) and `outer_area` for the particle surface (line 90).

2.3 Thin-shell limit and the $1/r^2$ factor

What we want. Shrink the shell to zero thickness to obtain an equation that holds at each radius. This assumes c, J_r and S are smooth (continuously differentiable) near r, which is true inside a layer but not across an interface; interfaces are treated in Part III.

Step 1: divide Eq. (2.2) by Δr. Valid because $\Delta r > 0$:

$$\frac{1}{\Delta r}\frac{\partial}{\partial t}\int_r^{r+\Delta r} c\,4\pi r'^2\,dr' = -\frac{A(r+\Delta r)J_r(r+\Delta r) - A(r)J_r(r)}{\Delta r} + \frac{1}{\Delta r}\int_r^{r+\Delta r} S\,4\pi r'^2\,dr'.$$

The flux terms were reordered as (outer minus inner) with one minus sign in front, so that the fraction has the shape of a difference quotient.

Step 2: the storage term. The limits of the integral, r and $r+\Delta r$, do not depend on t, and c is smooth, so the time derivative may be taken inside the integral: $\frac{\partial}{\partial t}\int c\,4\pi r'^2\,dr' = \int \frac{\partial c}{\partial t}\,4\pi r'^2\,dr'$. For a continuous integrand $f(r')$ the mean value theorem for integrals gives $\int_r^{r+\Delta r} f\,dr' = f(r'^*)\,\Delta r$ for some r'^* between r and $r+\Delta r$. In words: the area under a continuous curve over an interval equals the interval width times the average height, and a continuous curve actually takes its average height at some point r'^* of the interval. With $f = \frac{\partial c}{\partial t}\,4\pi r'^2$:

$$\frac{1}{\Delta r}\int_r^{r+\Delta r}\frac{\partial c}{\partial t}\,4\pi r'^2\,dr' = \frac{\partial c}{\partial t}(r'^*,t)\,4\pi(r'^*)^2 \;\longrightarrow\; 4\pi r^2\,\frac{\partial c}{\partial t}(r,t) \quad \text{as } \Delta r \to 0,$$

because r'^* is squeezed to r and the integrand is continuous. In words: the amount in a very thin shell changes at the rate $\partial c/\partial t$ times the shell volume $4\pi r^2\Delta r$, and dividing by Δr leaves $4\pi r^2\,\partial c/\partial t$.

Step 3: the source term. The same argument with $f = S\,4\pi r'^2$ gives $4\pi r^2\,S(r,t)$.

Step 4: the flux term, by the definition of the derivative. Define the total outward flow through the sphere of radius r,

$$\Phi(r,t) = A(r)\,J_r(r,t) = 4\pi r^2 J_r(r,t) \qquad (\text{mol s}^{-1}).$$

The fraction in Step 1 is $[\Phi(r+\Delta r) - \Phi(r)]/\Delta r$, and the derivative of Φ is defined as the limit of exactly this quotient:

$$\frac{A(r+\Delta r)J_r(r+\Delta r) - A(r)J_r(r)}{\Delta r} \;\longrightarrow\; \frac{\partial \Phi}{\partial r} = 4\pi\,\frac{\partial}{\partial r}(r^2 J_r).$$

Step 4 again, written out to see where the area enters. Expand $(r+\Delta r)^2 = r^2 + 2r\Delta r + \Delta r^2$, then split the outer-wall term into the part that has the inner-wall area $4\pi r^2$ and the extra area:

$$\begin{aligned} 4\pi(r+\Delta r)^2 J_r(r+\Delta r) - 4\pi r^2 J_r(r) &= 4\pi\big(r^2 + 2r\Delta r + \Delta r^2\big) J_r(r+\Delta r) \\ &\quad - 4\pi r^2 J_r(r) \\ &= 4\pi r^2 J_r(r+\Delta r) - 4\pi r^2 J_r(r) \\ &\quad + 4\pi\big(2r\Delta r + \Delta r^2\big) J_r(r+\Delta r) \\ &= 4\pi r^2 \big[J_r(r+\Delta r) - J_r(r)\big] \\ &\quad + 4\pi\big(2r\Delta r + \Delta r^2\big) J_r(r+\Delta r). \end{aligned}$$

Divide by Δr:

$$\begin{aligned} \frac{4\pi(r+\Delta r)^2 J_r(r+\Delta r) - 4\pi r^2 J_r(r)}{\Delta r} &= 4\pi r^2 \, \frac{J_r(r+\Delta r) - J_r(r)}{\Delta r} \\ &\quad + 4\pi \, (2r + \Delta r) \, J_r(r+\Delta r). \end{aligned}$$

Let $\Delta r \to 0$. The first quotient becomes $\partial J_r/\partial r$ by the definition of the derivative, $2r + \Delta r \to 2r$, and $J_r(r+\Delta r) \to J_r(r)$ because J_r is continuous:

$$\longrightarrow \; 4\pi r^2 \, \frac{\partial J_r}{\partial r} + 8\pi r \, J_r = 4\pi \left(r^2 \frac{\partial J_r}{\partial r} + 2r J_r \right) = 4\pi \, \frac{\partial}{\partial r}(r^2 J_r),$$

the last equality being the product rule $\frac{\partial}{\partial r}(r^2 J_r) = 2r J_r + r^2 \frac{\partial J_r}{\partial r}$. The two routes of Step 4 agree.

Step 5: assemble. Insert the limits of Steps 2 to 4 into Step 1:

$$4\pi r^2 \, \frac{\partial c}{\partial t} = -4\pi \, \frac{\partial}{\partial r}(r^2 J_r) + 4\pi r^2 S.$$

Step 6: divide by $4\pi r^2$. Valid for $r > 0$, where the area is positive (the centre $r = 0$ is treated in Part III, Section 3.1):

$$\frac{\partial c}{\partial t} = -\frac{1}{r^2} \frac{\partial}{\partial r}(r^2 J_r) + S \tag{2.3}$$

Where the $1/r^2$ comes from. It is the ratio of two geometric facts. The flow through a sphere is flux times area, $4\pi r^2 J_r$, so the derivative acts on $r^2 J_r$. The volume of a thin shell is area times thickness, $4\pi r^2 \Delta r$, so turning "moles per second in the shell" into "concentration change per second" divides by $4\pi r^2$. The expanded form makes the geometry visible:

$$\frac{1}{r^2} \frac{\partial}{\partial r}(r^2 J_r) = \frac{\partial J_r}{\partial r} + \frac{2}{r} J_r.$$

The term $\partial J_r/\partial r$ is the familiar slab effect: if more leaves per square metre than enters, the shell empties. The term $2J_r/r$ is purely spherical: the outer wall is larger than the inner wall by $4\pi(r+\Delta r)^2 - 4\pi r^2 = 8\pi r\,\Delta r + 4\pi\Delta r^2$, which is $8\pi r\,\Delta r$ once the tiny Δr^2 part is dropped for a thin shell, so even an outward flux that is the same at both walls removes more substance than it brings in. A picture: the light from a bulb. The same total power crosses every sphere around it, so the intensity per square metre falls like $1/r^2$. For diffusion the analogue is a steady outward stream with $4\pi r^2 J_r$ the same at every radius: then $r^2 J_r$ is constant, Eq. (2.3) gives $\partial c/\partial t = 0$ (for $S = 0$), and yet J_r itself decreases with r. Without the factor r^2 inside the derivative such a steady state would wrongly appear to accumulate substance.

Classification. Derived for STRATO; the spherical form is an established textbook result (Crank, 1975). DERIVED NOT IMPLEMENTED.

2.4 Divergence

Route (b) reaches Eq. (2.3) with a general tool that works for regions of any shape.

2.4.1 Divergence in plain words

Think of **J** as a field of arrows filling space, one arrow at each point showing where substance is heading. Around a point, surround a tiny region of volume δV and measure the net amount per second leaving through its surface. Divide by δV. As the region shrinks to the point, the ratio tends to a number called the divergence of **J** at that point, written $\nabla \cdot \mathbf{J}$:

$$\nabla \cdot \mathbf{J} = \lim_{\delta V \to 0} \frac{1}{\delta V} \oint_{\partial(\delta V)} \mathbf{J} \cdot \mathbf{n}\, dA \qquad (\text{mol m}^{-3}\ \text{s}^{-1}).$$

A positive divergence means arrows spread out from the point, like water from a tap; there is net outflow. A negative divergence means arrows converge, like water into a drain.

The small box in Cartesian coordinates. Take a box with corner (x, y, z) and sides Δx, Δy, Δz. The two faces perpendicular to x have area $\Delta y\,\Delta z$. Through the face at $x + \Delta x$ the outward normal is $+\mathbf{e}_x$, so $\mathbf{J} \cdot \mathbf{n} = J_x$; through the face at x it is $-\mathbf{e}_x$, so $\mathbf{J} \cdot \mathbf{n} = -J_x$. Treating J_x as constant over each small face (the error vanishes as the box shrinks), their net outflow is approximately

$$\begin{aligned} J_x(x+\Delta x)\,\Delta y\,\Delta z - J_x(x)\,\Delta y\,\Delta z &= \frac{J_x(x+\Delta x) - J_x(x)}{\Delta x}\,\Delta x\,\Delta y\,\Delta z \\ &\approx \frac{\partial J_x}{\partial x}\,\Delta x\,\Delta y\,\Delta z, \end{aligned}$$

where the middle step multiplies and divides by Δx and the last step uses the definition of the derivative.

The faces perpendicular to y and z contribute $\frac{\partial J_y}{\partial y}\Delta x\Delta y\Delta z$ and $\frac{\partial J_z}{\partial z}\Delta x\Delta y\Delta z$. Divide the sum by the volume $\Delta x\Delta y\Delta z$ and shrink the box:

$$\nabla \cdot \mathbf{J} = \frac{\partial J_x}{\partial x} + \frac{\partial J_y}{\partial y} + \frac{\partial J_z}{\partial z}.$$

Script check P2.7 (Section 2.10) computes the outflow through cubes of shrinking side around a point for a radial field and confirms that outflow divided by volume approaches the divergence, with error falling by a factor 4 each time the side is halved.

2.4.2 Divergence of a radial field

What we want. $\nabla \cdot \mathbf{J}$ for $\mathbf{J} = J_r(r)\,\mathbf{e}_r$, with $\mathbf{e}_r = (x, y, z)/r$ and $r = \sqrt{x^2 + y^2 + z^2}$. The time argument is fixed and omitted.

Step 1: Cartesian components. $J_x = J_r(r)\,x/r$, $J_y = J_r(r)\,y/r$, $J_z = J_r(r)\,z/r$.

Step 2: two derivatives needed. From $r^2 = x^2+y^2+z^2$, differentiating with respect to x gives $2r\,\partial r/\partial x = 2x$, so $\partial r/\partial x = x/r$. Then, by the quotient rule,

$$\frac{\partial}{\partial x}\left(\frac{x}{r}\right) = \frac{1 \cdot r - x\,\partial r/\partial x}{r^2} = \frac{r - x^2/r}{r^2} = \frac{1}{r} - \frac{x^2}{r^3}.$$

Step 3: differentiate J_x by the product rule and the chain rule.

$$\frac{\partial J_x}{\partial x} = \frac{dJ_r}{dr}\,\frac{\partial r}{\partial x}\,\frac{x}{r} + J_r\,\frac{\partial}{\partial x}\left(\frac{x}{r}\right) = \frac{dJ_r}{dr}\,\frac{x^2}{r^2} + J_r\left(\frac{1}{r} - \frac{x^2}{r^3}\right).$$

The y and z terms are the same with x replaced by y and z.

Step 4: add the three terms and use $x^2 + y^2 + z^2 = r^2$.

$$\begin{aligned}\nabla \cdot \mathbf{J} &= \frac{dJ_r}{dr}\,\frac{x^2+y^2+z^2}{r^2} + J_r\left(\frac{3}{r} - \frac{x^2+y^2+z^2}{r^3}\right)\\ &= \frac{dJ_r}{dr} + J_r\left(\frac{3}{r} - \frac{1}{r}\right) = \frac{dJ_r}{dr} + \frac{2}{r}J_r.\end{aligned}$$

Step 5: recognise the product rule. $\frac{1}{r^2}\frac{d}{dr}(r^2 J_r) = \frac{1}{r^2}\left(2rJ_r + r^2\frac{dJ_r}{dr}\right) = \frac{dJ_r}{dr} + \frac{2}{r}J_r$. Therefore, restoring time,

$$\nabla \cdot \left(J_r\,\mathbf{e}_r\right) = \frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2 J_r\right) \tag{2.4}$$

This is the spherical divergence formula for a radial field (established textbook result of vector calculus). The “3” in Step 4 counts the three space directions; one of them is taken up by the radial derivative, and the other two, the directions along the sphere, produce the $2/r$.

2.4.3 The divergence theorem

Cut any region V into many tiny boxes. For each box, net outflow is approximately divergence times box volume. Now add up all the boxes. Every inner wall is shared by two neighbouring boxes, and what leaves one box through that wall enters its neighbour, so the inner walls cancel in pairs. Only the walls on the outer surface of V survive. In the limit of infinitely many tiny boxes this becomes the divergence theorem (established textbook result of vector calculus):

$$\oint_{\partial V} \mathbf{J} \cdot \mathbf{n}\, dA = \int_V \nabla \cdot \mathbf{J}\, dV.$$

The box-counting picture above is a sketch, not a proof. The version used here assumes **J** continuously differentiable throughout V (weaker conditions exist but are not needed), so here V lies inside one layer.

2.4.4 Local conservation law

Step 1. Insert the divergence theorem into Eq. (2.1):

$$\frac{d}{dt}\int_V c\, dV = -\int_V \nabla \cdot \mathbf{J}\, dV + \int_V S\, dV.$$

Step 2. V is fixed in time and c is smooth, so the time derivative moves inside the integral as a partial derivative: $\frac{d}{dt}\int_V c\, dV = \int_V \frac{\partial c}{\partial t}\, dV$.

Step 3. Collect everything under one integral:

$$\int_V \left(\frac{\partial c}{\partial t} + \nabla \cdot \mathbf{J} - S \right) dV = 0.$$

Step 4: from "every region" to "every point". This holds for every control volume V inside a layer, however small. Suppose the bracket were positive at some point. Inside a layer $\partial c/\partial t$, $\nabla \cdot \mathbf{J}$ and S are continuous, so the bracket is continuous; it would therefore stay positive in a small ball around that point, and choosing that ball as V would make the integral positive, contradicting Step 3. A negative value is excluded the same way. Hence the bracket vanishes everywhere inside a layer:

$$\frac{\partial c}{\partial t} + \nabla \cdot \mathbf{J} = S \tag{2.5}$$

Eq. (2.5) says that at each point the rate of increase of concentration plus the net outflow per unit volume equals the creation per unit volume.

Agreement of the two routes. Insert Eq. (2.4) into Eq. (2.5): $\frac{\partial c}{\partial t} + \frac{1}{r^2}\frac{\partial}{\partial r}(r^2 J_r) = S$, which is Eq. (2.3) rearranged. The same answer also follows from the limit definition of divergence with the thin shell as the shrinking region: net outflow $A(r+\Delta r)J_r(r+\Delta r) - A(r)J_r(r)$ divided by the shell volume $V(r, r+\Delta r) = \frac{4\pi}{3}[(r+\Delta r)^3 - r^3] = 4\pi r^2 \Delta r + O(\Delta r^2)$ (Eq. (1.6)) tends to $4\pi\, \partial_r(r^2 J_r)/(4\pi r^2)$, the right-hand side of Eq.

(2.4). Script checks P2.6 (Cartesian computation in SymPy) and P2.8 (divergence theorem on a shell) confirm Eq. (2.4).

Classification. Eqs. (2.4) and (2.5): established textbook results, DERIVED NOT IMPLEMENTED. Neither code forms a divergence; Code B works only with face flows, which is the discrete form of the left side of the divergence theorem.

2.5 Spherical diffusion equation

What we want. A single equation for c alone, obtained by inserting Fick's law into Eq. (2.3).

Step 1: Fick's law, Eq. (1.4). $J_r = -D\,\partial c/\partial r$, where D may vary with r (it is piecewise constant in STRATO, one value per layer).

Step 2: multiply by r^2. $r^2 J_r = -r^2 D\,\partial c/\partial r$.

Step 3: insert into Eq. (2.3). The two minus signs cancel:

$$\begin{aligned}\frac{\partial c}{\partial t} &= -\frac{1}{r^2}\frac{\partial}{\partial r}\left(-r^2 D\,\frac{\partial c}{\partial r}\right) + S \\ &= \frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2 D\,\frac{\partial c}{\partial r}\right) + S\end{aligned} \tag{2.6}$$

This is the spherical diffusion equation in conservative form (established textbook result; Crank, 1975). In vector notation it is $\partial c/\partial t = \nabla \cdot (D \nabla c) + S$.

2.5.1 Why D must stay inside the derivative

The product rule splits the flux term into two parts:

$$\frac{\partial}{\partial r}\left(r^2 D\,\frac{\partial c}{\partial r}\right) = D\,\frac{\partial}{\partial r}\left(r^2\frac{\partial c}{\partial r}\right) + r^2\,\frac{\partial D}{\partial r}\,\frac{\partial c}{\partial r}.$$

Pulling D in front of the derivative keeps only the first part and silently deletes $r^2 D' c'$. Where D is constant the deleted part is zero and nothing is lost; this is the case inside each layer (Section 2.6). Where D changes, the deleted part carries real physics: the quantity that conservation controls is the flow $-4\pi r^2 D\,\partial c/\partial r$, and only the form with D inside the derivative states that this flow changes only where substance accumulates. At a layer interface D jumps, so D' is an infinitely tall, infinitely thin spike there and the product form is not even defined. The conservative form remains meaningful, because it came from the shell balance (2.2), which holds across an interface. Apply Eq. (2.2) with $S = 0$ to the shell from $R_\ell - \delta$ to $R_\ell + \delta$ around an interface at R_ℓ, with $\delta > 0$ small (in m):

$$\frac{\partial}{\partial t}\int_{R_\ell-\delta}^{R_\ell+\delta} c\,4\pi r'^2\,dr' = A(R_\ell-\delta)\,J_r(R_\ell-\delta) - A(R_\ell+\delta)\,J_r(R_\ell+\delta).$$

As $\delta \to 0$ the shell volume shrinks to zero while c and its rate of change stay bounded, so the left side tends to zero. The right side tends to $4\pi R_\ell^2\,[J_r(R_\ell^-) - J_r(R_\ell^+)]$, where

R_ℓ^- and R_ℓ^+ mean "just inside" and "just outside". Hence J_r, and with Fick's law $D\,\partial c/\partial r$, has the same value on both sides, which is the flux continuity condition, Part III Eq. (3.3).

Numerical illustration (derived for STRATO, synthetic numbers). Consider a steady state ($\partial c/\partial t = 0$, $S = 0$) in a two-layer shell from $a = 1.0 \times 10^{-4}$ m to $b = 3.0 \times 10^{-4}$ m, with the interface at $R_\ell = 2.0 \times 10^{-4}$ m, $D_1 = 1.0 \times 10^{-12}$ m^2 s^{-1} inside and $D_2 = 4.0 \times 10^{-10}$ m^2 s^{-1} outside (the kernel and buffer diffusivities of the synthetic input), and $c(a) = 1$ mol m^{-3}, $c(b) = 0$.

- *Correct (D inside).* Eq. (2.6) gives $\partial_r(r^2 D\,\partial_r c) = 0$, so $r^2 D\,\partial_r c$ does not change with r, in either layer or across the interface (the previous paragraph). Write it as $r^2 D\,\partial_r c = -\Phi/(4\pi)$, where $\Phi = -4\pi r^2 D\,\partial_r c$ is the total outward flow (in mol s^{-1}), the same through every sphere. Then $\partial_r c = -\Phi/(4\pi D r^2)$. Integrate across layer 1, using $\int_a^{R_\ell} r^{-2}\,dr = 1/a - 1/R_\ell$:

$$c(R_\ell) - c(a) = -\frac{\Phi}{4\pi D_1}\left(\frac{1}{a} - \frac{1}{R_\ell}\right).$$

In the same way across layer 2, $c(b) - c(R_\ell) = -\frac{\Phi}{4\pi D_2}\left(\frac{1}{R_\ell} - \frac{1}{b}\right)$. Concentration is continuous at the interface (Part III, Eq. (3.2)), so $c(R_\ell)$ is the same number in both lines; adding them cancels it and, after a change of sign,

$$c(a) - c(b) = \frac{\Phi}{4\pi}\left[\frac{1}{D_1}\left(\frac{1}{a} - \frac{1}{R_\ell}\right) + \frac{1}{D_2}\left(\frac{1}{R_\ell} - \frac{1}{b}\right)\right].$$

With the numbers above the bracket is $5.0\times10^{15}+4.1667\times10^{12} = 5.0042\times10^{15}$ m^{-3} s, so $\Phi = 4\pi \times 1$ mol m$^{-3}/(5.0042 \times 10^{15}$ m^{-3} s$) = 2.511181 \times 10^{-15}$ mol s^{-1}, the same on both sides of the interface. The two bracketed terms act like resistances in series, an idea Part V uses for the face conductance.

- *Wrong (D pulled outside).* Then $D\,\partial_r(r^2\partial_r c) = 0$, so $\partial_r(r^2\partial_r c) = 0$ everywhere and $r^2\partial_r c$ is one constant across the interface, written $-\mathcal{K}'$. Integrating $\partial_r c = -\mathcal{K}'/r^2$ from a to b gives $c(b) - c(a) = -\mathcal{K}'(1/a - 1/b)$, so $\mathcal{K}' = [c(a) - c(b)]/(1/a - 1/b) = 1$ mol m$^{-3}/(6666.7$ m$^{-1}) = 1.5 \times 10^{-4}$ mol m^{-2}. The flow arriving at the interface from layer 1 is $4\pi D_1\mathcal{K}' = 1.885 \times 10^{-15}$ mol s^{-1}, but the flow leaving into layer 2 is $4\pi D_2\mathcal{K}' = 7.540\times10^{-13}$ mol s^{-1}, 400 times more. The interface would create substance from nothing.

Script check P2.13 reproduces both numbers and checks the piecewise profile of the correct case against both end values and the continuity of c and of the flow at R_ℓ. This illustration is DEMONSTRATED NUMERICALLY for one synthetic case; the general statement is the product-rule identity above. Both STRATO codes keep D inside: Code B places the diffusivities of both neighbouring cells inside the face conductance (src/numerical/finite_volume.rs, line 86, Part V Eq. (5.9)), and Code C enforces continuity of $D(u'-u/r)$ at every interface (src/analytical/sturm_liouville.rs, lines 94 to 100, Part IV Eqs. (4.7) and (4.8)). Here $u = r\phi$, where $\phi(r)$ is the radial shape of one mode,

$c = \phi(r)e^{-\Lambda t}$ (Part IV, Eq. (4.1)). The derivative of the product $r\phi$ is $u' = \phi + r\phi'$, so $u' - u/r = \phi + r\phi' - \phi = r\phi'$ and $D(u' - u/r) = r\,D\phi'$. Since r has the same value on both sides of an interface, keeping $rD\phi'$ continuous keeps $D\phi'$, and with it $D\,\partial c/\partial r$, continuous.

Classification. Eq. (2.6) with $S \neq 0$ or with D varying inside a layer: FUTURE IMPROVEMENT (no source, no $D(T)$ in v1.0.0; the temperature `temperature_k` is read and validated in src/config.rs, lines 86 to 88 but enters no formula). Eq. (2.6) with $S = 0$ and piecewise-constant D is the model of v1.0.0: DERIVED NOT IMPLEMENTED as a formula, solved by Code B and Code C through Parts V and IV.

2.6 Layer equation of v1.0.0

Assumptions of v1.0.0. Inside layer ℓ ($R_{\ell-1} < r < R_\ell$, with $R_0 = 0$ the centre as in Part I) the diffusivity is the constant D_ℓ, and there is no source, $S = 0$.

Step 1. Inside the open layer $\partial D/\partial r = 0$, so the deleted term $r^2 D' c'$ of Section 2.5 is zero and D_ℓ may leave the derivative.

Step 2. Set $S = 0$ in Eq. (2.6) and write c_ℓ for c restricted to layer ℓ:

$$\frac{\partial c_\ell}{\partial t} = \frac{D_\ell}{r^2}\frac{\partial}{\partial r}\left(r^2\frac{\partial c_\ell}{\partial r}\right), \qquad R_{\ell-1} < r < R_\ell, \quad \ell = 1, \dots, 5 \tag{2.7}$$

This is the governing equation of Gelbard (2003), Eq. (IV.1), p. 27, which also has no source term, and where D_i is "constant within each layer, but may vary from layer to layer" (adapted from published literature, with Gelbard's layer index i written as ℓ; Gelbard labels the inner and outer radii of layer i as r_i and r_{i+1} with $r_1 = 0$, which are $R_{\ell-1}$ and R_ℓ here). Eq. (2.7) holds in the open interior of each layer only. The five copies are joined to one another, and closed at the centre and at the surface, by the conditions of Part III.

Step 3: expanded form, for intuition. Apply the product rule, $\frac{\partial}{\partial r}\left(r^2\frac{\partial c_\ell}{\partial r}\right) = 2r\frac{\partial c_\ell}{\partial r} + r^2\frac{\partial^2 c_\ell}{\partial r^2}$, and divide by r^2:

$$\frac{\partial c_\ell}{\partial t} = D_\ell\left(\frac{\partial^2 c_\ell}{\partial r^2} + \frac{2}{r}\frac{\partial c_\ell}{\partial r}\right) \tag{2.8}$$

Meaning. The second derivative $\partial^2 c/\partial r^2$ measures curvature of the profile. Where the profile is a valley (curving up), neighbours hold more substance and the point fills; where it is a hill, the point empties. This is the whole story in a flat slab. The term $(2/r)\,\partial c/\partial r$ is the spherical correction of Section 2.3: a profile that falls outward ($\partial c/\partial r < 0$) loses extra substance because it feeds an ever larger area.

Example application. Try the shape $c = \frac{\sin kr}{r}\,e^{-D_\ell k^2 t}$, with k in m^{-1}. Write $X = e^{-D_\ell k^2 t}$.

$$\frac{\partial c}{\partial r} = \left(\frac{k\cos kr}{r} - \frac{\sin kr}{r^2}\right) X, \qquad r^2\frac{\partial c}{\partial r} = (kr\cos kr - \sin kr)X,$$

$$\frac{\partial}{\partial r}\left(r^2\frac{\partial c}{\partial r}\right) = (k\cos kr - k^2 r\sin kr - k\cos kr)X = -k^2 r\sin kr\, X,$$

$$\frac{D_\ell}{r^2}\frac{\partial}{\partial r}\left(r^2\frac{\partial c}{\partial r}\right) = -D_\ell k^2\,\frac{\sin kr}{r}\,X = -D_\ell k^2\, c,$$

while $\partial c/\partial t = -D_\ell k^2\, c$. Both sides of Eq. (2.7) agree, so this shape fades at the rate $D_\ell k^2$ without changing form. Part IV builds the exact solution from shapes of this kind. Script check P2.11 confirms the substitution and also that this solution satisfies the finite-shell balance, Eq. (2.2), on a shell of finite thickness.

Classification. Eq. (2.7): DERIVED NOT IMPLEMENTED as a formula. It is the model that both codes solve: Code C through its transformed form $u'' + k_\ell^2 u = 0$, stated in the module documentation (src/analytical/sturm_liouville.rs, lines 12 to 15) and derived in Part IV; Code B through the cell balance of Part V. Eq. (2.8): DERIVED NOT IMPLEMENTED; neither code uses the expanded form. Code B works with the conservative form, which never divides by r and so has no trouble at the centre, where $2/r$ is unbounded.

2.7 Units check of every term

Each equation of this part must be dimensionally consistent: every term in one equation carries the same unit. Table 2.2 builds each unit from the symbols of Table 2.1.

Table 2.2 Units of every term. A derivative $\partial/\partial r$ contributes m^{-1} and $\partial/\partial t$ contributes s^{-1}.

Equation	Term	Units, built up	Result
(2.1)	$\frac{d}{dt}\int_V c\,dV$	$\mathrm{mol\ m^{-3} \times m^3 / s}$	$\mathrm{mol\ s^{-1}}$
(2.1)	$\oint \mathbf{J}\cdot\mathbf{n}\,dA$	$\mathrm{mol\ m^{-2}\ s^{-1} \times 1 \times m^2}$	$\mathrm{mol\ s^{-1}}$
(2.1)	$\int_V S\,dV$	$\mathrm{mol\ m^{-3}\ s^{-1} \times m^3}$	$\mathrm{mol\ s^{-1}}$
(2.2)	$A(r)J_r(r)$	$\mathrm{m^2 \times mol\ m^{-2}\ s^{-1}}$	$\mathrm{mol\ s^{-1}}$
(2.3), (2.5)	$\partial c/\partial t$	$\mathrm{mol\ m^{-3} / s}$	$\mathrm{mol\ m^{-3}\ s^{-1}}$
(2.3), (2.4)	$\frac{1}{r^2}\frac{\partial}{\partial r}(r^2 J_r)$	$\mathrm{m^{-2} \times m^{-1} \times m^2 \times mol\ m^{-2}\ s^{-1}}$	$\mathrm{mol\ m^{-3}\ s^{-1}}$
(2.6), (2.7)	$\frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2 D\frac{\partial c}{\partial r}\right)$	$\mathrm{m^{-2} \times m^{-1} \times m^2 \times m^2\ s^{-1} \times mol\ m^{-4}}$	$\mathrm{mol\ m^{-3}\ s^{-1}}$
(2.8)	$D_\ell\,\partial^2 c/\partial r^2$ and $D_\ell\,(2/r)\,\partial c/\partial r$	$\mathrm{m^2\ s^{-1} \times mol\ m^{-3} \times m^{-2}}$	$\mathrm{mol\ m^{-3}\ s^{-1}}$
(2.3), (2.6)	S	given	$\mathrm{mol\ m^{-3}\ s^{-1}}$

The integral equations (2.1) and (2.2) balance amounts per second; the local equations (2.3) to (2.8) balance concentration per second. Dividing by a volume ($4\pi r^2\Delta r$ in Section 2.3) is exactly what converts one into the other. Script check P2.12 repeats the table with symbolic units.

2.8 Size of diffusion times

What we want. A quick estimate of how long diffusion takes to cross a layer, before solving anything.

Dimensional argument. The only data that describe diffusion across a layer are its thickness $L_\ell = R_\ell - R_{\ell-1}$ (in m) and its diffusivity D_ℓ (in $\mathrm{m^2\ s^{-1}}$). Look for a time of the form $L_\ell^\alpha D_\ell^\beta$ with numbers α and β. Its unit is $\mathrm{m}^\alpha\,(\mathrm{m^2\ s^{-1}})^\beta = \mathrm{m}^{\alpha+2\beta}\,\mathrm{s}^{-\beta}$. For this to be s we need $-\beta = 1$ and $\alpha + 2\beta = 0$, so $\beta = -1$ and $\alpha = 2$. The only combination is therefore the square of the length divided by the diffusivity:

$$t_{\mathrm{D},\ell} = \frac{L_\ell^2}{D_\ell} \qquad (\mathrm{s}) \tag{2.9}$$

Any true time in the problem equals $t_{\mathrm{D},\ell}$ times a pure number that depends on geometry and on the conditions at the boundaries. The estimate fixes the order of magnitude, not that number. Because $t_{\mathrm{D},\ell}$ grows with the square of the distance, doubling a layer's thickness makes it four times slower to cross. The symbol $t_{\mathrm{D},\ell}$ is local to this section and is not part of the notation table.

Worked example (calculated from the synthetic input, not literature data). The synthetic five-layer case (inputs/synthetic/synthetic_five_layer.toml, lines 1 to 42) gives the diffusion times of Table 2.3:

Table 2.3 Diffusion time scales of the synthetic five-layer input (synthetic numerical benchmark, labelled in the input file as not literature TRISO data).

Layer ℓ	L_ℓ (m)	D_ℓ ($\mathrm{m^2\ s^{-1}}$)	$t_{\mathrm{D},\ell} = L_\ell^2/D_\ell$ (s)	Readable
1 Kernel	2.0×10^{-4}	1.0×10^{-12}	4.0×10^{4}	11.11 h
2 Buffer	1.0×10^{-4}	4.0×10^{-10}	25	25 s
3 IPyC	5.0×10^{-5}	8.0×10^{-13}	3125	52.08 min
4 SiC	5.0×10^{-5}	2.0×10^{-14}	1.25×10^{5}	34.72 h
5 OPyC	5.0×10^{-5}	7.0×10^{-13}	3571.4	59.52 min

For the kernel, step by step: $L_1^2 = (2.0 \times 10^{-4}\ \mathrm{m})^2 = 4.0 \times 10^{-8}\ \mathrm{m^2}$, and $4.0 \times 10^{-8}\ \mathrm{m^2} \,/\, 1.0 \times 10^{-12}\ \mathrm{m^2\ s^{-1}} = 4.0 \times 10^{4}$ s.

What the table says. The buffer is crossed almost instantly (25 s) and acts as a well-mixed holding volume. The SiC layer is the slowest (1.25×10^5 s), and its time is comparable to the simulated period $t_{\mathrm{end}} = 2.0 \times 10^5$ s (55.56 h). We therefore expect partial release by t_{end}, controlled mainly by the SiC layer. The full calculation agrees in order of magnitude: Code C gives a slowest decay rate $\Lambda_1 = 3.27325567118 \times 10^{-6}\ \mathrm{s^{-1}}$ (a 40-digit independent evaluation gives $3.273255671177023 \times 10^{-6}\ \mathrm{s^{-1}}$, a relative difference of 3.7×10^{-13}; Part IV, Section 4.7), a time $1/\Lambda_1 = 3.055 \times 10^5$ s (84.86 h), and a release fraction at t_{end} of $F(2 \times 10^5\ \mathrm{s}) = 0.43255563909286$ (golden file `tests/golden/v1.0/comparison_fv_vs_analytical.csv`); an independent high-precision evaluation gives 0.43255563909304 (Part IV, Section 4.10) and Code C agrees with it to 1.9×10^{-13} (calculated values for the synthetic input).

A caution on the pure number. For a single uniform sphere of radius R_1 with D_1 and $c = 0$ at its surface, the exact slowest time is $R_1^2/(\pi^2 D_1) = 4052.85$ s (Part IV, Eq. (4.29), with $n = 1$), about ten times shorter than $t_{\mathrm{D},1} = 4.0 \times 10^4$ s because of the factor $\pi^2 \approx 9.87$. Eq. (2.9) estimates orders of magnitude; it does not replace Parts IV and V.

Classification. Eq. (2.9): derived for STRATO by dimensional analysis, DERIVED NOT IMPLEMENTED (no code computes $t_{\mathrm{D},\ell}$). Code C uses a related combination, the sum of $L_\ell/\sqrt{D_\ell}$ (in $\mathrm{s}^{1/2}$), only to set the lower bracket $10^{-6}\left(\sum_\ell L_\ell/\sqrt{D_\ell}\right)^{-2}$ of its eigenvalue search (src/analytical/sturm_liouville.rs, lines 220 to 232, Part IV Eq. (4.14)).

2.9 Classification of Part II and link to the code

Table 2.4 lists every result of Part II with its classification, its status and the place where it acts in v1.0.0.

Table 2.4 Classification and status of the results of Part II. FV is src/numerical/finite_volume.rs and SL is src/analytical/sturm_liouville.rs, both at tag v1.0.0. A2 and A4 are the assumptions of Part I, Section 1.6.

Item	Content	Classification	Status	Where it acts in v1.0.0
Fick's law, Eq. (1.4), vector form	$\mathbf{J} = -D\nabla c$, $J_r = -D\,\partial c/\partial r$	established textbook result; used, not evaluated	CLAIMED ONLY (constitutive law); radial form PROVED from the vector form	face flows of Code B (Part V); interface rule of Code C (Part IV)
Eq. (2.1)	integral conservation	established textbook result; DERIVED NOT IMPLEMENTED as a separate computation	CLAIMED ONLY (physical principle)	cell balance (5.6), FV 162-182
Source S	volumetric creation	FUTURE IMPROVEMENT	not applicable	absent: FV 179 has no source term
Eq. (2.2)	exact thin-shell balance	derived for STRATO; DERIVED NOT IMPLEMENTED (continuous form)	PROVED from Eq. (2.1); DEMONSTRATED NUMERICALLY (P2.11b)	cell form (5.6); areas at FV 83 and FV 90
Eq. (2.3)	local balance in spherical form	DERIVED NOT IMPLEMENTED	PROVED (smooth fields, $r > 0$)	basis of Parts IV and V
Eq. (2.4)	spherical divergence of a radial field	established textbook result; DERIVED NOT IMPLEMENTED	PROVED; DEMONSTRATED NUMERICALLY (P2.6, P2.7)	none
Divergence theorem	surface flow equals volume integral of divergence	established textbook result	CLAIMED ONLY (sketch only)	none
Eq. (2.5)	local conservation law	established textbook result; DERIVED NOT IMPLEMENTED	PROVED, given the divergence theorem	none
Eq. (2.6)	conservative spherical diffusion equation	established textbook result; DERIVED NOT IMPLEMENTED; $S \neq 0$ or $D(r)$ within a layer FUTURE IMPROVEMENT	PROVED from Eq. (2.3) and (1.4)	D kept inside: FV 86, SL 94-100
Two-layer steady example	D inside conserves; D outside creates substance at the interface	derived for STRATO, synthetic numbers	DEMONSTRATED NUMERICALLY (P2.13)	none

Item	Content	Classification	Status	Where it acts in v1.0.0
Eq. (2.7)	layer equation, $S = 0$, constant D_ℓ	adapted from published literature (Gelbard, 2003, Eq. (IV.1)); DERIVED NOT IMPLEMENTED as a formula	PROVED from Eq. (2.6) under A2 and A4	solved by Code C (SL 12-15, Part IV) and Code B (Part V)
Eq. (2.8)	expanded layer equation	DERIVED NOT IMPLEMENTED	PROVED (product rule)	none (intuition only)
Eq. (2.9)	diffusion time scale	derived for STRATO; DERIVED NOT IMPLEMENTED	form PROVED by dimensional analysis; use as an order-of-magnitude estimate DEMONSTRATED NUMERICALLY for the synthetic case only	none

Differences from the delivered Phase 1 derivation. Section 2 of `docs/mathematics/phase1/TRISO_Finite_Volume_Derivation.md` states the integral law and the localisation argument in compressed form and describes the shell picture in words; its content agrees with this part. This part (i) replaces its working-document labels by the numbering of this document (Table 7.3, item I20): TRISO-GOV-020 is Eq. (2.1), TRISO-GOV-030 is Eq. (2.5), TRISO-GOV-034 is the vector form of Fick's law in Section 2.0.3, TRISO-GOV-040 is the vector form of Eq. (2.6), and TRISO-FV-100 is Eq. (2.6) written for layer i with a source S_i, which becomes Eq. (2.7) when $S_i = 0$; (ii) corrects "constant speed" to "constant diffusivity" (Table 7.3, item I18); (iii) keeps S in the general equations but states that v1.0.0 has $S = 0$ and marks the source as FUTURE (Table 7.3, item I21); and (iv) shows the thin-shell algebra and the divergence calculation step by step.

2.10 Verification

Script: `scripts/derivation_checks/v1_0_0_p2_conservation.py`, run from the repository root with `.venv/Scripts/python.exe -I -X utf8 <script>`. It writes nothing; it reads the synthetic input file only. Output (23 of 23 PASS):

```
PASS  P2.1 Fick: [D] = [J_r]/[dc/dr] = m^2 s^-1  [[D] = m**2/s]
PASS  P2.2 4pi(r+h)^2 J(r+h) - 4pi r^2 J(r) = 4pi[r^2 (J(r+h)-J(r)) + (2rh + h^2) J(r+h)]
PASS  P2.3 lim_{h->0} [A(r+h)J(r+h) - A(r)J(r)]/h = 4pi (r^2 J' + 2 r J) = 4pi d(r^2 J)/dr  [limit =
↪  4*pi*r*(2*J0 + J1*r)]
PASS  P2.3b same limit for three concrete fluxes (r e^-r, sin r + r^3, 1/(1+r^2))
PASS  P2.4 lim_{h->0} (1/h) d/dt int_r^{r+h} c 4pi rho^2 drho = 4pi r^2 dc/dt (three concrete c)
PASS  P2.5 (1/r^2) d(r^2 J)/dr = J' + 2J/r
PASS  P2.6 div(J_r(r) e_r) in Cartesian x,y,z equals (1/r^2) d(r^2 J_r)/dr
PASS  P2.7 net outflow of a cube / its volume -> divergence as side h -> 0 (error O(h^2))  [div =
↪  0.9192489216, errors 1.86e-03, 4.64e-04, 1.16e-04, 2.90e-05, observed order 2.00]
PASS  P2.8 int_shell div J dV = 4pi b^2 J(b) - 4pi a^2 J(a) (three concrete J_r)
PASS  P2.9 d(r^2 D c')/dr = D d(r^2 c')/dr + r^2 D' c' (D outside loses r^2 D' c')
PASS  P2.10 (D/r^2) d(r^2 c')/dr = D (c'' + 2c'/r)
PASS  P2.11a c = sin(kr)/r exp(-D k^2 t) satisfies Eq. (2.7) (SymPy residual)
PASS  P2.11b same c satisfies the finite-shell balance Eq. (2.2) on [0.3, 1.7]  [residual 0.0e-164]
PASS  P2.12 Eq. (2.6): [dc/dt] = [(1/r^2) d/dr (r^2 D dc/dr)] = [S] = mol m^-3 s^-1  [mol/(m**3*s),
↪  mol/(m**3*s), mol/(m**3*s)]
PASS  P2.12b Eq. (2.2): every term in mol s^-1
PASS  P2.13a D inside: piecewise steady profile has c(a)=1, c(b)=0, c continuous and equal flow on both
↪  sides of Ri  [Q = 2.511181e-15 mol s^-1, flow in 2.511181e-15, flow out 2.511181e-15]
PASS  P2.13b D outside: flow leaving the interface exceeds flow arriving by 4pi K'(D2 - D1) != 0
↪  [arriving 1.885e-15, leaving 7.540e-13 mol s^-1, ratio D2/D1 = 400]
PASS  P2.14a synthetic input parsed: five layers  [Kernel R=2.00e-04 D=1.0e-12, Buffer R=3.00e-04
↪  D=4.0e-10, IPyC R=3.50e-04 D=8.0e-13, SiC R=4.00e-04 D=2.0e-14, OPyC R=4.50e-04 D=7.0e-13]
```

```
PASS  P2.14b L_l^2/D_l = 4.0e4, 25, 3125, 1.25e5, 3571.4 s  [40000 s, 25 s, 3125 s, 125000 s, 3571.43 s]
PASS  P2.14c R^2/D with D_1 and with D_4: 2.025e5 s and 1.0125e7 s  [202500 s, 1.0125e+07 s; t_end =
↪  200000 s]
PASS  P2.14d bare kernel (radius R_1, D_1, c = 0 at R_1): 1/Lambda_1 = R_1^2/(pi^2 D_1) = 4053 s
↪  [1/Lambda_1 = 4052.85 s]
PASS  P2.14e five-layer 1/Lambda_1 = 3.055e5 s is of the order of the largest layer time L_4^2/D_4
↪  [1/Lambda_1 = 3.055e+05 s, max L^2/D = 1.25e+05 s]
PASS  P2.14f unit conversions: 11.11 h, 52.08 min, 34.72 h, 59.52 min, t_end 55.56 h, 1/Lambda_1 84.86 h
↪  [11.1111, 52.0833, 34.7222, 59.5238, 55.5556, 84.8628]
SUMMARY 23/23 PASS  ALL PASS
```

Check P2.14e uses a value of Λ_1 from an earlier independent evaluation ($3.273255671177023 \times 10^{-6}$ s^{-1}, which agrees with the Code C value $3.273255671175805 \times 10^{-6}$ s^{-1} to a relative 3.7×10^{-13}); it does not recompute it. Part IV's scripts and check P3.16 recompute Λ_1 independently. Check P2.14c computes R^2/D_1 and R^2/D_4 for the whole particle; these times are not used in the text and only confirm the input values.

Part III: Five-layer model and conditions

Part II ended with one diffusion equation per layer, Eq. (2.7). An equation of that kind describes how concentration changes inside a region, but on its own it has infinitely many solutions. To pick out the one solution that describes a TRISO particle we must also say what happens at the edges of every layer and what the particle looks like at the start. This part supplies that information, one condition at a time. It then derives the initial inventory and the two release quantities that the code reports.

The checks quoted as "P3.x" are the PASS lines of the script `scripts/derivation_checks/v1_0_0_p3_conditions.py` (Section 3.7).

3.0 Scope: the five-layer domain

Physical problem. A TRISO particle is a small ball made of five nested shells, like an onion. A fission product starts inside the kernel and spreads outwards through the four coating layers until it leaves through the outer surface.

What we want. A complete mathematical statement of this problem: the domain of every layer, the equation that holds inside it, and enough extra conditions to make the solution unique.

Radii and layers. The five outer radii are ordered,

$$0 = R_0 < R_1 < R_2 < R_3 < R_4 < R_5 = R,$$

and layer ℓ occupies the shell between $R_{\ell-1}$ and R_ℓ (Part I, Eqs. (1.1) and (1.2)). Table 3.1 lists the layers. The numerical values are the synthetic benchmark input of v1.0.0, file inputs/synthetic/synthetic_five_layer.toml, lines 6 to 34, whose first line states "SYNTHETIC NUMERICAL BENCHMARK - NOT LITERATURE TRISO DATA". They are test numbers chosen to exercise the code and describe no real particle or nuclide.

Table 3.1 The five layers, their radial domains and diffusivities. Synthetic values from the v1.0.0 input file (synthetic input, not literature data); check P3.1.

ℓ	Layer	Domain	Diffusivity	Synthetic R_ℓ (m)	Synthetic L_ℓ (m)	Synthetic D_ℓ (m^2 s^{-1})
1	Kernel	$0 \le r < R_1$	D_1	2.0×10^{-4}	2.0×10^{-4}	1.0×10^{-12}
2	Buffer	$R_1 < r < R_2$	D_2	3.0×10^{-4}	1.0×10^{-4}	4.0×10^{-10}
3	IPyC	$R_2 < r < R_3$	D_3	3.5×10^{-4}	5.0×10^{-5}	8.0×10^{-13}
4	SiC	$R_3 < r < R_4$	D_4	4.0×10^{-4}	5.0×10^{-5}	2.0×10^{-14}
5	OPyC	$R_4 < r \le R_5 = R$	D_5	4.5×10^{-4}	5.0×10^{-5}	7.0×10^{-13}

The layers along the radial axis are drawn to scale in Figure 3.1. The figure uses the AGR-1 baseline radii of Table 1.1, not the synthetic radii of Table 3.1, and it numbers the layers with the index i that the text calls ℓ.

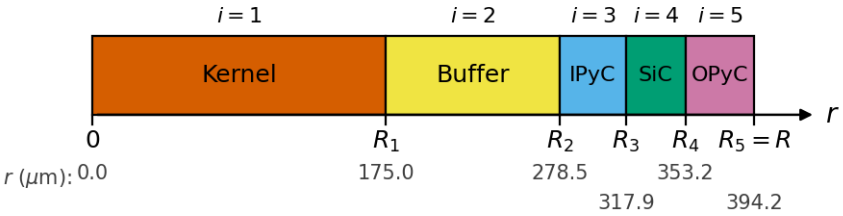


Figure 3.1: The five layers along the radius, drawn to scale with the AGR-1 baseline radii of Table 1.1 (Morris et al., 2016, Table 2). The figure numbers the layers 1 to 5 with the index i, which the text calls ell, and marks the outer radii R1 to R5 in micrometres.

The input reader requires exactly five regions with increasing finite radii and positive finite diffusivities: IMPLEMENTED in src/config.rs, lines 79 to 99. Code C on its own accepts any number of layers from two upwards (src/analytical/sturm_liouville.rs, lines 112 to 127); in v1.0.0 its layers always come from the configuration through `layers_from_config` (src/postprocessing/comparison.rs, lines 15 to 23) or from five-layer unit tests, so it always receives five.

The equation inside each layer. Inside layer ℓ the concentration $c_\ell(r, t)$ (in mol m^{-3}) obeys Eq. (2.7) of Part II:

$$\frac{\partial c_\ell}{\partial t} = \frac{D_\ell}{r^2}\frac{\partial}{\partial r}\left(r^2 \frac{\partial c_\ell}{\partial r}\right), \qquad R_{\ell-1} < r < R_\ell, \quad t > 0, \quad \ell = 1, \dots, 5.$$

Here r is the distance from the centre in m, t is time in s and D_ℓ is the constant diffusivity of layer ℓ in m^2 s^{-1}. There is no source and no decay term in v1.0.0. Code B discretises this equation (src/numerical/finite_volume.rs, lines 162 to 182); Code C solves its separated form (src/analytical/sturm_liouville.rs, lines 12 to 19).

How many conditions are needed. The equation contains a second derivative in r. A second-order equation leaves two free constants for every layer, in the same way that the path of a thrown ball is fixed only once you give both its starting position and its starting speed. Five layers therefore need $2 \times 5 = 10$ conditions in r. The equation contains a first derivative in t, so it needs one starting profile. The ten conditions are supplied as follows (check P3.2):

- 1 at the centre, Section 3.1;

- 2 at each of the four internal interfaces $R_1, \dots, R_4$, which makes 8, Section 3.2;
- 1 at the outer surface, Section 3.3.

The starting profile is Section 3.4.

Temperature. The input file also contains a temperature (`temperature_k`). It is read and checked to be positive and finite (src/config.rs, lines 86 to 88), but no formula in v1.0.0 uses it. The diffusivities in Table 3.1 are given directly. A temperature-dependent $D_\ell(T)$ is FUTURE IMPROVEMENT.

3.1 Centre: symmetry and regularity

Physical problem. The point $r = 0$ is the centre of the kernel. In the coordinate r it looks like an edge of the domain. Physically the centre is an ordinary interior point of the kernel: material on one side of the centre is just as much kernel as material on the other side. We need a condition that says so.

What we want. A statement about c_1 at $r = 0$ that follows from symmetry and from the absence of anything special (such as a source) at the centre.

Two plain-word ideas. *Regular* (or *bounded*) means the concentration stays a finite number everywhere, including at the centre. *Flow through a sphere* means the total amount per second that crosses a sphere of radius r; with the outward flux J_r in mol m^{-2} s^{-1} (Eq. (1.4)) and the sphere area $A(r) = 4\pi r^2$ (Eq. (1.5)) it equals $4\pi r^2 J_r$ in mol s^{-1}.

3.1.1 Argument 1: symmetry forbids a slope at the centre

Draw a straight line through the centre and let s be the signed position along it, so that $r = |s|$. Along this line the concentration is $f(s) = c_1(|s|, t)$. Suppose the radial slope at the centre were $b = \partial_r c_1(0^+, t)$.

Step 1. Moving a small distance $h > 0$ in the positive direction, $r = h$, so the slope seen from the right is

$$\lim_{h \to 0^+} \frac{f(h) - f(0)}{h} = \lim_{h \to 0^+} \frac{c_1(h,t) - c_1(0,t)}{h} = b.$$

Step 2. Moving the same distance in the negative direction, $s = -h$ but still $r = h$, so the slope seen from the left is

$$\lim_{h \to 0^+} \frac{f(-h) - f(0)}{-h} = -\lim_{h \to 0^+} \frac{c_1(h,t) - c_1(0,t)}{h} = -b.$$

Step 3. A smooth profile has one slope at each point, so the two must agree: $b = -b$, which gives $b = 0$. If $b \neq 0$ the profile would have a sharp point (a cone) at the centre, as the profile $c = a + br$ does (check P3.3). Nothing in the kernel can create such a point.

3.1.2 Argument 2: no flow through a vanishing sphere

Apply the conservation law (2.1), with no source, to the small ball $0 \le r \le \delta$ inside the kernel. The only surface of this ball is the sphere of radius δ, so

$$\frac{d}{dt}\int_0^\delta c_1\, 4\pi r^2\, dr = -4\pi\delta^2 J_r(\delta, t).$$

Step 1. The vertical bars $|x|$ denote the size of x with its sign dropped, so $|-3| = 3$. Suppose $|\partial_t c_1| \le \mathcal{K}$ near the centre, with $\mathcal{K}$ a finite constant in mol m^{-3} s^{-1} (the concentration does not change infinitely fast). The ball does not move, so the rate of change of its content is the sum of the rates of change of its thin shells, and each of these is at most $\mathcal{K}$ in size:

$$\left|\frac{d}{dt}\int_0^\delta c_1\, 4\pi r^2\, dr\right| = \left|\int_0^\delta \frac{\partial c_1}{\partial t}\, 4\pi r^2\, dr\right| \le \int_0^\delta \mathcal{K}\, 4\pi r^2\, dr = \mathcal{K}\,\frac{4\pi}{3}\delta^3.$$

Step 2. The left side equals $|-4\pi\delta^2 J_r(\delta, t)| = 4\pi\delta^2 |J_r(\delta, t)|$ by the balance above, so

$$4\pi\delta^2 |J_r(\delta, t)| \le \mathcal{K}\,\frac{4\pi}{3}\delta^3.$$

Step 3. Divide both sides by the sphere area $4\pi\delta^2$, which is positive:

$$|J_r(\delta, t)| \le \frac{\mathcal{K}\delta}{3}.$$

Step 4. Let $\delta \to 0$. The right side goes to zero, so $J_r(0^+, t) = 0$ (check P3.4). In words: the ball's volume shrinks like δ^3 while its surface shrinks only like δ^2, so a ball too small to store anything cannot pass a finite flux through its surface.

Step 5. Fick's law (1.4) in the kernel, $J_r = -D_1\,\partial_r c_1$, with $D_1 > 0$, turns zero flux into zero slope: $\partial_r c_1(0^+, t) = -J_r(0^+, t)/D_1 = 0$.

3.1.3 Why c must stay finite

Arguments 1 and 2 assumed that c_1 is smooth and that $\partial_t c_1$ is bounded. The following calculation shows what goes wrong otherwise. The steady, radially symmetric solutions of Eq. (2.7) are $c = a + B/r$ with constants a and B (check P3.5c). The term B/r is infinite at the centre, yet the amount it holds inside a small ball is finite:

$$\int_0^\delta \frac{B}{r}\, 4\pi r^2\, dr = 4\pi B\left[\frac{r^2}{2}\right]_0^\delta = 2\pi B\delta^2.$$

The flow it carries through a sphere of radius r is

$$-4\pi r^2 D\,\frac{\partial}{\partial r}\left(\frac{B}{r}\right) = -4\pi r^2 D\left(-\frac{B}{r^2}\right) = 4\pi D B,$$

the same for every r, however small. A fixed amount per second leaves every sphere around the centre, so the centre itself must be producing it: B/r describes a point source. The time-dependent singular shape $\cos(kr)/r$ behaves in the same way, with a centre flow tending to $4\pi D$ (check P3.5b). The regular shape $\sin(kr)/r$ tends to the finite value k at the centre, with zero slope and zero centre flow, and it solves Eq. (2.7) when multiplied by e^{-Dk^2t} (check P3.5a). Since v1.0.0 has no source of any kind, the singular shapes are excluded, and the boundedness requirement is exactly what excludes them.

3.1.4 Result

The centre condition is

$$\frac{\partial c_1}{\partial r}(0,t) = 0, \qquad \lim_{r\to 0^+} 4\pi r^2 D_1 \frac{\partial c_1}{\partial r} = 0, \qquad c_1(0,t) \text{ finite.} \tag{3.1}$$

This is adapted from published literature: Gelbard (2003, p. 27) requires c finite at $r = 0$. The derivation above is derived for STRATO; it is PROVED under its two assumptions (a smooth profile and a bounded $\partial_t c_1$). Eq. (3.1) is IMPLEMENTED in both codes, each in its own variables:

- Code B gives the first cell no inner face. The west conductance of the first cell is set to zero in src/numerical/finite_volume.rs, line 172, and the face loop in lines 78 to 87 only builds faces between neighbouring cells. This is the discrete form of "zero flow through a sphere of zero area"; no ghost value is used (Part V, Eq. (5.10)).
- Code C starts its shooting from $(u, q) = (0, 1)$ at $r = 0$ (src/analytical/sturm_liouville.rs, line 134, and the same start in the eigenvalue counter, line 168), where $\phi(r)$ is the radial shape of one mode (Part IV), $u = r\phi$ and $q = D\,du/dr$. A finite ϕ at the centre gives $u(0) = 0$, which selects the regular shape and removes the $\cos(kr)/r$ shape (Part IV, Eq. (4.9)).

Physical meaning. The centre is a point of symmetry, so nothing flows through it; the concentration profile is flat there, like the top of a smooth hill.

3.2 Interfaces

Physical problem. At each internal radius R_ℓ, $\ell = 1, \dots, 4$, two different materials touch. The diffusivity jumps from D_ℓ to $D_{\ell+1}$. Eq. (2.7) holds on each side, but not at the interface itself, because D has no single value there.

What we want. Two conditions per interface that connect the solution on the inner side to the solution on the outer side.

Notation for the two sides. $c_\ell(R_\ell^-, t)$ means the limit of $c_\ell(r, t)$ as r approaches R_ℓ from inside (from smaller r), and $c_{\ell+1}(R_\ell^+, t)$ the limit from outside. These *one-sided limits* let us talk about "just inside" and "just outside" the interface.

3.2.1 Concentration continuity (an assumption about contact)

We assume perfect contact with no partitioning: the concentration just inside the interface equals the concentration just outside,

$$c_\ell(R_\ell^-, t) = c_{\ell+1}(R_\ell^+, t), \qquad \ell = 1, \dots, 4, \quad t > 0. \tag{3.2}$$

Conservation alone does not give this condition. Its justification comes from how the two materials hold the species: if the species is equally at home on both sides and the layers touch without a gap, a jump in concentration across a surface of zero thickness would act like an infinitely steep slope and drive atoms across until the jump disappeared. If one material holds the species more strongly than the other (different solubility or trapping), a finite jump can persist in equilibrium. IAEA-TECDOC-978 (IAEA, 1997, p. 436) states that most diffusion models set this ratio of concentrations at contiguous surfaces (the *partition coefficient*) equal to one, and reports measured ratios from about 0.3 to 3 that may change with temperature and fast fluence (adapted from published literature). Eq. (3.2) is therefore a modelling choice of v1.0.0 (Part I, Section 1.6) whose error is set by how far the true ratio is from one. As an assumption it is neither PROVED nor DEMONSTRATED NUMERICALLY.

Classification: IMPLEMENTED. In Code C the condition becomes continuity of $u = r\phi$ (because r itself is continuous): the interface rule in src/analytical/sturm_liouville.rs, lines 94 to 100 leaves u unchanged across R_ℓ. In Code B the condition is built into the face conductance of src/numerical/finite_volume.rs, line 86: a single face value of the concentration, shared by both neighbouring cells, is eliminated (Part V, Eqs. (5.8) and (5.9)).

Partition coefficient: FUTURE IMPROVEMENT. Gelbard (2003, Eq. (IV.3), p. 27) writes the more general condition

$$\gamma_\ell \, c_\ell = \gamma_{\ell+1} \, c_{\ell+1} \quad \text{at } r = R_\ell,$$

with dimensionless partition factors $\gamma_\ell > 0$ (adapted from published literature; Gelbard numbers layers from his own index i). Solving for the outer value gives $c_{\ell+1} = (\gamma_\ell/\gamma_{\ell+1})\, c_\ell$, and setting every $\gamma_\ell = 1$ recovers Eq. (3.2) (check P3.14). Two findings of the STRATO source audit of Gelbard (2003), which this document did not repeat in full (Section 7.5), matter for any future implementation:

- With partition factors the orthogonality weight changes from r^2 to $\gamma_\ell r^2$ (Gelbard, 2003, Eq. (IV.7), p. 29), and the release weights of Part IV acquire γ_1 and a γ-weighted norm. The audit confirmed this weight numerically; this document checks only the algebra (check P3.14).
- Gelbard's Section V example states $2\gamma_{\text{SiC}} = \gamma_{\text{others}} = 1$. Under Eq. (IV.3) as printed this gives $c_{\text{SiC}} = 2\, c_{\text{IPyC}}$, a doubling (check P3.14), while the text says the concentration drops by a factor of two into the SiC. The audit reproduced Gelbard's Fig. V.2 only with $c_{\text{SiC}} = c_{\text{PyC}}/2$ (largest difference 0.0011 in F against 0.333 for the printed orientation). A future version must state its own convention explicitly, for example $c_{\ell+1} = K_{\ell,\ell+1}\, c_\ell$, and must not copy the orientation of (IV.3) without checking it.

3.2.2 Flux continuity (derived from conservation)

Unlike Eq. (3.2), the flux condition follows from conservation alone. The idea is that a surface has no volume, so it cannot store anything: whatever arrives at the interface from inside must leave on the outside at the same moment.

Governing principle. The conservation law (2.1) with no source, applied to a thin shell that straddles the interface.

Step 1: choose the control volume. Take the shell $R_\ell - \delta < r < R_\ell + \delta$ with $0 < \delta$ small enough that the shell contains no other interface. Its inner surface lies in layer ℓ, its outer surface in layer $\ell + 1$. By Eq. (2.1), using the sphere area of Eq. (1.5):

$$\frac{d}{dt}\int_{R_\ell-\delta}^{R_\ell+\delta} c\, 4\pi r^2\, dr = 4\pi(R_\ell-\delta)^2 J_r(R_\ell-\delta,t) - 4\pi(R_\ell+\delta)^2 J_r(R_\ell+\delta,t).$$

The first term on the right is the flow in through the inner surface, the second the flow out through the outer surface.

Step 2: bound the storage term. If $|\partial_t c| \le \mathcal{K}$ on both sides near the interface, the left side is at most $\mathcal{K}$ times the shell volume, Eq. (1.6), by the same reasoning as Step 1 of Section 3.1.2:

$$\begin{aligned}\left|\frac{d}{dt}\int_{R_\ell-\delta}^{R_\ell+\delta} c\, 4\pi r^2\, dr\right| &= \left|\int_{R_\ell-\delta}^{R_\ell+\delta} \frac{\partial c}{\partial t}\, 4\pi r^2\, dr\right| \le \int_{R_\ell-\delta}^{R_\ell+\delta} \mathcal{K}\, 4\pi r^2\, dr \\ &= \mathcal{K}\,\frac{4\pi}{3}\left[(R_\ell+\delta)^3 - (R_\ell-\delta)^3\right] \\ &= \mathcal{K}\,\frac{4\pi}{3}\left(6R_\ell^2\delta + 2\delta^3\right).\end{aligned}$$

The last line expands both cubes, $(R_\ell \pm \delta)^3 = R_\ell^3 \pm 3R_\ell^2\delta + 3R_\ell\delta^2 \pm \delta^3$. In the difference the terms R_ℓ^3 and $3R_\ell\delta^2$ (even powers of δ) cancel, and the odd-power terms double: $2 \times 3R_\ell^2\delta + 2\delta^3 = 6R_\ell^2\delta + 2\delta^3$ (check P3.7).

Step 3: shrink the shell. As $\delta \to 0$ the bound of Step 2 goes to zero, so the left side of Step 1 vanishes. On the right side, $4\pi(R_\ell \mp \delta)^2 \to 4\pi R_\ell^2$ and the fluxes tend to their one-sided limits:

$$0 = 4\pi R_\ell^2\left[J_r(R_\ell^-,t) - J_r(R_\ell^+,t)\right].$$

Step 4: divide by the area. $R_\ell > 0$ for $\ell \ge 1$, so we may divide by $4\pi R_\ell^2$:

$$J_r(R_\ell^-,t) = J_r(R_\ell^+,t).$$

Step 5: insert Fick's law on each side. Inside, $J_r = -D_\ell\,\partial_r c_\ell$; outside, $J_r = -D_{\ell+1}\,\partial_r c_{\ell+1}$. Equating and multiplying by -1:

$$D_\ell \frac{\partial c_\ell}{\partial r}(R_\ell^-, t) = D_{\ell+1} \frac{\partial c_{\ell+1}}{\partial r}(R_\ell^+, t), \qquad \ell = 1, \dots, 4, \quad t > 0. \tag{3.3}$$

Eqs. (3.2) and (3.3) are Gelbard's (2003) Eqs. (IV.3) and (IV.2) on p. 27 with all $\gamma_\ell = 1$ (adapted from published literature); the thin-shell derivation is derived for STRATO. Eq. (3.3) is PROVED from conservation and Fick's law, under the assumption of a bounded $\partial_t c$ and one-sided limits of the flux.

Classification: IMPLEMENTED. Code C keeps $D(u' - u/r)$ continuous through the jump rule $q^+ = q^- + (D_{\ell+1} - D_\ell)\, u/R_\ell$ in src/analytical/sturm_liouville.rs, lines 94 to 100, called at every interface by the shooting function in lines 138 to 140 and by the eigenvalue counter in lines 172 to 174 (the algebra is Part IV, Eqs. (4.7) and (4.8)). Code B computes one conductance per face (src/numerical/finite_volume.rs, lines 78 to 87) and uses that same number as the east conductance of the inner cell and the west conductance of the outer cell (lines 172 to 177). The amount that leaves one cell through a face is therefore exactly the amount that enters its neighbour, which is the discrete form of Eq. (3.3).

What Eq. (3.3) means for the shape of the profile. Where the inner slope is not zero, dividing Eq. (3.3) by $D_{\ell+1}\, \partial_r c_\ell$ gives the ratio of the slopes on the two sides:

$$\frac{\partial_r c_{\ell+1}(R_\ell^+)}{\partial_r c_\ell(R_\ell^-)} = \frac{D_\ell}{D_{\ell+1}}.$$

The concentration is continuous but bends sharply at an interface: the slope is steep in a slow layer and shallow in a fast one. A picture in words: water flowing through a wide pipe joined to a narrow pipe carries the same litres per second through both, but the narrow pipe needs a much larger pressure drop per metre. For the synthetic input the slope ratios are 0.0025 at R_1, 500 at R_2, 40 at R_3 and 0.02857 at R_4 (calculated from Table 3.1, synthetic input; check P3.15). The same ratios are measured on the slowest five-layer mode, built independently from closed-form pieces (DEMONSTRATED NUMERICALLY; check P3.16). The factor 500 at the buffer to IPyC interface means the profile in the IPyC starts 500 times steeper than it ends in the buffer.

Why slope continuity would be wrong. If one imposed $\partial_r c_\ell = \partial_r c_{\ell+1}$ instead of Eq. (3.3), the flow $4\pi r^2 J_r$ would jump at the interface whenever $D_\ell \neq D_{\ell+1}$, so the interface would create or destroy material. Check P3.8 confirms this on the steady two-layer shell $c = \zeta_\ell + \xi_\ell / r$: with Eqs. (3.2) and (3.3) the flow is the same constant in both layers; with slope continuity it differs (DEMONSTRATED NUMERICALLY for this example; the general statement follows directly from $4\pi R_\ell^2 D_\ell\, \partial_r c \neq 4\pi R_\ell^2 D_{\ell+1}\, \partial_r c$ when $D_\ell \neq D_{\ell+1}$ and $\partial_r c \neq 0$).

3.3 Outer surface: a perfect sink

Physical problem. Outside the OPyC layer lies the graphite matrix of the fuel element and, further out, the coolant. v1.0.0 does not model either of them.

What we want. One condition at $r = R$ that replaces everything outside the particle.

The condition. We assume that every atom reaching the outer surface is removed at once, so the concentration there is held at zero:

$$c_5(R,t) = 0, \qquad t > 0. \tag{3.4}$$

A condition of this kind, which fixes the value of the unknown on a boundary, is called a *Dirichlet* condition. The picture is a *perfect sink*: like a drain with infinite capacity, it never fills up and never slows the outflow. Gelbard (2003, p. 27) uses the same absorbing condition at the outer radius (adapted from published literature).

What it neglects.

- Mass-transfer resistance at the surface. A real surface passes atoms at a finite rate, which the Robin condition of Section 3.6 represents through a coefficient h in m s^{-1}. Eq. (3.4) is the limit $h \to \infty$ of that condition with zero ambient concentration (Section 3.6, check P3.12).
- Everything outside the particle: diffusion and sorption in the matrix graphite, and any build-up of the species around the particle that would reduce the concentration difference driving the release.

Because a finite surface resistance can only hold material back, Eq. (3.4) gives the fastest release that the five coating layers allow. For a homogeneous sphere this ordering is DEMONSTRATED NUMERICALLY in Section 3.6 (Table 3.2, checks P3.13 and P3.17); for the five-layer case it is CLAIMED ONLY on physical reasoning only.

Classification: IMPLEMENTED. Code B connects the last cell's volume centroid r_{N_c} (the volume-weighted average radius of the cell, $\frac{3}{4}(b^4 - a^4)/(b^3 - a^3)$ for a cell from a to b, src/numerical/finite_volume.rs, lines 42 to 48; Part V) to the value $c = 0$ at R through the half-cell conductance $G_R = 4\pi R^2 D_5/(R - r_{N_c})$ (src/numerical/finite_volume.rs, lines 88 to 91), used as the east conductance of the last cell (lines 173 to 177); the derivation is Part V, Eq. (5.11). In Code C the eigenvalues are the values of Λ at which $u(R;\Lambda) = 0$, which is Eq. (3.4) in the variable u (module statement in src/analytical/sturm_liouville.rs, line 15; Part IV, Eq. (4.10)). The code does not search for sign changes of $u(R;\Lambda)$ directly. It counts the zeros of u in $(0, R]$ (lines 166 to 192); this count rises by one exactly when Λ passes a value with $u(R;\Lambda) = 0$, and bisection on the count isolates each such value (lines 197 to 262). The shooting function (lines 132 to 162) returns $u(R;\Lambda)$ and the layer states used afterwards for the integrals.

3.4 Initial state

Physical problem. At the start of the calculation (for example, the start of a heating test) the fission product is in the kernel and the coatings are clean.

The condition. With c_0 the initial kernel concentration in mol m^{-3}:

$$c(r,0) = \begin{cases} c_0, & 0 \le r < R_1, \\ 0, & R_1 < r < R. \end{cases} \tag{3.5}$$

The value at exactly $r = R_1$ is left open on purpose: a single radius is a sphere of zero thickness, holds no amount, and does not affect any integral below. For every $t > 0$ the solution is continuous at R_1 by Eq. (3.2); only the starting profile jumps there. That jump has two consequences used later: the series of Part IV converges slowly in the number of modes, and Code B must resolve a steep internal layer at R_1 at early times.

The synthetic input uses $c_0 = 1$ mol m^{-3} (inputs/synthetic/synthetic_five_layer.toml, lines 36 to 37; synthetic input). The reader accepts any finite $c_0 \geq 0$ (src/config.rs, lines 108 to 111), and the solver then stops with an error if the initial inventory is not positive (src/numerical/finite_volume.rs, lines 186 to 189), so $c_0 = 0$ is rejected at run time.

Classification: IMPLEMENTED. Code B sets every cell of layer 1 to c_0 and every other cell to zero (src/numerical/finite_volume.rs, lines 101 to 112). Because the last kernel face is set exactly to R_1 (lines 55 to 62), the kernel cells cover $[0, R_1]$ exactly and the cell averages of Eq. (3.5) are exact. Code C uses Eq. (3.5) only through the kernel moment $I_{1,n} = \int_0^{R_1} r\, u_n\, dr$, an integral over the first layer (src/analytical/sturm_liouville.rs, lines 346 to 350; Part IV, Eq. (4.20)). A different starting profile (for example fission products already in the buffer) is FUTURE IMPROVEMENT.

3.5 Inventory and release

3.5.1 Initial inventory

What we want. The total amount M^0 (in mol) inside the particle at $t = 0$.

Step 1: start from the inventory, Eq. (1.7), at $t = 0$.

$$M^0 = M(0) = \int_0^R c(r, 0)\, 4\pi r^2\, dr.$$

Step 2: split the integral at R_1, where Eq. (3.5) changes form.

$$M^0 = \int_0^{R_1} c_0\, 4\pi r^2\, dr + \int_{R_1}^R 0 \cdot 4\pi r^2\, dr.$$

Step 3: the second integral is zero; take the constants out of the first.

$$M^0 = 4\pi c_0 \int_0^{R_1} r^2\, dr.$$

Step 4: integrate r^2, whose antiderivative is $r^3/3$.

$$M^0 = 4\pi c_0 \left[\frac{r^3}{3}\right]_0^{R_1} = 4\pi c_0\, \frac{R_1^3}{3}.$$

Result.

$$M^0 = \frac{4\pi}{3} R_1^3\, c_0. \tag{3.6}$$

Physical meaning. The initial inventory is the kernel volume, $\frac{4\pi}{3}R_1^3$ in m^3, times the kernel concentration in mol m^{-3}. For the synthetic input, $M^0 = \frac{4\pi}{3}(2.0 \times 10^{-4}\ \text{m})^3 \times$

$1\ \text{mol m}^{-3} = 3.351032163829113 \times 10^{-11}$ mol (calculated, synthetic input; check P3.6).

Classification: IMPLEMENTED in both codes, in different forms.

Code B computes $M^0 = \sum_j V_j C_j^0$ from the cell volumes (src/numerical/finite_volume.rs, line 186, using `inventory`, lines 114 to 120); Part V, Eq. (5.20). This equals Eq. (3.6) in exact arithmetic because the kernel cell volumes telescope. With faces $0 = r_{1/2} < r_{3/2} < \cdots < r_{n_1+1/2} = R_1$ and $V_j = \frac{4\pi}{3}(r_{j+1/2}^3 - r_{j-1/2}^3)$,

$$\sum_{j=1}^{n_1} V_j c_0 = \frac{4\pi}{3} c_0 \left(r_{n_1+1/2}^3 - r_{1/2}^3\right) = \frac{4\pi}{3} c_0 \left(R_1^3 - 0\right),$$

because every inner face cube appears once with a plus sign and once with a minus sign. For the synthetic mesh (40 kernel cells) the floating-point sum and the closed form agree to the last bit (relative difference 0; check P3.6b), and the first row of the v1.0.0 golden output `tests/golden/v1.0/timeseries.csv` holds the same number, $3.351032163829113 \times 10^{-11}$ mol.

Code C never forms M^0 as a number. Dividing by Eq. (3.6) is what puts the factor $3/R_1^3$ into the modal weight $w_n = 3I_{1,n}I_n/(R_1^3 N_n)$ (src/analytical/sturm_liouville.rs, line 352; Part IV, Eq. (4.22)).

3.5.2 Release rate

Definition. The *release rate* $\Phi_R(t)$ is the total amount per second that leaves the particle through its outer surface, in mol s^{-1}. It is the outward flow through the sphere of radius R: area times flux, with Fick's law in layer 5:

$$\Phi_R(t) = 4\pi R^2\, J_r(R,t) = -4\pi R^2 D_5 \frac{\partial c_5}{\partial r}(R,t). \tag{3.7}$$

Units: $\text{m}^2 \times \text{m}^2\ \text{s}^{-1} \times \text{mol m}^{-3}\ \text{m}^{-1} = \text{mol s}^{-1}$ (check P3.11).

Classification: IMPLEMENTED in discrete form. Code B approximates Eq. (3.7) by $\Phi_R \approx G_R\, C_{N_c}$, conductance times last-cell concentration ($\text{m}^3\ \text{s}^{-1} \times \text{mol m}^{-3} = \text{mol s}^{-1}$), at $t = 0$ in src/numerical/finite_volume.rs, line 202 and after every step in line 210 (Part V, Eqs. (5.11) and (5.22)). It is written to the output column `release_rate_mol_s` (src/main.rs, line 50). With the kernel-only start the last cell is empty at $t = 0$, so the first row reports $\Phi_R = 0$. Code C does not compute Φ_R.

The delivered Phase 1 derivation defines a dimensionless release rate $G_R C_N^n / M^0$ instead (Table 7.3, item I1). That quantity is the time derivative of F derived in Section 3.5.4 below; it is DERIVED NOT IMPLEMENTED, and the code outputs Φ_R in mol s^{-1}.

3.5.3 Global balance

What we want. How the inventory $M(t) = \int_0^R c\, 4\pi r^2\, dr$ (Eq. (1.7)) changes in time, using only the layer equation and the conditions of this part.

Step 1: split the inventory by layer.

$$M(t) = \sum_{\ell=1}^{5} \int_{R_{\ell-1}}^{R_\ell} c_\ell(r,t)\, 4\pi r^2 \, dr.$$

Step 2: differentiate in time. The layer limits $R_{\ell-1}$, R_ℓ do not move, so the time derivative of each integral is the integral of the time derivative. In words: the inventory is a sum of many thin shells, each of which changes at the rate $\partial_t c$ times its volume.

$$\frac{dM}{dt} = \sum_{\ell=1}^{5} \int_{R_{\ell-1}}^{R_\ell} \frac{\partial c_\ell}{\partial t}\, 4\pi r^2 \, dr.$$

Step 3: insert the layer equation (2.7). Multiply Eq. (2.7) by $4\pi r^2$; the factor r^2 cancels the $1/r^2$:

$$\frac{\partial c_\ell}{\partial t}\, 4\pi r^2 = 4\pi D_\ell \, \frac{\partial}{\partial r}\left(r^2 \frac{\partial c_\ell}{\partial r} \right).$$

Step 4: integrate over the layer. The integrand is a derivative with respect to r. The fundamental theorem of calculus says that adding up the rate of change of a quantity across an interval gives the total change between the two ends, $\int_a^b \frac{dg}{dr}\, dr = g(b) - g(a)$, so the integral is the difference of end values. Define the inward-counted flow $\Pi_\ell(r,t) = 4\pi r^2 D_\ell \, \partial_r c_\ell(r,t)$ in mol s^{-1} (it equals minus the outward flow $\Phi = 4\pi r^2 J_r$ of Section 2.3). Then

$$\begin{aligned} \int_{R_{\ell-1}}^{R_\ell} \frac{\partial c_\ell}{\partial t}\, 4\pi r^2 \, dr &= \int_{R_{\ell-1}}^{R_\ell} \frac{\partial}{\partial r}\left(4\pi D_\ell \, r^2 \frac{\partial c_\ell}{\partial r} \right) dr = \int_{R_{\ell-1}}^{R_\ell} \frac{\partial \Pi_\ell}{\partial r}\, dr \\ &= \Pi_\ell(R_\ell^-, t) - \Pi_\ell(R_{\ell-1}^+, t). \end{aligned}$$

Step 5: add the five layers and regroup by interface. Writing out the sum and collecting the two terms that belong to the same radius:

$$\begin{aligned} \frac{dM}{dt} &= \left[\Pi_1(R_1^-) - \Pi_1(0^+)\right] + \left[\Pi_2(R_2^-) - \Pi_2(R_1^+)\right] + \cdots + \left[\Pi_5(R^-) - \Pi_5(R_4^+)\right] \\ &= -\Pi_1(0^+) + \sum_{\ell=1}^{4} \left[\Pi_\ell(R_\ell^-) - \Pi_{\ell+1}(R_\ell^+)\right] + \Pi_5(R^-). \end{aligned}$$

This regrouping is a *telescoping sum*: picture a row of rooms joined by doors. Every inner door is counted twice, once as an exit from one room and once as an entrance to the next, so in the total only the two outer doors remain.

Step 6: use the conditions. The centre term vanishes by Eq. (3.1): $\Pi_1(0^+) = \lim_{r\to 0^+} 4\pi r^2 D_1\,\partial_r c_1 = 0$. Each interface bracket vanishes by Eq. (3.3) multiplied by $4\pi R_\ell^2$. Only the outer surface remains, and by Eq. (3.7) $\Pi_5(R^-) = -\Phi_R$:

$$\frac{dM}{dt} = -\Phi_R(t). \tag{3.8}$$

Physical meaning. The particle loses material only through its outer surface; nothing is created or lost at the centre or at the interfaces. Eq. (3.8) is PROVED for the continuous model. Only Eqs. (3.1) and (3.3) were used. The concentration condition (3.2) and the outer condition (3.4) were not needed, so Eq. (3.8) also holds with a partition factor or with the Robin surface of Section 3.6, provided Φ_R is defined as the outward flow at R.

Classification: DERIVED NOT IMPLEMENTED (a property of the continuous model; no code evaluates dM/dt). Its discrete counterpart, Part VI Eq. (6.1), is exact for Code B. Code B computes the time-integrated form every step as the balance residual $\varepsilon^n = M^n + M^n_{\text{rel}} - M^0$ (src/numerical/finite_volume.rs, lines 219 to 220) and writes it at the output rows; the verification driver asserts that the relative residual stays below 10^{-12} (examples/verify.rs, lines 217 to 222). The largest relative residual of the v1.0.0 runs is 2.806×10^{-13}: it is the largest value of $|\varepsilon^n|/M^0$ in the main-run golden output `tests/golden/v1.0/timeseries.csv` and the 200-cell line of the refinement study in `tests/golden/v1.0/verify_stdout.txt` (lines 28 to 32 there give 5.4×10^{-15} to 2.806×10^{-13} for 100 to 1600 cells). For the synthetic five-layer slowest mode, a shooting in the continuous pair $(\phi, \Pi = r^2 D\phi')$ gives $\Lambda_1 = 3.273255671177117 \times 10^{-6}$ s^{-1}, within 2.9×10^{-14} (relative) of the 40-digit reference value $3.273255671177023 \times 10^{-6}$ s^{-1} (Part IV, Section 4.7), and $-dM/dt$ equals Φ_R to a relative 4.4×10^{-16} (checks P3.9a and P3.9b). That balance is partly built into the shooting, which carries Π continuously, so check P3.16 repeats it with a mode built from closed-form pieces, $\phi_\ell = (\zeta_\ell \sin k_\ell r + \xi_\ell \cos k_\ell r)/r$ matched at each interface, and M found by numerical integration: $\Lambda_1 = 3.273255671175929 \times 10^{-6}$ s^{-1} (relative 3.3×10^{-13} from the reference value, the double-precision limit of this construction) and $-dM/dt = \Phi_R$ to a relative 1.0×10^{-13} (calculated, synthetic input; DEMONSTRATED NUMERICALLY). For a single mode of the homogeneous sphere the identity holds symbolically (check P3.10).

3.5.4 Cumulative release and release fraction

Step 1: define the cumulative release. The amount released up to time t, $M_{\text{rel}}(t)$ in mol, is the release rate summed over time:

$$M_{\text{rel}}(t) = \int_0^t \Phi_R(t')\,dt'.$$

Step 2: integrate Eq. (3.8) from 0 to t. The left side integrates to $M(t) - M(0) = M(t) - M^0$:

$$M(t) - M^0 = -\int_0^t \Phi_R(t')\,dt' = -M_{\text{rel}}(t).$$

Step 3: rearrange. What is still inside plus what has left equals what was there at the start:

$$M(t) + M_{\text{rel}}(t) = M^0.$$

Step 4: divide by $M^0 > 0$. The *release fraction* $F(t)$ is the dimensionless share of the initial inventory that has left the particle. Dividing Step 3 by M^0 gives two equivalent expressions for it. Together with Step 1:

$$M_{\text{rel}}(t) = \int_0^t \Phi_R(t')\,dt', \qquad F(t) = \frac{M_{\text{rel}}(t)}{M^0} = 1 - \frac{M(t)}{M^0}. \tag{3.9}$$

Classification: IMPLEMENTED, and the two codes use the two different forms of Eq. (3.9):

- Code B uses the first form. It accumulates M_{rel} step by step from $\Delta t\,\Phi_R^{n+1}$ with Kahan compensated summation (src/numerical/finite_volume.rs, lines 211 to 217) and divides by M^0 (line 227); Part V, Eqs. (5.22) to (5.24). It also computes the inventory M^n independently (line 219); the residual ε^n is the discrete gap between the two forms of Eq. (3.9), multiplied by M^0.
- Code C uses the second form. Its series gives $M(t)/M^0 = \sum_n w_n e^{-\Lambda_n t}$ (src/analytical/sturm_liouville.rs, lines 381 to 388), and $F = 1-$ that sum (lines 390 to 393); Part IV, Eq. (4.22).

So the comparison of Code A tests the two forms of Eq. (3.9) against each other: one built from the surface flux, the other from what remains inside. For the synthetic case at $t = 2\times10^5$ s Code C gives $F = 0.43255563909286$, which differs by 1.9×10^{-13} from the independent high-precision evaluation 0.43255563909304 (Part IV, Section 4.10), and the 200-cell finite-volume run with $\Delta t = 200$ s gives 0.4324286068785 (main run, golden file `tests/golden/v1.0/comparison_fv_vs_analytical.csv`); all three are calculated values for synthetic input.

Properties of F.

At the start, $F(0) = 0$, because $M_{\text{rel}}(0) = 0$ (PROVED). Differentiating the first form of Eq. (3.9) gives the release rate as a share of the initial inventory per second, in s^{-1} (DERIVED NOT IMPLEMENTED; see Section 3.5.2):

$$\frac{dF}{dt} = \frac{\Phi_R(t)}{M^0}.$$

Two further properties follow under stated conditions.

- If $c \geq 0$ everywhere, then $M \geq 0$ and so $F \leq 1$. Moreover $\Phi_R \geq 0$: since $c_5(R) = 0$ by Eq. (3.4), a positive slope $\partial_r c_5(R) > 0$ would make $c_5(R-h) \approx -h\,\partial_r c_5(R) < 0$ just inside the surface, which is impossible, so $\partial_r c_5(R) \leq 0$

and Eq. (3.7) gives $\Phi_R \geq 0$. Then F never decreases. These consequences are PROVED once $c \geq 0$ is granted, and Section 3.5.5 proves $c \geq 0$ for the continuous model. For Code B the discrete counterpart is proved in Part VI (Eqs. (6.2) to (6.4)).

- $F \to 1$ as $t \to \infty$: the whole inventory eventually leaves. Section 3.5.5 proves this from the equations of this part alone, with an explicit (if generous) bound on how slowly it can happen. The series of Part IV, Eq. (4.22), gives the same limit once its weights are known to add up to one (Part IV, Eq. (4.23)).

3.5.5 Proofs: the concentration stays nonnegative and the inventory empties

What we want. Proofs of the two statements used above: $c(r,t) \geq 0$ for all r and t, and $M(t) \to 0$, so $F(t) \to 1$, as $t \to \infty$.

Assumptions. Inside each layer c is smooth for $t > 0$; the conditions (3.1) to (3.4) hold; the start (3.5) has $c_0 \geq 0$, and the integrals $\mathcal{Z}$ and $\mathcal{E}$ defined below approach their values for the start (3.5) as $t \to 0$. Write $\Pi = 4\pi r^2 D\, \partial_r c$ for the inward-counted flow of Section 3.5.3 (in mol s^{-1}), with $D = D_\ell$ in layer ℓ, and $D_{\min}$ for the smallest of the five diffusivities (in $\mathrm{m}^2\ \mathrm{s}^{-1}$).

The idea. Both proofs follow one quantity that is a sum of squares, so it can never be negative, and show that it can only shrink. A quantity that starts at zero, can never be negative and can never grow must stay zero.

Tool: moving the derivative across the layers. Let $\Omega(r)$ be any function that is continuous at the interfaces, smooth inside each layer and zero at $r = R$.

Step 1. The product rule gives $\frac{\partial}{\partial r}(\Omega\Pi) = \frac{\partial \Omega}{\partial r}\Pi + \Omega\frac{\partial \Pi}{\partial r}$.

Step 2. Integrate over layer ℓ and use the fundamental theorem of calculus on the left side:

$$\int_{R_{\ell-1}}^{R_\ell} \Omega\, \frac{\partial \Pi}{\partial r}\, dr = \Big[\Omega\,\Pi\Big]_{R_{\ell-1}^+}^{R_\ell^-} - \int_{R_{\ell-1}}^{R_\ell} \frac{\partial \Omega}{\partial r}\, \Pi\, dr.$$

Step 3. Add the five layers. The end terms telescope exactly as in Step 5 of Section 3.5.3. At the centre $\Pi \to 0$ by Eq. (3.1) while Ω stays finite. At each interface Ω is continuous and Π is continuous by Eq. (3.3) multiplied by $4\pi R_\ell^2$, so the term from the inner layer cancels the term from the outer layer. At the surface $\Omega(R) = 0$. What is left is

$$\sum_{\ell=1}^{5} \int_{R_{\ell-1}}^{R_\ell} \Omega\, \frac{\partial \Pi}{\partial r}\, dr = -\int_0^R \frac{\partial \Omega}{\partial r}\, 4\pi r^2 D\, \frac{\partial c}{\partial r}\, dr.$$

Recall from Step 3 of Section 3.5.3 that $\frac{\partial c}{\partial t}\, 4\pi r^2 = \frac{\partial \Pi}{\partial r}$ inside each layer.

Proof 1: $c \geq 0$. Let $c^- = \min(c, 0)$ be the negative part of c: it equals c where $c < 0$ and is zero elsewhere. Define

$$\mathcal{Z}(t) = \int_0^R (c^-)^2 \, 4\pi r^2 \, dr \qquad (\text{mol}^2 \text{ m}^{-3}).$$

$\mathcal{Z} \geq 0$ always, and $\mathcal{Z} = 0$ exactly when $c \geq 0$ everywhere.

Step 1. At $t = 0$ every value of Eq. (3.5) is $c_0 \geq 0$ or 0, so $\mathcal{Z}(0) = 0$.

Step 2. Where $c < 0$, $(c^-)^2 = c^2$ changes at the rate $2c\,\partial_t c = 2c^-\,\partial_t c$; where $c > 0$, both $(c^-)^2$ and $2c^-\,\partial_t c$ are zero. So in both cases the rate is $2c^-\,\partial_t c$, and

$$\frac{d\mathcal{Z}}{dt} = \int_0^R 2c^- \frac{\partial c}{\partial t} \, 4\pi r^2 \, dr = 2 \sum_{\ell=1}^{5} \int_{R_{\ell-1}}^{R_\ell} c^- \frac{\partial \Pi}{\partial r} \, dr.$$

Step 3. Use the tool with $\Omega = c^-$. This is allowed: c^- is continuous at the interfaces because c is (Eq. (3.2)), and $c^-(R) = 0$ because $c(R) = 0$ (Eq. (3.4)). Its slope is $\partial_r c$ where $c < 0$ and 0 where $c > 0$; the isolated points where c crosses zero have no width and change no integral. Hence $\frac{\partial c^-}{\partial r}\frac{\partial c}{\partial r} = \left(\frac{\partial c^-}{\partial r}\right)^2$ everywhere, and

$$\frac{d\mathcal{Z}}{dt} = -2 \int_0^R 4\pi r^2 D \left(\frac{\partial c^-}{\partial r}\right)^2 dr \leq 0.$$

Step 4. $\mathcal{Z}$ starts at zero, is never negative and never increases, so $\mathcal{Z}(t) = 0$ for every t. Therefore $c(r,t) \geq 0$ everywhere and at all times (check P3.18a).

Proof 2: $M \to 0$ and $F \to 1$. Define the "energy"

$$\mathcal{E}(t) = \int_0^R c^2 \, 4\pi r^2 \, dr \qquad (\text{mol}^2 \text{ m}^{-3}).$$

Step 1. The same steps with $\Omega = c$ (continuous, zero at R) give

$$\frac{d\mathcal{E}}{dt} = 2 \sum_{\ell=1}^{5} \int_{R_{\ell-1}}^{R_\ell} c \frac{\partial \Pi}{\partial r} \, dr = -2 \int_0^R 4\pi r^2 D \left(\frac{\partial c}{\partial r}\right)^2 dr = -2\mathcal{W},$$

where $\mathcal{W} = \int_0^R 4\pi r^2 D \, (\partial_r c)^2 \, dr \geq 0$ (in $\text{mol}^2 \text{ m}^{-3} \text{ s}^{-1}$) measures how fast the profile is being smoothed out.

Step 2. A rule for sums of products (the Cauchy-Schwarz inequality). For any numbers a_i and b_i, $\left(\sum a_i b_i\right)^2 \leq \left(\sum a_i^2\right)\left(\sum b_i^2\right)$. Reason: for every number λ, $\sum (a_i\lambda + b_i)^2 \geq 0$. This is the quadratic $\lambda^2 \sum a_i^2 + 2\lambda \sum a_i b_i + \sum b_i^2$, which never dips below zero, so its discriminant cannot be positive: $4\left(\sum a_i b_i\right)^2 - 4 \sum a_i^2 \sum b_i^2 \leq 0$. An integral is a limit of such sums, so $\left(\int fg\right)^2 \leq \int f^2 \int g^2$ as well.

Step 3. Bound c at one radius by $\mathcal{W}$. Because $c(R) = 0$, the fundamental theorem of calculus gives $c(r) = -\int_r^R \partial_s c \, ds$. Write the integrand as a product, $\partial_s c = \left(s\sqrt{D}\,\partial_s c\right) \times \left(1/(s\sqrt{D})\right)$, and apply Step 2:

$$c(r)^2 \le \int_r^R \frac{ds}{D\,s^2} \int_r^R s^2 D\,(\partial_s c)^2\, ds \le \frac{1}{D_{\min}} \left(\frac{1}{r} - \frac{1}{R}\right) \frac{\mathcal{W}}{4\pi}.$$

The second inequality uses $D \ge D_{\min}$, $\int_r^R s^{-2}\, ds = 1/r - 1/R$, and the fact that the integral from r to R of a nonnegative quantity is at most the integral from 0 to R.

Step 4. Multiply by $4\pi r^2$ and integrate from 0 to R, using $\int_0^R r^2 \left(\frac{1}{r} - \frac{1}{R}\right) dr = \frac{R^2}{2} - \frac{R^2}{3} = \frac{R^2}{6}$:

$$\mathcal{E} \le \frac{R^2}{6D_{\min}}\,\mathcal{W}, \qquad \text{so} \qquad \frac{d\mathcal{E}}{dt} = -2\mathcal{W} \le -\frac{12 D_{\min}}{R^2}\,\mathcal{E}.$$

Step 5. Write $a = 12D_{\min}/R^2$ (in s^{-1}). Then $\frac{d}{dt}\left(\mathcal{E}\, e^{at}\right) = e^{at}\left(\frac{d\mathcal{E}}{dt} + a\mathcal{E}\right) \le 0$, so $\mathcal{E}(t)e^{at}$ never grows and $\mathcal{E}(t) \le \mathcal{E}(0)\, e^{-at}$. With Eq. (3.5), $\mathcal{E}(0) = c_0^2 \frac{4\pi}{3} R_1^3$.

Step 6. Bound the inventory by $\mathcal{E}$, again with Step 2 ($f = c$, $g = 1$, both weighted by $4\pi r^2$): $|M| = \left|\int_0^R c \cdot 1 \cdot 4\pi r^2\, dr\right| \le \sqrt{\mathcal{E}}\,\sqrt{\frac{4\pi}{3}R^3}$. Divide by $M^0 = \frac{4\pi}{3} R_1^3 c_0$ from Eq. (3.6), insert Step 5 and simplify:

$$0 \le 1 - F(t) = \frac{M(t)}{M^0} \le \left(\frac{R}{R_1}\right)^{3/2} \exp\left(-\frac{6D_{\min}t}{R^2}\right) \longrightarrow 0 \quad \text{as } t \to \infty.$$

The lower bound 0 is Proof 1. Hence $F(t) \to 1$.

Physical meaning. Diffusion smooths a profile, so it cannot push any value below zero. With an open outer surface the whole inventory eventually escapes, at a rate no slower than one set by the slowest layer spread over the whole radius. The bound is generous: for the synthetic input $6D_{\min}/R^2 = 5.926 \times 10^{-7}\ \mathrm{s}^{-1}$, while the true slowest decay rate is $\Lambda_1 = 3.273 \times 10^{-6}\ \mathrm{s}^{-1}$, and at $t = 2 \times 10^5$ s the bound $3.375 \times e^{-0.1185} = 3.0$ says nothing useful about the true value $1 - F = 1 - 0.43256 = 0.567$ of Section 3.5.4 (calculated, synthetic input; check P3.18d gives the rate and the prefactor $(R/R_1)^{3/2} = 3.375$). The bound serves only to prove the limit.

Classification: DERIVED NOT IMPLEMENTED (no code evaluates $\mathcal{Z}$, $\mathcal{E}$ or the bound); derived for STRATO. Both results are PROVED under the assumptions stated at the start of this section. The intermediate inequalities are also DEMONSTRATED NUMERICALLY on random test profiles (check P3.18b). The code counterparts are separate: Code B keeps every cell value nonnegative by its matrix structure (Part VI, Eqs. (6.2) to (6.4)), and the verification driver asserts that the smallest cell value of the final state is nonnegative (examples/verify.rs, line 215).

3.6 Case B reference: the Robin surface

The delivered Phase 1 documents describe a second test problem, Case B: a homogeneous sphere in which the species is produced continuously and the surroundings remove it at a limited rate. Case A (this part) and Case B start from different situations, so their results are never compared. Only one piece of Case B is in the v1.0.0 code, and it is used only by a unit test.

The condition. A finite surface resistance is described by a *Robin* condition: the outward flux at the surface is proportional to the difference between the surface concentration and the ambient concentration c_∞ (in mol m^{-3}), with mass-transfer coefficient h_m (in m s^{-1}):

$$-D_5 \frac{\partial c_5}{\partial r}(R,t) = h_\mathrm{m}\left[c_5(R,t) - c_\infty\right]. \tag{3.10}$$

Classification: FUTURE IMPROVEMENT for the five-layer problem. Neither Code B nor Code C applies Eq. (3.10) to the layered particle.

Two limits.

Step 1. Solve Eq. (3.10) for the surface value, writing $g = \partial_r c_5(R,t)$:

$$c_5(R,t) = c_\infty - \frac{D_5\, g}{h_\mathrm{m}}.$$

Step 2. As $h_\mathrm{m} \to \infty$ (no resistance) with g bounded, the last term vanishes and $c_5(R,t) \to c_\infty$. With $c_\infty = 0$ this is Eq. (3.4) (check P3.12). As $h_\mathrm{m} \to 0$ the right side of Eq. (3.10) vanishes, so no flux leaves: a sealed surface.

Step 3. Multiplying Eq. (3.10) by R/D_5 gives the dimensionless form

$$-R\frac{\partial c_5}{\partial r}(R,t) = \mathrm{Bi}\left[c_5(R,t) - c_\infty\right], \qquad \mathrm{Bi} = \frac{h_\mathrm{m} R}{D_5},$$

where the *Biot number* Bi compares the ease of leaving through the surface with the ease of diffusing across the particle (check P3.12).

What the code implements. For a homogeneous sphere with $c_\infty = 0$, the decay rates are set by the roots z of $z\cot z = 1 - \mathrm{Bi}$ (Part IV, Eq. (4.31)). This residual is IMPLEMENTED in `robin_residual` (src/analytical/sturm_liouville.rs, lines 31 to 36) and its roots in `robin_roots` (lines 38 to 79). Both are called only by the unit test in lines 447 to 454, never by the program or by the verification driver.

Numerical illustration: surface resistance slows release. For a homogeneous sphere of radius R with a uniform start and $c_\infty = 0$, the same projection that Part IV uses for the kernel (with $R_1 = R$) gives $F = 1 - \sum_n w_n e^{-z_n^2 Dt/R^2}$ with $w_n = 3I_n^2/(R^3 N_n)$, $I_n = R^2(\sin z_n - z_n\cos z_n)/z_n^2$ and $N_n = \frac{R}{2}\left(1 - \frac{\sin 2z_n}{2z_n}\right)$ (DERIVED NOT IMPLEMENTED; in the Dirichlet limit $z_n = n\pi$ it reproduces the uniform-sphere series of Part IV, Eq. (4.30), to 10^{-12}, check P3.13). Table 3.2 lists the resulting values at $Dt/R^2 = 0.1$. An independent finite-volume solve of the same homogeneous sphere

(800 cells, surface resistance $1/h_{\mathrm{m}}$ added in series to the last half cell) reproduces every entry to 2×10^{-5} (check P3.17).

Table 3.2 Release fraction of a homogeneous sphere with uniform start under the Robin condition (3.10) with $c_\infty = 0$. Calculated (derived for STRATO, not implemented); check P3.13 also confirms the ordering at $Dt/R^2 = 0.02$, 0.05 and 0.3, and check P3.17 reproduces the row by finite volume.

Bi	1	10	100	10^6	∞ (Eq. (3.4))
F at $Dt/R^2 = 0.1$	0.228635	0.653988	0.758821	0.770478	0.770479

The release grows with Bi and approaches the perfect-sink value from below, which supports the statement in Section 3.3 that Eq. (3.4) gives the fastest release for given internal transport (DEMONSTRATED NUMERICALLY for the homogeneous sphere at the four times checked). This is a homogeneous-sphere illustration; the five-layer Robin problem is FUTURE IMPROVEMENT.

3.7 Summary and verification

Table 3.3 lists every equation of Part III with its classification and the code that realises it.

Table 3.3 Equations of Part III, their classification and the code that realises them. FV = `src/numerical/finite_volume.rs`, SL = `src/analytical/sturm_liouville.rs`, VER = `examples/verify.rs`; links to every line range are given in the text above.

Eq.	Statement	Classification	Code (tag v1.0.0)
(3.1)	centre: zero slope, zero flow, c finite	IMPLEMENTED	FV line 172; SL lines 134, 168
(3.2)	concentration continuous at $R_1, \dots, R_4$	IMPLEMENTED (assumption $\gamma_\ell = 1$)	FV line 86; SL lines 94 to 100
(3.3)	$D\,\partial_r c$ continuous at $R_1, \dots, R_4$	IMPLEMENTED	FV lines 78 to 87, 172 to 177; SL lines 94 to 100, 138 to 140, 172 to 174
(3.4)	$c_5(R, t) = 0$	IMPLEMENTED	FV lines 88 to 91; SL line 15, lines 166 to 192, 197 to 262
(3.5)	kernel-only uniform start	IMPLEMENTED	FV lines 101 to 112; SL lines 346 to 350
(3.6)	$M^0 = \frac{4\pi}{3} R_1^3 c_0$	IMPLEMENTED (FV as cell sum, SL inside w_n)	FV lines 114 to 120, 186; SL line 352
(3.7)	$\Phi_R = -4\pi R^2 D_5\, \partial_r c_5(R)$	IMPLEMENTED as $G_R C_{N_c}$	FV lines 202, 210
(3.8)	$dM/dt = -\Phi_R$	DERIVED NOT IMPLEMENTED (time-integrated form computed as residual, FV lines 219 to 220; asserted in VER lines 217 to 222)	none
(3.9)	$M_{\mathrm{rel}} = \int \Phi_R\, dt$, $F = M_{\mathrm{rel}}/M^0 = 1 - M/M^0$	IMPLEMENTED (FV first form, SL second form)	FV lines 211 to 217, 227; SL lines 381 to 393
(3.10)	Robin surface	FUTURE IMPROVEMENT (layered); root equation only, unit test	SL lines 31 to 79, 447 to 454
none	partition factor γ_ℓ, Gelbard (IV.3)	FUTURE IMPROVEMENT	none
none	$dF/dt = \Phi_R/M^0$	DERIVED NOT IMPLEMENTED	none

Eq.	Statement	Classification	Code (tag v1.0.0)
none	$c \geq 0$; $0 \leq 1 - F \leq (R/R_1)^{3/2} e^{-6 D_{\min} t / R^2}$, so $F \to 1$ (Section 3.5.5)	DERIVED NOT IMPLEMENTED (PROVED)	none; Code B counterpart Part VI, VER line 215

The script `scripts/derivation_checks/v1_0_0_p3_conditions.py` (not part of tag `v1.0.0`; see Appendix A) checks every derived formula of this part. Run it from the repository root with

```
.venv/Scripts/python.exe -I -X utf8 scripts/derivation_checks/v1_0_0_p3_conditions.py
```

It writes nothing and printed (detail brackets shortened):

```
PASS  P3.1 five layers, increasing radii, R = R_5 = 4.5e-4 m
PASS  P3.2 10 conditions needed (2 per layer), 1 centre + 8 interface + 1 outer supplied
PASS  P3.3 c = a + b r seen along a line: right slope b, left slope -b, equal only if b = 0
PASS  P3.4 |J_r(eps)| <= K eps / 3 -> 0 as eps -> 0
PASS  P3.5a sin(kr)/r e^{-Dk^2 t} solves Eq. (2.7); value k, slope 0, centre flow 0
PASS  P3.5b cos(kr)/r: centre flow -> 4 pi D (point source)  [limit 4*pi*D]
PASS  P3.5c B/r: steady solution, flow 4 pi D B for every r, finite amount 2 pi B eps^2
PASS  P3.6a int_0^R1 c0 4 pi r^2 dr + int_R1^R 0 dr = (4/3) pi R1^3 c0
PASS  P3.6b synthetic M^0 = 3.351032163829e-11 mol; FV telescoping sum equals closed form  [rel diff
↪  0.0e+00]
PASS  P3.7 V(R-d, R+d) = (4pi/3)(6R^2 d + 2 d^3) -> 0; A(R +- d) -> 4 pi R^2
PASS  P3.8 with Eq. (3.3) flow equal in both layers; with slope continuity it jumps when D_1 != D_2
PASS  P3.9a slowest synthetic eigenvalue from (phi, P) shooting matches the 40-digit reference
↪  [Lambda_1 = 3.273255671177117e-06 s^-1, rel diff 2.9e-14]
PASS  P3.9b global balance -dM/dt = J_R for the five-layer mode (interface terms cancel)  [rel diff
↪  4.4e-16, J_R > 0: True]
PASS  P3.10 one mode: dM/dt + J_R = 0 and M(0) - M(t) = int_0^t J_R dt'
PASS  P3.11 [4 pi R^2 D dc/dr] = mol/s, [G_R] = m^3/s, [G_R C] = mol/s, [M^0] = mol, [F] = 1
PASS  P3.12 Robin: c_R = c_inf - D g/h -> c_inf as h -> inf; times R/D gives -R g = Bi (c_R - c_inf)
PASS  P3.13 homogeneous sphere: F(Bi=1) < F(Bi=10) < F(Bi=100) < F(Dirichlet) at tau = 0.02 to 0.3
PASS  P3.14 Gelbard (IV.3): c_(l+1) = (gamma_l/gamma_(l+1)) c_l; gamma = 1 gives (3.2); gamma_IPyC = 1,
↪  gamma_SiC = 1/2 gives c_SiC = 2 c_IPyC
PASS  P3.15 slope ratio (dc/dr)_(l+1)/(dc/dr)_l = D_l/D_(l+1) at R_1..R_4 (synthetic)  [R_1: 0.0025,
↪  R_2: 500, R_3: 40, R_4: 0.02857]
PASS  P3.16 closed-form mode: Lambda_1 vs 40-digit reference, slope ratio D_l/D_(l+1) on the mode,
↪  -dM/dt = J_R by quadrature  [Lambda_1 = 3.273255671175929e-06 s^-1 (rel 3.3e-13), ratios 0.0025,
↪  500, 40, 0.0285714, balance rel 1.0e-13]
PASS  P3.17 Table 3.2 (tau = 0.1) reproduced by an independent finite-volume solve to 2e-5  [Bi=1.0:
↪  0.228635, Bi=10.0: 0.653988, Bi=100.0: 0.758821, Bi=inf: 0.770478]
PASS  P3.18a dZ/dt step: sum int c^- dP/dr dr = -int 4 pi r^2 D (dc^-/dr)^2 dr (interface and end terms
↪  cancel)  [both = -544*pi/15]
PASS  P3.18b c^2 <= (1/r - 1/R) Phi / D_min pointwise and E <= R^2/(6 D_min) 4 pi Phi (100 random
↪  profiles, 3 layers)  [largest ratios 0.058, 0.037]
PASS  P3.18c int_0^R r^2 (1/r - 1/R) dr = R^2/6
PASS  P3.18d 6 D_min/R^2 <= Lambda_1 (synthetic); homogeneous sphere, R_1 = R: 1 - F(tau) <= exp(-6 tau)
↪  for tau in [1e-4, 3]  [6 D_min/R^2 = 5.9259e-07 s^-1, bound on (R/R_1)^1.5 = 3.3750]
SUMMARY 25/25 PASS  ALL PASS
```

Part IV: Analytical solution

Part III gave the complete problem: the layer equation (2.7) in each of the five layers, the centre condition (3.1), the interface conditions (3.2) and (3.3), the absorbing surface (3.4) and the kernel-only start (3.5). Part IV solves this problem exactly, as a sum of simple shapes that fade at their own rates, and follows the solution all the way to the release fraction $F(t)$ that Code C computes. Every step is matched to the Rust source in src/analytical/sturm_liouville.rs at tag `v1.0.0`, and to its use in src/postprocessing/comparison.rs.

What Code C computes, in one paragraph. For a given set of layers, Code C finds every decay constant Λ_n up to a requested bound, builds the matching shape $u_n(r)$ layer by layer, evaluates three integrals of each shape by Gauss-Legendre quadrature,

combines them into one weight w_n per shape and returns $F(t) = 1 - \sum_n w_n e^{-\Lambda_n t}$. It never forms the concentration $c(r, t)$ itself, and it never calls the finite-volume solver (src/analytical/sturm_liouville.rs, lines 1 to 2). That independence is the reason the series is a useful check on Code B.

Symbols used only in Part IV. Tables 0.1 to 0.4 list the shared symbols (Λ, ϕ, u, q, k_ℓ, s, a_ℓ, b_ℓ, $\mathbf{T}_\ell$, $q^{\pm}$, g, θ_0, $\mathcal{N}$, N_n, $I_{1,n}$, I_n, A_n, w_n, $N_{\mathrm{p},\ell}$, x_m, ω_m, $\Lambda_{\max}$, Λ_{cov}, K, $K_{\max}$, Λ_{lo}, $t_{\min}$, t_{valid}, ρ, ε_F, ϵ_{m}, z). Table 4.1 lists the few extra symbols, which appear only inside one proof.

Table 4.1 Symbols local to Part IV (all others are in Tables 0.1 to 0.4).

Symbol	Meaning	Unit	Section
$\mathbf{Q}_\ell$	2 by 2 interface matrix at $r = R_\ell$ (a compact way to write Eq. (4.8); the code applies the scalar rule)	mixed	4.4.1
$Y(r)$	$r^2 D\, d\phi/dr$, continuous through the whole particle	arbitrary $\times$ m^3 s^{-1}	4.6
$\psi(r)$, $\Xi(r)$	Pruefer angle and amplitude, $\phi = \Xi \sin\psi$, $Y/Y_0 = \Xi\cos\psi$, with the reference flow $Y_0 = 1$ m^3 s^{-1}	rad; arbitrary	4.6
ν	an integer counting multiples of π	1	4.6
$\mathcal{B}_{mn}(r)$	boundary bracket $r^2 D(\phi_n\phi_m' - \phi_m\phi_n')$ in the orthogonality proof	mixed	4.8
σ	an arbitrary nonzero scale factor of an eigenfunction	1	4.9

Synthetic input used for every number in Part IV. Radii $R_1, \dots, R_5 = 200, 300, 350, 400, 450$ μm; diffusivities $D_1, \dots, D_5 = 1.0 \times 10^{-12}$, 4.0×10^{-10}, 8.0×10^{-13}, 2.0×10^{-14}, 7.0×10^{-13} m^2 s^{-1}; output every 2000 s up to 2×10^5 s (inputs/synthetic/synthetic_five_layer.toml). These are synthetic numbers chosen to test the code. They are not literature data for any TRISO particle or fission product.

Verification. Two scripts check every formula of this part, `v1_0_0_p4_separation.py` (41 PASS lines) and `v1_0_0_p4_sturm_weights.py` (24 PASS lines), and a third, `v1_0_0_eigenfunctions.py`, draws Figure 4.1. Appendix A lists them with the run command. The PASS lines are quoted where they support a statement, in the form "(check: `4.8 jump keeps D(u' - u/r) ...` PASS)".

4.1 Separation of variables

4.1.1 Key ideas in plain words

Shapes that only fade. Pluck a guitar string and it vibrates in a complicated way. Yet the motion is a sum of simple "modes": the fundamental, in which the whole string swings as one arch, and overtones with one or more resting points. Each mode keeps its shape and only its size changes in time. Diffusion has the same structure, except that the size of a diffusion mode only decays with time and never swings back. A cup of hot tea is a good picture. Soon after it is poured, the temperature inside the cup is uneven, but after a while the uneven parts have smoothed out and the whole cup cools with one fixed shape of temperature, losing the same fraction of its excess heat every minute. That last, slowest shape is the first mode; the faster ones have already died away.

Separation of variables. This is the mathematical form of "a shape times a fading factor". We guess that a solution can be written as a function of r alone multiplied by a function of t alone. If the guess works, one equation in two variables splits ("separates") into two simpler equations in one variable each.

Eigenfunction and eigenvalue. Only special shapes can fade without changing form, and each such shape fades at its own special rate. The shape is called an eigenfunction (German "eigen" means "own"), and the rate is the matching eigenvalue. In Part IV the eigenvalue Λ_n is a decay constant in s^{-1}: the mode shrinks by a factor e every $1/\Lambda_n$ seconds. The boundary conditions decide which shapes are allowed, in the same way as the fixed ends of a string decide which notes it can play.

4.1.2 Derivation

Physical problem. Solve the layer equation (2.7) with the conditions (3.1) to (3.5).

What we want. Solutions of the product form, and the equation that the shape must satisfy.

Governing principle. The layer equation (2.7), valid for $R_{\ell-1} < r < R_\ell$,

$$\frac{\partial c_\ell}{\partial t} = \frac{D_\ell}{r^2}\frac{\partial}{\partial r}\left(r^2 \frac{\partial c_\ell}{\partial r}\right).$$

Symbols. $c_\ell(r,t)$ concentration in layer ℓ (mol m^{-3}); r radius (m); t time (s); D_ℓ constant diffusivity of layer ℓ ($\mathrm{m}^2\ \mathrm{s}^{-1}$); $\phi(r)$ shape (arbitrary unit); $\vartheta(t)$ fading factor (dimensionless, used only in this derivation); Λ separation constant (s^{-1}).

Assumptions. A1 to A8 of Section 1.6; in particular D_ℓ is constant inside each layer and there is no source or decay.

Step 1 (guess the product form). Try $c_\ell(r,t) = \phi(r)\,\vartheta(t)$ in layer ℓ. Then $\partial c_\ell/\partial t = \phi\,\vartheta'$ and $\partial c_\ell/\partial r = \phi'\vartheta$, where a prime means the derivative with respect to the function's own variable. Substituting into Eq. (2.7):

$$\phi(r)\,\vartheta'(t) = \vartheta(t)\,\frac{D_\ell}{r^2}\frac{d}{dr}\left(r^2\frac{d\phi}{dr}\right).$$

Step 2 (divide by $\phi\vartheta$). This is allowed wherever $\phi \neq 0$ and $\vartheta \neq 0$:

$$\frac{\vartheta'(t)}{\vartheta(t)} = \frac{D_\ell}{r^2\phi(r)}\frac{d}{dr}\left(r^2\frac{d\phi}{dr}\right).$$

Step 3 (both sides are one constant). The left side depends only on t, the right side only on r. If we keep r fixed and change t, the right side cannot change, so the left side cannot change either; the same argument with t fixed shows the right side is constant. Both sides therefore equal one number, which we call $-\Lambda$ (the minus sign is chosen so that $\Lambda > 0$ means fading; Section 4.8 proves $\Lambda > 0$).

Step 4 (the time equation). $\vartheta' = -\Lambda\vartheta$. Its solution with $\vartheta(0) = 1$ is $\vartheta = e^{-\Lambda t}$, because $\frac{d}{dt}e^{-\Lambda t} = -\Lambda e^{-\Lambda t}$.

Step 5 (why the same Λ in every layer). Suppose layer ℓ had constant Λ_ℓ and layer $\ell+1$ had $\Lambda_{\ell+1}$. Condition (3.2) at $r = R_\ell$ must hold at every time: $\phi_\ell(R_\ell)e^{-\Lambda_\ell t} = \phi_{\ell+1}(R_\ell)e^{-\Lambda_{\ell+1}t}$ for all $t \geq 0$, and the same holds for the fluxes (3.3). Two exponentials with different rates can be equal at all times only if both coefficients are zero. Then $\phi_{\ell+1}$ and $D_{\ell+1}\phi'_{\ell+1}$ would both vanish at R_ℓ, and a second-order equation with zero value and zero slope at one point has only the zero solution. A nonzero mode therefore needs $\Lambda_\ell = \Lambda_{\ell+1}$, and by repeating the argument at every interface one Λ serves the whole particle.

Result. The product form and the shape equation are

$$c(r,t) = \phi(r)\, e^{-\Lambda t}, \tag{4.1}$$

$$\frac{1}{r^2}\frac{d}{dr}\left(r^2 D_\ell \frac{d\phi}{dr}\right) + \Lambda\phi = 0, \qquad R_{\ell-1} < r < R_\ell, \quad \ell = 1, \ldots, 5. \tag{4.2}$$

Eq. (4.2) follows from Step 2 with the right side set to $-\Lambda$ and multiplied by ϕ; D_ℓ was moved inside the derivative, which is allowed because it is constant in the layer. (check: `4.1-4.2 c = phi e^{-Lambda t} turns (2.7) into -e^{-Lambda t} x (4.2)` PASS.)

Classification. Eqs. (4.1) and (4.2) are DERIVED NOT IMPLEMENTED as a computation: no routine evaluates them, and Code C never forms $c(r,t)$. They are the premise of every Code C routine and are stated in the module documentation (src/analytical/sturm_liouville.rs, lines 12 to 19). Source label: adapted from published literature, Gelbard (2003) Eqs. (IV.4) and (IV.5), p. 28, whose separation constant is $-\beta_n^2$, so that $\Lambda_n = \beta_n^2$.

Physical meaning. A mode is a concentration profile that keeps its shape while every point loses the same fraction per second. Eq. (4.2) says which shapes can do that: at each radius, the net diffusive inflow per unit volume (the first term, Section 2.4) must equal exactly Λ times the local concentration.

4.2 Radial transformation

Physical problem. Eq. (4.2) contains the awkward combination $r^{-2}(r^2\phi')'$, which comes from the area $4\pi r^2$ of a sphere growing with r (Section 2.3).

What we want. A change of variable that turns Eq. (4.2) into the equation of a vibrating string, whose solutions are sines and cosines.

Governing principle. The quotient rule and the product rule of differentiation.

Symbols. $u(r) = r\phi(r)$, the transformed eigenfunction (arbitrary unit times m); k_ℓ the wavenumber in layer ℓ (m^{-1}).

Assumptions. $r > 0$ (the centre is treated in Section 4.5); D_ℓ constant in the layer.

Step 1 (write ϕ through u). From $u = r\phi$, divide by $r > 0$: $\phi = u/r$.

Step 2 (derivative of the quotient). By the quotient rule, $\frac{d}{dr}\frac{u}{r} = \frac{u'\,r - u\cdot 1}{r^2}$. Splitting the fraction:

$$\phi' = \frac{ru' - u}{r^2} = \frac{1}{r}\left(u' - \frac{u}{r}\right). \tag{4.3}$$

(check: `4.3 phi' = (u' - u/r)/r` PASS.)

Step 3 (multiply by r^2). $r^2\phi' = ru' - u$. (check: `4.3 step: r^2 phi' = r u' - u` PASS.)

Step 4 (differentiate once more). By the product rule, $\frac{d}{dr}(ru') = u' + ru''$, and $\frac{d}{dr}u = u'$. Subtracting:

$$\frac{d}{dr}\left(r^2\phi'\right) = u' + ru'' - u' = r\,u''.$$

The two u' terms cancel. This cancellation is the whole point of the substitution. (check: `4.3 step: (r^2 phi')' = r u''` PASS.)

Step 5 (divide by r^2). $\frac{1}{r^2}\frac{d}{dr}(r^2\phi') = \frac{u''}{r}$. (check: `4.4 (1/r^2)(r^2 phi')' = u''/r` PASS; the expanded form of Eq. (2.8) gives the same, check `4.4 expanded form phi'' + (2/r) phi' = u''/r` PASS.)

Step 6 (insert into Eq. (4.2)). With D_ℓ constant, (4.2) becomes $D_\ell\,u''/r + \Lambda\,u/r = 0$. Multiply by r/D_ℓ, which is positive:

$$u'' + k_\ell^2\,u = 0, \qquad k_\ell = \sqrt{\frac{\Lambda}{D_\ell}}, \qquad R_{\ell-1} < r < R_\ell. \tag{4.4}$$

(check: `4.4 (4.2) times r/D equals u'' + (Lambda/D) u` PASS.) Units: Λ/D_ℓ has $\mathrm{s}^{-1}/(\mathrm{m}^2\,\mathrm{s}^{-1}) = \mathrm{m}^{-2}$, so k_ℓ is in m^{-1} (check: `4.4 k = sqrt(Lambda/D) has units 1/m` PASS).

Classification. Eq. (4.3) is DERIVED NOT IMPLEMENTED as a computation (the code never stores ϕ or ϕ'); it enters the code only through the interface rule (4.8). Eq. (4.4) is IMPLEMENTED: the wavenumber is computed as `(lambda / l.diffusivity).sqrt()` in `shoot` (src/analytical/sturm_liouville.rs, line 141) and in `eigenvalue_count` (line 175), and the solution of Eq. (4.4) is used in Section 4.3. Source label: established textbook transformation for spherical diffusion (for example Crank, 1975, where it reduces the homogeneous sphere to a slab); adapted from Gelbard (2003) Eq. (IV.6), p. 28, for layers.

Physical meaning. In a sphere, the amount that leaves a thin shell is spread over a larger area further out. Multiplying by r undoes this geometric spreading, so u behaves like the displacement of a string, with k_ℓ telling how fast the shape wiggles in layer ℓ. A slow layer (small D_ℓ) has a large k_ℓ and therefore a short "wavelength" $2\pi/k_\ell$: the shape bends sharply there. This is visible in the SiC layer of Figure 4.1.

4.3 Solution in one layer and the transfer matrix

Physical problem. Solve Eq. (4.4) inside one layer and describe how the solution carries information from the inner face of the layer to its outer face.

What we want. A formula that takes the value of u and a flux-like quantity at the inner face and returns them at the outer face.

Governing principle. The general solution of $u'' = -k^2 u$ is a combination of $\cos kr$ and $\sin kr$, because each of these functions returns minus k^2 times itself when differentiated twice.

Symbols. $s = r - R_{\ell-1}$, local coordinate (m), so $0 \le s \le L_\ell$ with $L_\ell = R_\ell - R_{\ell-1}$; a_ℓ, b_ℓ layer coefficients (unit of u); $q = D\,du/dr$, the flux-like state the code carries (unit of u times m s^{-1}).

Assumptions. $\Lambda > 0$, so that k_ℓ is real and positive (Section 4.8).

Step 1 (general solution). In layer ℓ,

$$u(s) = a_\ell \cos(k_\ell s) + b_\ell \sin(k_\ell s), \qquad 0 \le s \le L_\ell. \tag{4.5}$$

Check by differentiating twice: $u'' = -k_\ell^2 a_\ell \cos(k_\ell s) - k_\ell^2 b_\ell \sin(k_\ell s) = -k_\ell^2 u$ (check: `4.5 a cos ks + b sin ks solves u'' + k^2 u = 0` PASS). The local coordinate s is used because it makes a_ℓ and b_ℓ easy to read off at the inner face.

Step 2 (coefficients from the inner-face state). At $s = 0$: $u(0) = a_\ell$ because $\cos 0 = 1$ and $\sin 0 = 0$. The slope is $u'(s) = -k_\ell a_\ell \sin(k_\ell s) + k_\ell b_\ell \cos(k_\ell s)$, so $u'(0) = k_\ell b_\ell$ and $q(0) = D_\ell k_\ell b_\ell$. Solving:

$$a_\ell = u_{\text{in}}, \qquad b_\ell = \frac{q_{\text{in}}}{D_\ell k_\ell}.$$

(check: `4.5 a = u_in and b = q_in/(D k) give u(0) = u_in, D u'(0) = q_in` PASS.)

Step 3 (evaluate at the outer face $s = L_\ell$). Substituting a_ℓ and b_ℓ into $u(L_\ell)$ and $q(L_\ell) = D_\ell u'(L_\ell)$:

$$\begin{aligned} u_{\text{out}} &= \cos(k_\ell L_\ell)\, u_{\text{in}} + \frac{\sin(k_\ell L_\ell)}{D_\ell k_\ell}\, q_{\text{in}}, \\ q_{\text{out}} &= D_\ell \left[-k_\ell a_\ell \sin(k_\ell L_\ell) + k_\ell b_\ell \cos(k_\ell L_\ell)\right] = -D_\ell k_\ell \sin(k_\ell L_\ell)\, u_{\text{in}} + \cos(k_\ell L_\ell)\, q_{\text{in}}. \end{aligned}$$

Step 4 (matrix form). Collecting the coefficients:

$$\begin{pmatrix} u \\ q \end{pmatrix}_{\text{out}} = \mathbf{T}_\ell \begin{pmatrix} u \\ q \end{pmatrix}_{\text{in}}, \qquad \mathbf{T}_\ell = \begin{pmatrix} \cos(k_\ell L_\ell) & \dfrac{\sin(k_\ell L_\ell)}{D_\ell k_\ell} \\ -D_\ell k_\ell \sin(k_\ell L_\ell) & \cos(k_\ell L_\ell) \end{pmatrix}. \tag{4.6}$$

(checks: `4.6 T row 1 equals u(L)` PASS, `4.6 T row 2 equals D u'(L)` PASS.) Two properties are worth knowing. The determinant is $\cos^2 + \sin^2 = 1$ (check: `4.6 det T = 1` PASS), so the step never loses information. And cutting a layer into two thinner pieces of the same material and applying the two matrices one after the other gives the same result as one matrix for the whole layer (check: `4.6 T(L1 + L2) = T(L2) T(L1) (split a layer)` PASS); the Rust unit test `transfer_preserves_state_`

`across_equal_material_subdivision` checks the same property (src/analytical/sturm_liouville.rs, lines 463 to 470).

Classification. Eq. (4.6) is IMPLEMENTED exactly as written in `transfer_layer` (src/analytical/sturm_liouville.rs, lines 86 to 92), with `u2 = c * u + s * du_flux / (d * k)` and `q2 = -d * k * s * u + c * du_flux`. Eq. (4.5) with Step 2 is IMPLEMENTED as the closure `u_at` in `ModalRelease::new` (src/analytical/sturm_liouville.rs, lines 339 to 344), where `a = st.u` and `b = st.q / (st.d * st.k)`. Source label: derived for STRATO (a standard transfer-matrix construction).

Why $q = Du'$ and not u'. Inside a layer either choice works. The code keeps the factor D in the state because the interface rule of Section 4.4 is simplest in terms of Du'. The doc comment of `transfer_layer` warns that the pair (u, Du') is valid only inside one material (lines 84 to 85).

Physical meaning. $\mathbf{T}_\ell$ is a rotation in disguise: if we measure q in units of $D_\ell k_\ell$, the pair $(u, q/(D_\ell k_\ell))$ turns by the angle $k_\ell L_\ell$ as we cross the layer. The phase picture of Section 4.6 is built on this observation.

4.4 Interface matching

Physical problem. At each interface $r = R_\ell$ the two neighbouring layers have different diffusivities. The layer solutions (4.5) must be joined there so that concentration and flux agree on both sides.

What we want. The matching rule for (u, q) at $r = R_\ell$, and the full chain of matchings through the four interfaces of the TRISO particle.

Governing principle. The interface conditions (3.2) and (3.3) of Part III: equal concentration and equal flux on the two sides (perfect contact, partition coefficient $\gamma_\ell = 1$).

Symbols. u_ℓ, $u_{\ell+1}$ the layer solutions on the inner and outer sides of R_ℓ; $q^- = D_\ell u'_\ell(R_\ell)$ just inside; $q^+ = D_{\ell+1} u'_{\ell+1}(R_\ell)$ just outside.

Assumptions. A1 to A8; perfect contact; $R_\ell > 0$ for every interface.

Step 1 (concentration). With Eq. (4.1), condition (3.2) reads $\phi_\ell(R_\ell)e^{-\Lambda t} = \phi_{\ell+1}(R_\ell)e^{-\Lambda t}$. Divide by $e^{-\Lambda t} > 0$, then multiply by R_ℓ (the same number on both sides):

$$u_\ell(R_\ell) = R_\ell\,\phi_\ell(R_\ell) = R_\ell\,\phi_{\ell+1}(R_\ell) = u_{\ell+1}(R_\ell).$$

So u is continuous. Call the common value u.

Step 2 (flux). Condition (3.3), after the same division by $e^{-\Lambda t}$, reads $D_\ell \phi'_\ell(R_\ell) = D_{\ell+1}\phi'_{\ell+1}(R_\ell)$. Replace each ϕ' by Eq. (4.3):

$$\frac{D_\ell}{R_\ell}\left(u'_\ell - \frac{u}{R_\ell}\right) = \frac{D_{\ell+1}}{R_\ell}\left(u'_{\ell+1} - \frac{u}{R_\ell}\right).$$

Step 3 (multiply by R_ℓ). This removes the common factor $1/R_\ell$ and gives the result

$$u \text{ continuous}, \qquad D\left(u' - \frac{u}{r}\right) \text{ continuous at every } r = R_\ell. \tag{4.7}$$

Step 4 (expand and use $q = Du'$). Multiplying out both sides of Eq. (4.7): $D_\ell u'_\ell - D_\ell u/R_\ell = D_{\ell+1}u'_{\ell+1} - D_{\ell+1}u/R_\ell$, that is $q^- - D_\ell u/R_\ell = q^+ - D_{\ell+1}u/R_\ell$.

Step 5 (solve for q^+). Add $D_{\ell+1}u/R_\ell$ to both sides:

$$q^+ = q^- + (D_{\ell+1} - D_\ell)\,\frac{u(R_\ell)}{R_\ell}. \tag{4.8}$$

(check: `4.8 jump keeps D(u' - u/r): q+ - D+ u/R = q- - D- u/R` PASS.)

Classification. Eq. (4.8) is IMPLEMENTED exactly in `interface_state` (src/analytical/sturm_liouville.rs, lines 94 to 100), `q_inner + (d_outer - d_inner) * u / radius`, called before every layer except the first in `shoot` (lines 138 to 140) and in `eigenvalue_count` (lines 172 to 174). The variable `left` passed as `radius` holds R_ℓ, the outer radius of the previous layer. Eq. (4.7) is IMPLEMENTED through Eq. (4.8); the unit test `interface_state_conserves_flux_and_reduces_to_identity` checks it (lines 472 to 482). Source label: adapted from published literature, Gelbard (2003) Eqs. (IV.2) and (IV.3), p. 27, with $\gamma_\ell = 1$.

Why Du' is not continuous. The physical flux in the u language is $D(u' - u/r)$, a difference of two terms. The term Du/r jumps by $(D_{\ell+1} - D_\ell)u/R_\ell$ at an interface because D jumps while u and r do not. For the difference to stay continuous, Du' must jump by the same amount. Copying (u, Du') unchanged across an interface would give wrong eigenvalues and wrong release curves; the Phase 1 document stressed this point, and it is kept here.

Physical meaning of the matched quantity. From Eq. (4.3), $D(u' - u/r) = rD\phi'$ (check: `4.7 D(u' - u/r) = r D phi'` PASS). With $J_r = -D\,\partial c/\partial r = -D\phi' e^{-\Lambda t}$ (Eq. (1.4)), this is $-rJ_r e^{\Lambda t}$: the radius times the outward flux, up to the fading factor. Condition (4.7) is therefore flux continuity written in the new variable.

4.4.1 Worked chain through all four interfaces

The interface rule is a 2 by 2 matrix with determinant 1, $\mathbf{Q}_\ell = \begin{pmatrix} 1 & 0 \\ (D_{\ell+1} - D_\ell)/R_\ell & 1 \end{pmatrix}$, which becomes the identity when $D_{\ell+1} = D_\ell$ (check: `4.8 interface matrix Q has det 1 and reduces to I when D+ = D-` PASS). Starting from the centre state of Section 4.5, $(u, q) = (0, 1)$, the code walks outward as follows.

1. **Kernel**, $0 < r < R_1$: $(u, q)(R_1^-) = \mathbf{T}_1(0, 1)^\mathsf{T}$, so $u(R_1) = \sin(k_1 R_1)/(D_1 k_1)$ and $q^-(R_1) = \cos(k_1 R_1)$.
2. **Kernel/Buffer interface**, $r = R_1$: u unchanged; $q^+(R_1) = q^-(R_1) + (D_2 - D_1)\,u(R_1)/R_1$.
3. **Buffer**, $R_1 < r < R_2$: apply $\mathbf{T}_2$ to $(u(R_1), q^+(R_1))$.
4. **Buffer/IPyC interface**, $r = R_2$: $q^+(R_2) = q^-(R_2) + (D_3 - D_2)\,u(R_2)/R_2$.
5. **IPyC**, $R_2 < r < R_3$: apply $\mathbf{T}_3$.

6. **IPyC/SiC interface**, $r = R_3$: $q^+(R_3) = q^-(R_3) + (D_4 - D_3)\,u(R_3)/R_3$.
7. **SiC**, $R_3 < r < R_4$: apply $\mathbf{T}_4$.
8. **SiC/OPyC interface**, $r = R_4$: $q^+(R_4) = q^-(R_4) + (D_5 - D_4)\,u(R_4)/R_4$.
9. **OPyC**, $R_4 < r < R_5 = R$: apply $\mathbf{T}_5$; the first component is $u(R)$.

In matrix language the whole walk is one product, read from right to left:

$$\begin{pmatrix} u \\ q \end{pmatrix}(R) = \mathbf{T}_5\,\mathbf{Q}_4\,\mathbf{T}_4\,\mathbf{Q}_3\,\mathbf{T}_3\,\mathbf{Q}_2\,\mathbf{T}_2\,\mathbf{Q}_1\,\mathbf{T}_1 \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Table 4.2 gives the numbers for the first mode of the synthetic case at the scale the code uses, $q(0) = 1$. They were computed by the independent 40-digit implementation and agree with a float64 replica of `shoot` to 10^{-9} relative (checks: `4.7-4.8 mode 1: at all four interfaces u is continuous, q+ = q- + (D+ - D-) u/R, D(u'-u/r) continuous` PASS; `4.6-4.9 float64 replica of shoot (transfer_layer + interface_state) reproduces the chain to rel 1e-9` PASS).

Table 4.2 Interface chain for mode 1 of the synthetic case, $\Lambda_1 = 3.2732556711770 \times 10^{-6}$ s^{-1}, scale $q(0) = 1$ (calculated; u and q carry the arbitrary scale of Section 4.5). The last column is the flux-cancellation ratio of Section 4.13.

Interface	R_ℓ (m)	$u(R_\ell)$	q^-	q^+	$D(u' - u/r)$ inner side	$D(u' - u/r)$ outer side	ρ (outer side)
Kernel/Buffer	2.00×10^{-4}	1.9566414×10^{8}	0.93524606	391.28521	−0.04307465	−0.04307465	5.504×10^{-5}
Buffer/IPyC	3.00×10^{-4}	2.934761×10^{8}	391.20515	0.686285	−0.096317945	−0.096317945	0.06557
IPyC/SiC	3.50×10^{-4}	3.3479614×10^{8}	0.63482874	−0.11128836	−0.13041957	−0.13041957	1.0
SiC/OPyC	4.00×10^{-4}	8.9751451×10^{6}	−0.14041976	−0.12516202	−0.14086852	−0.14086852	1.0

At the Kernel/Buffer interface q jumps by a factor of about 400, because the buffer diffusivity is 400 times the kernel diffusivity, while the matched flux $D(u' - u/r)$ is the same on both sides to all printed digits. At that interface $q^+ = 391.28521$ and $D_2 u/R_1 = 391.32828$ nearly cancel and leave a flux of only -0.0431; this is the cancellation measured by ρ in Section 4.13. The matched flux is negative everywhere, which means ϕ decreases outward and material flows outward, as expected for the slowest mode of a particle that starts full in the centre. At the end of the chain the independent implementation gives $|u(R)|$ of order 10^{-30} on a scale of 3×10^8, which is zero to its 40-digit working precision.

4.5 Centre start and characteristic function

Physical problem. Section 4.4 tells us how to continue the solution outward once we know (u, q) somewhere. We still need a starting state at the centre and a test at the surface.

What we want. The state at $r = 0$, and one equation whose roots are the eigenvalues.

Governing principle. The centre condition (3.1) (bounded concentration, zero slope by symmetry) and the absorbing surface (3.4).

Step 1 ($u(0) = 0$). The concentration is bounded at the centre, so $u(0) = \lim_{r \to 0} r\,\phi(r) = 0 \cdot \phi(0) = 0$.

Step 2 (the kernel solution). In the kernel (4.5) with $s = r$ gives $u = a_1 \cos(k_1 r) + b_1 \sin(k_1 r)$. Step 1 forces $a_1 = 0$. The other choice would give $\phi = \cos(k_1 r)/r$, which grows without bound as $r \to 0$ (check: `4.9 cos branch: cos(k_1 r)/r unbounded at r = 0` PASS). So $\phi = b_1 \sin(k_1 r)/r$, whose limit at the centre is $b_1 k_1$, a finite number (check: `4.9 sin branch: phi(0) = b k_1 (finite)` PASS), and whose slope at the centre is 0, so the symmetry part of Eq. (3.1) holds automatically (check: `4.9 sin branch: phi'(0) = 0 (symmetry, (3.1))` PASS).

Step 3 (the free scale). Eq. (4.2) and all the conditions are linear and homogeneous: if ϕ is a solution, so is $\sigma\phi$ for any number $\sigma \neq 0$. The size of an eigenfunction is therefore free, and the code fixes it by choosing $q(0) = D_1 u'(0) = D_1 k_1 b_1 = 1$:

$$(u, q)\,(r = 0) = (0, 1). \tag{4.9}$$

Section 4.9 shows that this choice cancels from every result the code reports.

Step 4 (the surface test). Condition (3.4) gives $c(R, t) = 0$, so $\phi(R) = 0$ and $u(R) = R\,\phi(R) = 0$. Define the characteristic function as the value of u that arrives at the surface when the chain of Section 4.4.1 is run with a trial value of Λ:

$$g(\Lambda) = u(R; \Lambda), \qquad \text{eigenvalues: } g(\Lambda_n) = 0. \tag{4.10}$$

Shooting, in words. Think of firing a ball from the centre with a fixed launch slope. The trial value Λ sets how strongly the path bends in each layer. For most values of Λ the path does not land exactly at $u = 0$ on the surface; the special values that do are the eigenvalues. This is why the method is called shooting.

Classification. Eq. (4.9) is IMPLEMENTED at the start of `shoot` (src/analytical/sturm_liouville.rs, line 134) and of `eigenvalue_count` (line 168). Eq. (4.10) is IMPLEMENTED as the first return value of `shoot` (lines 129 to 162, return at line 161), with one qualification that the reader should know: the only call of `shoot` in v1.0.0 is in `ModalRelease::new` (line 329), which discards g (`let (_, starts) = ...`) and keeps only the layer start states. The eigenvalues are located with the Sturm count of Section 4.6, never by testing the sign of g. The `rescale` branch of `shoot` (lines 151 to 157) is not reached in v1.0.0 because `shoot` is always called with `rescale = false`. Source label: derived for STRATO. Gelbard (2003, p. 33) finds the same eigenvalues as zeros of a $(2L-1) \times (2L-1)$ determinant (Section 4.16).

Physical meaning. $g(\Lambda) = 0$ says: the shape that grows naturally out of a regular centre, bent layer by layer at the rate Λ, reaches zero concentration exactly at the surface, as the perfect sink requires.

4.6 Eigenvalue concepts and the Sturm count

Physical problem. A root finder that looks for sign changes of g can step over two roots that lie close together. Code C avoids this with a whole-number count.

What we want. A function $\mathcal{N}(\Lambda)$ that returns, exactly, how many eigenvalues are less than or equal to Λ; and the formula the code uses to compute it.

4.6.1 Concept: counting zeros

Look at Figure 4.1 (a). The first mode has no zero inside the particle, the second has one, the third has two. This is the Sturm oscillation property: the n-th eigenfunction crosses zero exactly $n-1$ times between the centre and the surface. As Λ grows, every k_ℓ grows, the shape wiggles faster and its zeros move toward the centre; a new zero enters through the surface exactly when $u(R)=0$, that is when Λ passes an eigenvalue. So counting the zeros of the shot solution tells us how many eigenvalues lie below the trial value.

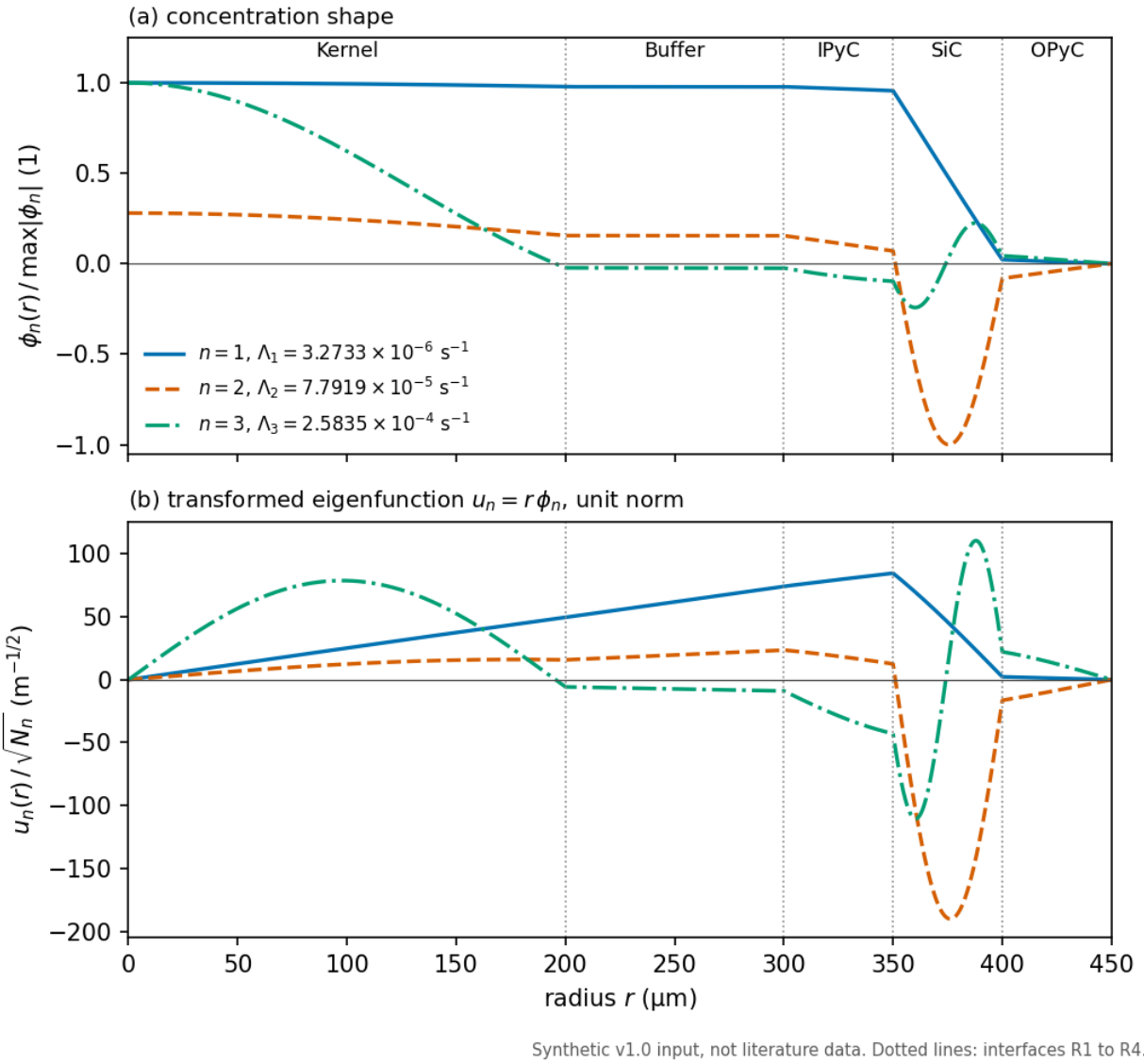


Figure 4.1: First three eigenfunctions of the synthetic five-layer case. (a) Concentration shape $\phi_n = u_n/r$ scaled to a maximum of 1; (b) $u_n/\sqrt{N_n}$, unit norm in m$^{-1/2}$. Dotted lines mark the interfaces R_1 to R_4. Mode n has $n-1$ zeros inside the particle. The shapes are nearly flat in the fast buffer and bend sharply in the slow SiC layer. Calculated by the script scripts/figures/v1_0_0_eigenfunctions.py; synthetic input, not literature data.

4.6.2 Step-by-step derivation of the count in one layer

Step 1 (amplitude-phase form). Inside layer ℓ write Eq. (4.5) as one shifted sine. Put $H_\ell = \sqrt{a_\ell^2 + b_\ell^2}$ and choose the angle θ_0 with $\sin\theta_0 = a_\ell/H_\ell$ and $\cos\theta_0 = b_\ell/H_\ell$.

By the addition formula $\sin(x+y) = \sin x \cos y + \cos x \sin y$,

$$H_\ell \sin(k_\ell s + \theta_0) = H_\ell \sin(k_\ell s)\frac{b_\ell}{H_\ell} + H_\ell \cos(k_\ell s)\frac{a_\ell}{H_\ell} = a_\ell \cos(k_\ell s) + b_\ell \sin(k_\ell s).$$

The angle with a given sine and cosine is the two-argument arctangent, so

$$u(s) = H_\ell \sin(k_\ell s + \theta_0), \qquad \theta_0 = \operatorname{atan2}(a_\ell, b_\ell), \qquad a_\ell = u_{\text{in}}, \quad b_\ell = \frac{q_{\text{in}}}{D_\ell k_\ell}. \tag{4.11}$$

(checks: `4.11 A sin(ks + theta0) = a cos ks + b sin ks ...` PASS; `4.11 atan2(a, b) gives the phase (2000 random a, b, ks)` PASS.) The function atan2 returns θ_0 in $(-\pi, \pi]$.

Step 2 (where the zeros are). $H_\ell \sin(k_\ell s + \theta_0) = 0$ exactly when $k_\ell s + \theta_0 = \nu\pi$ for an integer ν (assuming $H_\ell > 0$). A zero lies in the layer, $0 < s \le L_\ell$, exactly when $\theta_0 < \nu\pi \le \theta_0 + k_\ell L_\ell$.

Step 3 (count the integers). For real numbers $x < y$ the number of integers ν with $x < \nu \le y$ is $\lfloor y \rfloor - \lfloor x \rfloor$, where $\lfloor \cdot \rfloor$ rounds down. For example, with $x = 0.3$ and $y = 2.7$ the integers are 1 and 2, and indeed $\lfloor 2.7 \rfloor - \lfloor 0.3 \rfloor = 2 - 0 = 2$; with $x = -0.5$ and $y = 1.0$ they are 0 and 1, and $1 - (-1) = 2$. With $x = \theta_0/\pi$ and $y = (\theta_0 + k_\ell L_\ell)/\pi$:

$$\text{zeros of } u \text{ in } 0 < s \le L_\ell = \left\lfloor \frac{\theta_0 + k_\ell L_\ell}{\pi} \right\rfloor - \left\lfloor \frac{\theta_0}{\pi} \right\rfloor. \tag{4.12}$$

(check against a brute-force sign count: `4.12 floor((theta0 + kL)/pi) - floor(theta0/pi) = zeros of u in (0, L] (2000 random layers)` PASS.) A zero exactly at an interface is counted once, in the layer that ends there, because each layer counts $(0, L_\ell]$. The zero of u at the centre is never counted, because the kernel starts with $\theta_0 = \operatorname{atan2}(0, 1/(D_1 k_1)) = 0$ and the interval excludes $s = 0$.

Step 4 (sum over layers). Add Eq. (4.12) over the five layers, with the interface rule (4.8) applied between layers. Because $r > 0$ in every layer, the zeros of u and of $\phi = u/r$ are the same points. The total is the number of zeros of the shot solution in $(0, R]$, and the claim of Code C is

$$\mathcal{N}(\Lambda) = \sum_{\ell=1}^{5} \left(\left\lfloor \frac{\theta_{0,\ell} + k_\ell L_\ell}{\pi} \right\rfloor - \left\lfloor \frac{\theta_{0,\ell}}{\pi} \right\rfloor \right) = \#\{n : \Lambda_n \le \Lambda\}. \tag{4.13}$$

4.6.3 Why the zero count equals the eigenvalue count

This is the Sturm oscillation theorem, an established result of Sturm-Liouville theory. No textbook section has been verified for this document, so we give a self-contained sketch for the layered sphere and back it with numerical checks.

Step A (a pair that is continuous everywhere). Let $Y(r) = r^2 D\,\phi'(r)$. Condition (3.2) makes ϕ continuous and (3.3) makes $D\phi'$, hence Y, continuous at every interface.

Multiplying Eq. (4.2) by r^2 gives $Y' = -\Lambda r^2 \phi$, and the definition of Y gives $\phi' = Y/(r^2 D)$. So (ϕ, Y) obeys one first-order system on the whole interval $0 < r < R$, with coefficients that are only piecewise continuous (they jump where D jumps, and both ϕ and Y stay continuous there).

Step B (Pruefer angle). The pair (ϕ, Y) has two different units, so introduce the reference flow $Y_0 = 1\ \mathrm{m}^3\ \mathrm{s}^{-1}$ and write $\phi = \Xi \sin\psi$ and $Y/Y_0 = \Xi \cos\psi$ with $\Xi > 0$. Both components now carry the unit of ϕ. From $\tan\psi = \phi Y_0 / Y$, the derivative of the angle is

$$\psi' = \frac{\phi' Y - \phi Y'}{Y_0\, \Xi^2} = \frac{Y_0 \cos^2\psi}{r^2 D} + \frac{\Lambda r^2 \sin^2\psi}{Y_0} > 0.$$

Both terms have the unit m^{-1}, as the derivative of an angle with respect to r must: the first is $\mathrm{m}^3\ \mathrm{s}^{-1}/(\mathrm{m}^2 \cdot \mathrm{m}^2\ \mathrm{s}^{-1})$ and the second is $\mathrm{s}^{-1}\ \mathrm{m}^2/(\mathrm{m}^3\ \mathrm{s}^{-1})$. The two terms are never zero together (for $r > 0$ and $\Lambda > 0$, $\cos^2\psi$ and $\sin^2\psi$ cannot both vanish), so ψ increases strictly with r. At the centre $Y \to 0$ and $\phi \to \phi(0) = b_1 k_1 = 1/D_1 > 0$ at the code's scale (Section 4.5), so $\psi(0^+) = \pi/2$ for every Λ. The zeros of ϕ are the radii where ψ passes a multiple of π, and ψ passes each one only once, upward. Hence the number of zeros in $(0, r]$ is $\lfloor \psi(r)/\pi \rfloor$.

Step C (monotone in Λ). The right side of the ψ' equation increases with Λ (strictly where $\sin\psi \neq 0$) and the starting value $\pi/2$ does not depend on Λ. The comparison theorem for first-order equations then gives that $\psi(R;\Lambda)$ increases strictly and continuously with Λ; as $\Lambda \to 0^+$ the shape tends to a constant and $\psi(R) \to \pi/2$.

Step D (conclusion). Λ is an eigenvalue exactly when $\phi(R) = 0$, that is $\psi(R;\Lambda) = n\pi$ for an integer $n \geq 1$. Because $\psi(R;\Lambda)$ rises strictly from $\pi/2$, each n gives exactly one eigenvalue Λ_n, with $\Lambda_1 < \Lambda_2 < \ldots$, and $\lfloor \psi(R;\Lambda)/\pi \rfloor$ is the number of n with $\Lambda_n \leq \Lambda$. The n-th eigenfunction has $n-1$ zeros inside the particle plus the zero at R. The comparison theorem in Step C is quoted, not proved; this is why Section 7.5 lists a full proof for the layered case as a future improvement. Status: the equality $\mathcal{N}(\Lambda) = \#\{n : \Lambda_n \leq \Lambda\}$ is PROVED in exact arithmetic, with the comparison theorem quoted, and DEMONSTRATED NUMERICALLY for the synthetic case below.

Numerical confirmation. For the synthetic case the independent implementation found 95 eigenvalues up to $0.2\ \mathrm{s}^{-1}$ (check: `mode n has exactly n - 1 interior zeros, n = 1..95 (no root skipped by the scan)` PASS). A float64 replica of `eigenvalue_count` steps from $n-1$ to n across every one of them (check: `4.13 code count N(Lambda) steps from n-1 to n across each of 95 eigenvalues` PASS) and equals the number of independent eigenvalues $\leq \Lambda$ at 300 random values of Λ (check: `4.13 N(Lambda) = #{Lambda_n <= Lambda} at 300 random Lambda in [1e-7, 0.2]` PASS).

Classification. Eqs. (4.11) to (4.13) are IMPLEMENTED in `eigenvalue_count` (src/analytical/sturm_liouville.rs, lines 164 to 192): a, b at lines 176 to 177, `theta0 = a.atan2(b)` at line 179, the floor difference at line 181. Two details of the code: after every layer the state is divided by $m = \max(|u|, |q/(Dk)|) > 0$ (lines 183 to 187), which changes neither signs nor zeros and prevents overflow; and the final `n.max(0)`

(line 191) is only a guard, since every floor difference in Eq. (4.12) is already ≥ 0. The equality with $\#\{n : \Lambda_n \leq \Lambda\}$ is the established Sturm result above, checked numerically and in the unit test `sturm_count_matches_root_count_and_is_monotone` (lines 484 to 517). The code counts "less than or equal", as its doc comment says (lines 164 to 165); the Phase 1 document said "below".

4.7 Root isolation

Physical problem. Find every eigenvalue up to an upper bound, each to near machine precision, without skipping any.

What we want. The bracketing and bisection rules, with their stopping tests, exactly as coded in `eigenvalues` (src/analytical/sturm_liouville.rs, lines 194 to 264).

Concept: bisection on a count. If $\mathcal{N}(a) = 3$ and $\mathcal{N}(b) = 5$, then exactly two eigenvalues lie in $a < \Lambda \leq b$, because $\mathcal{N}$ counts eigenvalues $\leq \Lambda$. Evaluate $\mathcal{N}$ at a point m between them; the numbers $\mathcal{N}(m) - \mathcal{N}(a)$ and $\mathcal{N}(b) - \mathcal{N}(m)$ say how many lie in each half. Halves that contain none are dropped; halves that contain one are shrunk until they are tiny. Since the counts of the two halves always add up to the count of the whole, no eigenvalue can be lost.

The code first fixes the upper end of the search (Step 3 below, lines 202 to 217), then the lower end (Step 1), then bisects (Step 2). The steps are presented from the bottom up because the lower bracket is the simplest.

Step 1 (lower bracket). The code needs a value Λ_{lo} with no eigenvalue below it. If u behaved like a pure sine through all layers, the total phase gained from centre to surface would be $\sum_\ell k_\ell L_\ell = \sqrt{\Lambda} \sum_\ell L_\ell / \sqrt{D_\ell}$. The code makes this phase 10^{-3} rad, far below π:

$$\Lambda_{\text{lo}} = 10^{-6} \left(\sum_{\ell=1}^{5} \frac{L_\ell}{\sqrt{D_\ell}} \right)^{-2}, \quad \text{then } \Lambda_{\text{lo}} \leftarrow 0.01\, \Lambda_{\text{lo}} \text{ until } \mathcal{N}(\Lambda_{\text{lo}}) = 0. \tag{4.14}$$

The phase estimate ignores the interface jumps, so it is a heuristic; the loop makes it safe, and the code stops with an error if Λ_{lo} falls below 10^{-300}. IMPLEMENTED: src/analytical/sturm_liouville.rs, lines 220 to 238. Synthetic case: $\sum_\ell L_\ell/\sqrt{D_\ell} = 674.217\ \text{s}^{1/2}$, $\Lambda_{\text{lo}} = 2.1999 \times 10^{-12}\ \text{s}^{-1}$, six decades below Λ_1, and $\mathcal{N}(\Lambda_{\text{lo}}) = 0$ on the first try (check: `4.14 Lambda_lo = 1e-6 (sum L/sqrt(D))^-2 has N = 0 and lies below Lambda_1` PASS).

Step 2 (bisection on the count). The code keeps a stack of intervals (a, b, n_a, n_b), starting with $(\Lambda_{\text{lo}}, \Lambda_{\text{cov}}, 0, \mathcal{N}(\Lambda_{\text{cov}}))$. For each interval popped from the stack: if $n_b \leq n_a$ it holds no eigenvalue and is dropped; otherwise it is split at

$$m = \begin{cases} \sqrt{ab}, & b > 4a, \\ (a+b)/2, & b \leq 4a, \end{cases} \qquad \text{stop when } b - a \leq 4 \times 10^{-15}\, b. \tag{4.15}$$

The geometric midpoint $\sqrt{ab}$ halves the interval on a logarithmic scale, which matters at the start when the interval spans ten decades. The count $n_m = \mathcal{N}(m)$ is clamped to

$[n_a, n_b]$, a guard against rounding at the edges of the floor function, and both halves are pushed back. When the stopping test holds: if $n_b - n_a = 1$ the eigenvalue is recorded as $(a+b)/2$, whose distance from the true root is at most $(b-a)/2 \leq 2\times10^{-15}b$; if $n_b - n_a > 1$ the code stops with the error "unresolved eigenvalue cluster". Finally the roots are sorted. IMPLEMENTED: src/analytical/sturm_liouville.rs, lines 240 to 263.

Step 3 (upper end and mode cap). The requested bound Λ_{max} comes from Section 4.12. If $\mathcal{N}(\Lambda_{\text{max}}) \leq K_{\text{max}} = 20\,000$ the code covers it in full. Otherwise it searches geometrically between $\text{lo} = 10^{-12}\Lambda_{\text{max}}$ and $\text{hi} = \Lambda_{\text{max}}$ with $m = \sqrt{\text{lo}\cdot\text{hi}}$, moves hi to m where the count exceeds K_{max} and lo to m where it does not, stops after 200 steps or when $\text{hi} - \text{lo} \leq 10^{-12}\text{hi}$, and keeps lo:

$$\Lambda_{\text{cov}} = \begin{cases} \Lambda_{\text{max}}, & \mathcal{N}(\Lambda_{\text{max}}) \leq K_{\text{max}}, \\ \text{lo of the geometric search, so that } \mathcal{N}(\Lambda_{\text{cov}}) \leq K_{\text{max}}, & \text{otherwise.} \end{cases} \tag{4.16}$$

IMPLEMENTED: src/analytical/sturm_liouville.rs, lines 202 to 217; K_{max} is `MAX_MODES` in src/postprocessing/comparison.rs, line 13. The code does not test $\mathcal{N}(10^{-12}\Lambda_{\text{max}}) \leq K_{\text{max}}$ before the search; for any realistic input that count is small. For the synthetic main run $\mathcal{N}(0.0225\ \text{s}^{-1}) = 32$, so the cap is not reached and $\Lambda_{\text{cov}} = \Lambda_{\text{max}}$.

Why nothing is skipped. Every split satisfies $(n_m - n_a) + (n_b - n_m) = n_b - n_a$, so the counts of all final intervals add up to $\mathcal{N}(\Lambda_{\text{cov}})$, and each final interval yields exactly one root. Provided $\mathcal{N}$ is evaluated correctly, the code returns all $\mathcal{N}(\Lambda_{\text{cov}})$ eigenvalues, each once. Status: PROVED for an exactly evaluated count. In double precision the count is evaluated with rounding, so for the code as run the property is DEMONSTRATED NUMERICALLY: for the synthetic case the Rust run keeps 32 modes (`analytical_modes=32` in `tests/golden/v1.0/main_run_stdout.txt`), the same number as the independent count of eigenvalues up to $0.0225\ \text{s}^{-1}$, and the first six agree one to one (Table 4.3).

Accuracy reached. The bisection tolerance is 2×10^{-15} relative, but the count itself is evaluated in double precision and inherits the cancellation of Section 4.13. Against the independent 40-digit values the Rust eigenvalues of the synthetic case agree to 3.7×10^{-13} relative for Λ_1 and 1.2×10^{-15} for Λ_2 (checks: `Lambda_1 = 3.273255671175805e-6 s^-1 (Rust) within rel 1e-12 (independent 3.273255671177022e-6, rel diff 3.7e-13) PASS`; `Lambda_2 = 7.791893309400181e-5 s^-1 (Rust) within rel 1e-12 (independent 7.79189330940019e-5, rel diff 1.2e-15) PASS`). The larger error of Λ_1 fits the small flux ratio $\rho = 5.5\times10^{-5}$ of mode 1 at the Kernel/Buffer interface (Table 4.2): $\epsilon_{\text{m}}/\rho \approx 4\times10^{-12}$.

Bisection tolerance and actual accuracy. Near Λ_1 the double-precision count is not a clean step from 0 to 1. A float64 transcription of `eigenvalue_count` changes its value five times, instead of once, between $\Lambda_1(1-2\times10^{-12})$ and $\Lambda_1(1+2\times10^{-12})$: rounding in the cancelling flux decides the floor in Eq. (4.12) inside that narrow band. The bisection then settles somewhere inside the band, and where exactly depends on the bracket it started from. The same transcription reproduces the Rust main-run value $3.2732556711758045\times10^{-6}\ \text{s}^{-1}$ (bracket up to $0.0225\ \text{s}^{-1}$) and returns

$3.2732556711761027\times10^{-6}$ s^{-1} for the refinement-study bracket up to 2.25×10^{-4} s^{-1}; both lie within 4×10^{-13} relative of the independent value. The clamp of n_m to $[n_a, n_b]$ keeps the root count right in such a band. Status: the eigenvalue accuracy of about $\epsilon_{\mathrm{m}}/\rho_{\min}$ is DEMONSTRATED NUMERICALLY for this case only, without a proof.

Table 4.3 First six eigenvalues and weights of the synthetic case (calculated). Rust values from a one-off print of the 32 modes of the v1.0.0 library (not stored in the repository, Section 7.5), reproduced to all quoted digits by a float64 transcription of the Rust routines; independent values from `v1_0_0_p4_sturm_weights.py`, computed for the inputs as stored in binary64, which is how Rust reads the TOML file. With the exact decimal inputs the independent eigenvalues move by less than 10^{-15} relative ($\Lambda_1 = 3.2732556711770232\times10^{-6}$).

n	Λ_n, Rust v1.0.0 (s^{-1})	Λ_n, independent 40-digit (s^{-1})	w_n (independent)
1	$3.2732556711758 05\times10^{-6}$	$3.2732556711770218\times10^{-6}$	1.092030273751
2	$7.7918933094001 81\times10^{-5}$	$7.7918933094001902\times10^{-5}$	−0.1423582853238
3	$2.5834695316556 20\times10^{-4}$	$2.5834695316556170\times10^{-4}$	0.2269692966236
4	$3.0316036088526 74\times10^{-4}$	$3.0316036088526755\times10^{-4}$	−0.1826220508228
5	$5.8333747203909 83\times10^{-4}$	$5.8333747203909809\times10^{-4}$	−0.02170242475855
6	$8.0883270985862 79\times10^{-4}$	$8.0883270985862859\times10^{-4}$	0.1112726747645

4.8 Orthogonality

4.8.1 Concept: perpendicular shapes

Two arrows in the plane are perpendicular when their dot product is zero: multiply matching components and add. A function can be thought of as an arrow with very many components, one value per radius. The dot product of two functions becomes an integral of their product, and the "weight" in the integral says how much each radius counts. For a sphere the natural weight is r^2, because a shell at radius r has volume $4\pi r^2\,dr$. Two eigenfunctions that are orthogonal in this sense do not overlap, and that is what allows a starting profile to be split cleanly into its modes (Section 4.9).

4.8.2 Proof

Physical problem. Show that eigenfunctions of different eigenvalues are orthogonal with weight r^2 for ϕ, that is weight 1 for u.

Governing principle. The self-adjoint form of Eq. (4.2), integration over each layer, and the interface conditions.

Symbols. (Λ_m, ϕ_m), (Λ_n, ϕ_n) two eigenpairs with $\Lambda_m \neq \Lambda_n$; $\varpi(r) = r^2 D(r)$ only as shorthand inside this proof.

Step 1 (self-adjoint form). Multiply Eq. (4.2) by r^2: $(r^2 D_\ell \phi')' + \Lambda r^2 \phi = 0$. For the two eigenpairs,

$$(r^2 D\phi_m')' = -\Lambda_m r^2 \phi_m, \qquad (r^2 D\phi_n')' = -\Lambda_n r^2 \phi_n.$$

Step 2 (cross-multiply and subtract). Multiply the first by ϕ_n, the second by ϕ_m, and subtract:

$$\phi_n (r^2 D\phi_m')' - \phi_m (r^2 D\phi_n')' = (\Lambda_n - \Lambda_m)\, r^2 \phi_m \phi_n.$$

Step 3 (the left side is a derivative). Define $\mathcal{B}_{mn}(r) = r^2 D\,(\phi_n\phi_m' - \phi_m\phi_n')$. Differentiating with the product rule, $\mathcal{B}_{mn}' = \phi_n' r^2 D\phi_m' + \phi_n (r^2 D\phi_m')' - \phi_m' r^2 D\phi_n' - \phi_m (r^2 D\phi_n')'$. The first and third terms cancel, so the left side of Step 2 equals $\mathcal{B}_{mn}'$ (check: `4.17 Lagrange identity ...` PASS).

Step 4 (integrate over one layer). Inside layer ℓ everything is smooth, so

$$\mathcal{B}_{mn}(R_\ell^-) - \mathcal{B}_{mn}(R_{\ell-1}^+) = (\Lambda_n - \Lambda_m) \int_{R_{\ell-1}}^{R_\ell} r^2 \phi_m \phi_n \, dr.$$

Step 5 (add the five layers). The left side becomes a sum of differences: $[\mathcal{B}(R_1^-) - \mathcal{B}(0^+)] + [\mathcal{B}(R_2^-) - \mathcal{B}(R_1^+)] + \cdots + [\mathcal{B}(R^-) - \mathcal{B}(R_4^+)]$.

Step 6 (interface terms cancel). Write $\mathcal{B}_{mn} = r^2\,[\phi_n\,(D\phi_m') - \phi_m\,(D\phi_n')]$. It is built only from ϕ (continuous by Eq. (3.2)) and $D\phi'$ (continuous by Eq. (3.3)), so $\mathcal{B}_{mn}(R_\ell^-) = \mathcal{B}_{mn}(R_\ell^+)$ and each interior pair cancels. This is the step where the interface conditions with $\gamma_\ell = 1$ are essential.

Step 7 (end terms vanish). At the centre $r^2 \to 0$ while ϕ and ϕ' stay bounded, so $\mathcal{B}_{mn}(0^+) = 0$. At the surface $\phi_m(R) = \phi_n(R) = 0$ by Eq. (3.4), so $\mathcal{B}_{mn}(R^-) = 0$.

Step 8 (conclusion). $(\Lambda_n - \Lambda_m)\int_0^R r^2\phi_m\phi_n\,dr = 0$ and $\Lambda_n \neq \Lambda_m$. Since $r^2\phi_m\phi_n = (r\phi_m)(r\phi_n) = u_m u_n$ (check: `4.17 weight: r^2 phi_m phi_n = u_m u_n and r^2 phi'^2 = (u' - u/r)^2` PASS):

$$\int_0^R r^2 \phi_m \phi_n \, dr = \int_0^R u_m \, u_n \, dr = 0, \qquad m \neq n. \tag{4.17}$$

Two consequences.

- *Eigenvalues are simple.* Two eigenfunctions with the same Λ both start from $u(0) = 0$; after scaling to the same $q(0)$ they solve the same initial-value problem and are identical. So each eigenvalue has one shape, up to scale.
- *Eigenvalues are positive.* Multiply the self-adjoint form by ϕ and integrate over each layer, using $(Y\phi)' = Y'\phi + Y\phi'$ with $Y = r^2 D\phi'$ (check: `4.17 Rayleigh step (p phi' phi)' = (p phi')' phi + p phi'^2` PASS). The boundary and interface terms vanish exactly as in Steps 6 and 7, leaving

$$\Lambda \int_0^R r^2\phi^2\,dr = \int_0^R r^2 D\,(\phi')^2\,dr, \qquad \text{that is} \qquad \Lambda = \frac{\int_0^R D\,(u' - u/r)^2\,dr}{\int_0^R u^2\,dr} > 0.$$

Both integrals are positive for a nonzero shape that vanishes at R, so $\Lambda > 0$ and every k_ℓ is real, as assumed in Section 4.3 (check: `4.17 Rayleigh quotient int D(u' - u/r)^2 / int u^2 = Lambda_n > 0 (modes 1 to 4)` PASS).

Numerical confirmation. For the 32 modes of the synthetic case the normalised overlap matrix $\int u_m u_n\,dr/\sqrt{N_m N_n}$ has off-diagonal entries below 1.6×10^{-14}; with

the wrong weight (weight 1 for ϕ) the largest off-diagonal entry is 0.97 (check: `4.17 int u_m u_n dr = 0 for m != n (32 modes, normalised Gram off-diagonal < 1e-10)` PASS).

Classification. Eq. (4.17) is DERIVED NOT IMPLEMENTED: no code computes an overlap integral, but the weight formula (4.22) is valid only because Eq. (4.17) holds, and the module documentation states it (src/analytical/sturm_liouville.rs, line 16). Source label: adapted from published literature, Gelbard (2003) Eq. (IV.7), p. 29, whose weight is $\gamma_\ell r^2$ and reduces to r^2 for $\gamma_\ell = 1$. Gelbard defers the proof to Ozisik (1980, pp. 331 to 334, as cited in Gelbard, 2003, p. 29), a book that was not consulted for this document; the proof above is derived for STRATO. Status: orthogonality, positivity of Λ and simplicity of the eigenvalues are PROVED above and DEMONSTRATED NUMERICALLY for the synthetic case.

4.9 Expansion of the initial state

Physical problem. The particle starts with $c = c_0$ in the kernel and $c = 0$ in the coatings, Eq. (3.5). Write this start as a sum of modes.

What we want. The amount A_n of each mode, and the integrals that Code C evaluates.

Governing principle. Orthogonality (4.17), used like the dot product with a unit arrow to pick out one component.

Assumption (completeness). Every reasonable starting profile can be written as $c(r,0) = \sum_n A_n \phi_n(r)$, the sum converging in the mean-square sense with weight r^2. For a regular Sturm-Liouville problem this is established theory; for the layered sphere v1.0.0 relies on it without proof and checks it numerically (Section 4.10). A proof is a FUTURE IMPROVEMENT (Section 7.5).

Step 1 (multiply by $r^2\phi_m$ and integrate). $\int_0^R r^2 c(r,0)\phi_m \, dr = \sum_n A_n \int_0^R r^2 \phi_n \phi_m \, dr$.

Step 2 (orthogonality removes all terms but one). By Eq. (4.17) only $n = m$ survives: $\int_0^R r^2 c(r,0)\phi_m \, dr = A_m \int_0^R r^2 \phi_m^2 \, dr$.

Step 3 (change to u). Put $\phi_m = u_m / r$. Then $r^2\phi_m^2 = r^2 u_m^2 / r^2 = u_m^2$ and $r^2 c\, \phi_m = r^2 c\, u_m / r = r\, c\, u_m$, so $\int_0^R r\, c(r,0)\, u_m \, dr = A_m \int_0^R u_m^2 \, dr$.

Step 4 (insert the kernel-only start). $c(r,0) = c_0$ for $r < R_1$ and 0 beyond, so the left side is $c_0 \int_0^{R_1} r\, u_m \, dr$. Dividing by $\int_0^R u_m^2 \, dr$ and renaming m as n, the coefficient is

$$A_n = \frac{c_0 \int_0^{R_1} r\, u_n \, dr}{\int_0^R u_n^2 \, dr} = \frac{c_0\, I_{1,n}}{N_n}, \tag{4.18}$$

and with Eq. (4.1) the concentration series is

$$c(r,t) = \sum_{n=1}^{\infty} A_n \, \frac{u_n(r)}{r} \, e^{-\Lambda_n t}. \tag{4.19}$$

Step 5 (the integrals Code C evaluates). The two integrals in Eq. (4.18), together with the whole-particle moment needed in Section 4.10, are

$$I_{1,n} = \int_0^{R_1} r\, u_n \, dr, \qquad I_n = \int_0^{R} r\, u_n \, dr, \qquad N_n = \int_0^{R} u_n^2 \, dr. \tag{4.20}$$

I_n is not needed for A_n; it appears in Section 4.10. Units: $I_{1,n}$ and I_n carry (unit of u) $\times$ m^2, N_n carries (unit of u)$^2 \times$ m, so $A_n u_n / r$ is in mol m^{-3} whatever the unit of u.

Normalisation, and why the code needs none. If u_n is replaced by σu_n, then $I_{1,n} \to \sigma I_{1,n}$, $N_n \to \sigma^2 N_n$, $A_n \to A_n/\sigma$, and the product $A_n u_n$ in Eq. (4.19) is unchanged (check: `4.18 A_n u_n and w_n are unchanged when u_n -> sigma u_n PASS`). The scale fixed by $q(0) = 1$ (Eq. (4.9)) therefore drops out. A reader who prefers orthonormal shapes can use $\hat{u}_n = u_n / \sqrt{N_n}$, for which $\int_0^R \hat{u}_n^2 \, dr = 1$; Figure 4.1 (b) plots these.

Classification. Eq. (4.20) is IMPLEMENTED in `ModalRelease::new` (src/analytical/sturm_liouville.rs, lines 337 to 351): `norm` sums $\int u^2$ over the five layers (line 345), `moment` sums $\int r\, u$ (lines 346 to 347), and `i1` keeps the kernel part only (lines 348 to 350); the quadrature rule is described in Section 4.11. Normalisation is IMPLEMENTED implicitly: the scale is set at line 134 and divided out through N_n at line 352. Eq. (4.18) is DERIVED NOT IMPLEMENTED (the code never forms A_n; it cancels into w_n). Eq. (4.19) is DERIVED NOT IMPLEMENTED: no code reconstructs $c(r,t)$ in v1.0.0; concentration profiles come only from Code B. Source label: adapted from published literature, Gelbard (2003) Eq. (IV.10), p. 30, with $\gamma_\ell = 1$, and Eq. (IV.12), p. 31, for N_n.

Physical meaning. A_n measures how much of the starting profile looks like mode n. $I_{1,n}$ is the overlap of mode n with the filled kernel; N_n is the size of the mode itself, so dividing by it makes the answer independent of how large we happened to draw the mode.

4.10 Release fraction and weights

Physical problem. Turn the series (4.19) into the fraction of the initial inventory that has left the particle.

What we want. $F(t)$ in the form Code C evaluates.

Governing principle. The inventory integral (1.7), $M(t) = \int_0^R c\, 4\pi r^2 \, dr$, and the definition (3.9), $F = 1 - M/M^0$.

Step 1 (inventory of the series). Insert Eq. (4.19) into Eq. (1.7) and integrate term by term. Exchanging the sum and the integral is allowed here because the series converges in the mean-square sense with weight r^2 and the inventory integral is the

weighted integral of c against the constant function 1 on a finite interval (Cauchy-Schwarz inequality; stated without proof). Using $\frac{u_n}{r} \cdot 4\pi r^2 = 4\pi r\, u_n$:

$$M(t) = \sum_n A_n e^{-\Lambda_n t} \int_0^R \frac{u_n}{r}\, 4\pi r^2\, dr = 4\pi \sum_n A_n\, I_n\, e^{-\Lambda_n t}. \tag{4.21}$$

Step 2 (divide by the initial inventory). With $M^0 = \frac{4\pi}{3} R_1^3 c_0$ (Eq. (3.6)) and A_n from Eq. (4.18):

$$\frac{M(t)}{M^0} = \sum_n \frac{4\pi\, c_0 I_{1,n} I_n / N_n}{\frac{4\pi}{3} R_1^3 c_0}\, e^{-\Lambda_n t} = \sum_n \frac{3\, I_{1,n} I_n}{R_1^3 N_n}\, e^{-\Lambda_n t}.$$

Both 4π and c_0 cancel (check: `4.22 4 pi A_n I_n / M^0 = 3 I_1 I_n / (R_1^3 N_n)` PASS).

Step 3 (release fraction). By Eq. (3.9),

$$F(t) = 1 - \sum_{n=1}^{K} w_n\, e^{-\Lambda_n t}, \qquad w_n = \frac{3\, I_{1,n}\, I_n}{R_1^3\, N_n}, \tag{4.22}$$

where the code keeps the K modes with $\Lambda_n \le \Lambda_{\text{cov}}$ (Section 4.12).

Step 4 (the weights must add up to one). At $t = 0$ nothing has left, $F(0) = 0$, so with all modes present the weights must sum to one:

$$\lim_{K\to\infty} \sum_{n=1}^{K} w_n = 1. \tag{4.23}$$

This follows from completeness and is not proved here for the layered sphere: for the five-layer problem (4.23) is CLAIMED ONLY (established Sturm-Liouville theory, not proved in this document) and DEMONSTRATED NUMERICALLY by Table 4.4. In the homogeneous sphere with a uniform start it becomes $\sum_n 6/(n^2\pi^2) = 1$, which is true (Section 4.14).

Classification. Eq. (4.22) is IMPLEMENTED: the weight at src/analytical/sturm_liouville.rs, lines 352 to 356 (`3.0 * i1 * moment / (r1.powi(3) * norm)`, with an error if it is not finite), the sum $\sum_n w_n e^{-\Lambda_n t}$ in `remaining_fraction` (lines 381 to 388) and $F = 1 - \sum$ in `release_fraction` (lines 390 to 393). Eq. (4.21) is DERIVED NOT IMPLEMENTED (Code C never computes $M(t)$; the code needs no c_0, and `Layer` has no concentration field). Eq. (4.23) is DERIVED NOT IMPLEMENTED; it is checked by `modal_completeness` in examples/verify.rs, lines 228 to 246, and its proof for layers is a FUTURE IMPROVEMENT. Source label: adapted from published literature, Gelbard (2003) Eqs. (IV.14) and (IV.15), p. 32 to 33, with $\gamma_\ell = 1$ (the reduction to this case is part of the source audit noted in Section 4.16).

Numbers for the synthetic case. The independent implementation, run with the inputs as stored in binary64 (as Rust reads them), gives

$F(2 \times 10^5\ \mathrm{s}) = 0.43255563909304174$; with the exact decimal inputs the same series gives 0.43255563909304190, a difference of 1.7×10^{-16} that comes only from the binary rounding of the inputs and is far below the accuracy floor of Section 4.13 (checks: `4.22 F(2e5 s) = 0.43255563909304 (abs < 5e-15)` PASS; `4.22 F(2e5 s) equals 50-digit oracle 0.43255563909304174 (abs < 1e-16)` PASS; `4.22 F(2e5 s) is within 1e-10 of the repo test constant 0.4325556390930383 (TST1 22-30)` PASS). The repo test constant itself lies 3.4×10^{-15} below the binary64-input value; the integration test compares it with `exact_release_at` (src/postprocessing/comparison.rs, lines 223 to 227), which builds a two-mode series for $t = 2 \times 10^5$ s (Section 4.12), with a tolerance of 10^{-10}. The weights of Table 4.3 alternate in sign: the step-shaped start (3.5) is built from modes that add in the kernel and cancel in the coatings. Table 4.4 shows how the truncated weight sum approaches 1.

Table 4.4 Truncated weight sum for the synthetic case (calculated, independent implementation; the Rust 32-mode sum is 1.000007275592173 and the 40-digit sum 1.000007275592095, check `4.23 sum of 32 weights = 1.000007275592095 ...` PASS).

$\Lambda_{\max}$ (s^{-1})	K	$1 - \sum_{n \le K} w_n$
2×10^{-3}	9	-2.202×10^{-3}
2×10^{-2}	30	1.457×10^{-3}
2.25×10^{-2} (main run)	32	-7.276×10^{-6}
2×10^{-1}	95	5.401×10^{-6}

The convergence is slow and not monotone: the sum overshoots and undershoots 1 while the envelope shrinks (check: `4.23 |1 - sum w| decreases for Lambda_max = 2e-3, 2e-2, 2e-1 (VER 228-246 rule)` PASS). This is the reason the truncated series is not used at $t = 0$: there it would give $F(0) = 1 - \sum w_n = -7.3 \times 10^{-6}$ instead of 0. At the times where it is used, the factors $e^{-\Lambda_n t}$ suppress every omitted mode (Section 4.12), so the slow convergence of $\sum w_n$ does not limit the accuracy of F.

4.11 Evaluating the integrals

4.11.1 Concept: Gauss-Legendre quadrature

A quadrature rule replaces an integral by a weighted sum of function values. The trapezoid rule uses equally spaced points; Gauss-Legendre places its 16 points at the roots x_m of the Legendre polynomial of degree 16 on $[-1, 1]$ and gives them weights ω_m, so that

$$\int_{-1}^{1} f(x)\,dx \approx \sum_{m=1}^{16} \omega_m\, f(x_m)$$

is exact for every polynomial of degree up to $2 \cdot 16 - 1 = 31$. This is an established textbook result. A smooth function that changes little over the interval is integrated almost to rounding error.

4.11.2 What the code does

Step 1 (nodes and weights). `gauss_legendre_16` computes the nodes by Newton's method on the three-term recurrence $P_k(\xi) =$

$[(2k-1)\xi P_{k-1}(\xi) - (k-1)P_{k-2}(\xi)]/k$, with the derivative $P'_{16}(\xi) = 16\,[\xi P_{16}(\xi) - P_{15}(\xi)]/(\xi^2 - 1)$, starting values $\xi = \cos(\pi(i+0.75)/16.5)$, $i = 0, \ldots, 15$, and stopping when the Newton step is below 10^{-15} or after 100 iterations. The weights are $\omega = 2/\left[(1-\xi^2)P'_{16}(\xi)^2\right]$. The result agrees with `numpy.polynomial.legendre.leggauss(16)` to 1.4×10^{-17} in the nodes and 2.2×10^{-15} in the weights, and is exact to 4.4×10^{-15} for degree ≤ 31 but not for degree 32 (error 7.2×10^{-10}) (checks: `4.24 code nodes and weights equal numpy leggauss(16)` PASS; `4.24 rule exact for degree <= 31, not for degree 32` PASS).

Step 2 (panels). On $[a, b]$ the code uses $N_{\mathrm{p},\ell}$ equal panels of width $h = (b-a)/N_{\mathrm{p},\ell}$; on each panel the node x_m maps to $\mathrm{mid} + \frac{h}{2}x_m$ and the sum is multiplied by $h/2$. In layer ℓ of mode n

$$\int_{R_{\ell-1}}^{R_\ell} f(r)\,dr \approx \sum_{i=1}^{N_{\mathrm{p},\ell}} \frac{h}{2} \sum_{m=1}^{16} \omega_m\, f\left(\mathrm{mid}_i + \frac{h}{2}x_m\right), \qquad N_{\mathrm{p},\ell} = \left\lceil \frac{k_\ell L_\ell}{\pi/2} \right\rceil + 1, \tag{4.24}$$

applied to $f = u_n^2$ (for N_n) and $f = r\,u_n$ (for $I_{1,n}$ and I_n). The panel rule guarantees $k_\ell h < \pi/2$: each panel covers less than a quarter wavelength of u_n (check: `4.24 panel rule gives phase k h < pi/2 per panel (10000 random kL)` PASS). For mode 1 of the synthetic case the phases $k_\ell L_\ell$ are 0.362, 0.009, 0.101, 0.640 and 0.108 rad, all below $\pi/2$, so every layer gets 2 panels (script CHAIN output).

Classification. Eq. (4.24) is IMPLEMENTED: `gauss_legendre_16` (src/analytical/sturm_liouville.rs, lines 266 to 292), `integrate` (lines 294 to 304) and the panel count (line 340). Source label: established textbook result (Gauss-Legendre); the panel rule is derived for STRATO. A float64 replica of this rule reproduces the closed-form weights of all 32 modes to 2.0×10^{-13} (check: `4.24 composite GL16 with ceil(kL/(pi/2)) + 1 panels reproduces the closed-form w_n (32 modes, abs < 1e-12)` PASS). Status: the panel property $k_\ell h < \pi/2$ is PROVED (because $N_{\mathrm{p},\ell} > k_\ell L_\ell/(\pi/2)$); exactness of the 16-point rule up to degree 31 is an established textbook result, DEMONSTRATED NUMERICALLY for the code's nodes; the size of the quadrature error in w_n is DEMONSTRATED NUMERICALLY for the synthetic case only, and no error bound is proved.

4.11.3 Closed-form alternative

Because u is a sine and a cosine in each layer, the integrals have closed forms. With Eq. (4.5) on $0 \leq s \leq L$ in layer ℓ, $r = R_{\ell-1} + s$, and the identities $\cos^2 x = \frac{1}{2}(1 + \cos 2x)$, $\sin^2 x = \frac{1}{2}(1 - \cos 2x)$, $2\sin x \cos x = \sin 2x$:

$$\int_0^L u^2\,ds = \frac{(a^2+b^2)L}{2} + \frac{(a^2-b^2)\sin(2kL)}{4k} + \frac{ab\,(1-\cos(2kL))}{2k},$$

$$\int_0^L (R_{\ell-1}+s)\,u\,ds = \frac{R_{\ell-1}}{k}\left[a\sin(kL)+b\,(1-\cos(kL))\right] + a\left[\frac{L\sin(kL)}{k}+\frac{\cos(kL)-1}{k^2}\right] + b\left[\frac{\sin(kL)}{k^2}-\frac{L\cos(kL)}{k}\right]. \tag{4.25}$$

(checks: `4.25 int_0^L u^2 ds closed form` PASS; `4.25 int_0^L (R_{l-1} + s) u ds closed form` PASS.) Classification: DERIVED NOT IMPLEMENTED (v1.0.0 uses quadrature only). Using Eq. (4.25) would remove the quadrature error altogether; this is listed in Section 7.5. Gelbard (2003) gives the equivalent antiderivatives in absolute r as Eqs. (IV.11) and (IV.13), p. 31 to 32; see Section 4.16 for two misprints there.

4.12 Truncation and validity time

Physical problem. The code keeps only finitely many modes. The omitted modes have larger Λ_n and fade faster, but at very early times they still matter.

What we want. The rule that decides how many modes to keep and from what time on the truncated series is trusted.

Step 1 (size of an omitted mode). Every omitted mode has $\Lambda_n > \Lambda_{\text{cov}}$, so at time t its factor obeys $e^{-\Lambda_n t} < e^{-\Lambda_{\text{cov}} t}$.

Step 2 (validity time). The code accepts a time only if $\Lambda_{\text{cov}} t \geq 40$, so each omitted term is below $e^{-40} = 4.25 \times 10^{-18}$ of its weight, under the double-precision rounding of F.

Step 3 (requested bound). To make the earliest requested time t_{min} valid with margin, the code asks for all modes up to $45/t_{\text{min}}$:

$$\Lambda_{\text{max}} = \frac{45}{t_{\text{min}}}, \qquad t_{\text{valid}} = \frac{40}{\Lambda_{\text{cov}}}, \qquad F^{\text{ex}}(t) \text{ is reported only for } t \geq t_{\text{valid}}. \tag{4.26}$$

When the cap of Eq. (4.16) is not reached, $t_{\text{valid}} = \frac{40}{45} t_{\text{min}} < t_{\text{min}}$, so every requested time is valid and the omitted modes are below $e^{-45} = 2.9 \times 10^{-20}$ at t_{min}. When the cap is reached, $\Lambda_{\text{cov}} < \Lambda_{\text{max}}$, t_{valid} moves later, and earlier times return no value (`None`). The doc comment of `exact_release` (lines 396 to 398) describes only the first case.

Classification. IMPLEMENTED: `min_valid_time` (src/analytical/sturm_liouville.rs, lines 375 to 379), `exact_release` (lines 399 to 418), `compare_release` (src/postprocessing/comparison.rs, lines 58 to 64) and `convergence_study` (lines 186 to 190); `exact_release` is reached through `exact_release_at` (lines 223 to 227),

which the tests and `examples/verify.rs` call. The rule bounds each omitted term separately; a bound on the whole tail $\sum_{n>K}|w_n|e^{-\Lambda_n t}$ is not computed (FUTURE IMPROVEMENT). Status: the per-term bound is PROVED; the claim that the whole omitted tail is below double-precision rounding is CLAIMED ONLY in general, and DEMONSTRATED NUMERICALLY for the synthetic case by the two-mode versus 32-mode comparison below. Source label: derived for STRATO.

Synthetic case. Main run: the first positive output time is $t_{\text{min}} = 10 \times 200\ \text{s} = 2000\ \text{s}$, so $\Lambda_{\text{max}} = \Lambda_{\text{cov}} = 0.0225\ \text{s}^{-1}$, $K = 32$ and $t_{\text{valid}} = 1777.78$ s (check: `4.26 t_valid = 40/0.0225 = 1777.78 s < first output 2000 s` PASS). Refinement study: the reference is built with $t_{\text{min}} = t_{\text{end}} = 2 \times 10^5$ s, so $\Lambda_{\text{max}} = 2.25 \times 10^{-4}\ \text{s}^{-1}$ and only $K = 2$ modes are kept; the third mode would contribute $0.227\, e^{-51.7} \approx 8 \times 10^{-24}$, and in exact arithmetic the two-mode value equals the 32-mode value to 8.3×10^{-24} (check: `4.26 study reference (Lambda_max = 45/t_end): 2 modes give the same F(2e5 s) to 1e-20` PASS). In double precision the two runs differ slightly more, because Λ_1 is found from a different bracket (Section 4.7): a float64 transcription of the Rust routines gives 0.4325556390928056 for the study reference and 0.43255563909285666 for the main run, 5×10^{-14} apart and both within 2.4×10^{-13} of the independent value, well inside $\varepsilon_F = 4.0 \times 10^{-12}$. The study-reference figure comes from the transcription; the Rust program was not re-run for it.

Early time. At $t = 2000$ s the independent 40-digit value of the 32-mode series is $F = 1.6 \times 10^{-15}$, while Rust returns -8.55×10^{-14} (first row of `tests/golden/v1.0/comparison_fv_vs_analytical.csv`; the float64 transcription gives -8.57×10^{-14}; this 2×10^{-16} difference is not explained here and is far below ε_F). The Rust value is slightly negative, which no physical release fraction can be, but it lies inside the accuracy floor of Section 4.13. Clamping or flagging negative series values is a FUTURE IMPROVEMENT.

4.13 Round-off: flux ratio and accuracy floor

4.13.1 Concept: cancellation

Subtracting two nearly equal numbers destroys leading digits. If $x = 391.28521$ and $y = 391.32828$ are each known to 16 significant digits, their difference -0.04308 is known only to about 12 digits, because the first four digits cancelled. In double precision each stored number carries a relative error of about $\epsilon_{\text{m}} = 2.2 \times 10^{-16}$ (the code's literal; it is the spacing of doubles near 1, 2^{-52}). The relative error of a difference is about ϵ_{m} times the size of the numbers divided by the size of the difference.

4.13.2 Derivation

Step 1 (where the code subtracts). The physical flux in the u language is $q - Du/r$ (Section 4.4). When the two terms are large and nearly equal, the flux, and every later step built on it, loses digits.

Step 2 (the ratio the code records). For every mode and every interface the code computes

$$\rho = \frac{|q - D\,u/R_\ell|}{|q| + |D\,u/R_\ell|}, \qquad \rho_{\text{min}} = \min_{n,\,\ell} \rho, \tag{4.27}$$

using the post-jump value $q = q^+$ and the outer layer's diffusivity $D = D_{\ell+1}$. The numerator is the same on both sides of the interface, by Eq. (4.7); the denominator is not. For the synthetic case the outer side gives $\rho_{\min} = 5.504 \times 10^{-5}$ (reached for mode 1 at the Kernel/Buffer interface, Table 4.2), while the inner side would give 1.231×10^{-4}.

Step 3 (accuracy floor). The code turns this ratio into an estimate of the absolute accuracy of F:

$$\varepsilon_F = \frac{2.2 \times 10^{-16}}{\min(\rho_{\min}, 1)}. \tag{4.28}$$

For the synthetic case $\varepsilon_F = 3.997 \times 10^{-12}$ (check: `4.27-4.28 rho_min = 5.504e-5 and eps_F = 3.997e-12 (main run) within rel 1e-3` PASS; the same line reports the inner-side alternative, floor 1.79×10^{-12}).

How good is the estimate. The Rust value $F(2 \times 10^5 \text{ s}) = 0.43255563909285666$ (last row of `tests/golden/v1.0/comparison_fv_vs_analytical.csv`) differs from the independent 40-digit value by 1.85×10^{-13}, about 21 times below ε_F (check: `4.28 Rust F(2e5 s) = 0.43255563909285666 (audit 01) lies within eps_F of the independent value` PASS). This is a single case. Eq. (4.28) is a heuristic estimate for which no bound has been proved, and the doc-comment statement that the true error stays "about 10 to 20 times below" (lines 366 to 370) rests on test inputs, not on an analysis. A sweep over diffusivity contrasts is a FUTURE IMPROVEMENT. Status: ε_F as a bound on the error of F is CLAIMED ONLY; that it exceeds the actual error is DEMONSTRATED NUMERICALLY for the synthetic case at $t = 2 \times 10^5$ s.

Classification. IMPLEMENTED: ρ in `ModalRelease::new` (src/analytical/sturm_liouville.rs, lines 330 to 336, where `st.left` is R_ℓ and `starts.iter().skip(1)` skips the centre) and ε_F in `accuracy_floor` (lines 366 to 373). Code A uses $3\varepsilon_F$ as the threshold below which a difference between the methods counts as unresolved (Part VII, Eq. (7.2)). Source label: derived for STRATO.

Physical meaning. A large jump in diffusivity, such as kernel to buffer (factor 400) or IPyC to SiC (factor 40), makes the two parts of the flux large and nearly equal on one side of the interface. The series is then exact in principle but limited in practice to about ε_F.

4.14 Homogeneous checks

Physical problem. If all five layers have the same diffusivity D, the particle is a uniform sphere and the series must reduce to known closed forms. This gives a test with an answer known in advance.

Step 1 (no jumps). With $D_{\ell+1} = D_\ell$, Eq. (4.8) gives $q^+ = q^-$ and the interface matrices become the identity. The wavenumber is one k everywhere, so the shot solution is the kernel solution continued: $u = b \sin(kr)$ on $0 \le r \le R$.

Step 2 (eigenvalues). $u(R) = b \sin(kR) = 0$ requires $kR = n\pi$, so $k = n\pi/R$ and $\Lambda_n = Dk^2$.

Step 3 (integrals). With the scale $b = 1$ and $k_n = n\pi/R$ (the code's scale differs by a factor that cancels, Section 4.9), direct integration gives

$$u_n = \sin\frac{n\pi r}{R}, \qquad \Lambda_n = \frac{n^2\pi^2 D}{R^2}, \qquad N_n = \frac{R}{2},$$
$$I_{1,n} = \frac{\sin(k_n R_1) - k_n R_1 \cos(k_n R_1)}{k_n^2}, \qquad I_n = \frac{R^2(-1)^{n+1}}{n\pi}. \tag{4.29}$$

(checks: `4.29 u_n = sin(n pi r/R) vanishes at r = R` PASS, `4.29 N_n = R/2` PASS, `4.29 I_n = R^2 (-1)^(n+1)/(n pi)` PASS, `4.29 I_1n = (sin(alpha R1) - alpha R1 cos(alpha R1))/alpha^2` PASS; the independent five-layer matching code finds $g(n^2\pi^2 D/R^2) = 0$, check `4.29 equal D: g(n^2 pi^2 D/R^2) = 0 with the five-layer matching (n = 1..5)` PASS.)

Step 4 (uniform start). If the substance fills the whole sphere, $R_1 = R$, then $I_{1,n} = I_n$ and $w_n = 3I_n^2/(R^3 \cdot R/2) = 6/(n^2\pi^2)$ (check: `4.30 R_1 = R gives w_n = 6/(n^2 pi^2)` PASS), so

$$F(t) = 1 - \frac{6}{\pi^2}\sum_{n=1}^{\infty}\frac{1}{n^2}\exp\left(-\frac{n^2\pi^2 D t}{R^2}\right). \tag{4.30}$$

This is the classical release from a uniform sphere, an established textbook result (Crank, 1975). Here the weight sum can be done exactly, $\sum_n 6/(n^2\pi^2) = 1$ (check: `4.30 sum_n 6/(n^2 pi^2) = 1 (completeness holds here)` PASS), which confirms Eq. (4.23) in this special case.

Classification. Eq. (4.29) is IMPLEMENTED in the unit-test helper `homogeneous_kernel_release` (src/analytical/sturm_liouville.rs, lines 434 to 445), against which the test `modal_release_matches_closed_form_for_equal_diffusivity` (lines 519 to 532) requires agreement to 10^{-9} at $t = 20$, 100, 500 and 2000 s for $D = 10^{-10}$ m^2 s^{-1}. Eq. (4.30) is IMPLEMENTED as the verification function `sphere_release_uniform` (examples/verify.rs, lines 48 to 60), where it serves only as a reference for Code B.

4.15 Robin sphere root equation (Case B reference)

Physical problem. Case B of the project replaces the perfect sink by a partly blocking surface, $-D\,\partial c/\partial r = h_\mathrm{m}(c - c_\infty)$ at $r = R$ (Eq. (3.10)). For a single homogeneous sphere with $c_\infty = 0$ the eigenvalue equation is short, and Code C contains a solver for it.

Step 1 (trial shape). For one material the regular shape is $\phi = \sin(zr/R)/r$ with a dimensionless z; then $k = z/R$ and $\Lambda = Dz^2/R^2$.

Step 2 (derivative at the surface). By the quotient rule $\phi' = \frac{(z/R)\cos(zr/R)}{r} - \frac{\sin(zr/R)}{r^2}$, so at $r = R$: $\phi'(R) = (z\cos z - \sin z)/R^2$ and $\phi(R) = \sin z/R$.

Step 3 (insert into the Robin condition). $-D\,(z\cos z - \sin z)/R^2 = h_\mathrm{m}\sin z/R$. Multiply by R^2/D: $\sin z - z\cos z = (h_\mathrm{m}R/D)\sin z$.

Step 4 (divide by $\sin z$). With $\mathrm{Bi} = h_\mathrm{m}R/D$ and $\cot z = \cos z/\sin z$:

$$z\cot z = 1 - \mathrm{Bi}. \tag{4.31}$$

(check: `4.31 -D phi'(R) - h phi(R) = (D sin z / R^2)(1 - z cot z - Bi)` PASS.)

Classification. IMPLEMENTED: `robin_residual` evaluates $z/\tan z - (1 - \mathrm{Bi})$ (src/analytical/sturm_liouville.rs, lines 31 to 36) and `robin_roots` (lines 38 to 79) scans each interval $[\nu\pi + 10^{-8}, (\nu+1)\pi - 10^{-8}]$ in 128 segments, bisects each sign change 80 times, and accepts roots above 10^{-6} that differ from the previous root by more than 10^{-6}. The offsets 10^{-8} are what keep the poles of $\cot z$ at $\nu\pi$ out of the scan; the comment at line 39 attributes this to the finite-value test. Both functions are called only by the unit test `robin_roots_satisfy_residual` (lines 447 to 454); the program and the verification driver never use them. The layered Robin problem is a FUTURE IMPROVEMENT. Source label: derived for STRATO (a standard homogeneous-sphere result).

4.16 Relation to Gelbard (2003)

Gelbard (2003, Section IV, p. 26 to 34) solves the same layered problem by the same separation of variables, and is the published source closest to Code C. The correspondence and the differences are as follows; page numbers are the printed pages of the report.

- **Notation.** Gelbard's separation constant is $-\beta_n^2$ (Eq. (IV.4), p. 28; time factor Eq. (IV.8), p. 29), so $\Lambda_n = \beta_n^2$ and $k_\ell = \beta_n/\sqrt{D_\ell}$. His layer shapes $\Psi = (A\sin + B\cos)/r$ (Eq. (IV.6), p. 28) are u/r in absolute r; Code C uses the local coordinate s instead.
- **Root finding (DERIVED NOT IMPLEMENTED).** Gelbard writes the $2L$ end and interface conditions as a homogeneous linear system, reduces it to $(2L-1)$ unknowns after $B_{1,n} = 0$, and finds the zeros of its determinant with a secant method, checking that the sign of the determinant's derivative alternates (p. 33). The determinant vanishes exactly when $g(\Lambda) = 0$, so the two formulations have the same roots. Code C does not use the determinant; the Sturm count of Section 4.6 guarantees that no root is skipped, which a derivative-sign check does not.
- **Partition factor (FUTURE IMPROVEMENT).** Gelbard's interface condition (IV.3), p. 27, is $\gamma_\ell c_\ell = \gamma_{\ell+1} c_{\ell+1}$. With it, u jumps by the factor $\gamma_\ell/\gamma_{\ell+1}$, the orthogonality weight becomes $\gamma_\ell r^2$ (Eq. (IV.7), p. 29), N_n becomes $\sum_\ell \gamma_\ell \int u_n^2\, dr$ (Eq. (IV.12), p. 31), and the weight acquires γ_1 in the numerator while I_n stays unweighted (source audit). STRATO v1.0.0 is the special case $\gamma_\ell = 1$.
- **Errata found in the source (reported by the STRATO source audit; this document did not repeat the check, Section 7.5).** In Eq. (IV.11), p. 31, the cosine term is printed with $\cos x - x\sin x$; the antiderivative of $r\cos(\beta r/\sqrt{D})$ requires $\cos x + x\sin x$. In Eq. (IV.13), p. 32, the coefficients inside $\sum_j$ are printed $A_{i,n}$, $B_{i,n}$ and should be $A_{j,n}$, $B_{j,n}$. In Section V the stated partition values and the orientation of (IV.3) disagree with the plotted profiles; the figures are reproduced only with $c_{\mathrm{SiC}} = c_{\mathrm{PyC}}/2$. None of these affects v1.0.0, which has $\gamma_\ell = 1$ and a kernel-only start ($B_{1,n} = 0$).
- **Number of modes.** Gelbard states that 5 to 10 eigenvalues give four significant figures (p. 33). The source audit found this true after about 1 h in his Section V example but not at early times; Code C sidesteps the issue with the rule (4.26).

4.17 Summary and verification

Table 4.5 lists every equation of Part IV with its classification and the Rust lines. Table 4.6 gives the numerical properties of Code C with their status, and Table 4.7 collects the verified numbers.

Table 4.5 Equations of Part IV, their classification and the Rust lines (SL = src/analytical/sturm_liouville.rs, CMP = src/postprocessing/comparison.rs, VER = examples/verify.rs).

Eq.	Content	Classification	Code (tag `v1.0.0`)
(4.1), (4.2)	separation, shape equation	DERIVED NOT IMPLEMENTED (premise)	SL 12-19 (doc)
(4.3)	$\phi' = (u' - u/r)/r$	DERIVED NOT IMPLEMENTED (used via 4.8)	none
(4.4)	$u'' + k_\ell^2 u = 0$, $k_\ell = \sqrt{\Lambda/D_\ell}$	IMPLEMENTED	SL 141, SL 175
(4.5), (4.6)	layer solution, transfer matrix	IMPLEMENTED	SL 86-92, SL 339-344
(4.7), (4.8)	interface matching and jump of q	IMPLEMENTED	SL 94-100, SL 138-140, SL 172-174
(4.9), (4.10)	centre start, characteristic function	IMPLEMENTED (g returned, then discarded)	SL 132-162, SL 168, SL 329
(4.11) to (4.13)	phase form, zero count, Sturm count	IMPLEMENTED	SL 164-192
(4.14) to (4.16)	lower bracket, bisection, mode cap	IMPLEMENTED	SL 197-264, CMP 13
(4.17)	orthogonality, weight 1 in u	DERIVED NOT IMPLEMENTED (property)	SL 16 (doc)
(4.18), (4.19)	A_n, $c(r,t)$	DERIVED NOT IMPLEMENTED	none
(4.20)	$I_{1,n}$, I_n, N_n	IMPLEMENTED	SL 337-351
(4.21)	$M(t)$ series	DERIVED NOT IMPLEMENTED	none
(4.22)	w_n and $F(t)$	IMPLEMENTED	SL 352-356, SL 381-393
(4.23)	$\sum w_n \to 1$	DERIVED NOT IMPLEMENTED; proof FUTURE IMPROVEMENT	VER 228-246 (check)
(4.24)	composite Gauss-Legendre, panel rule	IMPLEMENTED	SL 266-304, SL 340
(4.25)	closed-form layer integrals	DERIVED NOT IMPLEMENTED	none
(4.26)	$\Lambda_{\max} = 45/t_{\min}$, $t_{\text{valid}} = 40/\Lambda_{\text{cov}}$	IMPLEMENTED	SL 375-379, SL 399-418, CMP 58-64, CMP 186-190
(4.27), (4.28)	flux ratio, accuracy floor	IMPLEMENTED (heuristic)	SL 330-336, SL 366-373
(4.29), (4.30)	homogeneous closed forms, Crank sphere	IMPLEMENTED in tests and verification only	SL 434-445, SL 519-532, VER 48-60
(4.31)	Robin root equation	IMPLEMENTED, unit test only	SL 31-79, SL 447-454
none	determinant formulation	DERIVED NOT IMPLEMENTED	none
none	partition factors, layered Robin problem, tail bound, clamp of negative F, completeness proof	FUTURE IMPROVEMENT	none

Table 4.6 Numerical properties of Code C and their status.

Property	Status	Where
eigenvalues real, positive and simple; eigenfunctions orthogonal with weight r^2 for ϕ	PROVED; DEMONSTRATED NUMERICALLY (32 modes)	4.8
$\mathcal{N}(\Lambda) = \#\{n : \Lambda_n \le \Lambda\}$ (Sturm count)	PROVED in exact arithmetic with a quoted comparison theorem; DEMONSTRATED NUMERICALLY (95 modes)	4.6
bisection on the count misses no eigenvalue	PROVED for an exact count; DEMONSTRATED NUMERICALLY for the float64 code	4.7
eigenvalue accuracy about $\epsilon_{\mathrm{m}}/\rho_{\min}$ rather than the 2×10^{-15} tolerance	DEMONSTRATED NUMERICALLY (synthetic case)	4.7
completeness, $\sum_n w_n \to 1$ for the layered sphere	CLAIMED ONLY (theory not proved here); DEMONSTRATED NUMERICALLY (Table 4.4); PROVED for the homogeneous sphere (4.30)	4.9, 4.10
panel rule gives $k_\ell h < \pi/2$	PROVED	4.11
quadrature error of w_n near 10^{-13}	DEMONSTRATED NUMERICALLY (synthetic case)	4.11
each omitted mode below e^{-40} of its weight for $t \ge t_{\mathrm{valid}}$	PROVED	4.12
whole omitted tail below rounding	CLAIMED ONLY in general; DEMONSTRATED NUMERICALLY (synthetic case)	4.12
ε_F bounds the error of F^{ex}	CLAIMED ONLY (heuristic); DEMONSTRATED NUMERICALLY for one case	4.13

Table 4.7 Verified numbers of Part IV for the synthetic input (all calculated, none measured; Rust values from the v1.0.0 run in `tests/golden/v1.0/` and from the one-off print of the library noted at Table 4.3).

Quantity	Value	Source
modes kept in the main run, K	32	calculated; independent and Rust agree
Λ_1	$3.273255671175805 \times 10^{-6}$ s^{-1} (Rust); $3.2732556711770218 \times 10^{-6}$ (40-digit)	calculated
Λ_2	$7.791893309400181 \times 10^{-5}$ s^{-1} (Rust); $7.7918933094001902 \times 10^{-5}$ (40-digit)	calculated
$\sum_{n \le 32} w_n$	1.000007275592095 (40-digit); 1.000007275592173 (Rust)	calculated
$F^{\mathrm{ex}}(2 \times 10^5$ s$)$	0.43255563909304174 (40-digit, binary64 inputs); 0.43255563909304190 (40-digit, exact decimal inputs); 0.43255563909285666 (Rust)	calculated
$\rho_{\min}$, ε_F	5.504×10^{-5}, 3.997×10^{-12}	calculated
t_{valid} (main run)	1777.78 s	calculated
Λ_{lo}	2.1999×10^{-12} s^{-1}	calculated

Part V: Discrete finite-volume solution

Part IV built the answer as a sum of fading shapes. Part V builds the same answer a second way, the way Code B (src/numerical/finite_volume.rs) does it: cut the particle into many thin spherical shells and keep an exact account of how much substance each shell holds, moving that account forward in time in small steps. Every equation

in this part is matched to the line of Rust that computes it, or is labelled as a step that the code does not compute.

What this part assumes from earlier parts. The flux law $J_r = -D\,\partial c/\partial r$, Eq. (1.4); the sphere area $A(r) = 4\pi r^2$, Eq. (1.5); the shell volume, Eq. (1.6); the inventory integral, Eq. (1.7); the layer equation, Eq. (2.7); the centre condition, Eq. (3.1); flux continuity at interfaces, Eq. (3.3); the absorbing surface $c(R,t) = 0$, Eq. (3.4); the kernel-only start, Eq. (3.5); and the release relations, Eqs. (3.7) to (3.9). The source term S of Eq. (2.6) is zero throughout, as in the code (assumption list of Section 1.6).

Verification lines such as `P5-10` refer to the PASS lines of the two scripts named in Section 5.12.

5.1 Control volumes and cell averages

The idea in plain words. Picture an onion. Instead of asking for the concentration at every single radius, which is infinitely many numbers, we slice the particle into N_c thin onion skins and ask only for the *average* concentration in each skin. Each skin is called a *control volume*, or a *cell*: a fixed region of space whose contents we keep track of, like a warehouse whose stock we count. The walls between neighbouring skins are called *faces*. Substance can only enter or leave a cell by crossing one of its two faces.

Why this gives exact bookkeeping. Whatever leaves cell j through its outer face enters cell $j+1$ through the same face, because the face is shared. If we compute that one flow once and use it with a minus sign in one cell and a plus sign in the other, nothing can be created or lost between cells, whatever approximation we use for the flow itself. Approximations enter only through the *size* of each face flow; where the substance goes is fixed by the shared faces. This is the defining property of the finite-volume method (established textbook result; see Patankar, 1980).

Definition. Cell j ($j = 1, \dots, N_c$, counted outward from the centre; Rust index `i` $= j-1$) is the shell $r_{j-1/2} \le r \le r_{j+1/2}$. Its inner face is at $r_{j-1/2}$ and its outer face at $r_{j+1/2}$, both in m. The half-integer index says "the face between cell j and cell $j+1$ is face $j+1/2$". The cell average concentration $\bar{c}_j(t)$, in mol m^{-3}, is the amount in the cell divided by its volume. The amount in a thin shell of thickness dr is $c\,4\pi r^2\,dr$, Eq. (1.3), so the amount in the cell is the integral of that, and

$$\bar{c}_j(t) = \frac{1}{V_j} \int_{r_{j-1/2}}^{r_{j+1/2}} c(r,t)\, 4\pi r^2\, dr. \tag{5.1}$$

Classification: DERIVED NOT IMPLEMENTED. This is the *meaning* of the number the code stores for each cell (the vectors `c`, `old`, `next` in src/numerical/finite_volume.rs, lines 162 to 236). The code never evaluates this integral; it evolves the averages directly.

Physical meaning. $V_j \bar{c}_j$ is the number of moles in cell j. The sum of $V_j \bar{c}_j$ over all cells is therefore the inventory $M(t)$ of Eq. (1.7), with no approximation: the cells together fill the sphere exactly, so the integral over $[0, R]$ splits exactly into the integrals over the cells.

5.2 The mesh

What we want. A list of faces $0 = r_{1/2} < r_{3/2} < \cdots < r_{N_c+1/2} = R$ such that every layer interface R_ℓ is one of the faces.

Why faces must sit on interfaces. Inside a cell we will treat the diffusivity as one constant number. That is only true if the cell lies entirely inside one layer. If a cell straddled the kernel and the buffer, it would contain two values of D (here 10^{-12} and 4×10^{-10} m^2 s^{-1} for the synthetic input), and no single number could represent it. Placing faces on interfaces also means the flux continuity condition, Eq. (3.3), is applied exactly where it holds: at a face.

How the code builds it. Layer ℓ (thickness $L_\ell = R_\ell - R_{\ell-1}$, Eq. (1.2)) receives n_ℓ cells from the input (the configuration requires $n_\ell \geq 1$, src/config.rs, lines 90 to 97), all of the same width. Let the $(\iota + 1)$-th cell of layer ℓ, with $\iota = 0, 1, \ldots, n_\ell - 1$, be global cell $j = n_1 + \cdots + n_{\ell-1} + \iota + 1$. Its faces are

$$
\begin{aligned}
w_\ell &= \frac{R_\ell - R_{\ell-1}}{n_\ell} = \frac{L_\ell}{n_\ell}, \\
r_{j-1/2} &= R_{\ell-1} + \iota\, w_\ell, \\
r_{j+1/2} &= \begin{cases} R_{\ell-1} + (\iota+1)\, w_\ell, & \iota < n_\ell - 1, \\ R_\ell, & \iota = n_\ell - 1. \end{cases}
\end{aligned}
\tag{5.2}
$$

Classification: IMPLEMENTED, src/numerical/finite_volume.rs, lines 55 to 62 (width at line 55, faces at lines 57 to 62); the next layer starts at the input radius R_ℓ (line 72). Each cell also stores its layer number and D_ℓ (lines 63 to 70). The mesh must have at least two cells in total (lines 74 to 76).

Why the last face is "snapped". In exact arithmetic $R_{\ell-1} + n_\ell w_\ell = R_\ell$, so the second case of Eq. (5.2) would be unnecessary. In floating-point arithmetic the product and sum are rounded, and the result can miss R_ℓ by a unit in the last place. Example: a kernel of radius 2×10^{-4} m cut into 13 cells gives $13\, w_1 = 1.9999999999999998 \times 10^{-4}$ m (check `P5R-03b`). Setting the last face to the input value R_ℓ guarantees that the outer face of the last cell of layer ℓ and the inner face of the first cell of layer $\ell + 1$ are the same number, bit for bit. For the synthetic input the faces are contiguous and every interface face equals R_ℓ exactly (checks `P5R-02`, `P5R-03`). The delivered Phase 1 document did not state the uniform width or this rule (Table 7.3, item I4).

Physical meaning. The cells are equally wide *within* a layer, but the width changes from layer to layer: for the synthetic input (40 cells per layer) $w_1 = 5.0 \times 10^{-6}$ m, $w_2 = 2.5 \times 10^{-6}$ m and $w_3 = w_4 = w_5 = 1.25 \times 10^{-6}$ m (check `P5R-01`).

5.3 Cell volume, centre and face areas

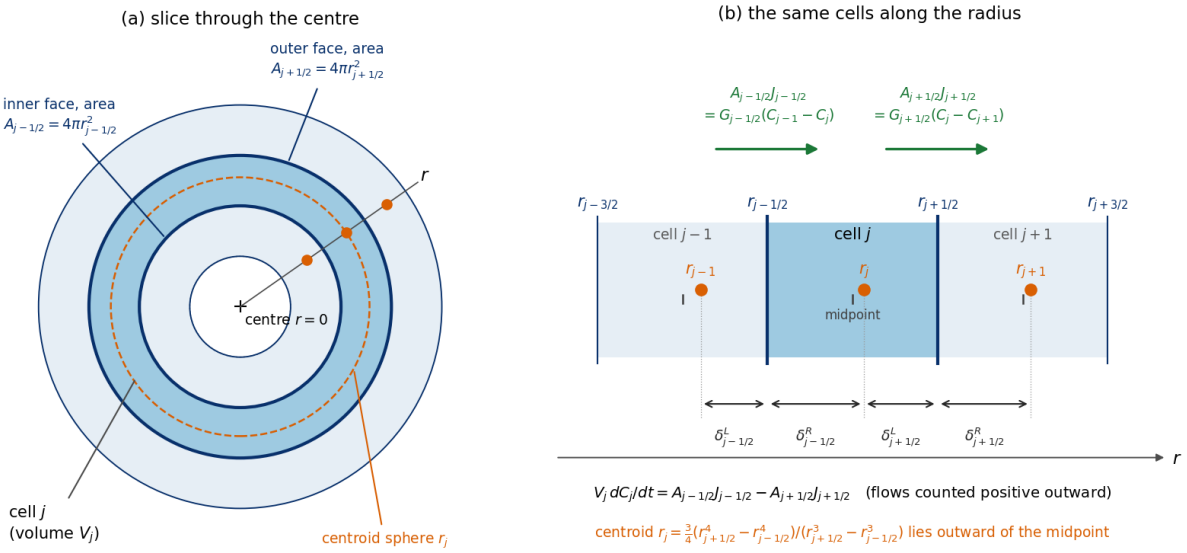


Figure 5.1: One spherical control volume, cell j, between its inner face r_{j-1/2} and outer face r_{j+1/2}. (a) A slice through the particle centre: the shaded shell is cell j, the dashed circle is the centroid sphere r_j, and the dots on the ray are the centroids of cells j-1, j and j+1. (b) The same three cells along the radius, with the face areas, the centroids, the cell midpoints (short ticks) for comparison, the four centre-to-face distances used in the conductances and the two face flows of the cell balance. Radii are illustrative (faces at 1, 2, 3, 4), chosen so the outward shift of the centroid is visible.

5.3.1 Volume

Problem. Find the volume V_j (m^3) of the shell between $a = r_{j-1/2}$ and $b = r_{j+1/2}$ (the letters a, b are shorthand for this section only).

Governing principle. A shell of radius r and tiny thickness dr is, to first order, a sheet of area $4\pi r^2$, Eq. (1.5), and thickness dr, so its volume is $dV = 4\pi r^2\, dr$. The cell is a stack of such sheets, so its volume is their sum, an integral.

Steps. Write the sum as an integral:

$$V_j = \int_a^b 4\pi r^2\, dr.$$

The constant 4π comes out, and an antiderivative of r^2 is $r^3/3$ (its derivative is $3r^2/3 = r^2$):

$$V_j = 4\pi \left[\frac{r^3}{3}\right]_a^b = 4\pi \left(\frac{b^3}{3} - \frac{a^3}{3}\right),$$

and collecting the factor $1/3$,

$$V_j = \frac{4\pi}{3}\left(r^3_{j+1/2} - r^3_{j-1/2}\right). \tag{5.3}$$

Classification: IMPLEMENTED, `shell_volume`, src/numerical/finite_volume.rs, lines 38 to 40, called at line 67. This is Eq. (1.6) applied to one cell. Check `P5-01` (symbolic). Because each term $r_{j+1/2}^3$ appears once with a plus sign (cell j) and once with a minus sign (cell $j+1$), the cell volumes add up to $\frac{4\pi}{3}R^3$; the replica gives a relative difference of 2.7×10^{-16} (check `P5R-07`).

Physical meaning. The volume grows with the cube of radius, so equally wide cells far from the centre hold much more material than equally wide cells near it. The first kernel cell of the synthetic mesh, $[0, 5 \times 10^{-6}]$ m, has volume 5.24×10^{-16} m^3. A cell of the same width at the kernel edge, $[1.95, 2.00] \times 10^{-4}$ m, is much larger. To see by how much, measure radii in units of the width $w_1 = 5 \times 10^{-6}$ m: the first cell is $[0, 1]$ and the edge cell is $[39, 40]$. By Eq. (5.3) the factor $4\pi/3$ and the unit w_1^3 cancel in the ratio, which is therefore

$$\frac{40^3 - 39^3}{1^3 - 0^3} = 64000 - 59319 = 4681.$$

The replica gives 4681.0 to round-off (check `P5R-09c`).

5.3.2 Cell centre: the volume centroid

Problem. Each cell average C_j needs a radius r_j at which it "lives", because the face flows below are built from distances between these radii and the faces. Which radius should we choose?

Concept: centroid. The centroid of a region is its volume-weighted mean position: chop the region into tiny pieces, multiply each piece's radius by its volume, add up, and divide by the total volume. It is the balance point. For a shell, more volume lies near the outer face (the area $4\pi r^2$ is larger there), so the balance point sits outward of the geometric midpoint $(a + b)/2$.

Initial equation.

$$r_j = \frac{\int_a^b r \, dV}{\int_a^b dV} = \frac{\int_a^b r \, 4\pi r^2 \, dr}{V_j}.$$

Step 1, numerator. $r \cdot 4\pi r^2 = 4\pi r^3$, and an antiderivative of r^3 is $r^4/4$:

$$\int_a^b 4\pi r^3 \, dr = 4\pi \, \frac{b^4 - a^4}{4} = \pi \, (b^4 - a^4).$$

Step 2, divide by Eq. (5.3).

$$r_j = \frac{\pi \, (b^4 - a^4)}{\frac{4\pi}{3}(b^3 - a^3)} = \frac{3}{4} \, \frac{b^4 - a^4}{b^3 - a^3}.$$

Step 3, the cell touching the centre. For $j = 1$, $a = r_{1/2} = 0$ and the formula reduces to $\frac{3}{4}\, b^4/b^3 = \frac{3}{4}\, b$. Together:

$$r_j = \frac{3}{4}\,\frac{r_{j+1/2}^4 - r_{j-1/2}^4}{r_{j+1/2}^3 - r_{j-1/2}^3} \quad (j \geq 2), \qquad r_1 = \frac{3}{4}\, r_{3/2}. \tag{5.4}$$

Classification: IMPLEMENTED, `volume_centroid`, src/numerical/finite_volume.rs, lines 42 to 48: the branch `a == 0.0` at lines 43 to 44 returns `0.75 * b`, the general case at line 46; called at line 66. Checks `P5-02` and `P5-04`. In exact arithmetic the branch gives the same value as the general formula would; it saves two powers and a division and removes any doubt about the case $a = 0$.

Where the centroid sits (DERIVED NOT IMPLEMENTED, a property of Eq. (5.4)). Factor $b^4 - a^4 = (b-a)(b+a)(b^2+a^2)$ and $b^3 - a^3 = (b-a)(b^2+ab+a^2)$ and cancel $b - a$:

$$r_j = \frac{3}{4}\,\frac{(a+b)(a^2+b^2)}{a^2+ab+b^2}.$$

Subtracting the midpoint over the common denominator $4(a^2+ab+b^2)$:

$$r_j - \frac{a+b}{2} = \frac{(a+b)\left[3(a^2+b^2) - 2(a^2+ab+b^2)\right]}{4(a^2+ab+b^2)} = \frac{(a+b)(b-a)^2}{4(a^2+ab+b^2)} \geq 0.$$

So the centroid is always on the outer side of the midpoint (check `P5-05`), and it stays strictly inside the cell (check `P5-06`, which also gives the closed forms of $r_j - a$ and $b - r_j$). For a thin cell of width $w = b - a$ around midpoint m, the shift is $w^2/(6m)$ plus terms of order w^4 (check `P5-07`): small far from the centre, large near it. For the first cell the centroid is at three quarters of the radius (3.75×10^{-6} m, midpoint 2.5×10^{-6} m); for the second cell, $[5, 10] \times 10^{-6}$ m, it is at 8.0357×10^{-6} m, midpoint 7.5×10^{-6} m (checks `P5R-05`, `P5R-06`).

Why the centroid is the right choice (DERIVED NOT IMPLEMENTED, justification). Suppose that inside the cell the concentration is a straight line in r, $c = \zeta_0 + \zeta_1 r$ (with constants ζ_0 in mol m^{-3} and ζ_1 in mol m^{-4}). Put it into Eq. (5.1):

$$C_j = \frac{1}{V_j}\int_a^b (\zeta_0 + \zeta_1 r)\, 4\pi r^2\, dr = \zeta_0\,\frac{\int_a^b 4\pi r^2\, dr}{V_j} + \zeta_1\,\frac{\int_a^b r\, 4\pi r^2\, dr}{V_j} = \zeta_0 + \zeta_1\, r_j = c(r_j).$$

The first fraction is 1 by Eq. (5.3) and the second is r_j by the definition of the centroid. So for a straight-line profile the cell average *is* the point value at the centroid, exactly (check `P5-08`). For a curved profile the two differ by a term of order w^2. For example, for $c = \zeta_2 r^2$ (constant ζ_2 in mol m^{-5}) on a cell of width w around the midpoint m, the cell average minus the centroid value is $\zeta_2 w^2/12$ plus terms of order w^4/m^2 (check `P5-09` shows the difference is nonzero; expanding both averages for a thin cell about

its midpoint gives the leading term $w^2/12$ times ζ_2). This is why the code places C_j at the centroid: the straight-line half-cell profiles of Section 5.5 then connect point values that the cell averages represent to second order.

5.3.3 Face areas

Problem. Find the area through which substance crosses face $j + 1/2$.

Step. The face is the whole sphere of radius $r_{j+1/2}$, so by Eq. (1.5)

$$A_{j+1/2} = 4\pi r_{j+1/2}^2. \tag{5.5}$$

Classification: IMPLEMENTED, src/numerical/finite_volume.rs, lines 82 to 83 for internal faces, where `face = left.right` is the outer face of the inner cell; line 90 for the outer surface $A(R) = 4\pi R^2$.

Why the inner and outer faces of one cell differ. The two faces of cell j have areas $4\pi r_{j-1/2}^2$ and $4\pi r_{j+1/2}^2$, and their ratio is $(r_{j+1/2}/r_{j-1/2})^2 > 1$. For the second kernel cell, $[5, 10] \times 10^{-6}$ m, the outer face is four times larger than the inner one. In a flat slab both faces of a slice have the same area; in a sphere the outer face is always the larger. This is the discrete form of the effect that produced the $1/r^2$ factor in Part II, Section 2.3: the same flow, spreading outward, passes through ever larger spheres, so the flow per unit area falls. At the centre the inner face of cell 1 has radius 0 and therefore area 0, which Section 5.6 uses.

5.4 Exact balance of one cell

Physical problem. How fast does the amount in cell j change?

Governing principle. Conservation, Eq. (2.1): the rate of change of the amount inside equals what flows in minus what flows out (no source in v1.0.0).

Initial equation. Inside cell j the diffusivity is the constant D_j of its layer (Section 5.2), and Eq. (2.7) holds:

$$\frac{\partial c}{\partial t} = \frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2 D_j \frac{\partial c}{\partial r}\right), \qquad r_{j-1/2} < r < r_{j+1/2}.$$

Step 1, multiply both sides by $4\pi r^2$. This turns concentration into amount per unit radius, Eq. (1.3), and it cancels the $1/r^2$ on the right:

$$4\pi r^2 \frac{\partial c}{\partial t} = 4\pi \frac{\partial}{\partial r}\left(r^2 D_j \frac{\partial c}{\partial r}\right).$$

Step 2, integrate over the cell, from $r_{j-1/2}$ to $r_{j+1/2}$. The right side is the derivative of something, so by the fundamental theorem of calculus its integral is that something evaluated at the two ends:

$$\int_{r_{j-1/2}}^{r_{j+1/2}} 4\pi r^2 \frac{\partial c}{\partial t}\, dr = \left[4\pi r^2 D_j \frac{\partial c}{\partial r}\right]_{r_{j-1/2}}^{r_{j+1/2}}.$$

Step 3, left side. The faces do not move, so the time derivative can be taken outside the integral; by Eq. (5.1) the integral is $V_j \bar{c}_j$, and V_j is constant:

$$\frac{d}{dt} \int_{r_{j-1/2}}^{r_{j+1/2}} c \, 4\pi r^2 \, dr = \frac{d}{dt} \left(V_j \bar{c}_j \right) = V_j \frac{d\bar{c}_j}{dt}.$$

Step 4, right side. Use $4\pi r^2 = A(r)$, Eq. (1.5), and $D_j \, \partial c / \partial r = -J_r$, Eq. (1.4). Write $J_{j\pm1/2}$ for the flux J_r at face $j \pm 1/2$ (positive outward, in mol m^{-2} s^{-1}):

$$\left[4\pi r^2 D_j \frac{\partial c}{\partial r} \right]_{r_{j-1/2}}^{r_{j+1/2}} = -A_{j+1/2} J_{j+1/2} + A_{j-1/2} J_{j-1/2}.$$

Result.

$$V_j \frac{d\bar{c}_j}{dt} = A_{j-1/2} J_{j-1/2} - A_{j+1/2} J_{j+1/2}. \tag{5.6}$$

Classification: DERIVED NOT IMPLEMENTED. Eq. (5.6) is an exact identity, the starting point of the scheme; the code evaluates only its approximated, time-stepped form, Eq. (5.14).

Physical meaning. Each term has units m^3 · mol m^{-3} s^{-1} = mol s^{-1}. The first term on the right is the flow into the cell through the inner face (moles per second); the second is the flow out through the outer face. No approximation has been made yet: if we knew the two face fluxes exactly, Eq. (5.6) would give the exact cell averages. At an interface face the flux is the same seen from either layer, by Eq. (3.3), so $J_{j+1/2}$ is one number shared by cells j and $j+1$. The delivered Phase 1 document carried a source term $S_j V_j$ here (TRISO-FV-123); the code has none (Table 7.3, item I21), and a discrete source is a FUTURE IMPROVEMENT.

5.5 Face flux and the series conductance

Physical problem. Estimate the flow $A_{j+1/2} J_{j+1/2}$ through face $j+1/2$ from the two cell values C_j and C_{j+1} on either side.

Distances. The face lies between the centroid r_j of the inner cell and the centroid r_{j+1} of the outer cell. Define the two half-cell distances (in m):

$$\delta^L_{j+1/2} = r_{j+1/2} - r_j, \qquad \delta^R_{j+1/2} = r_{j+1} - r_{j+1/2}. \tag{5.7}$$

Classification: IMPLEMENTED, `dl` and `dr`, src/numerical/finite_volume.rs, lines 84 to 85. Both are positive because the centroid lies inside its cell. They are generally unequal, even inside one layer, because each centroid sits outward of its midpoint (Section 5.3.2): $\delta^L_{j+1/2}$ is the shorter "outer part" of cell j and $\delta^R_{j+1/2}$ the longer "inner part" of cell $j+1$. At the synthetic kernel/buffer face, $\delta^L = 2.478904 \times 10^{-6}$ m (kernel cell, width 5×10^{-6} m) and $\delta^R = 1.255176 \times 10^{-6}$ m (buffer cell, width 2.5×10^{-6} m) (check `P5R-09`).

Assumptions.

1. Across each half cell the concentration is a straight line in r. This is the approximation of the scheme; Section 5.3.2 showed it is consistent with placing C_j at the centroid.
2. The face carries one concentration $C_{j+1/2}$, the same on both sides (perfect contact, Eq. (3.2)).
3. The same flux crosses both halves (flux continuity, Eq. (3.3); inside a layer this is simply the fact that the face flux is one number).
4. Both half cells are evaluated with the face area $A_{j+1/2}$.

Step 1, flux across the inner half. With a straight line from C_j at r_j to $C_{j+1/2}$ at the face, the slope is $(C_{j+1/2} - C_j)/\delta^L_{j+1/2}$, and Fick's law, Eq. (1.4), with D_j gives

$$J_{j+1/2} = D_j \, \frac{C_j - C_{j+1/2}}{\delta^L_{j+1/2}}.$$

Step 2, flux across the outer half, with D_{j+1} and the slope from $C_{j+1/2}$ to C_{j+1}:

$$J_{j+1/2} = D_{j+1} \, \frac{C_{j+1/2} - C_{j+1}}{\delta^R_{j+1/2}}.$$

By assumption 3 the two expressions are equal, which gives the two-sided flux statement

$$J_{j+1/2} = D_j \, \frac{C_j - C_{j+1/2}}{\delta^L_{j+1/2}} = D_{j+1} \, \frac{C_{j+1/2} - C_{j+1}}{\delta^R_{j+1/2}}. \tag{5.8}$$

Classification: DERIVED NOT IMPLEMENTED. The face value $C_{j+1/2}$ is unknown and the code never forms it; it is eliminated next.

Step 3, solve each half for its concentration drop. Multiply the first equality by $\delta^L_{j+1/2}/D_j$ and the second by $\delta^R_{j+1/2}/D_{j+1}$:

$$J_{j+1/2} \, \frac{\delta^L_{j+1/2}}{D_j} = C_j - C_{j+1/2}, \qquad J_{j+1/2} \, \frac{\delta^R_{j+1/2}}{D_{j+1}} = C_{j+1/2} - C_{j+1}.$$

Step 4, add the two equations. The unknown $C_{j+1/2}$ appears once with each sign and cancels:

$$J_{j+1/2} \left(\frac{\delta^L_{j+1/2}}{D_j} + \frac{\delta^R_{j+1/2}}{D_{j+1}} \right) = C_j - C_{j+1}.$$

Step 5, divide by the bracket and multiply by the face area.

$$A_{j+1/2} J_{j+1/2} = G_{j+1/2}\,(C_j - C_{j+1}), \qquad G_{j+1/2} = \frac{A_{j+1/2}}{\delta^L_{j+1/2}/D_j + \delta^R_{j+1/2}/D_{j+1}}. \quad (5.9)$$

Classification: IMPLEMENTED, src/numerical/finite_volume.rs, lines 78 to 87, formula at line 86. The same formula is used at every internal face, inside a layer and across an interface; the code has no separate branch for either case. Check `P5-10` (symbolic elimination). Source label: adapted from published literature (the two-point and harmonic-mean interface conductances of Patankar, 1980), re-derived here for spherical shells with centroid distances (derived for STRATO).

Units. δ/D has units $\mathrm{m}/(\mathrm{m}^2\,\mathrm{s}^{-1}) = \mathrm{s\,m}^{-1}$, so $G = \mathrm{m}^2/(\mathrm{s\,m}^{-1}) = \mathrm{m}^3\,\mathrm{s}^{-1}$, and $G\,(C_j - C_{j+1})$ is in $\mathrm{m}^3\,\mathrm{s}^{-1} \cdot \mathrm{mol\,m}^{-3} = \mathrm{mol\,s}^{-1}$, a flow, as it must be.

Physical meaning: resistors in series. Read $\delta/(AD)$ as a *resistance* to diffusion and C as a "pressure". The flow through two pipes joined end to end is the pressure difference divided by the sum of their resistances; Eq. (5.9) says exactly this, with one "pipe" for each half cell. G is the inverse of the total resistance, a *conductance*. The face value, which we eliminated, can be recovered (DERIVED NOT IMPLEMENTED) as

$$C_{j+1/2} = \frac{(D_j/\delta^L_{j+1/2})\,C_j + (D_{j+1}/\delta^R_{j+1/2})\,C_{j+1}}{D_j/\delta^L_{j+1/2} + D_{j+1}/\delta^R_{j+1/2}},$$

a weighted average of the two cell values, weighted toward the better-conducting side (check `P5-11`). It always lies between C_j and C_{j+1}.

Special case 1: equal half distances gives the harmonic mean. If $\delta^L_{j+1/2} = \delta^R_{j+1/2} = \delta$, then

$$G_{j+1/2} = \frac{A_{j+1/2}}{\delta\,(1/D_j + 1/D_{j+1})} = \frac{A_{j+1/2}}{2\delta} \cdot \frac{2D_jD_{j+1}}{D_j + D_{j+1}},$$

so the face behaves like one material of diffusivity $D_h = 2D_jD_{j+1}/(D_j + D_{j+1})$, the *harmonic mean*, over the distance 2δ between centres (check `P5-12`; established textbook result, Patankar, 1980). In the code the two distances are generally unequal (Section 5.5, distances), so the harmonic mean appears only as a special case of the code's formula.

Special case 2: one material. If $D_{j+1} = D_j = D$, then $G_{j+1/2} = A_{j+1/2}D/(\delta^L + \delta^R) = A_{j+1/2}D/(r_{j+1} - r_j)$, the ordinary two-point difference between the centres (check `P5-13`).

Special case 3: a bottleneck. If one side conducts far less, say $D_{j+1} \to 0$, then $G_{j+1/2} \to A_{j+1/2}D_{j+1}/\delta^R_{j+1/2}$: the poorly conducting half alone controls the flow (check `P5-14`). At the synthetic kernel/buffer face, $D_1 = 10^{-12}$ and $D_2 = 4 \times 10^{-10}\ \mathrm{m}^2\,\mathrm{s}^{-1}$, so the kernel half dominates: $G = 2.025166 \times 10^{-13}\ \mathrm{m}^3\,\mathrm{s}^{-1}$ against

2.027730×10^{-13} m^3 s^{-1} for the kernel half alone (check `P5R-09`). An arithmetic mean of the two diffusivities would instead be dominated by the buffer and would overstate the flow by a factor of about 133 (check `P5R-09b`); this is why the series form is used.

Remark on assumption 4 (DERIVED NOT IMPLEMENTED; possible FUTURE IMPROVEMENT). In steady state, the exact resistance of a spherical shell between radii $r_P < r_E$ with constant D is $(1/r_P - 1/r_E)/(4\pi D)$, because the area changes across the shell. Eq. (5.9) uses the face area for both halves instead. Inside one material the ratio of the code's resistance to the exact one is

$$\frac{r_P r_E}{r_{j+1/2}^2} = 1 + \frac{\delta^R - \delta^L}{r_{j+1/2}} - \frac{\delta^L \delta^R}{r_{j+1/2}^2}$$

(check `P5-16`). To see the size of the two correction terms inside one layer of width w, note that each half distance differs from $w/2$ by a centroid shift of Section 5.3.2: $\delta^L_{j+1/2} = w/2 - s_j$ and $\delta^R_{j+1/2} = w/2 + s_{j+1}$, with $s \approx w^2/(6r)$. Hence $\delta^R - \delta^L = s_j + s_{j+1} \approx w^2/(3r_{j+1/2})$ and $\delta^L \delta^R \approx w^2/4$, so the ratio is

$$1 + \frac{w^2}{3r_{j+1/2}^2} - \frac{w^2}{4r_{j+1/2}^2} + O\left(\frac{w^4}{r^4}\right) = 1 + \frac{w^2}{12\, r_{j+1/2}^2} + O\left(\frac{w^4}{r^4}\right),$$

that is, 1 plus a term of order w^2/r^2 (evaluated from Eqs. (5.2) and (5.4) for this mesh: at the face between kernel cells 21 and 22, $r_{21+1/2} = 1.05 \times 10^{-4}$ m, the ratio minus one is 1.898×10^{-4} against $w^2/(12r^2) = 1.890 \times 10^{-4}$). This is consistent with second-order spatial accuracy for F (Part VI, Section 6.5). Across an interface the two widths differ, $\delta^R - \delta^L$ is of order w there, and the first-order terms no longer cancel; they affect only the four interface faces. Status: DEMONSTRATED NUMERICALLY that this costs no order on the synthetic case (Part VI, Table 6.3: observed spatial orders 2.00 on the synthetic input). The spherical half-cell resistance is not used by the code.

5.6 Centre and outer boundary

Centre. The inner face of cell 1 is at $r_{1/2} = 0$, where Eq. (5.5) gives $A_{1/2} = 4\pi \cdot 0^2 = 0$. A face of zero area carries no flow, whatever the concentrations, so the inner (west) conductance of the first cell is

$$g_1^W = 0. \tag{5.10}$$

Classification: IMPLEMENTED, src/numerical/finite_volume.rs, line 172 (`gw[0] = 0.0`; the same at line 243 in the diagnostic). This is the discrete form of the symmetry condition, Eq. (3.1): nothing crosses the centre, so no imaginary "ghost cell" beyond $r = 0$ is needed.

Outer surface. At $r = R$ the surface is a perfect sink, $c(R, t) = 0$, Eq. (3.4). The last cell, $j = N_c$, has its centroid r_{N_c} a distance $R - r_{N_c}$ inside the surface. Apply Eq. (5.9) to the outer face with these changes: the "outer cell" is the sink, with concentration 0,

and it sits *on* the face, so its half distance is $\delta^R = 0$; the inner half has $\delta^L = R - r_{N_c}$ and diffusivity D_5. Then

$$G_R = \frac{A(R)}{(R - r_{N_c})/D_5 + 0} = \frac{4\pi R^2 D_5}{R - r_{N_c}}, \qquad \Phi_R \approx G_R\,(C_{N_c} - 0) = G_R\,C_{N_c}. \quad (5.11)$$

Classification: IMPLEMENTED, src/numerical/finite_volume.rs, lines 88 to 91: R is the last input radius (line 89), the area is $4\pi R^2$ (line 90) and D_5 is the diffusivity of the last cell (line 91). The release rate $\Phi_R \approx G_R C_{N_c}$, in mol s^{-1}, is the discrete form of Eq. (3.7), evaluated at line 202 and line 210. Check `P5-15` confirms that Eq. (5.11) is Eq. (5.9) with $\delta^R = 0$. For the synthetic mesh $R - r_{N_c} = 6.244205 \times 10^{-7}$ m and $G_R = 2.852698 \times 10^{-12}$ m^3 s^{-1} (check `P5R-10`).

Physical meaning. The last half cell is a resistor connecting the last cell to a reservoir that is always empty. Everything that flows through it leaves the particle. The delivered Phase 1 document wrote the release rate as $G_R C_N / M^0$ (dimensionless, per second); the code stores $G_R C_{N_c}$ in mol s^{-1} (Table 7.3, item I1).

5.7 The semi-discrete system

Concept. "Semi-discrete" means discrete in space (a finite list of cells) but still continuous in time: one ordinary differential equation per cell. Eq. (5.6) holds for the exact cell average $\bar{c}_j$. From here on C_j denotes the numerical approximation of $\bar{c}_j$, and the face flows of Eq. (5.6) are replaced by the conductance form of Eq. (5.9).

Notation for the two conductances of cell j. Let g_j^W be the conductance of its inner (west) face and g_j^E that of its outer (east) face:

$$g_1^W = 0, \quad g_j^W = G_{j-1/2}\ (j \geq 2); \qquad g_j^E = G_{j+1/2}\ (j < N_c), \quad g_{N_c}^E = G_R.$$

Because a face is shared, $g_{j+1}^W = g_j^E$ for $j < N_c$. These are `gw[i]` and `ge[i]` in src/numerical/finite_volume.rs, lines 172 to 177.

Step 1, inner face. By Eq. (5.9) applied to face $j - 1/2$ (inner cell $j-1$, outer cell j): $A_{j-1/2} J_{j-1/2} = g_j^W (C_{j-1} - C_j)$. For $j = 1$ this is zero by Eq. (5.10).

Step 2, outer face. By Eq. (5.9) applied to face $j + 1/2$: $A_{j+1/2} J_{j+1/2} = g_j^E (C_j - C_{j+1})$, and therefore $-A_{j+1/2} J_{j+1/2} = g_j^E (C_{j+1} - C_j)$. For $j = N_c$ this is Eq. (5.11) if we set $C_{N_c+1} = 0$, the sink value.

Step 3, insert both into Eq. (5.6).

$$V_j \frac{dC_j}{dt} = g_j^W\,(C_{j-1} - C_j) + g_j^E\,(C_{j+1} - C_j), \qquad j = 1, \ldots, N_c, \quad C_{N_c+1} = 0. \quad (5.12)$$

Classification: DERIVED NOT IMPLEMENTED as a separate computation (the code has no ordinary-differential-equation integrator); it is the basis of the time-stepped form Eq. (5.14), which is IMPLEMENTED. Check P5-17.

Physical meaning. Picture a row of water tanks of volume V_j joined by pipes of conductance g. The level in each tank rises if its neighbours are fuller and falls if they are emptier, at a rate set by the pipe conductances. The first tank has no pipe on its inner side; the last tank drains into an empty reservoir through the pipe G_R.

5.8 Implicit time stepping (Backward Euler)

5.8.1 Explicit and implicit steps on a simple example

The question. Eq. (5.12) tells us the rate of change *now*. To move from time t^n to $t^{n+1} = t^n + \Delta t$ we must decide *which* concentrations to use for the rate during the step.

Test problem. A single quantity decaying at rate $\mu > 0$ (s^{-1}): $dy/dt = -\mu\, y$, $y(0) = 1$. The exact solution is $y = e^{-\mu t}$, which shrinks by the factor $e^{-\mu \Delta t}$ each step and never changes sign.

Explicit (forward Euler) step. Use the rate at the *start* of the step: $(y^{n+1} - y^n)/\Delta t = -\mu\, y^n$, so

$$y^{n+1} = (1 - \mu \Delta t)\, y^n.$$

The new value is given *explicitly* by the old one, which is cheap. But if $\mu \Delta t > 1$ the factor is negative, and if $\mu \Delta t > 2$ its size exceeds 1. With $\mu = 1\ \mathrm{s}^{-1}$ and $\Delta t = 3$ s the sequence is $1, -2, 4, -8$: a decaying quantity becomes negative and grows (check P5-28).

Implicit (Backward Euler) step. Use the rate at the *end* of the step: $(y^{n+1} - y^n)/\Delta t = -\mu\, y^{n+1}$. The unknown y^{n+1} now appears on both sides, so it is defined *implicitly*; collect it on the left, $y^{n+1}(1 + \mu \Delta t) = y^n$, and divide:

$$y^{n+1} = \frac{y^n}{1 + \mu \Delta t}.$$

The factor lies between 0 and 1 for every $\Delta t > 0$. With the same numbers the sequence is $1, 0.25, 0.0625, 0.015625$, against the exact factor $e^{-3} = 0.049787$ per step (check P5-29): too slow a decay at this large step, but positive, shrinking and stable. Expanding, $1/(1 + x) - e^{-x} = x^2/2 + O(x^3)$ with $x = \mu \Delta t$ (check P5-30), so each step makes an error of order Δt^2 and the accumulated error after a fixed time is of order Δt: Backward Euler is *first-order accurate in time* (Part VI, Section 6.5).

Why this matters for Code B. Each cell of Eq. (5.12) behaves like such a decay with a rate of about $(g_j^W + g_j^E)/V_j$. For the synthetic mesh the largest of these rates is the inverse of the forward Euler bound of Part VI, Eq. (6.6): $1/\Delta t_{\mathrm{FE}}$ with $\Delta t_{\mathrm{FE}} = 7.812107958751 \times 10^{-3}$ s (check P5R-11). The step actually used is 200 s, about 2.56×10^4 times larger. An explicit scheme would need about 2.6×10^7 steps to reach 2×10^5 s;

Backward Euler takes 1000. The code uses Backward Euler only; no explicit scheme exists in v1.0.0 (Table 7.3, item I6). The bound Δt_{FE} itself is computed as a diagnostic by `explicit_dt_bound`, src/numerical/finite_volume.rs, lines 238 to 252, and printed by src/main.rs, line 78; it does not influence the time step.

5.8.2 Backward Euler for every cell

Step 1. Replace dC_j/dt in Eq. (5.12) by the difference quotient over one step, and evaluate the right side with the new values C^{n+1}:

$$V_j \frac{C_j^{n+1} - C_j^n}{\Delta t} = g_j^W \left(C_{j-1}^{n+1} - C_j^{n+1}\right) + g_j^E \left(C_{j+1}^{n+1} - C_j^{n+1}\right), \qquad C_{N_c+1}^{n+1} = 0. \quad (5.13)$$

Classification: IMPLEMENTED, in the rearranged form below; see the doc comment of the solver, src/numerical/finite_volume.rs, lines 122 to 124, and `backward_euler_step`, lines 162 to 182.

Step 2, name the storage coefficient. Let $v_j = V_j/\Delta t$, in $\mathrm{m}^3\ \mathrm{s}^{-1}$, the same units as a conductance (code `v`, line 178). The left side of Eq. (5.13) is $v_j C_j^{n+1} - v_j C_j^n$.

Step 3, move every unknown to the left. Expand the right side, $g_j^W C_{j-1}^{n+1} - g_j^W C_j^{n+1} + g_j^E C_{j+1}^{n+1} - g_j^E C_j^{n+1}$, subtract it from both sides, and add $v_j C_j^n$ to both sides:

$$-g_j^W\, C_{j-1}^{n+1} + \left(v_j + g_j^W + g_j^E\right) C_j^{n+1} - g_j^E\, C_{j+1}^{n+1} = v_j\, C_j^n = \mathrm{rhs}_j. \quad (5.14)$$

Classification: IMPLEMENTED. The coefficients are assembled in src/numerical/finite_volume.rs, lines 171 to 180 (`gw`, `ge`, `v`, and `rhs = v * old` at line 179) and the rows are solved by `solve_diffusion_system`, lines 132 to 160. Check `P5-18` (Eq. (5.13) rearranges to Eq. (5.14)). The right side contains no source term (Table 7.3, item I21).

The first and the last cell written out. For $j = 1$, Eq. (5.10) removes the first term:

$$\left(v_1 + g_1^E\right) C_1^{n+1} - g_1^E\, C_2^{n+1} = v_1 C_1^n.$$

For $j = N_c$, $g_{N_c}^E = G_R$ and the sink value $C_{N_c+1}^{n+1} = 0$ removes the last term, while G_R stays on the diagonal:

$$-g_{N_c}^W\, C_{N_c-1}^{n+1} + \left(v_{N_c} + g_{N_c}^W + G_R\right) C_{N_c}^{n+1} = v_{N_c} C_{N_c}^n.$$

Matrix form. Collect the N_c equations. Let $\mathbf{C}^n$ be the column of cell values, $\mathbf{V} = \mathrm{diag}(V_1, \ldots, V_{N_c})$, and let $\mathbf{A}$ be the conductance matrix with entries $A_{jj} = g_j^W + g_j^E$, $A_{j,j-1} = -g_j^W$, $A_{j,j+1} = -g_j^E$ and zeros elsewhere. Then

$$\left(\frac{\mathbf{V}}{\Delta t} + \mathbf{A}\right) \mathbf{C}^{n+1} = \frac{\mathbf{V}}{\Delta t}\, \mathbf{C}^n. \quad (5.15)$$

Classification: IMPLEMENTED implicitly. No matrix is stored; the code keeps only the three vectors `v`, `gw`, `ge` that define it (lines 167 to 180). For later use write $\mathbf{B} = \mathbf{V}/\Delta t + \mathbf{A}$, with entries $B_{jj} = v_j + g_j^W + g_j^E$, $B_{j,j-1} = -g_j^W$, $B_{j,j+1} = -g_j^E$.

5.9 The time step actually used

Problem. The input gives a requested step Δt_{in} (`dt_s`) and an end time t_{end} (`end_time_s`). If t_{end} is not a whole multiple of Δt_{in}, equal steps of Δt_{in} would overshoot or miss the end time.

Rule. Take the smallest whole number N_t of equal steps whose size does not exceed the requested size, then shrink the step so that N_t steps end exactly at t_{end}:

$$N_t = \left\lceil \frac{t_{\text{end}}}{\Delta t_{\text{in}}} \right\rceil, \qquad \Delta t = \frac{t_{\text{end}}}{N_t} \le \Delta t_{\text{in}}. \tag{5.16}$$

Here $\lceil x \rceil$ is the ceiling, the smallest integer $\ge x$. Classification: IMPLEMENTED, src/numerical/finite_volume.rs, lines 190 to 191. The delivered Phase 1 documents used Δt_{in} directly (Table 7.3, item I3).

Synthetic input. $t_{\text{end}}/\Delta t_{\text{in}} = 2 \times 10^5/200 = 1000$ exactly, so $N_t = 1000$ and $\Delta t = 200$ s $= \Delta t_{\text{in}}$ (checks `P5-31`, `P5R-14`).

Floating-point caveat (DERIVED NOT IMPLEMENTED; FUTURE IMPROVEMENT). The quotient is computed in floating point before the ceiling is taken. When the true quotient is a whole number but the computed one lands just above it, the ceiling adds one step. Example: $t_{\text{end}} = 2.1$ s, $\Delta t_{\text{in}} = 0.3$ s gives the computed quotient 7.000000000000001, so $N_t = 8$ and $\Delta t = 0.2625$ s instead of 7 steps of 0.3 s (check `P5-32`). The result is still a valid, slightly finer time grid; it matters only where exact step ratios are assumed, as in the Richardson combination and the temporal order of Part VI (Table 7.3, items K2 and K3). A tolerance before the ceiling would remove it.

5.10 The tridiagonal matrix and its solution

5.10.1 What the matrix looks like, and where each entry comes from

Concept: tridiagonal matrix. Each row of Eq. (5.14) involves only cell j and its two neighbours, so in the matrix $\mathbf{B}$ every row has at most three nonzero numbers: one on the main diagonal and one on each side of it. All other entries are zero. Such a matrix is called *tridiagonal*. For $N_c = 4$:

$$\mathbf{B} = \begin{pmatrix} v_1 + g_1^E & -g_1^E & 0 & 0 \\ -g_2^W & v_2 + g_2^W + g_2^E & -g_2^E & 0 \\ 0 & -g_3^W & v_3 + g_3^W + g_3^E & -g_3^E \\ 0 & 0 & -g_4^W & v_4 + g_4^W + G_R \end{pmatrix}.$$

Because $g_{j+1}^W = g_j^E$, the entry below the diagonal in row $j+1$ equals the entry above it in row j: the matrix is *symmetric* (check `P5-19`, symbolic; `P5R-12`, synthetic mesh). Table 5.1 lists every entry of $\mathbf{B}$ with its physical origin and its place in the code.

Table 5.1 Entries of the Backward Euler matrix **B** of Eq. (5.15) and the physical term each comes from; line numbers refer to src/numerical/finite_volume.rs.

Entry of **B**	Value	Physical origin	Code
diagonal, storage part	$v_j = V_j/\Delta t$	the amount already in cell j, carried into the new step	`v[i]`, line 178
diagonal, inner face part	g_j^W	flow lost to the inner neighbour when C_j rises	`gw[i]`, line 172
diagonal, outer face part	g_j^E (G_R for $j = N_c$)	flow lost to the outer neighbour, or to the sink	`ge[i]`, lines 173 to 177
below the diagonal	$-g_j^W$	flow gained from the inner neighbour	`gw[i]`
above the diagonal	$-g_j^E$	flow gained from the outer neighbour	`ge[i]`
right side	$\text{rhs}_j = v_j C_j^n$	the old content of the cell	`rhs[i]`, line 179

Each row's diagonal exceeds the sum of the sizes of its off-diagonal entries by exactly v_j (plus G_R in the last row) (checks `P5-20`, `P5R-13`). This margin is what Part VI, Section 6.2, uses to show that the solution is never negative.

5.10.2 Gaussian elimination for a tridiagonal system

Idea. Gaussian elimination removes one unknown at a time. For a tridiagonal system this takes one pass downward (each row loses its link to the previous unknown) and one pass upward (each unknown is read off from the next one). This specialisation is often called the Thomas algorithm (established textbook result; see Patankar, 1980).

Step 1, first row. Write $d_1 = B_{11} = v_1 + g_1^E$ and divide row 1 by it:

$$C_1^{n+1} = y_1 + \frac{g_1^E}{d_1}\, C_2^{n+1}, \qquad y_1 = \frac{\text{rhs}_1}{d_1}.$$

Step 2, the general row. Suppose row $j-1$ has already been reduced to $C_{j-1}^{n+1} = y_{j-1} + (g_{j-1}^E/d_{j-1})\, C_j^{n+1}$. Row j of Eq. (5.14) is

$$-g_j^W\, C_{j-1}^{n+1} + B_{jj}\, C_j^{n+1} - g_j^E\, C_{j+1}^{n+1} = \text{rhs}_j.$$

Substitute the reduced row $j-1$ for C_{j-1}^{n+1}:

$$-g_j^W\, y_{j-1} - \frac{g_j^W\, g_{j-1}^E}{d_{j-1}}\, C_j^{n+1} + B_{jj}\, C_j^{n+1} - g_j^E\, C_{j+1}^{n+1} = \text{rhs}_j.$$

Collect the C_j^{n+1} terms and move the known term $-g_j^W y_{j-1}$ to the right:

$$\left(B_{jj} - \frac{g_j^W\, g_{j-1}^E}{d_{j-1}} \right) C_j^{n+1} - g_j^E\, C_{j+1}^{n+1} = \text{rhs}_j + g_j^W\, y_{j-1}.$$

Call the bracket d_j, the *pivot*, and divide by it. Row j becomes $C_j^{n+1} = y_j + (g_j^E/d_j)\, C_{j+1}^{n+1}$ with $y_j = (\text{rhs}_j + g_j^W y_{j-1})/d_j$, which has the same form as the assumption, so the step repeats down to $j = N_c$.

Step 3, the textbook pivot in conductances. Insert $B_{jj} = v_j + g_j^W + g_j^E$ and use $g_{j-1}^E = g_j^W$ (shared face):

$$d_1 = v_1 + g_1^E, \qquad d_j = v_j + g_j^W + g_j^E - \frac{\left(g_j^W\right)^2}{d_{j-1}} \quad (j \geq 2). \tag{5.17}$$

Classification: DERIVED NOT IMPLEMENTED. This is the textbook pivot; the code computes the same numbers by Eq. (5.18) instead (check `P5-22` for the form, `P5-21` for the equality).

Step 4, the last row and the upward pass. Row N_c has no $C_{N_c+1}^{n+1}$ term (the sink value is zero), so after reduction $C_{N_c}^{n+1} = y_{N_c}$. Then every earlier unknown follows from the one after it, from $j = N_c - 1$ down to 1, by the reduced rows.

5.10.3 The subtraction-free form actually coded

The difficulty with Eq. (5.17). It contains a minus sign. Subtracting two nearly equal numbers on a computer loses the leading digits they share, a loss called *cancellation*. Decimal example with 6 significant digits: $1.00004 - 1.00001 = 0.00003$ has only one meaningful digit left, although both inputs had six. In Eq. (5.17) this happens when the conductances are much larger than the storage term, $g \gg v$, that is when the cell Fourier number $\mathrm{Fo} = D\Delta t/w^2$ (Part VI, Eq. (6.13)) is large. Then $d_{j-1} \approx g_j^W + e_{j-1}$ with a small e_{j-1}, the subtracted term $(g_j^W)^2/d_{j-1}$ is close to g_j^W, and the small, physically important part of the pivot is obtained as the difference of numbers of size g.

Step 1, name the part of the pivot that does not belong to the outer face. Define $e_j = d_j - g_j^E$, in $\mathrm{m^3\ s^{-1}}$. Then $d_j = e_j + g_j^E$, and for the previous row, using $g_{j-1}^E = g_j^W$,

$$d_{j-1} = e_{j-1} + g_{j-1}^E = e_{j-1} + g_j^W.$$

Step 2, rewrite Eq. (5.17) for e_j. Subtract g_j^E from both sides of Eq. (5.17) and insert the expression for d_{j-1}:

$$e_j = v_j + g_j^W - \frac{\left(g_j^W\right)^2}{e_{j-1} + g_j^W}.$$

Step 3, combine the last two terms over one denominator. Factor out g_j^W:

$$g_j^W - \frac{\left(g_j^W\right)^2}{e_{j-1} + g_j^W} = g_j^W \, \frac{\left(e_{j-1} + g_j^W\right) - g_j^W}{e_{j-1} + g_j^W} = \frac{g_j^W \, e_{j-1}}{e_{j-1} + g_j^W}.$$

The subtraction $\left(e_{j-1} + g_j^W\right) - g_j^W = e_{j-1}$ has been done *exactly, on paper*, once and for all.

Step 4, first row. $e_1 = d_1 - g_1^E = v_1$.

Result.

$$e_1 = v_1, \qquad e_j = v_j + \frac{g_j^W\, e_{j-1}}{e_{j-1} + g_j^W} \quad (j \geq 2), \qquad d_j = e_j + g_j^E. \tag{5.18}$$

Classification: IMPLEMENTED, src/numerical/finite_volume.rs, lines 141 to 146 (`e`, `den`; the previous `e` is kept in `e_prev`, line 152). Check `P5-21`: Eq. (5.18) equals the textbook pivot Eq. (5.17) for every row of a symbolic 5×5 system. The first row needs its own branch (lines 141 to 142) because the general formula with $g_1^W = 0$ and the starting value `e_prev = 0` would evaluate $0 \cdot 0/(0 + 0)$, which is undefined (check `P5-25`). The code stops with an error if a pivot is not positive and finite (lines 147 to 149); by Eq. (5.18) every $e_j \geq v_j > 0$, so this cannot happen for positive, finite volumes and conductances, and the guard only catches invalid input or overflow.

Physical meaning: a ladder of resistors. $g\,e/(g + e) = 1/(1/g + 1/e)$ is the formula for two conductances in series. So e_j is the storage conductance v_j of cell j in parallel with the face g_j^W in series with everything inward of it, e_{j-1}. It is the effective conductance that cell j "sees" when it looks toward the centre, and d_j adds the outer face. Every quantity is a sum, product or quotient of positive numbers.

The two sweeps. With the signs of Eq. (5.14) the forward and backward passes of Section 5.10.2 are

$$\begin{aligned} y_1 &= \frac{\text{rhs}_1}{d_1}, \qquad y_j = \frac{\text{rhs}_j + g_j^W\, y_{j-1}}{d_j} \quad (j = 2, \dots, N_c), \\ C_{N_c}^{n+1} &= y_{N_c}, \quad C_j^{n+1} = y_j + \frac{g_j^E}{d_j}\, C_{j+1}^{n+1} \quad (j = N_c - 1, \dots, 1). \end{aligned} \tag{5.19}$$

Classification: IMPLEMENTED, forward sweep src/numerical/finite_volume.rs, lines 150 to 151 (`carry` $= g_j^W y_{j-1}$, `dp` $= y_j$), back substitution lines 154 to 158. Check `P5-23`: Eqs. (5.18) and (5.19) solve a symbolic 5×5 system of the form (5.14) exactly; check `P5R-12`: on the synthetic mesh the sweep agrees with a dense solve to 4.3×10^{-15} mol m^{-3}. Status of "the sweep reproduces the dense solve": PROVED in exact arithmetic (check `P5-23`), DEMONSTRATED NUMERICALLY in floating point. An independent re-check, made while this document was prepared and not stored in the repository (Section 7.5), repeats this with a general dense LU solver: on the synthetic mesh ($\Delta t = 200$ s, five random nonnegative old states) the largest relative difference is 7.2×10^{-14}, and on the unit-test mesh at Fo $= 2.5 \times 10^5$ the sweep agrees with a 60-digit dense solve to a relative 3.7×10^{-14}.

Why cancellation cannot occur. Every operation in Eqs. (5.18) and (5.19) is an addition, multiplication or division, and when $C^n \geq 0$ every operand is nonnegative (check `P5-24`: no minus sign appears anywhere in the symbolic expressions). Adding

nonnegative numbers never cancels leading digits, so each rounding error stays relative to the size of the quantity being computed, whatever the Fourier number. Status: "no subtraction occurs" is PROVED (check `P5-24`); the resulting accuracy bound is sketched in Part VI, Section 6.8, whose constants are CLAIMED ONLY, and the accuracy is DEMONSTRATED NUMERICALLY below.

A small demonstration with $v = 1$ and $g = 10^6$ m^3 s^{-1} (all faces equal, 30 rows, exact rational arithmetic as reference): Eq. (5.18) gives e_j with relative error at most about 2×10^{-16} (one unit in the last place), while e_j recovered from the textbook pivot Eq. (5.17) as $d_j - g_j^E$ has relative error up to 1.7×10^{-11} (checks `P5-26`, `P5-27`). The textbook pivot itself is accurate relative to its size, which is about g; the information lost is in its small storage part.

How much of this loss reaches the solution and the balance is a separate question, and the measured answer is: little. At Fo $= 2.5 \times 10^5$ (the unit-test mesh, one step) both the textbook and the subtraction-free sweep agree with a 60-digit dense solve to a relative 3×10^{-14} to 4×10^{-14} (independent replica, not stored in the repository, Section 7.5). Over the 100 steps of the unit test, the worst relative balance residual is of order 10^{-16} to 10^{-15} with the subtraction-free pivot and of order 10^{-14} to 10^{-13} with the textbook pivot; the exact digits depend on the order of floating-point operations in each Python replica (Part VI, Section 6.8, and the independent replicas report different values within these ranges). Both are far below the test threshold 10^{-12}, and the effect is much smaller than the code comment at lines 126 to 131 suggests (Table 7.3, items I5 and K4). The guaranteed benefits of Eq. (5.18) are its Fourier-number-independent pivot accuracy and the exact nonnegativity of the computed solution (Part VI, Section 6.3).

5.11 Inventory, release, balance and release fraction

5.11.1 Initial state and initial inventory

Initial cell values. Eq. (3.5) puts $c = c_0$ in the kernel and $c = 0$ in the coatings. Since the face R_1 is a cell face (Section 5.2), every cell lies wholly in one region, and Eq. (5.1) gives the cell averages exactly. The initial inventory is the sum of the cell amounts:

$$C_j^0 = \begin{cases} c_0, & \text{cell } j \text{ in layer 1,} \\ 0, & \text{otherwise,} \end{cases} \qquad M^0 = \sum_{j=1}^{N_c} V_j \, C_j^0. \tag{5.20}$$

Classification: IMPLEMENTED, `initial_concentration`, src/numerical/finite_volume.rs, lines 101 to 112, and `initial = inventory(...)` at line 186; the run stops if $M^0 \le 0$ (lines 187 to 189). In exact arithmetic the kernel volumes telescope to $\frac{4\pi}{3} R_1^3$, so M^0 equals Eq. (3.6). The code uses the summed cell volumes, not Eq. (3.6): for the synthetic input $M^0 = 3.351032163829113 \times 10^{-11}$ mol, against $3.351032163829112 \times 10^{-11}$ mol from Eq. (3.6) evaluated with a library power R_1^3, a round-off difference in the last digit (check `P5R-08`; evaluating R_1^3 as $R_1 R_1 R_1$ instead gives $3.351032163829113 \times 10^{-11}$ mol, so the last digit depends only on the order of operations). Using the summed volumes keeps M^0 consistent with the inventory of Eq. (5.21), which is what the balance check needs.

5.11.2 Inventory at each step

$$M^n = \sum_{j=1}^{N_c} V_j \, C_j^n. \tag{5.21}$$

Classification: IMPLEMENTED, `inventory`, src/numerical/finite_volume.rs, lines 114 to 120 (a plain sum in cell order), called after every step at line 219. This is the discrete form of Eq. (1.7), exact for cell averages by Section 5.1.

5.11.3 Release through the outer face

Release rate. After the step, the flow through the outer face is evaluated with the new last-cell value, the same value Backward Euler used in the last row of Eq. (5.14):

$$\Phi_R^{n+1} = G_R \, C_{N_c}^{n+1}, \qquad M_{\text{rel}}^{n+1} = M_{\text{rel}}^n + \Delta t \, \Phi_R^{n+1}, \qquad M_{\text{rel}}^0 = 0. \tag{5.22}$$

Classification: IMPLEMENTED, rate at src/numerical/finite_volume.rs, line 210, increment $\Delta t \, \Phi_R^{n+1}$ at line 213, accumulated with compensation as in Eq. (5.23). The row at $t = 0$ reports $J_R^0 = G_R C_{N_c}^0$ (line 202), which is 0 for the kernel-only start (check `P5R-17`).

Why this choice closes the balance. The term $G_R C_{N_c}^{n+1}$ is exactly the amount per second that the last row of Eq. (5.14) removes from the last cell. When all rows of Eq. (5.14) are added, every internal face appears twice with opposite signs and cancels, leaving $M^{n+1} + \Delta t \, G_R C_{N_c}^{n+1} = M^n$; this is derived in full in Part VI, Eq. (6.1). Booking the same quantity as released therefore makes $M^n + M_{\text{rel}}^n = M^0$ hold in exact arithmetic. Status: PROVED in exact arithmetic (Part VI, Eq. (6.1); the cancellation is the statement that every column of the conductance matrix $\mathbf{A}$ sums to zero except the last, which sums to G_R, checked symbolically by `v1_0_0_p6_properties.py`). In floating point it holds to round-off, DEMONSTRATED NUMERICALLY in Section 5.11.5.

5.11.4 Compensated (Kahan) summation of the release

The problem. M_{rel} is a running total of N_t small increments. A computer stores about 16 significant digits. When a small number is added to a much larger total, the digits of the small number that fall below the last stored digit of the total are rounded away. Over many steps these losses accumulate. An extreme demonstration: adding 10^{-16} to 1.0 ten thousand times leaves a plain floating-point sum at exactly 1.0, because each increment is below half the spacing of numbers near 1 (check `P5-33`).

The remedy. Keep a second variable κ (code `release_compensation`, in mol) that holds the negative of what was rounded away, and feed it back into the next addition. With $\kappa = 0$ initially, each step performs

$$\begin{aligned} \upsilon &= \Delta t \, \Phi_R^{n+1} - \kappa, \\ \Sigma &= M_{\text{rel}}^n + \upsilon, \\ \kappa &\leftarrow (\Sigma - M_{\text{rel}}^n) - \upsilon, \\ M_{\text{rel}}^{n+1} &\leftarrow \Sigma. \end{aligned} \tag{5.23}$$

Classification: IMPLEMENTED, src/numerical/finite_volume.rs, lines 211 to 217 (the variable called `t` in the code is Σ here). Established result: compensated summation (Kahan, 1965).

Why it works, line by line. Line 1 corrects the new increment by the error left over from the previous addition. Line 2 is the ordinary, rounded addition. In line 3, $\Sigma - M^n_{\mathrm{rel}}$ is the part of υ that actually made it into the total; subtracting υ leaves minus the part that was lost. In binary floating point with round-to-nearest, both subtractions of line 3 are themselves exact whenever the running total is at least as large in size as the new term, $|M^n_{\mathrm{rel}}| \geq |\upsilon|$ (established result of floating-point error analysis; not proved here). That is the usual situation after the first few steps; in the first few steps, while the release rate is still rising, the increment can exceed the total and the correction is then only approximate. The rounding errors made there are relative to the small early totals, and the synthetic run shows no visible effect (check `P5R-19` below). In exact arithmetic κ would always be 0 and Eq. (5.23) reduces to Eq. (5.22). In the demonstration above the compensated sum is 1.000000000001, the exact value (check `P5-33`). For the synthetic run the plain and compensated totals differ by only 4.3×10^{-16} of M^0 (check `P5R-19`), so the compensation is a safeguard for long runs rather than a visible correction here. The delivered Phase 1 documents showed the plain sum only (Table 7.3, item I2).

5.11.5 Balance residual and release fraction

$$\varepsilon^n = M^n + M^n_{\mathrm{rel}} - M^0, \qquad F^n = \frac{M^n_{\mathrm{rel}}}{M^0}. \tag{5.24}$$

Classification: IMPLEMENTED, residual at src/numerical/finite_volume.rs, line 220 (every step, stored on output rows), release fraction at line 227. Both are in the units shown: ε^n in mol, F^n dimensionless.

Meaning of the residual. M^n is computed from the cell values; M^n_{rel} is accumulated from the outer-face flow alone. The two are computed independently, so their sum minus M^0 measures everything the arithmetic lost or created: solver round-off, summation round-off and any coding error. By Section 5.11.3 it is zero in exact arithmetic. For the synthetic run, the largest $|\varepsilon^n|/M^0$ over all 1000 steps is 2.81×10^{-13} (check `P5R-16`) and equal to the 2.806×10^{-13} read from the v1.0.0 output file `tests/golden/v1.0/verify_stdout.txt`. Status: DEMONSTRATED NUMERICALLY for this input; no general floating-point bound on ε^n is proved in this part.

Meaning of the release fraction. F^n is the discrete Eq. (3.9). In exact arithmetic $F^n = 1 - M^n/M^0$ as well, by the balance; the code uses the release form (the alternative $1 - M^n/M^0$ is DERIVED NOT IMPLEMENTED). For the synthetic input (200 cells, 1000 steps) the replica gives $F(2 \times 10^5 \text{ s}) = 0.43242860687853$, matching the main-run value 0.4324286068785 (check `P5R-15`). The series of Part IV gives 0.43255563909304; the difference of 1.27×10^{-4} is the discretisation error analysed in Part VI and compared in Part VII.

5.11.6 Output rows

The code writes a row at $t = 0$ and then after step n whenever

$$n \bmod n_{\mathrm{out}} = 0 \quad \text{or} \quad n = N_t, \qquad t_n = n\,\Delta t, \tag{5.25}$$

with the values M^n, M^n_{rel}, F^n, J^n_R, ε^n and $\min_j C^n_j$. Classification: IMPLEMENTED, src/numerical/finite_volume.rs, lines 196 to 206 (row at $t = 0$) and lines 221 to 233 (later rows; time $n\,\Delta t$ at line 223). The minimum concentration is only recorded, not checked, inside `run`; the checks that it is nonnegative are assertions in the verification driver and the unit tests (Table 7.3, item I7). For the synthetic input ($n_{\mathrm{out}} = 10$) there are 101 rows and $\min_j C^n_j \geq 0$ holds on every row with no tolerance (checks `P5R-14`, `P5R-18`).

5.12 Summary and verification

Table 5.2 collects the numbers quoted in this part, and Table 5.3 classifies every equation against the Rust source.

Table 5.2 Numbers quoted in Part V, all calculated from the synthetic five-layer input (synthetic values, not literature data) by the Python replica of Code B.

Quantity	Value	Source
cells, widths w_1 to w_5	200; 5.0×10^{-6}, 2.5×10^{-6}, 1.25×10^{-6} (three layers) m	`P5R-01`
r_1, r_2	3.75×10^{-6}, 8.0357×10^{-6} m	`P5R-05`, `P5R-06`
δ^L, δ^R, G at the kernel/buffer face	2.478904×10^{-6}, 1.255176×10^{-6} m; 2.025166×10^{-13} m^3 s^{-1}	`P5R-09`
$R - r_{N_c}$, G_R	6.244205×10^{-7} m, 2.852698×10^{-12} m^3 s^{-1}	`P5R-10`
Δt_{FE}, $\Delta t / \Delta t_{\mathrm{FE}}$	$7.812107958751 \times 10^{-3}$ s, 2.56×10^4	`P5R-11`
N_t, Δt	1000, 200 s	`P5R-14`
M^0	$3.351032163829113 \times 10^{-11}$ mol	`P5R-08`
worst $\lvert\varepsilon^n\rvert / M^0$	2.81×10^{-13}	`P5R-16`
$F(2 \times 10^5$ s)	0.43242860687853	`P5R-15`

Table 5.3 Classification of the Part V equations against src/numerical/finite_volume.rs at tag v1.0.0.

Eq.	Content	Classification	Code (finite_volume.rs)
(5.1)	cell average	DERIVED NOT IMPLEMENTED (meaning of c)	none
(5.2)	mesh faces, snapped last face	IMPLEMENTED	55 to 62, 72
(5.3)	cell volume	IMPLEMENTED	38 to 40, 67
(5.4)	volume centroid	IMPLEMENTED	42 to 48, 66
(5.5)	face area	IMPLEMENTED	82 to 83, 90
(5.6)	exact cell balance	DERIVED NOT IMPLEMENTED	none
(5.7)	half-cell distances	IMPLEMENTED	84 to 85
(5.8)	two-sided face flux	DERIVED NOT IMPLEMENTED (eliminated)	none
(5.9)	series conductance	IMPLEMENTED	78 to 87
(5.10)	centre, $g^W_1 = 0$	IMPLEMENTED	172
(5.11)	outer conductance, release rate	IMPLEMENTED	88 to 91, 202, 210
(5.12)	semi-discrete system	DERIVED NOT IMPLEMENTED (basis of 5.14)	none

Eq.	Content	Classification	Code (finite_ volume.rs)
(5.13), (5.14)	Backward Euler row	IMPLEMENTED	122 to 124, 162 to 182
(5.15)	matrix form	IMPLEMENTED implicitly (three vectors)	167 to 180
(5.16)	time step rule	IMPLEMENTED	190 to 191
(5.17)	textbook pivot	DERIVED NOT IMPLEMENTED	none
(5.18)	subtraction-free pivot	IMPLEMENTED	141 to 149
(5.19)	sweeps	IMPLEMENTED	150 to 158
(5.20), (5.21)	initial state, inventory	IMPLEMENTED	101 to 120, 186, 219
(5.22), (5.23)	release, Kahan sum	IMPLEMENTED	210 to 217
(5.24)	residual, release fraction	IMPLEMENTED	220, 227
(5.25)	output rule	IMPLEMENTED	196 to 206, 221 to 233
none	source term, decay, $D(T)$, Robin outer face, spherical half-cell resistance, tolerance in the step rule	FUTURE IMPROVEMENT	none

Verification. The script `scripts/derivation_checks/v1_0_0_p5_fv_scheme.py` has 33 checks, `P5-01` to `P5-33` (SymPy derivations of Eqs. (5.3) to (5.19), exact-rational demonstrations of cancellation, of the step rule and of Kahan summation, and the decay example). The script `scripts/derivation_checks/v1_0_0_p5_fv_replica.py` has 22 checks, `P5R-01` to `P5R-19` together with `P5R-03b`, `P5R-09b` and `P5R-09c` (a line-by-line Python replica of Code B on the synthetic input). Appendix A lists both with the run command. Figure 5.1 is drawn by `scripts/figures/v1_0_0_control_volume.py`.

Part VI: Numerical properties and stability

Part V built the discrete method of Code B: cell averages C_j, the conductances $G_{j+1/2}$ and G_R, the Backward Euler (BE) system (5.14) and the subtraction-free tridiagonal solve (5.18), (5.19). This part asks what that method guarantees. Each property below applies to the implemented method only, and each carries one status label (see How to read this document).

All numbers are calculated values for the synthetic five-layer input `inputs/synthetic/synthetic_five_layer.toml` (40 cells per layer, $N_c = 200$, $\Delta t = 200$ s, $t_{\text{end}} = 2 \times 10^5$ s). They are not literature data.

Four scripts check this part. Each is run as `.venv/Scripts/python.exe -I -X utf8 <script>` from the repository root, writes nothing, and ends with `ALL PASS`:

Table 6.1 Verification scripts of Part VI (84 PASS lines in total, no FAIL).

Script	What it checks	PASS lines
`scripts/derivation_checks/v1_0_0_p6_properties.py`	telescoping, symmetry, positive definiteness, M-matrix, positivity, maximum principle, BE and forward Euler stability, Fourier numbers, pivot bounds, round-off of both pivot forms (NumPy replica of Code B)	32

Script	What it checks	PASS lines
`scripts/derivation_checks/v1_0_0_p6_truncation.py`	face truncation errors (interior, interface, outer face), cell residuals, BE local error (SymPy)	16
`scripts/derivation_checks/v1_0_0_p6_orders.py`	Richardson weights, observed orders, contamination model, energy estimate, study cost	20
`scripts/derivation_checks/v1_0_0_p6_series.py`	truncation and validity time of the series, weight and eigenvalue accuracy against a 40-digit oracle, early-time cancellation (replica of Code C)	16

The system matrix $\mathbf{B} = \mathbf{V}/\Delta t + \mathbf{A}$ of one BE step was defined in Section 5.8.2 and keeps its name here. Its entries are $B_{jj} = v_j + g_j^W + g_j^E$, $B_{j,j-1} = -g_j^W$ and $B_{j,j+1} = -g_j^E$, all in $\mathrm{m}^3\ \mathrm{s}^{-1}$, with $\mathbf{V} = \mathrm{diag}(V_1, \dots, V_{N_c})$ and $\mathbf{A}$ the conductance matrix of Eq. (5.15). One symbol is new in this part: the scaled matrix $\mathbf{S} = \mathbf{V}^{-1/2}\mathbf{A}\mathbf{V}^{-1/2}$ (units s^{-1}), which appears in Section 6.4.

Rounding model. Floating-point statements use the standard model of IEEE 754 double precision with round to nearest: for $\circ \in \{+, -, \times, /\}$ the computed result is $\mathrm{fl}(a \circ b) = (a \circ b)(1+\theta)$ with $|\theta| \le \epsilon_\mathrm{m}/2$, where $\epsilon_\mathrm{m} = 2^{-52} \approx 2.2 \times 10^{-16}$ is the machine epsilon (the literal `2.2e-16` of the code) and $\epsilon_\mathrm{m}/2$ the unit roundoff. Round to nearest also maps a non-negative exact result to a non-negative floating-point number (an underflow gives $+0$ or a subnormal). This is an established textbook result (Higham, 2002).

6.1 Discrete conservation

Physical problem. Nothing is created or destroyed inside the particle in v1.0.0 (no source, no decay). The amount inside can only fall by what crosses the outer surface, Eq. (3.8).

What we want. We want to show that the discrete scheme keeps the same bookkeeping exactly: after every time step, the drop in the discrete inventory equals the amount sent out through the outer face in that step.

Governing principle. The discrete cell balance (5.14), one row per cell. Every symbol is from Part V: V_j the cell volume (m^3), $v_j = V_j/\Delta t$ ($\mathrm{m}^3\ \mathrm{s}^{-1}$), g_j^W and g_j^E the inner and outer conductances ($\mathrm{m}^3\ \mathrm{s}^{-1}$), C_j^n the cell average at time level n ($\mathrm{mol\ m}^{-3}$).

Assumptions. Exact arithmetic (floating point is treated in Section 6.8). The conductance used on the inner side of cell $j+1$ is the same number as the one used on the outer side of cell j, $g_{j+1}^W = g_j^E = G_{j+1/2}$. This holds bit for bit in the code, because both read the same stored value `mesh.conductance[i]` (src/numerical/finite_volume.rs, lines 171 to 177).

Step 1: write the row as a balance of face flows. Row j of Eq. (5.14) is

$$-g_j^W C_{j-1}^{n+1} + (v_j + g_j^W + g_j^E) C_j^{n+1} - g_j^E C_{j+1}^{n+1} = v_j C_j^n.$$

Move $v_j C_j^{n+1}$ to the right and the neighbour terms to the left, then group each conductance with its own pair of concentrations:

$$v_j\left(C_j^{n+1}-C_j^n\right)=g_j^W\left(C_{j-1}^{n+1}-C_j^{n+1}\right)-g_j^E\left(C_j^{n+1}-C_{j+1}^{n+1}\right).$$

Step 2: name the face flows. Define the amount per second that crosses face $j+1/2$ outward during the step (mol s^{-1}):

$$\Phi_{j+1/2}=G_{j+1/2}\left(C_j^{n+1}-C_{j+1}^{n+1}\right)\ (1\le j\le N_c-1),$$
$$\Phi_{1/2}=0,\qquad \Phi_{N_c+1/2}=G_R C_{N_c}^{n+1}.$$

$\Phi_{1/2}=0$ because the centre face has zero area, so $g_1^W=0$ (Eq. (5.10)). $\Phi_{N_c+1/2}$ is the flow into the perfect sink, where the concentration is 0 (Eq. (5.11)). With $g_j^W=G_{j-1/2}$ and $g_j^E=G_{j+1/2}$ the row of Step 1 reads "change inside the cell equals inflow minus outflow":

$$v_j\left(C_j^{n+1}-C_j^n\right)=\Phi_{j-1/2}-\Phi_{j+1/2}.$$

Step 3: multiply by Δt. Because $\Delta t\, v_j=V_j$,

$$V_jC_j^{n+1}-V_jC_j^n=\Delta t\left(\Phi_{j-1/2}-\Phi_{j+1/2}\right).$$

Step 4: add all cells (telescoping). A telescoping sum is a sum in which every inner term appears once with a plus sign and once with a minus sign, so it cancels, like a collapsing telescope of which only the two end sections remain. Picture the cells as a row of rooms joined by doors: what goes through a door leaves one room and enters the next, so when all rooms are added the inner doors drop out and only the outside door is left. With three cells the sum of the right-hand sides is

$$\left(\Phi_{1/2}-\Phi_{3/2}\right)+\left(\Phi_{3/2}-\Phi_{5/2}\right)+\left(\Phi_{5/2}-\Phi_{7/2}\right)=\Phi_{1/2}-\Phi_{7/2},$$

and the same happens for any N_c. Summing Step 3 over $j=1,\dots,N_c$ and using the inventory $M^n=\sum_j V_jC_j^n$ (5.21) gives $M^{n+1}-M^n=\Delta t\,(\Phi_{1/2}-\Phi_{N_c+1/2})=-\Delta t\,G_RC_{N_c}^{n+1}$, that is,

$$M^{n+1}+\Delta t\,G_RC_{N_c}^{n+1}=M^n. \tag{6.1}$$

Eq. (6.1) is DERIVED NOT IMPLEMENTED: it is a property of the scheme that no line of code evaluates (derived for STRATO). In matrix language it says that the column sums of $\mathbf{A}$ are $(0,\dots,0,G_R)$: $\mathbf{1}^{\mathsf{T}}\mathbf{A}=G_R\,\mathbf{e}_{N_c}^{\mathsf{T}}$.

Step 5: the inventory identity behind the balance residual. The code accumulates the release with the same G_R and the same $C_{N_c}^{n+1}$ as the matrix row (src/numerical/finite_volume.rs, lines 210 to 217): $M_{\text{rel}}^{n+1}=M_{\text{rel}}^n+\Delta t\,G_RC_{N_c}^{n+1}$ with $M_{\text{rel}}^0=0$ (Eq. (5.22)). Add this to Eq. (6.1):

$$M^{n+1} + M_{\text{rel}}^{n+1} = \left(M^n - \Delta t\, G_R C_{N_c}^{n+1}\right) + \left(M_{\text{rel}}^n + \Delta t\, G_R C_{N_c}^{n+1}\right)$$
$$= M^n + M_{\text{rel}}^n.$$

The sum $M^n + M_{\text{rel}}^n$ is therefore the same at every level, and at $n = 0$ it equals M^0. Hence, in exact arithmetic,

$$\varepsilon^n = M^n + M_{\text{rel}}^n - M^0 = 0 \quad \text{for every } n.$$

The code computes ε^n every step (IMPLEMENTED, src/numerical/finite_volume.rs, lines 219 to 220). Since the exact value is zero, the computed residual measures only rounding and programming errors. It says nothing about discretisation error: any symmetric choice of conductances would pass the same check.

Corollaries. With positivity (Section 6.2), $C_{N_c}^{n+1} \geq 0$ and $G_R > 0$, so (6.1) gives $M^{n+1} \leq M^n$: the inventory never increases. The code reports $F^n = M_{\text{rel}}^n / M^0$ (src/numerical/finite_volume.rs, line 227); in exact arithmetic this equals $1 - M^n/M^0$, and $0 \leq M^n \leq M^0$ gives $0 \leq F^n \leq 1$.

Physical meaning. The discrete particle loses material only through its surface, exactly as the continuous one does (Eq. (3.8)), and the release fraction is a true fraction.

Status: PROVED. Script checks: `v1_0_0_p6_properties.py` lines "PASS 6.1 dt * (sum of the N = 5 BE rows) = M^{n+1} + dt G_R C_N^{n+1} - M^n (telescoping)", "PASS 6.1 column sums of A are (0, ..., 0, G_R)", "PASS 6.1 replica: inventory M^n is non-increasing at every step". Numerical evidence for the floating-point residual: in the main run (200 cells) the worst relative residual $|\varepsilon^n|/M^0$ over all output rows is 2.806×10^{-13}, reached on the last row at $t = 2 \times 10^5$ s (`tests/golden/v1.0/timeseries.csv`). For meshes of 100 to 1600 cells, `verify` prints the last-row values 5.400×10^{-15} to 2.806×10^{-13} (`tests/golden/v1.0/verify_stdout.txt`, lines `CONSERVATION`) and asserts that the worst value over all output rows stays below 10^{-12} (examples/verify.rs, lines 199 to 223); that worst value itself is not printed. Two independent Python replicas give 1.46×10^{-13} and 5.2×10^{-14} over all 1000 steps of the main run: the rounding residual depends on the order of the floating-point operations and is not reproduced bit for bit.

6.2 M-matrix, positivity and the maximum principle

Physical problem. A concentration cannot be negative, and pure diffusion towards an absorbing surface cannot create a new peak higher than the highest value already present.

What we want. We want both statements for the discrete solution, for every time step $\Delta t > 0$.

Three matrix words in plain language.

- A **Z-matrix** is a square matrix whose off-diagonal entries are all zero or negative. In **B** the off-diagonal entries are $-g_j^W$ and $-g_j^E$, and every conductance is positive (Eq. (5.9)), so **B** is a Z-matrix. Physically, a larger neighbour value pushes material into cell j, which appears with a minus sign on the left of Eq. (5.14).

- A row is **strictly diagonally dominant** when its diagonal entry is larger than the sum of the absolute values of the other entries in that row.
- An **M-matrix**, for the purpose of this part, is a nonsingular Z-matrix whose inverse has no negative entry. A strictly diagonally dominant Z-matrix with positive diagonal is an M-matrix (Varga, 2000; established textbook result). The proof below does not rely on that theorem.

Row j of $\mathbf{B}$ has the dominance margin

$$B_{jj} - \sum_{k \neq j} |B_{jk}| = (v_j + g_j^W + g_j^E) - g_j^W - g_j^E = v_j \quad (j < N_c),$$

and $v_{N_c} + G_R$ in the last row, because that row has no east neighbour inside the mesh. Both margins are positive, so every row is strictly diagonally dominant. (The delivered Phase 1 derivation compares with "the sum of the other entries"; the comparison must use their absolute values, Table 7.3, item I17.) A picture in words: every cell is held more strongly by its own old value (the margin v_j) than it is pulled by its neighbours.

Step 1: a negative entry cannot be the smallest one. Let $\mathbf{Bx} = \mathbf{b}$ with $\mathbf{b} \geq 0$ entrywise. Pick j^* where x_{j^*} is smallest and suppose $x_{j^*} < 0$. Row j^* written with the neighbours on the right is

$$(v_{j^*} + g_{j^*}^W + g_{j^*}^E)\, x_{j^*} = b_{j^*} + g_{j^*}^W x_{j^*-1} + g_{j^*}^E x_{j^*+1},$$

where $x_{N_c+1} = 0$ stands for the sink and the term with x_0 is absent ($g_1^W = 0$).

Step 2: bound the neighbours from below. Each neighbour is at least x_{j^*} (it is the smallest entry), and the sink value 0 is larger than $x_{j^*} < 0$. The conductances are non-negative, so

$$(v_{j^*} + g_{j^*}^W + g_{j^*}^E)\, x_{j^*} \geq b_{j^*} + (g_{j^*}^W + g_{j^*}^E)\, x_{j^*}.$$

Step 3: cancel. Subtract $(g_{j^*}^W + g_{j^*}^E)x_{j^*}$ from both sides: $v_{j^*} x_{j^*} \geq b_{j^*} \geq 0$. Since $v_{j^*} > 0$, this gives $x_{j^*} \geq 0$, which contradicts the assumption. Hence $\mathbf{b} \geq 0 \Rightarrow \mathbf{x} \geq 0$.

Step 4: nonsingularity. If $\mathbf{Bx} = \mathbf{0}$, Step 3 applied to $\mathbf{x}$ and to $-\mathbf{x}$ gives $\mathbf{x} \geq 0$ and $-\mathbf{x} \geq 0$, so $\mathbf{x} = \mathbf{0}$. $\mathbf{B}$ is therefore invertible.

Step 5: the inverse. Column i of $\mathbf{B}^{-1}$ solves $\mathbf{Bx} = \mathbf{e}_i$ with $\mathbf{e}_i \geq 0$, so every column is non-negative:

$$\left(\frac{\mathbf{V}}{\Delta t} + \mathbf{A}\right)^{-1} \geq 0 \quad \text{entrywise, for every } \Delta t > 0. \tag{6.2}$$

Eq. (6.2) is DERIVED NOT IMPLEMENTED (a property; the code never forms the inverse). The textbook result (Varga, 2000) is established; the elementary proof above is derived for STRATO.

Step 6: positivity and the maximum principle. The right-hand side of one BE step is $\mathbf{b} = (\mathbf{V}/\Delta t)\mathbf{C}^n$, so $\mathbf{C}^n \geq 0$ gives $\mathbf{b} \geq 0$ and, by Eq. (6.2), $\mathbf{C}^{n+1} \geq 0$. For the upper bound, divide row j of Eq. (5.14) by its diagonal:

$$C_j^{n+1} = \frac{v_j\, C_j^n + g_j^W\, C_{j-1}^{n+1} + g_j^E\, C_{j+1}^{n+1}}{v_j + g_j^W + g_j^E}, \qquad C_{N_c+1}^{n+1} = 0.$$

The new value is a weighted average, with non-negative weights that add to one, of the old value in the same cell, the new values of the neighbours and the sink value 0. Let $m = \max_j C_j^{n+1}$, reached at j^*. If $m \leq 0$ there is nothing to show. If $m > 0$, every neighbour value and the sink value are at most m, so

$$(v_{j^*} + g_{j^*}^W + g_{j^*}^E)\, m \leq v_{j^*} C_{j^*}^n + (g_{j^*}^W + g_{j^*}^E)\, m \quad \Longrightarrow \quad m \leq C_{j^*}^n \leq \max_k C_k^n.$$

Together:

$$\mathbf{C}^n \geq 0 \implies 0 \leq C_j^{n+1} \leq \max_k C_k^n \quad \text{for every } j \text{ and every } \Delta t > 0. \tag{6.3}$$

Eq. (6.3) is DERIVED NOT IMPLEMENTED (derived for STRATO). The code only records $\min_j C_j^n$ on output rows (src/numerical/finite_volume.rs, line 230); it checks neither bound.

Physical meaning. For any time step, the scheme never produces negative concentrations and never creates a new maximum. Since $\mathbf{C}^{n+1} \geq 0$, (6.3) also says $\max_j |C_j^{n+1}| \leq \max_j |C_j^n|$: the scheme is non-expansive in the maximum norm.

Status: PROVED. Script checks: “PASS 6.2 B = V/dt + A is a Z-matrix with positive diagonal”, “PASS 6.2 row margin a_jj - sum |a_jk| = v_j (j < N), v_N + G_R (j = N): strictly diagonally dominant”, “PASS 6.2 every column of B^{-1} computed by the subtraction-free sweep is >= 0 (exactly) and solves B x = e_k”, “PASS 6.3 replica: max_j C^{n+1}_j <= max_j C^n_j at every step (maximum principle)“. In the repository, `verify` asserts $\min_j C_j \geq 0$ on the final row for five meshes (examples/verify.rs, line 215).

6.3 Positivity in floating point and the tridiagonal solver

A **tridiagonal matrix** has non-zero entries only on the main diagonal and the two diagonals next to it: each cell talks only to its two neighbours. Gaussian elimination on such a matrix needs one downward sweep, which removes the entry below the diagonal row by row, and one upward sweep (back substitution). This is the Thomas algorithm. Code B uses the subtraction-free form derived in Part V, Eqs. (5.18) and (5.19) (src/numerical/finite_volume.rs, lines 140 to 158), IMPLEMENTED:

$$e_1 = v_1, \qquad e_j = v_j + \frac{g_j^W\, e_{j-1}}{e_{j-1} + g_j^W}\ (j \geq 2), \qquad d_j = e_j + g_j^E,$$

$$y_1 = \frac{\text{rhs}_1}{d_1}, \quad y_j = \frac{\text{rhs}_j + g_j^W\, y_{j-1}}{d_j}, \qquad C_{N_c}^{n+1} = y_{N_c}, \quad C_j^{n+1} = y_j + \frac{g_j^E}{d_j}\, C_{j+1}^{n+1}.$$

6.3.1 Pivots are positive, so no pivoting is needed

Claim. $v_j \leq e_j \leq v_j + g_j^W$ and $d_j \geq v_j + g_j^E > 0$ for every j.

Proof by induction. $e_1 = v_1 > 0$. If $e_{j-1} > 0$, then $g_j^W e_{j-1}/(e_{j-1} + g_j^W)$ is a non-negative number, and it is at most g_j^W because $e_{j-1}/(e_{j-1} + g_j^W) \leq 1$. Adding v_j gives the bounds on e_j, and $e_j > 0$ carries the induction forward. Then $d_j = e_j + g_j^E \geq v_j + g_j^E > 0$.

Consequences. Pivoting, the swapping of rows to avoid a zero or tiny pivot, is never needed. The guard that stops the run on a pivot $d_j \leq 0$ (src/numerical/finite_volume.rs, lines 147 to 149) cannot fire for a valid input unless an overflow produces a non-finite number. The back-substitution multipliers satisfy $0 < g_j^E/d_j < 1$, so the upward sweep shrinks, and never amplifies, what it carries from cell $j+1$ to cell j. That elimination without pivoting is stable for diagonally dominant matrices is an established textbook result (Higham, 2002).

Status: PROVED. Script checks: "PASS 6.4b pivots: v_j <= e_j <= v_j + g^W_j and d_j = e_j + g^E_j > 0 for all j (no pivoting needed)", "PASS 6.4b back-substitution multipliers g^E_j/d_j lie in (0, 1)" (largest multiplier 0.999528 in the main run).

6.3.2 Operation count

Counted from the code, one BE step costs per cell: 2 operations for the assembly (v_j and rhs_j, lines 178 to 179), 8 in the forward sweep (4 for e_j, 1 for d_j, 1 for the carry $g_j^W y_{j-1}$, 2 for y_j, lines 144 to 151) and 3 in the back substitution (line 157), so 13 floating-point operations per cell, plus 2 per cell for the inventory sum (lines 114 to 120). Work and memory grow in proportion to N_c, written $O(N_c)$, against about $\frac{2}{3}N_c^3$ operations for dense Gaussian elimination. The main run needs about $13 \times 200 \times 1000 = 2.6 \times 10^6$ operations for all its solves. **Status: PROVED** (by counting the code lines cited).

6.3.3 Positivity holds exactly in floating point

Claim. If $\mathbf{C}^n \geq 0$, the computed y_j and C_j^{n+1} are non-negative floating-point numbers, with no tolerance.

Proof. Every quantity in the sweep is built from non-negative numbers by addition, multiplication and division by a positive number: $v_j > 0$ and $\text{rhs}_j = v_j C_j^n \geq 0$; e_j and d_j are positive (Section 6.3.1, and its argument repeats in floating point because each operation acts on non-negative inputs); y_j is a sum of non-negative terms divided by

$d_j > 0$; C_j^{n+1} is y_j plus a product of non-negative numbers. No subtraction occurs. By the rounding model, each exact intermediate result is non-negative and so is its rounded value. (The argument needs the computed $v_j = \mathrm{fl}(V_j/\Delta t)$ to be positive, that is, no underflow to zero; for the synthetic input the smallest v_j belongs to the centre cell, $v_1 = \frac{4\pi}{3}(5 \times 10^{-6}\ \mathrm{m})^3/(200\ \mathrm{s}) = 2.6 \times 10^{-18}\ \mathrm{m^3\ s^{-1}}$, far above the smallest double, about 10^{-308}. The pivot guard of lines 147 to 149 would stop the run if d_j ever came out as zero.) Induction over the cells, then over the time levels starting from $C_j^0 \in \{c_0, 0\}$ with $c_0 \geq 0$ (validated in `Config::validate`, src/config.rs, lines 108 to 109), gives

$$y_j \geq 0, \qquad C_j^{n+1} \geq 0 \quad \text{exactly in floating point, for every } j \text{ and } n. \tag{6.4}$$

Eq. (6.4) is DERIVED NOT IMPLEMENTED (a property of the IMPLEMENTED sweep; derived for STRATO). The exact-arithmetic proof of the delivered Phase 1 derivation does not cover floating point (Table 7.3, item I22). The unit test src/numerical/finite_volume.rs, line 310 accepts values down to -10^{-13}, which is looser than the property.

Status: PROVED (under the IEEE rounding model). Script check: "PASS 6.4 replica: every computed concentration is >= 0 exactly (1000 steps, dt = 200 s)", smallest value 1.237×10^{-17} mol m^{-3}.

6.4 Stability of Backward Euler and the explicit bound

Stability in plain words. A time-stepping method is stable when an error made at one step (a rounding error, or the error of the step itself) is not magnified by the later steps. Accuracy is a different question: how close each step is to the true solution. A method can be stable and inaccurate at the same time, and Section 6.4.5 shows exactly that situation.

6.4.1 The scalar test equation

Physical problem. Every pattern in which the concentration can fade (each eigenmode of Part IV, and each eigenvector of the discrete system below) decays like $e^{-\mu t}$ with some rate $\mu > 0$ (s^{-1}). The test equation isolates one such pattern:

$$\frac{dy}{dt} = -\mu\, y, \qquad y(t + \Delta t) = e^{-\mu \Delta t}\, y(t).$$

One BE step. BE evaluates the right-hand side at the new time: $y^{n+1} = y^n - \mu\, \Delta t\, y^{n+1}$. Collect y^{n+1}: $(1 + \mu \Delta t)\, y^{n+1} = y^n$, so

$$y^{n+1} = \mathcal{R}(\zeta)\, y^n, \qquad \mathcal{R}(\zeta) = \frac{1}{1 - \zeta}, \qquad \zeta = -\mu\, \Delta t.$$

$\mathcal{R}$ is the amplification factor. For every $\mu \Delta t > 0$, $0 < \mathcal{R} < 1$: the computed mode always shrinks and never changes sign.

A-stability. A method is A-stable when $|\mathcal{R}(\zeta)| \le 1$ for every complex ζ with $\mathrm{Re}\,\zeta \le 0$, so that no decaying mode, however fast, is ever amplified (Hairer and Wanner, 1996; established textbook result). For BE write $\zeta = x + iy$ with $x \le 0$:

$$|1-\zeta|^2 - 1 = (1-x)^2 + y^2 - 1 = x^2 - 2x + y^2 \ge 0,$$

because $x^2 \ge 0$, $y^2 \ge 0$ and $-2x \ge 0$. Hence $|1-\zeta| \ge 1$ and $|\mathcal{R}(\zeta)| \le 1$. BE is also L-stable: $\mathcal{R}(\zeta) \to 0$ as $\zeta \to -\infty$, so very fast modes are damped almost completely in one step.

Forward Euler for contrast. Forward Euler evaluates the right-hand side at the old time, $y^{n+1} = (1+\zeta)\,y^n$. On the real axis $|1+\zeta| \le 1$ only for $-2 \le \zeta \le 0$, that is, $\Delta t \le 2/\mu$: the step must be small compared with the fastest decay time.

Status: PROVED (established textbook result, checked by "PASS 6.4 |1 - z|^2 - 1 = x^2 - 2x + y^2 >= 0 for Re z = x <= 0 (A-stability of BE)", "PASS 6.4 BE is L-stable: R(z) -> 0 as z -> -infinity", "PASS 6.4 FE: |1 + z| <= 1 on the real axis only for -2 <= z <= 0").

6.4.2 The full system is symmetric positive definite after scaling by volumes

The semi-discrete system (5.12) in matrix form is $\mathbf{V}\,d\mathbf{C}/dt = -\mathbf{A}\mathbf{C}$.

Symmetry. $A_{j,j+1} = -g_j^E = -G_{j+1/2}$ and $A_{j+1,j} = -g_{j+1}^W = -G_{j+1/2}$, so $\mathbf{A} = \mathbf{A}^\mathsf{T}$.

Positive definiteness in plain words. A symmetric matrix is positive definite when the number $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$, a kind of "energy" of the vector $\mathbf{x}$, is positive for every non-zero $\mathbf{x}$.

Step 1: expand the energy. $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = \sum_j x_j \left[(g_j^W + g_j^E)x_j - g_j^W x_{j-1} - g_j^E x_{j+1}\right]$.

Step 2: group by faces. Face $j+1/2$ ($1 \le j \le N_c - 1$) contributes $G_{j+1/2}(x_j^2 - x_j x_{j+1})$ from row j and $G_{j+1/2}(x_{j+1}^2 - x_{j+1}x_j)$ from row $j+1$. Their sum is $G_{j+1/2}(x_j - x_{j+1})^2$. The outer face contributes $G_R x_{N_c}^2$ from the last row only. Hence

$$\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = \sum_{j=1}^{N_c-1} G_{j+1/2}\left(x_j - x_{j+1}\right)^2 + G_R\, x_{N_c}^2 .$$

Step 3: positivity. Every term is non-negative. The sum is zero only if $x_{N_c} = 0$ and all neighbouring differences vanish, which forces $\mathbf{x} = \mathbf{0}$. So $\mathbf{A}$ is symmetric positive definite.

Step 4: scale by volumes. Put $\hat{\mathbf{C}} = \mathbf{V}^{1/2}\mathbf{C}$, so $\hat{C}_j = \sqrt{V_j}\,C_j$. Then $\mathbf{V}\,d\mathbf{C}/dt = -\mathbf{A}\mathbf{C}$ becomes $d\hat{\mathbf{C}}/dt = -\mathbf{S}\hat{\mathbf{C}}$ with $\mathbf{S} = \mathbf{V}^{-1/2}\mathbf{A}\mathbf{V}^{-1/2}$. $\mathbf{S}$ is symmetric, and $\hat{\mathbf{C}}^\mathsf{T}\mathbf{S}\hat{\mathbf{C}} = \mathbf{C}^\mathsf{T}\mathbf{A}\mathbf{C} > 0$, so $\mathbf{S}$ is symmetric positive definite. By the spectral theorem (established textbook result) $\mathbf{S}$ has an orthonormal set of eigenvectors with real eigenvalues $0 < \mu_{\min} \le \mu_k \le \mu_{\max}$. The length of $\hat{\mathbf{C}}$ is the volume-weighted norm of $\mathbf{C}$,

$$\|\mathbf{C}\|_V^2 = \mathbf{C}^\mathsf{T}\mathbf{V}\mathbf{C} = \sum_j V_j\, C_j^2,$$

the discrete counterpart of $\int c^2\,dV$ (units mol^2 m^{-3}).

For the synthetic mesh, $\mu_{\min} = 3.2733 \times 10^{-6}$ s^{-1} and $\mu_{\max} = 255.61$ s^{-1} (calculated). The slowest discrete rate agrees, to the five digits printed, with the slowest physical decay constant $\Lambda_1 = 3.2733 \times 10^{-6}$ s^{-1} of Part IV; the relative difference, 1.7×10^{-5}, is the spatial discretisation error of the slowest mode on 200 cells.

Status: PROVED. Script checks: "PASS 6.4 A is symmetric because g^W_{j+1} = g^E_j = G_{j+1/2}", "PASS 6.4 x^T A x = sum_f G_f (x_j - x_{j+1})^2 + G_R x_N^2 (so A is positive definite)", "PASS 6.4 S = V^{-1/2} A V^{-1/2} is symmetric positive definite (all 200 eigenvalues > 0)".

6.4.3 Every BE step is non-expansive in the volume-weighted norm

Eigenvector route. One BE step $(\mathbf{V} + \Delta t\,\mathbf{A})\mathbf{C}^{n+1} = \mathbf{V}\mathbf{C}^n$ becomes, after multiplying by $\mathbf{V}^{-1/2}$ and inserting $\mathbf{V}^{1/2}\mathbf{V}^{-1/2}$, $(\mathbf{I} + \Delta t\,\mathbf{S})\hat{\mathbf{C}}^{n+1} = \hat{\mathbf{C}}^n$. In the eigenvector basis each component evolves separately, exactly as in the test equation with $\mu = \mu_k$:

$$\begin{aligned} \hat{C}_k^{n+1} &= \frac{1}{1 + \Delta t\,\mu_k}\,\hat{C}_k^n, \qquad 0 < \frac{1}{1 + \Delta t\,\mu_k} < 1, \\ \|\mathbf{C}^{n+1}\|_V &\le \frac{1}{1 + \Delta t\,\mu_{\min}}\,\|\mathbf{C}^n\|_V < \|\mathbf{C}^n\|_V \quad \text{for every } \Delta t > 0. \end{aligned} \tag{6.5}$$

Eq. (6.5) is DERIVED NOT IMPLEMENTED (derived for STRATO; no code computes μ_k).

Energy route (no eigenvectors). Multiply the BE step on the left by $(\mathbf{C}^{n+1})^{\mathsf{T}}$:

$$\|\mathbf{C}^{n+1}\|_V^2 + \Delta t\,(\mathbf{C}^{n+1})^{\mathsf{T}}\mathbf{A}\,\mathbf{C}^{n+1} = (\mathbf{C}^{n+1})^{\mathsf{T}}\mathbf{V}\,\mathbf{C}^n \le \|\mathbf{C}^{n+1}\|_V\,\|\mathbf{C}^n\|_V,$$

where the inequality is the Cauchy-Schwarz inequality for the inner product $\mathbf{a}^{\mathsf{T}}\mathbf{V}\mathbf{b}$. The second term on the left is non-negative (Section 6.4.2), so $\|\mathbf{C}^{n+1}\|_V^2 \le \|\mathbf{C}^{n+1}\|_V\|\mathbf{C}^n\|_V$; dividing by $\|\mathbf{C}^{n+1}\|_V$ (when it is not zero) gives $\|\mathbf{C}^{n+1}\|_V \le \|\mathbf{C}^n\|_V$.

Unconditional stability. No restriction on Δt entered either route. The same bound applies to the difference of two solutions, so a perturbation (for instance a rounding error) can never grow from one step to the next: BE is unconditionally stable for this system.

Status: PROVED. Script checks: "PASS 6.5 BE amplification factors 1/(1 + dt mu_k) lie in (0, 1) at dt = 200 s" (range 1.956×10^{-5} to 0.999346), "PASS 6.5 replica: volume-weighted norm ||C^{n+1}||_V <= ||C^n||_V at every step".

6.4.4 Forward Euler bounds (diagnostic only)

Code B contains no forward Euler solver; only one bound is computed and printed. The delivered Phase 1 documents describe an explicit "verification scheme" that does not exist (Table 7.3, item I6).

Positivity bound. Forward Euler would update $\mathbf{C}^{n+1} = \mathbf{C}^n - \Delta t\,\mathbf{V}^{-1}\mathbf{A}\mathbf{C}^n$. Row j:

$$C_j^{n+1} = \left(1 - \frac{\Delta t\,(g_j^W + g_j^E)}{V_j}\right) C_j^n + \frac{\Delta t\, g_j^W}{V_j}\, C_{j-1}^n + \frac{\Delta t\, g_j^E}{V_j}\, C_{j+1}^n.$$

The neighbour coefficients are non-negative. The own coefficient is non-negative exactly when $\Delta t \le V_j/(g_j^W + g_j^E)$. Requiring this in every cell gives

$$\Delta t_{\mathrm{FE}} = \min_j \frac{V_j}{g_j^W + g_j^E}, \tag{6.6}$$

IMPLEMENTED as a diagnostic in src/numerical/finite_volume.rs, lines 238 to 252 and printed by src/main.rs, line 78 (derived for STRATO). Main run: $\Delta t_{\mathrm{FE}} = 7.812107958751 \times 10^{-3}$ s.

L^2 bound. In the eigenvector basis of $\mathbf{S}$ forward Euler multiplies component k by $1 - \Delta t\,\mu_k$, which lies in $[-1, 1]$ for all k exactly when

$$\Delta t \le \frac{2}{\mu_{\max}}, \tag{6.7}$$

DERIVED NOT IMPLEMENTED (derived for STRATO). Synthetic mesh: $2/\mu_{\max} = 7.824293 \times 10^{-3}$ s, slightly larger than Δt_{FE}: keeping every coefficient non-negative asks a little more than keeping the norm bounded.

The two bounds are ordered on every mesh. That $\Delta t_{\mathrm{FE}} \le 2/\mu_{\max}$ is not an accident of the synthetic input. The proof takes three steps.

Step 1: change to a matrix with the same eigenvalues. $\mathbf{V}^{-1}\mathbf{A} = \mathbf{V}^{-1/2}\,\mathbf{S}\,\mathbf{V}^{1/2}$. If $\mathbf{S}\mathbf{y} = \mu\mathbf{y}$, then $\mathbf{V}^{-1}\mathbf{A}\,(\mathbf{V}^{-1/2}\mathbf{y}) = \mathbf{V}^{-1/2}\mathbf{S}\mathbf{y} = \mu\,(\mathbf{V}^{-1/2}\mathbf{y})$, so every eigenvalue μ_k of $\mathbf{S}$ is an eigenvalue of $\mathbf{V}^{-1}\mathbf{A}$. Row j of $\mathbf{V}^{-1}\mathbf{A}$ has the diagonal entry $a_j = (g_j^W + g_j^E)/V_j$ and the off-diagonal entries $-g_j^W/V_j$ and $-g_j^E/V_j$ (only $-g_{N_c}^W/V_{N_c}$ in the last row, whose $g_{N_c}^E = G_R$ couples to the sink and not to a cell).

Step 2: bound one eigenvalue (the Gershgorin argument). Let $\mathbf{V}^{-1}\mathbf{A}\,\mathbf{x} = \mu\mathbf{x}$ with $\mathbf{x} \ne \mathbf{0}$, and pick the row j where $|x_j|$ is largest. Row j reads $(\mu - a_j)\,x_j = -\frac{g_j^W}{V_j}x_{j-1} - \frac{g_j^E}{V_j}x_{j+1}$ (with the missing neighbours set to zero). Take absolute values and use $|x_{j\pm1}| \le |x_j|$:

$$|\mu - a_j|\,|x_j| \le \frac{g_j^W + g_j^E}{V_j}\,|x_j| = a_j\,|x_j|.$$

Divide by $|x_j| > 0$: $|\mu - a_j| \le a_j$, so $\mu \le 2a_j$.

Step 3: compare with Eq. (6.6). Since $a_j \le \max_k a_k = 1/\Delta t_{\mathrm{FE}}$ by Eq. (6.6), every eigenvalue satisfies $\mu \le 2/\Delta t_{\mathrm{FE}}$. Applied to $\mu_{\max}$:

$$\Delta t_{\mathrm{FE}} \le \frac{2}{\mu_{\max}} \quad \text{for every mesh.}$$

This is DERIVED NOT IMPLEMENTED (derived for STRATO; the circle argument of Step 2 is the established Gershgorin theorem, proved here in full). It also shows that Δt_{FE}, the number the code prints, is a safe forward Euler step in both senses.

Status: PROVED. Script checks: "PASS 6.6 FE update C_j^{n+1} = (1 - dt (g^W+gE)/V_j) C_j^n + ...: coefficient >= 0 iff dt <= V_j/(g^W+gE)", "PASS 6.6 Delta t_FE = min_j V_j/(g^W_j + g^E_j) = 7.812107958751e-3 s (main run prints 7.812107958751e-3)", "PASS 6.7 forward Euler L2 limit 2/mu_max = 7.824293e-3 s > Delta t_FE (inventory B15)".

6.4.5 Stable is not the same as accurate

The main run uses $\Delta t = 200$ s, which is 2.56×10^4 times Δt_{FE}. In terms of the cell Fourier number $\mathrm{Fo} = D_\ell \Delta t / w_\ell^2$ (Eq. (6.13) below), the layers have the values of Table 6.2.

Table 6.2 Cell Fourier numbers of the synthetic input (calculated; synthetic data, not literature values). The ratio $\Delta t / \Delta t_{\mathrm{FE}} = 2.56 \times 10^4$ is about twice the largest Fourier number, as expected from $\Delta t_{\mathrm{FE}} \approx w^2/(2D)$ for an interior cell.

Layer	D_ℓ (m^2 s^{-1})	w_ℓ (m)	Fo at $\Delta t = 200$ s
1 Kernel	1.0×10^{-12}	5.0×10^{-6}	8
2 Buffer	4.0×10^{-10}	2.5×10^{-6}	1.28×10^4
3 IPyC	8.0×10^{-13}	1.25×10^{-6}	102.4
4 SiC	2.0×10^{-14}	1.25×10^{-6}	2.56
5 OPyC	7.0×10^{-13}	1.25×10^{-6}	89.6

At these Fourier numbers BE keeps every property proved above, Eqs. (6.1) to (6.5). Its accuracy is a separate matter. Per step and per mode, BE replaces e^{-x} by $1/(1+x)$ with $x = \mu_k \Delta t$, and

$$\frac{1}{1+x} - e^{-x} = \frac{x^2}{2} + O(x^3).$$

For slow modes ($x \ll 1$) this local error of order Δt^2 adds up over $t/\Delta t$ steps to an error of order Δt (Section 6.5.3). For fast modes ($x \gg 1$) both factors are close to zero: the mode is removed, although its decay is not timed correctly. Early in the transient the solution still contains many fast modes, and the release fraction is small, so the relative error is large. The largest difference between the 200-cell FV result and the series is 2.537522×10^{-4} at $t = 2.2 \times 10^4$ s, where $F^{\mathrm{FV}} = 9.1969 \times 10^{-3}$ and $F^{\mathrm{ex}} = 8.9431 \times 10^{-3}$, a relative error of $2.8\,\%$ (`tests/golden/v1.0/main_run_stdout.txt` and `comparison_fv_vs_analytical.csv`). At t_{end} the error 1.270322×10^{-4} splits into a temporal part 1.2156×10^{-4} (96 %) and a spatial part 5.476×10^{-6}.

Status: DEMONSTRATED NUMERICALLY. Script checks: "PASS 6.4 dt/Delta t_FE = 2.560e4 for dt = 200 s", "PASS 6.13 layer Fourier numbers D dt/w^2 = 8, 1.28e4, 102.4, 2.56, 89.6 at dt = 200 s (max in the buffer)", "PASS 6.4 per-step amplification

error 1/(1+x) - e^{-x} = x^2/2 + O(x^3), x = mu dt", "PASS 6.4 final error 1.270322e-4 = temporal 1.2156e-4 + spatial 5.476e-6 (temporal part is 96 %)".

6.5 Consistency and order

Truncation error and consistency in plain words. Take the exact solution and put it into the discrete equations. It will not satisfy them exactly; what is left over is the truncation error. A scheme is consistent when that leftover tends to zero as the mesh width w and the step Δt tend to zero. The order says how fast: an error proportional to w^2 is second order in space. The tool for computing the leftover is the Taylor expansion, which writes a smooth function near a point r_f as $c(r) = c(r_f) + c'(r_f)(r - r_f) + \frac{1}{2}c''(r_f)(r - r_f)^2 + \frac{1}{6}c'''(r_f)(r - r_f)^3 + \ldots$.

6.5.1 Where the spatial approximation sits

The exact cell averages $\bar{c}_j(t)$ of Eq. (5.1) satisfy the exact cell balance (5.6) with no approximation:

$$V_j \frac{d\bar{c}_j}{dt} = A_{j-1/2} J_r(r_{j-1/2}, t) - A_{j+1/2} J_r(r_{j+1/2}, t).$$

The only spatial approximation of Code B replaces the exact face flow $A_{j+1/2} J_r(r_{j+1/2}, t)$ by $G_{j+1/2}(C_j - C_{j+1})$, Eq. (5.9). Define the face truncation error (mol s^{-1}) by inserting exact averages:

$$\begin{aligned} \tau_{j+1/2} &= G_{j+1/2}\left(\bar{c}_j - \bar{c}_{j+1}\right) - A_{j+1/2} J_r(r_{j+1/2}, t), \\ \tau_R &= G_R\, \bar{c}_{N_c} - A(R)\, J_r(R, t), \qquad \tau_{1/2} = 0. \end{aligned}$$

$\tau_{1/2} = 0$ exactly because the centre face has zero area. Replacing each exact face flow by $G(\bar{c}_j - \bar{c}_{j+1}) - \tau$ in the exact balance gives

$$V_j \frac{d\bar{c}_j}{dt} = g_j^W \left(\bar{c}_{j-1} - \bar{c}_j\right) + g_j^E \left(\bar{c}_{j+1} - \bar{c}_j\right) + \left(\tau_{j+1/2} - \tau_{j-1/2}\right),$$

with $\bar{c}_{N_c+1} = 0$ and $\tau_{N_c+1/2} = \tau_R$. The exact averages therefore satisfy the semi-discrete system (5.12) up to the cell residual $\tau_{j+1/2} - \tau_{j-1/2}$.

6.5.2 Truncation error at each kind of face

Method. Around the face r_f, expand c in a Taylor polynomial on each side. Compute the cell averages with the shell weight $4\pi r^2$ and the exact centroids (5.4), insert them in τ, and expand in powers of the cell width. One useful intermediate result: a cell of width w with midpoint m has its volume centroid at

$$r_j = m + \frac{w^2}{6m} + O(w^4),$$

so the centroid sits slightly outward of the midpoint, because the outer half of a shell holds more volume. The distances δ^L, δ^R of Eq. (5.7) are therefore unequal by about $w^2/(3r_f)$ even on a uniform mesh, and the face area $A_{j+1/2} = 4\pi r_{j+1/2}^2$ differs from the areas at the two centroids. Both effects are carried exactly by the SymPy computation; the results are the following, derived for STRATO.

(a) Interior face, one material, equal widths w:

$$\tau_{j+1/2} = -\frac{2\pi}{3}\, D\, w^2\, r_f \left(c'' + \frac{r_f}{2}\, c''' \right) \Bigg|_{r_f} + O(w^3).$$

The terms of order 1 and w cancel. Relative to the face flow $4\pi r_f^2 Dc'$ this is an error of order w^2: second-order consistency.

(b) Material interface at $r_f = R_\ell$, widths w_L (inside) and w_R (outside), diffusivities D_L, D_R, with c and Dc' continuous (Eqs. (3.2), (3.3)) and c'' allowed to jump:

$$\tau_{j+1/2} = \frac{4\pi r_f^2\, D_L D_R}{3}\, \frac{w_L^2\, c_L'' - w_R^2\, c_R''}{D_R\, w_L + D_L\, w_R} + O(w^2).$$

The term of order 1 cancels (consistency), and the error is first order. The jump of c'' is fixed by the physics: the layer equation (2.8) gives $D\, c'' = \partial c/\partial t - 2Dc'/r$, and both terms on the right are continuous across the interface (c is continuous at all times, so its time derivative is too; Dc' is continuous by Eq. (3.3)). Hence $D_L c_L'' = D_R c_R''$, and substituting $c_R'' = D_L c_L''/D_R$:

$$\tau_{j+1/2} = \frac{4\pi r_f^2\, D_L\, c_L''}{3}\, \frac{D_R\, w_L^2 - D_L\, w_R^2}{D_R\, w_L + D_L\, w_R} + O(w^2).$$

This first-order term vanishes only when $w_L/w_R = \sqrt{D_L/D_R}$, that is, when w^2/D (a diffusion time across one cell) is the same on both sides. The synthetic kernel and buffer have $w_L/w_R = 2$ against $\sqrt{D_L/D_R} = 0.05$, so the term is present.

(c) Outer face (half cell to $c = 0$ at R, Eq. (5.11)), width w of the last cell:

$$\tau_R = \frac{4\pi R^2\, D_5\, w}{3}\, c''(R^-) + O(w^2),$$

first order, because the half cell is one-sided.

(d) Faces near the centre. Formula (a) is an expansion for a face at a fixed radius r_f while $w \to 0$. The first few faces sit at $r_f = w, 2w, 3w, \ldots$, which move towards the centre as w shrinks, so (a) does not apply to them directly. For a smooth solution the concentration contains only even powers of r near the centre, $c = c(0) + \frac{1}{2}c''(0)\, r^2 + O(r^4)$: Eq. (3.1) removes the linear term, and every regular kernel mode of Part IV,

$\sin(kr)/r$, is an even function of r. A direct SymPy expansion with this form and the exact centroid $\frac{3}{4}w$ of the centre cell gives, for $r_f = kw$,

$$\tau_{k+1/2} = -\beta_k \, \pi \, D \, c''(0) \, w^3 + O(w^4), \qquad \beta_1 = \frac{4}{5}, \quad \beta_2 = \frac{608}{425}, \quad \beta_3 = \frac{1188}{575},$$

so these faces have $\tau = O(w^3)$, one order smaller than an interior face at fixed radius. They have $G = O(w)$, which matters in Section 6.10.

Cell residuals. In the interior, τ varies smoothly from face to face, so $\tau_{j+1/2} - \tau_{j-1/2} = O(w^3)$ and the residual per unit volume is $O(w^2)$. In the two cells next to each interface and in the outermost cell, $\tau = O(w)$ while $V_j = O(w)$, so the residual per unit volume stays of order one as $w \to 0$. Pointwise, the scheme is therefore consistent in the flux sense (every $\tau_f / A_f \to 0$) and inconsistent in the cell-residual sense at nine cells (two at each of the four interfaces, plus the outermost). The solution still converges at second order in the observed quantity F; this effect is known as supra-convergence, and Section 6.10 gives the energy argument that explains why an $O(1)$ cell residual of this flux-difference form still yields a convergent scheme.

Status: PROVED (Taylor expansion to the order shown, assuming c is smooth inside each layer). Script checks in `v1_0_0_p6_truncation.py`: “PASS 6.5a interior face: O(1) and O(w) terms of tau vanish”, “PASS 6.5a interior face: tau = -(2 pi/3) D w^2 r_f (c’’ + r_f c’’’/2) + O(w^3)“,”PASS 6.5a interior cell residual (tau_W - tau_E)/V_j = O(w^2)“,”PASS 6.5b interface face: O(1) term of tau vanishes (consistency)“,”PASS 6.5b interface face: tau = (4 pi r_f^2 D_L D_R/3) (w_L^2 c’’_L - w_R^2 c’’_R)/(D_R w_L + D_L w_R) + O(w^2)“,”PASS 6.5b that O(w) term vanishes exactly when w_L/w_R = sqrt(D_L/D_R) (equal w^2/D on both sides)“,”PASS 6.5b interface cell residual tau/V_j tends to a non-zero O(1) constant”, “PASS 6.5c outer face: tau = (4 pi R^2 D w/3) c’’(R^-) + O(w^2)”, “PASS 6.5 centre face: A(0) = 0, so tau_{1/2} = 0 exactly”; and in `v1_0_0_p6_orders.py`: “PASS 6.5 volume centroid of a cell of width w and midpoint m: r_j = m + w^2/(6m) + O(w^4)”.

6.5.3 Local error of Backward Euler

Let $y(t)$ be a smooth solution of $dy/dt = f(y)$ and $h = \Delta t$. Expand both $y(t+h)$ and $y'(t+h)$ about t:

$$y(t+h) = y + h\,y' + \frac{h^2}{2}\,y'' + O(h^3), \qquad y'(t+h) = y' + h\,y'' + O(h^2).$$

BE demands $y(t+h) - y(t) - h\,y'(t+h) = 0$. Insert the expansions; the terms $h\,y'$ cancel and

$$y(t+h) - y(t) - h\,y'(t+h) = \left(\frac{h^2}{2} - h^2\right) y'' + O(h^3) = -\frac{h^2}{2}\,y'' + O(h^3).$$

The local error of one step is of order Δt^2; per unit time it is $-\frac{1}{2}\Delta t\,y''$, first order. Over $t/\Delta t$ steps, with errors not amplified by Eq. (6.5), the global time error is of order Δt (Section 6.10 makes this precise in words).

Status: PROVED. Script checks: “PASS 6.5 BE local error: y(t+h) - h y’(t+h) - y(t) = -h^2 y’ ‘/2 + O(h^3)“,”PASS 6.5 BE truncation per unit time [y(t+h) - y(t)]/h - y’(t+h) = -h y’’/2 + O(h^2): first order”.

6.5.4 Observed order of the release fraction

The analysis above and the measurements of Section 6.7 combine into the error model

$$F^{\mathrm{FV}}(t_n) - F(t_n) = O(w^2) + O(\Delta t), \tag{6.8}$$

DERIVED NOT IMPLEMENTED. The temporal term rests on the proved local error (Section 6.5.3); the spatial order 2 is observed, and the proof of Section 6.10 guarantees only order $3/2$ for smooth solutions. **Status: DEMONSTRATED NUMERICALLY** (orders 2.00 in space and 1.00 in time, Section 6.7).

6.6 Richardson time extrapolation

Idea in plain words. If the time error is a known series in Δt, results at a few step sizes can be combined so that the leading error terms cancel.

Error expansion. For a fixed mesh and a fixed $t_{\mathrm{end}} > 0$, one BE mode after $t/\Delta t$ steps equals

$$(1+\mu\Delta t)^{-t/\Delta t} = \exp\!\left(-\frac{t}{\Delta t}\ln(1+\mu\Delta t)\right) = e^{-\mu t}\left(1 + \frac{\mu^2 t\,\Delta t}{2} + O(\Delta t^2)\right),$$

a smooth function of Δt. The FV release fraction is a finite sum of such modes, so, as an asymptotic series for small Δt,

$$F(\Delta t) = F_0 + \chi_1\Delta t + \chi_2\Delta t^2 + \chi_3\Delta t^3 + \ldots \tag{6.9}$$

with F_0 the time-converged value on that mesh and coefficients χ_1, χ_2, χ_3 (s^{-1}, s^{-2}, s^{-3}) that depend on the mesh. DERIVED NOT IMPLEMENTED (derived for STRATO; a series, not a closed form).

Where the series is valid. For one mode, $\ln(1+\mu\Delta t)$ has a convergent power series only when $\mu\Delta t < 1$. The mesh has fast modes with $\mu\Delta t$ up to $255.61 \times 200 \approx 5 \times 10^4$, for which (6.9) is not a convergent expansion. Those modes do not matter at the time where Richardson is applied: a mode with $\mu \geq 1/\Delta t = 5 \times 10^{-3}\ \mathrm{s}^{-1}$ has decayed by $e^{-\mu t_{\mathrm{end}}} \leq e^{-1000}$ in the exact solution and by at most $(1+\mu\Delta t)^{-t_{\mathrm{end}}/\Delta t} \leq 2^{-1000}$ in the BE solution, both far below double precision. The modes that still carry weight at t_{end} have $\mu\Delta t \ll 1$, where (6.9) holds.

Finding the weights, one equation per requirement. Seek $\varsigma_4 F(\Delta t/4) + \varsigma_2 F(\Delta t/2) + \varsigma_1 F(\Delta t)$ that keeps F_0 and removes χ_1 and χ_2:

$$\varsigma_4 + \varsigma_2 + \varsigma_1 = 1, \qquad \frac{\varsigma_4}{4} + \frac{\varsigma_2}{2} + \varsigma_1 = 0, \qquad \frac{\varsigma_4}{16} + \frac{\varsigma_2}{4} + \varsigma_1 = 0.$$

Subtract the third from the second: $\frac{3}{16}\varsigma_4 + \frac{1}{4}\varsigma_2 = 0$, so $\varsigma_2 = -\frac{3}{4}\varsigma_4$. The second then gives $\varsigma_1 = \frac{3}{8}\varsigma_4 - \frac{1}{4}\varsigma_4 = \frac{1}{8}\varsigma_4$. The first gives $\varsigma_4(1 - \frac{3}{4} + \frac{1}{8}) = \frac{3}{8}\varsigma_4 = 1$, so $\varsigma_4 = \frac{8}{3}$, $\varsigma_2 = -2$, $\varsigma_1 = \frac{1}{3}$. The χ_3 term keeps the factor $\frac{8}{3} \cdot \frac{1}{64} - 2 \cdot \frac{1}{8} + \frac{1}{3} = \frac{1-6+8}{24} = \frac{1}{8}$:

$$F_{\mathrm{RE}} = \frac{1}{3}\left(8F(\Delta t/4) - 6F(\Delta t/2) + F(\Delta t)\right) = F_0 + \frac{1}{8}\chi_3 \Delta t^3 + \ldots \tag{6.10}$$

IMPLEMENTED in `time_extrapolated_release`, src/postprocessing/comparison.rs, lines 150 to 157 (derived for STRATO).

Condition. The cancellation needs step sizes exactly in the ratio $1 : \frac{1}{2} : \frac{1}{4}$. Under the step rule (5.16) this holds when $t_{\mathrm{end}}/\Delta t_{\mathrm{in}}$ is an integer, which is true for every v1.0.0 input (Table 7.3, item K3).

Status: PROVED for the cancellation (given Eq. (6.9)). Script checks: "PASS 6.9 one BE mode after t/h steps: (1 + mu h)^(-t/h) = e^{-mu t}(1 + mu^2 t h/2 + O(h^2)), smooth in h", "PASS 6.10 (8F(h/4) - 6F(h/2) + F(h))/3 = F0 + a3 h^3/8 + 7 a4 h^4/32", "PASS 6.10 weights (8/3, -2, 1/3) are the unique ones that keep F0 and cancel h and h^2".

6.7 Observed order of accuracy and convergence

Idea. If the error behaves like $\mathrm{err} = \mathcal{C}\, h^p$ for a step h (cell width or time step), halving h divides the error by 2^p. Taking the base-2 logarithm of the ratio of successive errors gives the observed order:

$$p_k = \log_2 \frac{\mathrm{err}_k}{\mathrm{err}_{k+1}}, \tag{6.11}$$

IMPLEMENTED in `ConvergenceStudy::orders`, src/postprocessing/comparison.rs, lines 112 to 118. The formula assumes that each refinement halves h exactly. For the spatial study that requires every n_ℓ to be even (the factor 0.5 is rounded, line 132); for the temporal study it requires $t_{\mathrm{end}}/\Delta t_{\mathrm{in}}$ to stay an integer. Both hold for the synthetic input (Table 7.3, item K2).

An order computed from errors close to the accuracy of the reference is meaningless, so the study is declared resolved only if

$$\mathrm{err}_k > 3\,\varepsilon_F \quad \text{for every } k, \tag{6.12}$$

IMPLEMENTED in `resolved`, src/postprocessing/comparison.rs, lines 120 to 126. The study builds its own series with $\Lambda_{\max} = 45/t_{\mathrm{end}} = 2.25 \times 10^{-4}\ \mathrm{s}^{-1}$ (src/postprocessing/comparison.rs, line 186), which keeps only the two slowest modes; its smallest flux ratio is the same 5.504×10^{-5} as for the 32-mode series of the main run (independent recomputation), so $\varepsilon_F = 3.997 \times 10^{-12}$ there as well. Hence $3\varepsilon_F = 1.199 \times 10^{-11}$, while the smallest error is 8.556×10^{-8}.

Measured values.

Table 6.3 Observed orders for the synthetic input (calculated). Rows 1 and 2 are the v1.0.0 output (`tests/golden/v1.0/convergence_study.csv` and `main_run_stdout.txt`); row 3 uses the 200-cell time-extrapolated FV value 0.4325501631858728 as reference (the spatial row for 200 cells of `convergence_study.csv`), which v1.0.0 does not use as a reference.

Study	Steps	Errors err_k	Orders p_k
Space (time-extrapolated, $\Delta t = 200$ s)	100, 200, 400, 800, 1600 cells	2.190×10^{-5}, 5.476×10^{-6}, 1.369×10^{-6}, 3.423×10^{-7}, 8.556×10^{-8}	2.00, 2.00, 2.00, 2.00
Time, against the series (printed)	$\Delta t = 800$, 400, 200, 100, 50 s, 200 cells	4.912×10^{-4}, 2.485×10^{-4}, 1.270×10^{-4}, 6.626×10^{-5}, 3.587×10^{-5}	0.98, 0.97, 0.94, 0.89
Time, against the same-mesh reference	same runs	4.857×10^{-4}, 2.430×10^{-4}, 1.216×10^{-4}, 6.079×10^{-5}, 3.040×10^{-5}	0.999, 1.000, 1.000, 1.000

Why the printed temporal orders fall from 0.98 to 0.89. The temporal study measures each error against the exact series, so each error contains the spatial error of the 200-cell mesh as well. Let $e_{\mathrm{sp}} = F^{\mathrm{ex}} - F_0 = 5.476 \times 10^{-6}$ be that spatial error (F_0 the time-converged 200-cell value) and $\chi_1 \approx -6.08 \times 10^{-7}$ s^{-1} the BE coefficient of Eq. (6.9) (negative: F grows as Δt shrinks). Both parts have the same sign, so

$$\mathrm{err}_k = |\chi_1|\,\Delta t_k + e_{\mathrm{sp}}.$$

With $s = |\chi_1|\Delta t_{k+1}$ and $\Delta t_k = 2\Delta t_{k+1}$:

$$p_k = \log_2 \frac{2s + e_{\mathrm{sp}}}{s + e_{\mathrm{sp}}} < 1 \quad \text{for } e_{\mathrm{sp}} > 0,$$

and p_k falls towards 0 as $\Delta t \to 0$, because the constant e_{sp} takes over. With the numbers above this model gives 0.984, 0.969, 0.939, 0.885, the printed values. Backward Euler itself is first order; the printed numbers understate it. The window $[0.8, 1.2]$ asserted in examples/verify.rs, lines 189 to 191 accepts the contaminated values. Measuring against a same-mesh reference is a FUTURE IMPROVEMENT (Table 7.4).

Convergence in words. Consistency (Section 6.5: the leftover tends to zero) and stability (Section 6.4: leftovers are not amplified) together give convergence: the discrete solution tends to the exact one as w and Δt tend to zero. For linear finite-difference schemes this is the content of the Lax equivalence theorem (Strikwerda, 2004; established textbook result). Section 6.10 proves a version of it directly for Code B.

Status. Spatial order 2 and temporal order 1: **DEMONSTRATED NUMERICALLY**. The explanation of the printed temporal orders: **PROVED** (algebra) and checked against the data. Script checks: "PASS 6.11 spatial orders from convergence_study.csv round to 2.00, 2.00, 2.00, 2.00", "PASS 6.11 printed temporal orders 0.98, 0.97, 0.94, 0.89 are reproduced", "PASS 6.11 against the same-mesh reference the errors are 4.857e-4 ... 3.040e-5 and the orders 0.999 to 1.000", "PASS 6.11 model e_k = K dt_k + E_s with K = 6.08e-7 s^-1 reproduces the printed orders to 0.01", "PASS 6.12 resolution test: every printed error exceeds 3 eps_F = 1.199e-11". The two-layer controls of `verify` also give spatial orders 2.0 (`tests/golden/v1.0/verify_stdout.txt`, lines `TWO_LAYER_MODAL`).

6.8 Round-off in the elimination and in the balance

6.8.1 Fourier number

The cell Fourier number compares the time step with the time diffusion needs to cross one cell:

$$\mathrm{Fo} = \frac{D\,\Delta t}{w^2}, \tag{6.13}$$

DERIVED NOT IMPLEMENTED (it appears only in a code comment, src/numerical/finite_volume.rs, lines 126 to 131). For an interior cell $g_j^W \approx A\,D/w$ and $v_j \approx A\,w/\Delta t$, so $g_j^W / v_j \approx \mathrm{Fo}$: at large Fourier number the conductances dwarf the volume term v_j.

6.8.2 Where the textbook pivot loses digits

The textbook Thomas pivot is $b_j - a_j c'_{j-1}$, which in the present notation is

$$d_j = \left(v_j + g_j^W + g_j^E\right) - \frac{g_j^W\, g_{j-1}^E}{d_{j-1}}.$$

With $g_{j-1}^E = g_j^W$ and $d_{j-1} = e_{j-1} + g_j^W$, the subtraction hides inside e_j:

$$g_j^W - \frac{\left(g_j^W\right)^2}{e_{j-1} + g_j^W} = \frac{g_j^W\left(e_{j-1} + g_j^W\right) - \left(g_j^W\right)^2}{e_{j-1} + g_j^W} = \frac{g_j^W\, e_{j-1}}{e_{j-1} + g_j^W}.$$

The left side subtracts two numbers of size g_j^W to produce a result that can be far smaller (as small as e_{j-1}). Its absolute rounding error is about $\frac{1}{2}\epsilon_{\mathrm{m}}\, g_j^W$, which relative to $e_j \geq v_j$ can be as large as about $\frac{1}{2}\epsilon_{\mathrm{m}}\,\mathrm{Fo}$. The right side is the subtraction-free form used by Code B, Eq. (5.18).

6.8.3 Why the subtraction-free form is accurate

Step 1: damping of earlier errors. Let $h(e) = g\,e/(e+g)$ with $g > 0$. If e carries a small relative error δ, the relative error of h is δ times the logarithmic derivative

$$\frac{d\ln h}{d\ln e} = 1 - \frac{e}{e+g} = \frac{g}{e+g} \in (0,1),$$

so it is smaller than δ: an error inherited from cell $j-1$ is damped, never amplified.

Step 2: sums of non-negative numbers. The relative error of a sum of non-negative numbers is at most the largest relative error among the terms, plus one rounding.

Step 3: the recurrence. Write $u = \frac{1}{2}\epsilon_{\mathrm{m}}$ for the unit roundoff. The product $g_j^W e_{j-1}$, the sum $e_{j-1} + g_j^W$ and the quotient each add one rounding; to first order the relative

error of the quotient is at most $\frac{g_j^W}{e_{j-1}+g_j^W}\delta(e_{j-1}) + 3u \le \delta(e_{j-1}) + 3u$ (Step 1 for the inherited part). The stored $v_j = \mathrm{fl}(V_j/\Delta t)$ carries its own rounding u, and the final addition adds one more. By Step 2, $\delta(e_j) \le \delta(e_{j-1}) + 4u = \delta(e_{j-1}) + 2\epsilon_\mathrm{m}$, starting from $\delta(e_1) = \delta(v_1) \le u$. Hence $\delta(e_j) \le u + 2(j-1)\,\epsilon_\mathrm{m}$, and d_j adds one rounding. The pivots are thus accurate to about $2N_c\,\epsilon_\mathrm{m} \approx 9 \times 10^{-14}$ for $N_c = 200$ in this worst case, with no Fourier number in the bound. Repeating the argument for y_j and C_j^{n+1} gives a worst-case relative error that grows at most like $N_c^2\epsilon_\mathrm{m}$, again free of Fo.

Status. No subtraction and exact sign: **PROVED** (Section 6.3.3). The first-order bounds of this subsection: derived for STRATO as a sketch; their constants are **CLAIMED ONLY**.

Measurement. On the unit-test case of src/numerical/finite_volume.rs, lines 375 to 389 (Fo $= 2.5 \times 10^5$, 250 cells, 100 steps) two independent Python replicas give a worst relative balance drift of 1.9×10^{-16} and 3.9×10^{-16} with the subtraction-free pivot and 9.5×10^{-14} with the textbook pivot. The subtraction-free form is better by a factor of roughly 250 to 500 (the exact factor depends on the rounding order of each replica), and both stay far below the test threshold 10^{-12}. The Rust test asserts only the threshold; it prints no drift value. The code comments that the textbook pivot "breaks the inventory balance" (line 128) and "loses ~6 digits" at this Fourier number (line 377), and the delivered Phase 1 statement of a drift "to about 10^{-9}", are **CLAIMED ONLY**: they are not reproduced (Table 7.3, items I5 and K4). **Status of the measurement: DEMONSTRATED NUMERICALLY** ("PASS 6.13 balance drift at Fo = 2.5e5: both pivots stay far below 1e-12 (no 'six digits lost')", "PASS 6.13 subtraction-free min C >= 0 at Fo = 2.5e5 (textbook value shown for comparison)").

6.8.4 Round-off in the balance residual

In exact arithmetic $\varepsilon^n = 0$ (Section 6.1). In floating point three sources make it non-zero:

- **The inventory sum** $M^n = \sum_j V_j C_j^n$ (lines 114 to 120) adds N_c non-negative terms in sequence; its relative error is at most about $N_c\,\epsilon_\mathrm{m}/2$ (established textbook bound, Higham, 2002). It is recomputed from the cell values at every step, so this error does not accumulate over steps.
- **The release sum** uses Kahan compensated summation (lines 211 to 217). The compensation term κ (Eq. (5.23)) stores the low-order bits lost in each addition and feeds them back into the next one, so the summation error stays near $\epsilon_\mathrm{m} M_\mathrm{rel}$ instead of growing with the number of steps (established textbook result, Higham, 2002).
- **The linear solves.** Repeating the telescoping of Section 6.1 with the computed $\tilde{\mathbf{C}}^m$ shows that ε^n collects $\Delta t \sum_{m \le n} \mathbf{1}^\mathsf{T}\sigma^m$, where $\sigma^m = \mathbf{B}\tilde{\mathbf{C}}^m - (\mathbf{V}/\Delta t)\tilde{\mathbf{C}}^{m-1}$ is the algebraic residual of step m, together with the rounding of $\Delta t \cdot \mathrm{fl}(V_j/\Delta t) \ne V_j$. This part accumulates over the N_t steps.

No bound on the accumulated value is proved here. **Status: DEMONSTRATED NUMERICALLY.** The worst relative residual over all output rows of the main run is 2.806×10^{-13}, about $1.3 \times 10^3\,\epsilon_\mathrm{m}$, on the last row (`tests/golden/v1.0/timeseries.csv`); for 100 to 1600 cells `verify` prints last-row values from 5.400×10^{-15} to $2.806 \times$

10^{-13}. Two independent replicas give 1.46×10^{-13} and 5.2×10^{-14} over all 1000 steps, which shows that the size of this residual depends on the rounding order ("PASS 6.1 replica: worst relative balance residual |eps$^{n|/M}$0 < 1e-12 (Rust: 2.806e-13 on output rows)"). The printed key `max_final_balance_residual_mol` $= 9.403\times10^{-24}$ mol is the final row only (Table 7.3, item I8); divided by M^0 it is the same 2.806×10^{-13}.

6.9 Cost of the refinement study

Because each BE step costs $O(N_c)$ (Section 6.3.2), one run costs in proportion to the number of cell-steps:

$$W = \frac{t_{\text{end}}}{\Delta t_{\text{in}}}\,N_c, \qquad W_{\text{study}} = 1.75 \sum_{\lambda_x\in\{0.5,1,2,4,8\}} W(\lambda_x) + \sum_{\lambda_t\in\{4,2,1,1/2,1/4\}} W(\lambda_t), \quad (6.14)$$

IMPLEMENTED in `run_cost` and `study_cost`, src/postprocessing/comparison.rs, lines 159 to 176, where $W(\lambda_x)$ is the cost with n_ℓ scaled by λ_x and $W(\lambda_t)$ the cost with Δt_{in} scaled by λ_t. The study is skipped if $W_{\text{study}} > W_{\text{lim}} = 4\times10^9$ (src/main.rs, line 21).

The factor 1.75 is wrong (Table 7.3, item K1): each spatial point calls `time_extrapolated_release`, which runs $\Delta t/4$, $\Delta t/2$ and Δt, costing $4+2+1 = 7$ times $W(\lambda_x)$. For the synthetic case the formula gives $1.75\times3.1\times10^6 + 1.55\times10^6 = 6.98\times10^6$ cell-steps; the true cost is $7\times3.1\times10^6 + 1.55\times10^6 = 2.33\times10^7$, still far below W_{lim}. Only the cost guard is affected, never a result. The fix (factor 7) is a FUTURE IMPROVEMENT. **Status: PROVED** (by counting). Script checks: "PASS 6.14 implemented study cost 1.75 * 3.1e6 + 1.55e6 = 6.975e6 cell-steps", "PASS 6.14 true spatial cost of three Richardson runs is 7 * 3.1e6 = 2.17e7 (factor 4 above the estimate)".

6.10 Convergence of the semi-discrete scheme: an energy argument

What we want. A proof that the semi-discrete solution of Code B converges to the exact cell averages, in the spirit of the Lax equivalence theorem: consistency (Section 6.5) plus stability (Section 6.4) implies convergence.

Assumptions. The exact solution is smooth inside each layer on the time interval considered, so that c'' and c''' are bounded there and the expansions of Section 6.5.2 hold. The constants hidden in the $O(\cdot)$ remainders of Section 6.5.2 are bounded uniformly over all faces with $r_f \geq w$. This uniformity is checked by the direct expansions of Section 6.5.2(d) for the first three faces, and it is assumed, not proved, for the faces in between. Exact arithmetic. Time is kept continuous (the semi-discrete system (5.12)).

Step 1: the error equation. Let $\eta_j(t) = C_j(t) - \bar{c}_j(t)$ (mol m^{-3}). Subtract the equation of Section 6.5.1 for $\bar{c}_j$ from Eq. (5.12) for C_j. The conductance terms are linear, so they act on the difference:

$$V_j\frac{d\eta_j}{dt} = -\left(\mathbf{A}\eta\right)_j - \left(\tau_{j+1/2} - \tau_{j-1/2}\right).$$

Step 2: multiply by η_j and add over cells. Since $\frac{d}{dt}\frac{1}{2}\eta_j^2 = \eta_j\, d\eta_j/dt$,

$$\frac{d}{dt}\,\frac{1}{2}\|\eta\|_V^2 = -\eta^{\mathsf{T}}\mathbf{A}\eta - \sum_{j=1}^{N_c}\eta_j\left(\tau_{j+1/2} - \tau_{j-1/2}\right).$$

Step 3: summation by parts. This is the discrete form of integration by parts: regroup the sum by faces instead of by cells. With $\tau_{1/2} = 0$ and $\tau_{N_c+1/2} = \tau_R$,

$$\sum_{j=1}^{N_c}\eta_j\left(\tau_{j+1/2} - \tau_{j-1/2}\right) = \sum_{j=1}^{N_c-1}\tau_{j+1/2}\left(\eta_j - \eta_{j+1}\right) + \tau_R\,\eta_{N_c}.$$

Step 4: use the energy of A. By Section 6.4.2, $\eta^{\mathsf{T}}\mathbf{A}\eta = \sum_f G_f\,\Delta\eta_f^2 + G_R\,\eta_{N_c}^2$ with $\Delta\eta_f = \eta_j - \eta_{j+1}$. Each face term of Step 3 is bounded by Young's inequality,

$$|\tau\,\Delta| \le \frac{G}{2}\,\Delta^2 + \frac{\tau^2}{2G},$$

which holds because $\frac{G}{2}\Delta^2 + \frac{\tau^2}{2G} - \tau\Delta = \frac{1}{2}\left(\sqrt{G}\,\Delta - \tau/\sqrt{G}\right)^2 \ge 0$ (and the same with $-\tau$). Apply it to every internal face and to the outer face:

$$\begin{aligned}\frac{d}{dt}\,\frac{1}{2}\|\eta\|_V^2 &\le -\frac{1}{2}\left(\sum_f G_f\Delta\eta_f^2 + G_R\eta_{N_c}^2\right) + \frac{1}{2}\left(\sum_f \frac{\tau_f^2}{G_f} + \frac{\tau_R^2}{G_R}\right)\\ &\le \frac{1}{2}\left(\sum_f \frac{\tau_f^2}{G_f} + \frac{\tau_R^2}{G_R}\right).\end{aligned}$$

Step 5: integrate in time. Multiply by 2 and integrate from 0 to t:

$$\|\eta(t)\|_V^2 \le \|\eta(0)\|_V^2 + \int_0^t\left(\sum_{j=1}^{N_c-1}\frac{\tau_{j+1/2}^2}{G_{j+1/2}} + \frac{\tau_R^2}{G_R}\right)dt'. \tag{6.15}$$

DERIVED NOT IMPLEMENTED (derived for STRATO). In v1.0.0, $\eta(0) = \mathbf{0}$: the kernel boundary R_1 is a face, so the initial cell values (5.20) are the exact cell averages of Eq. (3.5).

Step 6: size of the right-hand side. An interior face has $\tau = O(w^2)$ (Section 6.5.2) and $G = O(1/w)$, so $\tau^2/G = O(w^5)$, and there are about L/w such faces: together $O(w^4)$. More precisely, (a) gives $\tau \approx -\frac{2\pi}{3}Dw^2 r_f(\dots)$ and $G \approx 4\pi r_f^2 D/w$, so $\tau^2/G \approx \frac{\pi}{9}D\,w^5(\dots)^2$: the factor r_f^2 cancels and faces at small radius are not penalised. The faces next to the centre (Section 6.5.2(d)) have $\tau = O(w^3)$ and $G = O(w)$, so $\tau^2/G = O(w^5)$ each, and there are only a few of them. Each interface face and the outer face has $\tau = O(w)$ and $G = O(1/w)$, so $\tau^2/G = O(w^3)$, and there are five

of them. Hence $\|\eta(t)\|_V = O(w^{3/2})$. The interface faces, where the cell residual is of order one, enter only through τ^2/G, which is small: this is the mechanism behind supraconvergence.

Step 7: from the error to the release fraction. In exact arithmetic both inventories obey their balances, the cells tile the sphere, and the discrete and exact M^0 agree, so $F^{\mathrm{FV}} - F = -\sum_j V_j \eta_j / M^0$. The Cauchy-Schwarz inequality gives

$$\left|\sum_j V_j \eta_j\right| \le \|\eta\|_V \left(\sum_j V_j\right)^{1/2} = \|\eta\|_V \left(\frac{4\pi}{3} R^3\right)^{1/2},$$

so the release error is at most of order $w^{3/2}$.

Fully discrete version (sketch). Replacing d/dt by the BE difference and using $2a(a-b) = a^2 - b^2 + (a-b)^2 \ge a^2 - b^2$ in place of Step 2 gives the same estimate with a sum over steps and an extra term from the BE local error (Section 6.5.3), hence a bound of order $w^{3/2} + \Delta t$. This version is not written out here.

Limits of the argument. The bound is not sharp: the observed spatial order of F is 2.00 (Section 6.7), and a sharper proof of second order is not given. The smoothness assumption fails near $t = 0$ in v1.0.0, because the initial concentration jumps at R_1; there c'' grows without bound as $t \to 0$ and the integral in Eq. (6.15) is not guaranteed finite. The proof therefore covers smooth solutions; for the v1.0.0 initial state, convergence rests on the measurements.

Status: PROVED for solutions that are smooth in each layer (semi-discrete), under the uniform-remainder assumption stated above. For the v1.0.0 initial step and the fully discrete scheme: **CLAIMED ONLY** as a proof, **DEMONSTRATED NUMERICALLY** by the observed orders. Script checks: “PASS 6.15 summation by parts: sum_j e_j (tau_{j+1/2} - tau_{j-1/2}) = sum_f tau_f (e_j - e_{j+1}) + tau_R e_N”, “PASS 6.15 Young: |tau delta| <= G delta^2/2 + tau^2/(2G) since (sqrt(G) delta - tau/sqrt(G))^2 >= 0”, “PASS 6.15 interior faces contribute O(w^4) (about L/w faces, tau = O(w^2), G = O(1/w))”, “PASS 6.15 each interface or outer face contributes O(w^3), so ||e||_V = O(w^(3/2)) at least”.

6.11 Accuracy of the analytical series (Code C)

Code C evaluates $F(t) = 1 - \sum_{n \le K} w_n e^{-\Lambda_n t}$, Eq. (4.22) (src/analytical/sturm_liouville.rs, lines 381 to 393). Its accuracy is limited by the modes left out (Section 6.11.1) and by rounding in the modes that are kept (Sections 6.11.2 and 6.11.3), which matters most when F is small.

6.11.1 Truncation, mode cap and validity time

What the code does (IMPLEMENTED, Eq. (4.26)). `exact_release` requests $\Lambda_{\max} = 45/t_{\min}$ (line 412). If more than $K_{\max} = 20\,000$ eigenvalues lie below $\Lambda_{\max}$, the bound is lowered to Λ_{cov} (lines 202 to 217); otherwise $\Lambda_{\mathrm{cov}} = \Lambda_{\max}$. Every eigenvalue $\le \Lambda_{\mathrm{cov}}$ is found, because roots are isolated by bisection on the Sturm count (Part IV, Eqs. (4.13) and (4.15)). The series is used only for $t \ge t_{\mathrm{valid}} = 40/\Lambda_{\mathrm{cov}}$ (lines 375 to 379; earlier times return no value, lines 413 to 416).

Bound on the omitted modes. Every omitted mode has $\Lambda_n > \Lambda_{\text{cov}}$, so $e^{-\Lambda_n t} \le e^{-\Lambda_{\text{cov}} t}$ for $t > 0$. For $t \ge t_{\text{valid}}$, $\Lambda_{\text{cov}} t \ge 40$, and

$$\left| \sum_{n>K} w_n \, e^{-\Lambda_n t} \right| \le e^{-\Lambda_{\text{cov}} t} \sum_{n>K} |w_n| \le e^{-40} \sum_{n>K} |w_n|, \qquad e^{-40} = 4.25 \times 10^{-18}. \quad (6.16)$$

DERIVED NOT IMPLEMENTED (derived for STRATO). The per-term factor is proved. The sum $\sum_{n>K} |w_n|$ is not bounded by any proof in v1.0.0: completeness of the eigenfunctions and a tail bound are FUTURE IMPROVEMENTS (Table 7.4). When the cap is not reached, $t_{\text{valid}} = \frac{40}{45} t_{\text{min}} < t_{\text{min}}$, so every requested time is valid and each omitted term carries at most $e^{-45} = 2.86 \times 10^{-20}$ at t_{min}. When the cap is reached, $\Lambda_{\text{cov}} < 45/t_{\text{min}}$ and the earliest times may return no value; the code comment at lines 396 to 398 does not mention this case (Table 7.3, item K6).

Synthetic case. $t_{\text{min}} = 2000$ s, 32 modes, cap not reached, $\Lambda_{\text{cov}} = 0.0225\ \text{s}^{-1}$, $t_{\text{valid}} = 1777.78$ s. Modes 33 to 95 (all eigenvalues up to $0.2\ \text{s}^{-1}$) contribute 1.444×10^{-22} at t_{valid}. At $t = 0$ the same 32-mode series would give $F(0) = 1 - \sum w_n = -7.28 \times 10^{-6}$ instead of 0, which shows why times below t_{valid} must be excluded.

Status: per-term bound **PROVED**; smallness of the omitted sum **DEMONSTRATED NUMERICALLY** for the synthetic case. Script checks: "PASS 6.16 replica: 32 modes below Lambda_max = 45/t_min = 0.0225 s^-1 (main run: analytical_modes=32)", "PASS 6.16 t_valid = 40/Lambda_cov = 1777.78 s < t_min = 2000 s (= (40/45) t_min)", "PASS 6.16 per-term factor of every omitted mode: e^{-40} = 4.25e-18 at t_valid, e^{-45} = 2.86e-20 at t_min", "PASS 6.16 truncated series at t = 0 gives F(0) = 1 - sum w = -7.28e-6 (not 0): why t < t_valid is excluded", "PASS 6.16 omitted modes 33 to 95 contribute < e^{-40} sum|w| at t_valid (observed below 1e-18)". The weight deficit falling as more modes are kept is asserted in examples/verify.rs, lines 228 to 246.

6.11.2 Errors in the kept eigenvalues and weights

Step 1: perturb F. Let the computed values be $w_n + \delta w_n$ and $\Lambda_n + \delta\Lambda_n$. To first order,

$$\delta F(t) = -\sum_n \delta w_n \, e^{-\Lambda_n t} + \sum_n w_n \, t \, \delta\Lambda_n \, e^{-\Lambda_n t}.$$

Step 2: rewrite the eigenvalue term. Multiply and divide by Λ_n: $w_n \, t \, \delta\Lambda_n \, e^{-\Lambda_n t} = w_n \, (\delta\Lambda_n / \Lambda_n) \, x_n e^{-x_n}$ with $x_n = \Lambda_n t$.

Step 3: bound xe^{-x}. Its derivative $(1 - x)e^{-x}$ vanishes at $x = 1$, so $xe^{-x} \le e^{-1}$ for all $x > 0$. Hence

$$|\delta F(t)| \le \sum_n |\delta w_n| \, e^{-\Lambda_n t} + e^{-1} \max_n \frac{|\delta\Lambda_n|}{\Lambda_n} \sum_n |w_n|. \quad (6.17)$$

DERIVED NOT IMPLEMENTED (derived for STRATO; first order in the perturbations).

Numbers. The weights are of order one and alternate in sign, $\sum_n |w_n| = 2.023$ for the 32 modes. Against an independent 40-digit refinement in the script, the double-precision replica has eigenvalue errors up to 3.72×10^{-13} (relative) and weight errors up to 7.66×10^{-14} (absolute); its release fraction errs by 7.8×10^{-14}, 4.7×10^{-14} and 9.8×10^{-14} at $t = 2 \times 10^3$, 2×10^4 and 2×10^5 s, inside the bound (6.17) in each case. For the Rust code itself, a one-off comparison of the library with a 50-digit oracle (not stored in the repository, Section 7.5) gave eigenvalue errors up to 3.7×10^{-13} (relative), weight errors up to 9.0×10^{-14} and $|\delta F(2 \times 10^5 \text{ s})| = 1.9 \times 10^{-13}$ against a 50-digit oracle.

The accuracy floor. The code reports $\varepsilon_F = 2.2 \times 10^{-16} / \min(\rho_{\min}, 1) = 3.997 \times 10^{-12}$, Eq. (4.28) (lines 366 to 373). It is a heuristic: it uses one cancellation ratio, evaluated on one side of each interface (Table 7.3, item I15), and no proof links it to Eq. (6.17). The observed Rust error is about 21 times below it. The comment that "the true error stayed about 10 to 20 times below this estimate" (lines 369 to 370) rests on this single case and is **CLAIMED ONLY** (Table 7.3, item I16).

Status: Eq. (6.17) **PROVED** (first order); the measured errors **DEMONSTRATED NUMERICALLY**; the floor as an upper bound **CLAIMED ONLY**. Script checks: "PASS 6.17 replica eigenvalues agree with the 40-digit oracle (relative error < 1e-12; audit Rust vs 50-digit: 3.7e-13)", "PASS 6.17 replica weights agree with the oracle (absolute error < 1e-12; audit Rust: 9.0e-14)", "PASS 6.17 max over x > 0 of x e^{-x} is e^{-1} (at x = 1)", three lines "PASS 6.17 replica error at t = ... s is within the propagation bound and below eps_F = 3.997e-12", "PASS 6.17 oracle F(2e5 s) = 0.43255563909304 (audit 50-digit: 0.43255563909304174)".

6.11.3 Slightly negative values at early time

At early time almost nothing has been released, and $F = 1 - \sum_n w_n e^{-\Lambda_n t}$ is the difference of two nearly equal numbers. At $t = 2 \times 10^3$ s the replica sum is 0.99999999999992 while the true F is 1.6×10^{-15} (40-digit oracle; the 50-digit evaluation of the one-off comparison gives the same value). The subtraction itself is exact to within about $\frac{1}{2}\epsilon_{\mathrm{m}} \sum |w_n| \approx 2 \times 10^{-16}$, but the weight and eigenvalue errors of Section 6.11.2, about 10^{-13}, are more than fifty times larger than the true value, so even the sign of the computed F is not reliable. Rust returns $F(2 \times 10^3 \text{ s}) = -8.55 \times 10^{-14}$ (first row of `tests/golden/v1.0/comparison_fv_vs_analytical.csv`, $-8.548717289613705 \times 10^{-14}$); the replica returns $+7.98 \times 10^{-14}$. Both are within ε_F. **Status: DEMONSTRATED NUMERICALLY** ("INFO replica F(2e3 s) = 7.983e-14; Rust (audit 01) -8.55e-14; oracle 1.63e-15", "PASS 6.17 oracle F(2e3 s) = 1.6e-15 > 0"). Clamping or flagging negative values is a FUTURE IMPROVEMENT (Table 7.3, item K9).

6.12 Summary of numerical properties

Table 6.4 summarises every property of this part with its status and the evidence for it.

Table 6.4 Numerical properties of the implemented v1.0.0 methods. "p6_properties", "p6_truncation", "p6_orders" and "p6_series" are the scripts of Table 6.1, quoted by the prefix of their PASS lines; VER, FV and SL are `examples/verify.rs`, `src/numerical/finite_volume.rs` and `src/analytical/sturm_liouville.rs` at tag v1.0.0.

Property	Statement	Status	Evidence
Discrete conservation	$M^{n+1} + \Delta t\, G_R C_{N_c}^{n+1} = M^n$; $\varepsilon^n = 0$ in exact arithmetic	PROVED	Eq. (6.1), Section 6.1; p6_properties "6.1"; VER 199-223; worst 2.806×10^{-13}
Monotone inventory, $0 \le F \le 1$	$M^{n+1} \le M^n$; $F^n \in [0,1]$ (exact arithmetic)	PROVED	Section 6.1 corollaries; p6_properties "6.1 replica: inventory"; VER 216
M-matrix	$\mathbf{B}$ Z-matrix, positive diagonal, row margin v_j; $\mathbf{B}^{-1} \ge 0$	PROVED	Eq. (6.2); p6_properties "6.2" (four lines)
Positivity and maximum principle	$0 \le C_j^{n+1} \le \max_k C_k^n$ for every Δt	PROVED	Eq. (6.3); p6_properties "6.3 replica"; VER 215
Positivity in floating point	$C_j^{n+1} \ge 0$ exactly, no tolerance	PROVED (IEEE model)	Eq. (6.4); p6_properties "6.4 replica"; FV test 310 is looser
Pivots and pivoting	$v_j \le e_j \le v_j + g_j^W$, $d_j > 0$, multipliers in $(0,1)$; no pivoting	PROVED	Section 6.3.1; p6_properties "6.4b"
Cost of a step	13 operations per cell, $O(N_c)$ work and memory	PROVED (count)	Section 6.3.2; FV 140-158, 178-179
A-stability, L-stability of BE	$\lvert\mathcal{R}(\zeta)\rvert \le 1$ for $\mathrm{Re}\,\zeta \le 0$; $\mathcal{R} \to 0$ at $-\infty$	PROVED (textbook)	Section 6.4.1; p6_properties "6.4" (three lines)
Symmetric positive definite	$\mathbf{A} = \mathbf{A}^{\mathsf{T}}$, $\mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x} > 0$; $\mathbf{S}$ SPD	PROVED	Section 6.4.2; p6_properties "6.4"; $\mu_{\min} = 3.2733 \times 10^{-6}$ s^{-1}
Unconditional stability	$\lVert\mathbf{C}^{n+1}\rVert_V \le \lVert\mathbf{C}^n\rVert_V$ for every Δt	PROVED	Eq. (6.5); p6_properties "6.5"
Forward Euler positivity bound (diagnostic)	$\Delta t_{\mathrm{FE}} = \min_j V_j/(g_j^W + g_j^E)$	PROVED; IMPLEMENTED value	Eq. (6.6); FV 238-252; 7.812×10^{-3} s
Forward Euler L^2 bound	$\Delta t \le 2/\mu_{\max}$; always $\Delta t_{\mathrm{FE}} \le 2/\mu_{\max}$ (Gershgorin)	PROVED; DERIVED NOT IMPLEMENTED	Eq. (6.7), Section 6.4.4; 7.824×10^{-3} s
Stable but first-order inaccurate at large Fo	Fo up to 1.28×10^4; 2.8 % relative error at 2.2×10^4 s	DEMONSTRATED NUMERICALLY	Section 6.4.5; `tests/golden/v1.0/comparison_fv_vs_analytical.csv`
Flux consistency, interior	$\tau = O(w^2)$	PROVED	Section 6.5.2(a); p6_truncation "6.5a"
Flux consistency, interface and outer face	$\tau = O(w)$; vanishes iff $w_L/w_R = \sqrt{D_L/D_R}$	PROVED	Section 6.5.2(b), (c); p6_truncation "6.5b", "6.5c"
Flux consistency, faces next to the centre	$\tau = O(w^3)$ at $r_f = w, 2w, 3w$	PROVED (Taylor, three faces)	Section 6.5.2(d)
BE local error	$-\frac{1}{2}\Delta t^2 y'' + O(\Delta t^3)$	PROVED	Section 6.5.3; p6_truncation "6.5 BE"
Convergence (energy estimate)	$\lVert\eta\rVert_V = O(w^{3/2})$ for smooth solutions	PROVED (smooth case, uniform remainders assumed); CLAIMED ONLY for the v1.0.0 initial step	Eq. (6.15); p6_orders "6.15"

Property	Statement	Status	Evidence
Spatial order 2 of F	$p_k = 2.00$ (five-layer), 2.0 (two-layer)	DEMONSTRATED NUMERICALLY	Eq. (6.8), Table 6.3; p6_orders "6.11"
Temporal order 1 of F	0.999 to 1.000 against a same-mesh reference; printed 0.98 to 0.89 explained by e_{sp}	DEMONSTRATED NUMERICALLY	Table 6.3; p6_orders "6.11" (five lines)
Richardson cancellation	removes $\chi_1 \Delta t$ and $\chi_2 \Delta t^2$, remainder $\chi_3 \Delta t^3/8$	PROVED (given Eq. (6.9))	Eq. (6.10); p6_orders "6.9", "6.10"
Subtraction-free accuracy	no subtraction; worst-case bounds free of Fo	PROVED (sign); CLAIMED ONLY (constants)	Section 6.8.3
Pivot round-off at $\text{Fo} = 2.5 \times 10^5$	drift 1.9×10^{-16} to 3.9×10^{-16} (subtraction-free, two replicas) against 9.5×10^{-14} (textbook)	DEMONSTRATED NUMERICALLY (replicas; Rust asserts only $< 10^{-12}$)	p6_properties "6.13"; FV 375-389
Textbook pivot "loses about 6 digits"	code comment FV 128 and 377	CLAIMED ONLY (not reproduced)	Section 6.8.3; inventory I5, K4
Balance round-off	relative residual near 10^{-13}, rounding-order dependent	DEMONSTRATED NUMERICALLY (no bound proved)	Section 6.8.4; main run 2.806×10^{-13}; VER asserts $< 10^{-12}$
Study cost estimate	factor 1.75 should be 7	PROVED (count)	Eq. (6.14); p6_orders "6.14"; inventory K1
Series truncation	omitted terms $\leq e^{-40} \sum_{n>K} \lvert w_n \rvert$ for $t \geq t_{\text{valid}}$	PROVED per term; tail sum DEMONSTRATED NUMERICALLY	Eq. (6.16); p6_series "6.16"
Series eigenvalue and weight accuracy	3.7×10^{-13} relative, 9.0×10^{-14} absolute (Rust)	DEMONSTRATED NUMERICALLY	Eq. (6.17); one-off comparison of the library (Section 7.5); `v1_0_0_p6_series.py`
Series accuracy floor	$\varepsilon_F = 3.997 \times 10^{-12}$ bounds the series error	CLAIMED ONLY (heuristic, one case)	Eq. (4.28); SL 366-373
Negative early-time series value	$F(2 \times 10^3 \text{ s}) = -8.55 \times 10^{-14}$ (true 1.6×10^{-15})	DEMONSTRATED NUMERICALLY	Section 6.11.3; `tests/golden/v1.0/comparison_fv_vs_analytical.csv`

Part VII: Connection between theory and implementation

Parts IV to VI end in equations. This part follows each of them into the Rust source of release v1.0.0 and says, for every equation, which function computes it and on which lines. The source of truth is the Git tag `v1.0.0` (commit `8c5d8a8`) of https://github.com/weewzr/STRATO. No Rust file was edited for this document, and every line number and link below is pinned to that tag. The numerical values quoted come from the golden files stored in the tag (written by the Rust program) and from the Python check scripts in `scripts/derivation_checks`; the Rust program itself was not run again while writing this part.

The classification labels are used as in the reading guide. IMPLEMENTED means that the code computes the stated expression and that the function and lines are given. DERIVED NOT IMPLEMENTED means that the statement follows from the mathematics and no code computes it. FUTURE means that v1.0.0 does not contain it. Numerical properties carry one of PROVED, DEMONSTRATED NUMERICALLY or CLAIMED ONLY. Every number in this part was calculated for the synthetic benchmark `inputs/synthetic/synthetic_five_layer.toml`, which is a numerical test case and contains

no literature TRISO data.

Index conventions are those of Part V. A cell is $j = 1, \dots, N_c$ in the equations and `i = j - 1` in Rust. A layer is $\ell = 1, \dots, 5$ in the equations and `material = l - 1` in Rust. Table 7.1 lists the quantities that recur below with the Rust name that carries each one.

Table 7.1 Mathematical quantities and the Rust names that carry them.

Symbol	Rust name	Where	Unit
$r_{j-1/2}$, $r_{j+1/2}$	`left`, `right`	`Cell`	m
r_j (volume centroid)	`centroid`	`Cell`	m
V_j	`volume`	`Cell`	m^3
D_j	`diffusivity`	`Cell`	$\mathrm{m}^2\ \mathrm{s}^{-1}$
$G_{j+1/2}$	`conductance[j - 1]`	`Mesh`	$\mathrm{m}^3\ \mathrm{s}^{-1}$
G_R	`outer_conductance`	`Mesh`	$\mathrm{m}^3\ \mathrm{s}^{-1}$
g_j^W, g_j^E	`gw[i]`, `ge[i]`	`backward_euler_step`	$\mathrm{m}^3\ \mathrm{s}^{-1}$
$v_j = V_j/\Delta t$, rhs_j	`v[i]`, `rhs[i]`	`backward_euler_step`	$\mathrm{m}^3\ \mathrm{s}^{-1}$, $\mathrm{mol}\ \mathrm{s}^{-1}$
e_j, d_j, y_j	`e` (`e_prev`), `den[i]`, `dp[i]`	`solve_diffusion_system`	$\mathrm{m}^3\ \mathrm{s}^{-1}$, $\mathrm{m}^3\ \mathrm{s}^{-1}$, $\mathrm{mol}\ \mathrm{m}^{-3}$
Δt_{in}, t_{end}, n_{out}	`dt_s`, `end_time_s`, `output_every_steps`	`Config`	s, s, 1
N_t, Δt	`steps`, `dt`	`run`	1, s
M^n, M^0	`inv`, `initial`	`run`	mol
M^n_{rel}, κ	`released`, `release_compensation`	`run`	mol
Φ_R^n	`release_rate`	`run`	$\mathrm{mol}\ \mathrm{s}^{-1}$
ε^n, F^n	`residual`, `release_fraction`	`run`	mol, 1
Λ, k_ℓ	`lambda` (`lam`), `k`	`shoot`, `eigenvalue_count`	s^{-1}, m^{-1}
$R_{\ell-1}$, R_ℓ, L_ℓ	`left`, `outer_radius` (`right`), `length`	`Layer`, `LayerStart`	m
(u, q)	`(u, q)`	`shoot`	scale fixed by $q(0) = 1$
θ_0, $\mathcal{N}(\Lambda)$	`theta0`, return value of `eigenvalue_count`	`eigenvalue_count`	rad, 1
$\Lambda_{\max}$, Λ_{cov}, $K_{\max}$	`lambda_max`, `lambda_covered` (`hi`), `max_modes` (`MAX_MODES`)	`eigenvalues`	s^{-1}, s^{-1}, 1
w_n	`weights`	`ModalRelease`	1
$\rho_{\min}$, ε_F, t_{valid}	`min_flux_ratio`, `accuracy_floor()`, `min_valid_time()`	`ModalRelease`	1, 1, s
$e(t_n)$	`err`, `abs_error`	`compare_release`, `ConvergencePoint`	1

7.1 From the derivation to Code B

Code B is `src/numerical/finite_volume.rs`. It imports only `Config` from the rest of the crate (src/numerical/finite_volume.rs, line 4), so everything it needs about the particle arrives through the configuration. The driver calls it in three statements, `Config::load`, `build_mesh` and `run` (src/main.rs, lines 37 to 39). The subsections follow the order of execution.

7.1.1 Input and validation

`Config::load`, src/config.rs, lines 62 to 77 rejects any value that is still the placeholder `INSERT_VALUE_HERE`, parses the TOML text and calls `validate`. `Config::validate`, src/

config.rs, lines 79 to 113 enforces the geometric statements of Part I: exactly five regions (src/config.rs, lines 80 to 82), outer radii strictly increasing from $R_0 = 0$ with $D_\ell > 0$ and at least one cell per layer (src/config.rs, lines 89 to 99), positive finite time controls (src/config.rs, lines 100 to 107) and $c_0 \geq 0$ (src/config.rs, lines 108 to 111). The temperature is read and checked (src/config.rs, lines 86 to 88) and then used in no formula (item K11 of Table 7.3).

7.1.2 Mesh

`build_mesh`, src/numerical/finite_volume.rs, lines 50 to 99 builds the cells and the face quantities in one pass. It validates the configuration again (src/numerical/finite_volume.rs, line 51), so every configuration made by a refinement study is checked as well.

Eq. (5.2) is the loop of src/numerical/finite_volume.rs, lines 54 to 73. The width of a layer is `width = (outer_radius_m - inner) / cells` (line 55), the left face of the cell with offset ι in its layer is `inner + k * width` (line 57; `k` is the loop variable that Section 5.2 writes as ι) and the right face is `inner + (k + 1) * width` except for the last cell of the layer, whose right face is set to `outer_radius_m` itself (lines 58 and 59). The last-face rule keeps every layer boundary exactly on a cell face in floating point (Section 5.2). The cell volume of Eq. (5.3) is `shell_volume`, src/numerical/finite_volume.rs, lines 38 to 40, called at src/numerical/finite_volume.rs, line 67. The centroid of Eq. (5.4) is `volume_centroid`, src/numerical/finite_volume.rs, lines 42 to 48, called at src/numerical/finite_volume.rs, line 66; the branch `a == 0.0` returns $\frac{3}{4}b$, which the general expression gives as well when $a = 0$.

The face quantities of Eqs. (5.5), (5.7) and (5.9) are computed for every pair of neighbouring cells:

```
    for pair in cells.windows(2) {
        let left = &pair[0];
        let right = &pair[1];
        let face = left.right;
        let area = 4.0 * PI * face * face;
        let dl = face - left.centroid;
        let dr = right.centroid - face;
        conductance.push(area / (dl / left.diffusivity + dr / right.diffusivity));
    }
```

Here `face` is $r_{j+1/2}$, `area` is $A_{j+1/2}$ of Eq. (5.5), `dl` and `dr` are $\delta^L_{j+1/2}$ and $\delta^R_{j+1/2}$ of Eq. (5.7), and the pushed value is $G_{j+1/2}$ of Eq. (5.9). The code has no branch for equal diffusivities, so the series-resistor formula is used at every internal face. The code is at src/numerical/finite_volume.rs, lines 78 to 87. The outer conductance of Eq. (5.11) is src/numerical/finite_volume.rs, lines 88 to 91: `outer_radius` is R, `outer_area` is $4\pi R^2$, and `outer_conductance` is $G_R = 4\pi R^2 D_5/(R - r_{N_c})$ with D_5 and r_{N_c} taken from the last cell. The centre condition of Eq. (5.10) has no counterpart in the mesh, because `conductance` has no entry for $r = 0$. It appears in the next step as `gw[0] = 0`.

7.1.3 Initial state and step size

`initial_concentration`, src/numerical/finite_volume.rs, lines 101 to 112 implements Eq. (5.20): c_0 in every cell with `material == 0` and zero elsewhere. `inventory`, src/numerical/finite_volume.rs, lines 114 to 120 computes $\sum_j V_j C_j$ (Eq. (5.21)), and `run` uses it once for M^0 (src/numerical/finite_volume.rs, lines 186 to 189, with a guard

that $M^0 > 0$). The step size is Eq. (5.16), src/numerical/finite_volume.rs, lines 190 to 191: `steps = ceil(end_time_s / dt_s)` and `dt = end_time_s / steps`. For the synthetic input the quotient is exactly 1000, so $\Delta t = \Delta t_{\text{in}} = 200$ s. For decimal inputs the quotient of two doubles may land just above an integer and add one step (items I3 and K13 of Table 7.3). The row at $t = 0$ is stored first (src/numerical/finite_volume.rs, lines 196 to 206).

7.1.4 One Backward Euler step

`backward_euler_step`, src/numerical/finite_volume.rs, lines 162 to 182 assembles the coefficients of Eq. (5.14) cell by cell and never forms the matrix of Eq. (5.15):

```
    for i in 0..n {
        gw[i] = if i == 0 { 0.0 } else { mesh.conductance[i - 1] };
        ge[i] = if i + 1 == n {
            mesh.outer_conductance
        } else {
            mesh.conductance[i]
        };
        v[i] = mesh.cells[i].volume / dt;
        rhs[i] = v[i] * old[i];
    }
```

The array `gw` holds g_j^W (zero at the centre, Eq. (5.10)), `ge` holds g_j^E (the outer conductance in the last cell), `v` holds $v_j = V_j/\Delta t$ and `rhs` holds $v_j C_j^n$. The source term of the Phase 1 documents is absent: `rhs = v * old` has no source (item I21).

The linear system is solved by `solve_diffusion_system`, src/numerical/finite_volume.rs, lines 132 to 160. The forward sweep of Eqs. (5.18) and (5.19) is lines 139 to 153 and the back substitution is lines 154 to 158:

```
    let mut e_prev = 0.0;
    for i in 0..n {
        let e = if i == 0 {
            v[0]
        } else {
            v[i] + gw[i] * e_prev / (e_prev + gw[i])
        };
        den[i] = e + ge[i];
        if den[i] <= 0.0 || !den[i].is_finite() {
            bail!("singular or non-finite pivot at cell {i}");
        }
        let carry = if i == 0 { 0.0 } else { gw[i] * dp[i - 1] };
        dp[i] = (rhs[i] + carry) / den[i];
        e_prev = e;
    }
    let mut x = vec![0.0; n];
    x[n - 1] = dp[n - 1];
    for i in (0..n - 1).rev() {
        x[i] = dp[i] + ge[i] / den[i] * x[i + 1];
    }
```

The variable `e` is e_j, `den[i]` is the pivot $d_j = e_j + g_j^E$, `dp[i]` is y_j, and `carry` is $g_j^W y_{j-1}$. The explicit branch `i == 0` is needed because $e_1 = v_1$ and the general expression would evaluate $0 \cdot 0/(0+0)$. The guard at src/numerical/finite_volume.rs, lines 147 to 149 rejects a pivot that is not positive and finite; Section 6.3.1 proves that it never fires for validated input. The textbook pivot of Eq. (5.17) is not coded at all. The equivalence of the two pivots (Section 5.10.3) holds when $g_{j+1}^W = g_j^E$. The solver takes `gw` and `ge` as independent slices and does not assert that identity, so correctness rests on `backward_euler_step`, which builds both from the same `conductance` entries.

7.1.5 Release, inventory, balance and output

The remainder of the loop in `run` (src/numerical/finite_volume.rs, lines 208 to 234) converts the new concentration vector into the quantities of Section 5.11. The release rate $\Phi_R^{n+1} = G_R C_{N_c}^{n+1}$ of Eq. (5.22) is `mesh.outer_conductance * next[next.len() - 1]` (src/numerical/finite_volume.rs, line 210); the value is in mol s^{-1}, as stated in item I1. The cumulative release does not use the plain recurrence of Eq. (5.22) but the compensated form of Eq. (5.23):

```
        // Kahan compensated accumulation prevents long-run roundoff in the
        // independently accumulated boundary-release conservation check.
        let increment = dt * release_rate;
        let y = increment - release_compensation;
        let t = released + y;
        release_compensation = (t - released) - y;
        released = t;
```

`increment` is $\Delta t\, \Phi_R^{n+1}$, `y` and `t` are the υ and Σ of Eq. (5.23), `release_compensation` is κ and `released` is M_{rel}^{n}. The inventory M^{n+1} is src/numerical/finite_volume.rs, line 219 and the balance residual $\varepsilon^{n+1} = M^{n+1} + M_{\mathrm{rel}}^{n+1} - M^{0}$ of Eq. (5.24) is src/numerical/finite_volume.rs, line 220. The release fraction $F = M_{\mathrm{rel}}/M^{0}$ is src/numerical/finite_volume.rs, line 227. A row is stored when `step % output_every_steps == 0 || step == steps` at time `step as f64 * dt`, which is the output rule of Eq. (5.25) (src/numerical/finite_volume.rs, lines 221 to 233). For the synthetic input this gives 101 rows.

7.1.6 Diagnostics and what is not coded

`explicit_dt_bound`, src/numerical/finite_volume.rs, lines 238 to 252 evaluates Eq. (6.6), $\Delta t_{\mathrm{FE}} = \min_j V_j/(g_j^W + g_j^E)$, from the same conductances. The driver prints it once (src/main.rs, line 78). The synthetic value is $7.812107958751 \times 10^{-3}$ s. No forward Euler solver exists in v1.0.0, so Eq. (6.7) and the scheme behind it are DERIVED NOT IMPLEMENTED (item I6). The properties of Part VI that concern Code B are also not computed by the program: the conservation identity (6.1) is checked through the residual of Eq. (5.24), the M-matrix and maximum-principle statements (6.2) to (6.5) are properties of the sweep above, and the Fourier number (6.13) is not evaluated. Section 7.4 lists them with their labels.

7.2 From the derivation to Code C

Code C is `src/analytical/sturm_liouville.rs`. It imports nothing from the rest of the crate (src/analytical/sturm_liouville.rs, lines 21 to 22) and its header states that it never calls the finite-volume solver (src/analytical/sturm_liouville.rs, lines 1 to 2). The independence of the two codes is therefore structural, which gives their agreement weight (Section 7.3). The module comment (src/analytical/sturm_liouville.rs, lines 12 to 19) states the modal problem of Eqs. (4.1) to (4.4), the interface conditions of Eq. (4.7) and the release formula of Eq. (4.22). Those equations are the derivation behind the code and have no separate computation. The subsections follow the call order of `ModalRelease::new`.

7.2.1 Layers

Code A passes the series only the pairs (R_ℓ, D_ℓ), built by `layers_from_config`, src/postprocessing/comparison.rs, lines 15 to 23 into `Layer` records (src/analytical/sturm_

liouville.rs, lines 24 to 29). The series receives neither c_0 nor the cell counts nor any time step. `validate_layers`, src/analytical/sturm_liouville.rs, lines 112 to 127 is called at the start of `eigenvalues` (src/analytical/sturm_liouville.rs, line 198) and requires at least two layers, increasing radii and $D_\ell > 0$.

7.2.2 Propagation through layers and interfaces

In one layer the state (u, q) is carried by the matrix $\mathbf{T}_\ell$ of Eq. (4.6), coded in `transfer_layer`, src/analytical/sturm_liouville.rs, lines 86 to 92 as `u2 = c * u + s * du_flux / (d * k)` and `q2 = -d * k * s * u + c * du_flux` with `c` and `s` the cosine and sine of $k_\ell L_\ell$. Across an interface the jump of Eq. (4.8) is `interface_state`, src/analytical/sturm_liouville.rs, lines 98 to 100. The shooting routine `shoot`, src/analytical/sturm_liouville.rs, lines 132 to 162 starts from $(u, q) = (0, 1)$ at the centre (Eq. (4.9), src/analytical/sturm_liouville.rs, line 134), applies the jump for every layer after the first (src/analytical/sturm_liouville.rs, lines 138 to 140), computes $k_\ell = \sqrt{\Lambda / D_\ell}$ of Eq. (4.4) (src/analytical/sturm_liouville.rs, line 141), records the state at the start of the layer (src/analytical/sturm_liouville.rs, lines 142 to 149) and transfers it through the layer (src/analytical/sturm_liouville.rs, line 150). It returns $u(R; \Lambda)$, which is the characteristic function $g(\Lambda)$ of Eq. (4.10), together with the recorded states. The single call to `shoot` in the library discards the returned $u(R)$ and uses only the recorded states (src/analytical/sturm_liouville.rs, line 329). The characteristic function is therefore computed and thrown away, and the eigenvalues come from the count alone (item I13). The `rescale` branch of `shoot` (src/analytical/sturm_liouville.rs, lines 151 to 157) is never executed (item K16).

7.2.3 Counting eigenvalues

`eigenvalue_count`, src/analytical/sturm_liouville.rs, lines 166 to 192 repeats the same propagation and counts zeros instead of returning $u(R)$. For each layer it forms the phase of Eq. (4.11) and adds the number of zeros of Eq. (4.12):

```
        let a = u;
        let b = q / (l.diffusivity * k);
        // u(s) = A sin(k s + theta0); zeros at k s + theta0 = m pi for s in (0, L].
        let theta0 = a.atan2(b);
        let length = l.outer_radius - left;
        n += ((theta0 + k * length) / PI).floor() as i64 - (theta0 / PI).floor() as i64;
```

`a` and `b` are $a_\ell = u_{\text{in}}$ and $b_\ell = q_{\text{in}}/(D_\ell k_\ell)$, `theta0` is θ_0, and the running sum `n` is the count $\mathcal{N}(\Lambda)$ of Eq. (4.13). The function returns `n.max(0)` (src/analytical/sturm_liouville.rs, line 191). After each layer the state is rescaled by $m = \max(|u|, |q/(D_\ell k_\ell)|)$ (src/analytical/sturm_liouville.rs, lines 183 to 187). A positive scale factor leaves $\operatorname{atan2}(a, b)$ unchanged, so the count is unaffected.

7.2.4 Isolating the eigenvalues

`eigenvalues`, src/analytical/sturm_liouville.rs, lines 197 to 264 turns the count into roots in four stages. The argument check is src/analytical/sturm_liouville.rs, lines 199 to 201. The mode cap of Eq. (4.16) is src/analytical/sturm_liouville.rs, lines 202 to 217: if $\mathcal{N}(\Lambda_{\max}) > K_{\max}$, a geometric bisection lowers the upper limit to a value Λ_{cov} whose count does not exceed $K_{\max} = 20\,000$ (the constant `MAX_MODES` at src/postprocessing/comparison.rs, line 13). The cap is not active in the main run, which has 32 modes. The lower bracket of Eq. (4.14) is src/analytical/sturm_liouville.rs, lines 220 to 238:

`lo = 1e-6 * (1.0 / phase_rate).powi(2)`, reduced by a factor 0.01 until the count is zero. The bisection of Eqs. (4.13) and (4.15) is src/analytical/sturm_liouville.rs, lines 240 to 263. A stack of intervals $(a, b]$ with known counts is split at $m = \sqrt{ab}$ when $b > 4a$ and at $(a + b)/2$ otherwise (src/analytical/sturm_liouville.rs, lines 253 to 257). An interval with no new eigenvalue is dropped (src/analytical/sturm_liouville.rs, lines 243 to 245) and an interval of relative width at most 4×10^{-15} yields its midpoint as a root (src/analytical/sturm_liouville.rs, lines 246 to 252). An interval of that width that still holds more than one eigenvalue stops the program with an error (src/analytical/sturm_liouville.rs, lines 247 to 249).

7.2.5 Eigenfunctions, integrals and weights

For every eigenvalue, `ModalRelease::new`, src/analytical/sturm_liouville.rs, lines 323 to 364 shoots once more to obtain the layer states (src/analytical/sturm_liouville.rs, line 329), computes the flux ratio of Eq. (4.27) at every interface (src/analytical/sturm_liouville.rs, lines 330 to 336) and builds the eigenfunction of Eq. (4.5) in each layer from the recorded state (src/analytical/sturm_liouville.rs, lines 339 to 344). The integrals of Eq. (4.20) are summed layer by layer with composite 16-point Gauss-Legendre quadrature, Eq. (4.24) (src/analytical/sturm_liouville.rs, lines 337 to 351). The panel count $N_{\mathrm{p},\ell} = \lceil k_\ell L_\ell/(\pi/2) \rceil + 1$ is src/analytical/sturm_liouville.rs, line 340, and the rule itself is `gauss_legendre_16`, src/analytical/sturm_liouville.rs, lines 266 to 292 and `integrate`, src/analytical/sturm_liouville.rs, lines 294 to 304. The weight of Eq. (4.22) follows:

```
            let w = 3.0 * i1 * moment / (r1.powi(3) * norm);
            if !w.is_finite() {
                bail!("non-finite modal weight at Lambda = {lam:.6e}");
            }
            weights.push(w);
```

`i1` is $I_{1,n}$, `moment` is I_n, `norm` is N_n and `r1` is R_1 (src/analytical/sturm_liouville.rs, line 325). The eigenfunction scale is fixed by $q(0) = 1$ and cancels in the ratio, so no normalisation step exists. The quantities A_n of Eq. (4.18), the concentration series (4.19), the inventory series (4.21) and the closed-form integrals (4.25) are DERIVED NOT IMPLEMENTED.

7.2.6 Release fraction, validity and accuracy floor

The sum of Eq. (4.22) is `ModalRelease::remaining_fraction`, src/analytical/sturm_liouville.rs, lines 382 to 388, with `ModalRelease::release_fraction`, src/analytical/sturm_liouville.rs, lines 391 to 393 returning one minus that sum. No clamp to $[0, 1]$ is applied, so a slightly negative value can appear at early time (item K9). The truncation rule of Eq. (4.26) has three parts. The requested limit $\Lambda_{\max} = 45/t_{\min}$ is passed in src/postprocessing/comparison.rs, line 59 (and src/analytical/sturm_liouville.rs, line 412 for `exact_release`). The validity time $40/\Lambda_{\mathrm{cov}}$ is `ModalRelease::min_valid_time`, src/analytical/sturm_liouville.rs, lines 377 to 379. The gate that withholds a value before it is src/postprocessing/comparison.rs, line 64. The accuracy floor of Eq. (4.28) is `ModalRelease::accuracy_floor`, src/analytical/sturm_liouville.rs, lines 371 to 373, with the literal `2.2e-16` standing for machine epsilon.

7.2.7 Parts that only tests exercise

The Robin root equation (4.31) is coded in `robin_residual`, src/analytical/sturm_liouville.rs, lines 34 to 36 and `robin_roots`, src/analytical/sturm_liouville.rs, lines 40 to 79. Only the unit test `robin_roots_satisfy_residual`, src/analytical/sturm_liouville.rs, lines 448 to 454 calls them (item I23). The homogeneous closed forms of Eq. (4.29) appear in the test helper `homogeneous_kernel_release`, src/analytical/sturm_liouville.rs, lines 435 to 445, used by the test `modal_release_matches_closed_form_for_equal_diffusivity`, src/analytical/sturm_liouville.rs, lines 520 to 532. The Crank series of Eq. (4.30) is `sphere_release_uniform`, examples/verify.rs, lines 50 to 59 in the verification driver. These three are IMPLEMENTED in test code only, not in the library path.

7.3 From the derivation to Code A: comparison, extrapolation and observed orders

Code A is the part of the program that touches both B and C: `src/main.rs`, `src/config.rs` and `src/postprocessing/comparison.rs`, with the figures drawn by `src/postprocessing/plots.rs`. It contains no physics beyond the Richardson combination, the order formula and the cost model. Every physical quantity is computed in B or C.

7.3.1 Driver

`main`, src/main.rs, lines 29 to 200 reads the command-line flags `--fv-only` and `--no-convergence` and the input path (src/main.rs, lines 30 to 36), runs Code B (src/main.rs, lines 37 to 41), writes `timeseries.csv` and `final_profile.csv` (src/main.rs, lines 43 to 73) and prints the Code B results (src/main.rs, lines 75 to 87). With `--fv-only` it stops after the profile figure (src/main.rs, lines 99 to 101). Otherwise it runs the cross-check against Code C (src/main.rs, lines 103 to 138), the cost guard (src/main.rs, lines 141 to 150) and the refinement study (src/main.rs, lines 151 to 198). If the cross-check fails, the program prints `analytical_cross_check=SKIPPED` and returns success, which also skips the study (src/main.rs, lines 135 to 138, item I25).

7.3.2 Comparison of the two methods

`compare_release`, src/postprocessing/comparison.rs, lines 48 to 91 compares the finite-volume release fraction with the series at every stored positive time. It takes the stored times from the finite-volume rows (src/postprocessing/comparison.rs, lines 50 to 54), sets $t_{\min}$ to the smallest (src/postprocessing/comparison.rs, line 58) and builds the series with $\Lambda_{\max} = 45/t_{\min}$ (src/postprocessing/comparison.rs, line 59). It then evaluates the series only for times at or after t_{valid} (src/postprocessing/comparison.rs, lines 63 to 64). At each such time the error of Eq. (7.1) is formed (src/postprocessing/comparison.rs, lines 65 to 69):

$$e(t_n) = \left| F^{\text{FV}}(t_n) - F^{\text{ex}}(t_n) \right|, \qquad t_n \geq t_{\text{valid}}, \qquad F^{\text{FV}}(t_n) = F^n, \tag{7.1}$$

where F^n is Eq. (5.24) and F^{ex} is Eq. (4.22). Only the absolute error is computed. If no stored time reaches t_{valid}, the function returns an error (src/postprocessing/comparison.rs, lines 76 to 80). The returned structure carries the number of modes, Λ_{cov}, t_{valid}, $\rho_{\min}$ and ε_F (src/postprocessing/comparison.rs, lines 81 to 90).

The driver then applies the warning rule of Eq. (7.2) (src/main.rs, lines 124 to 128):

$$\max_{n:\ t_n \geq t_{\text{valid}}} e(t_n) < 3\,\varepsilon_F \quad \Longrightarrow \quad \text{print a warning that the differences are not resolved,} \tag{7.2}$$

so a difference between the methods below three times the accuracy floor of Eq. (4.28) is reported as unresolved. For the synthetic case the largest difference is about 2.5×10^{-4} and $3\varepsilon_F = 1.199 \times 10^{-11}$, so the warning is not triggered. The same factor 3 appears in the resolution test of the refinement study (Eq. (6.12)).

7.3.3 Richardson time extrapolation

Eq. (6.10) is `time_extrapolated_release`, src/postprocessing/comparison.rs, lines 154 to 157, with the formula on src/postprocessing/comparison.rs, lines 155 to 156: $F_{\text{RE}} = \frac{1}{3}[8F(\Delta t/4) - 6F(\Delta t/2) + F(\Delta t)]$. Each value $F(m\Delta t)$ comes from a full run of Code B on one fixed mesh. The helper `with_dt`, src/postprocessing/comparison.rs, lines 137 to 142 sets the input step to $m\,\Delta t_{\text{in}}$ and `output_every_steps` to `usize::MAX`, so that only the final row is stored, and `final_release`, src/postprocessing/comparison.rs, lines 144 to 148 returns that row's release fraction. The three runs cost $4 + 2 + 1 = 7$ times one run, a fact that the cost model ignores (item K1, Section 7.3.5). With the expansion of Eq. (6.9), the coefficients of Δt^0 to Δt^3 in F_{RE} are 1, 0, 0 and $\frac{1}{8}$ (symbolic check in `scripts/derivation_checks/v1_0_0_inventory_code_audit.py`), so the extrapolated value removes the terms in Δt and Δt^2. The comment at src/postprocessing/comparison.rs, lines 150 to 153 presumes that the three runs have step counts in the ratio $4 : 2 : 1$, which holds only when $t_{\text{end}}/\Delta t_{\text{in}}$ is an integer (items K3 and K13).

7.3.4 Refinement study and observed orders

`convergence_study`, src/postprocessing/comparison.rs, lines 183 to 221 builds one series with $\Lambda_{\max} = 45/t_{\text{end}}$ (src/postprocessing/comparison.rs, line 186), checks that $t_{\text{end}} \geq t_{\text{valid}}$ (src/postprocessing/comparison.rs, lines 187 to 189) and takes $F^{\text{ex}}(t_{\text{end}})$ from it (src/postprocessing/comparison.rs, line 190). The spatial study (src/postprocessing/comparison.rs, lines 192 to 202) scales the cell count of every layer by $\lambda_{\text{x}} \in \{0.5, 1, 2, 4, 8\}$ with `scaled_cells`, src/postprocessing/comparison.rs, lines 129 to 135, $n_\ell(\lambda_{\text{x}}) = \max(1, \text{round}(\lambda_{\text{x}}\, n_\ell))$, giving 100, 200, 400, 800 and 1600 cells in total. Each point is the extrapolated value F_{RE} on the scaled mesh at the configured Δt_{in}, so the time error does not mask the spatial error. The temporal study (src/postprocessing/comparison.rs, lines 204 to 214) uses the configured mesh and the plain value $F(\lambda_{\text{t}}\,\Delta t_{\text{in}})$ for $\lambda_{\text{t}} \in \{4, 2, 1, \frac{1}{2}, \frac{1}{4}\}$.

The observed order of Eq. (6.11) is `ConvergenceStudy::orders`, src/postprocessing/comparison.rs, lines 113 to 118, with $p_k = \log_2(\text{err}_k/\text{err}_{k+1})$ on line 116. It is valid when each refinement halves the mesh width or the step (item K2). The resolution test of Eq. (6.12) is `ConvergenceStudy::resolved`, src/postprocessing/comparison.rs, lines 122 to 126: every error must exceed $3\varepsilon_F$. The driver prints both order lists with two decimals and appends a warning when a list is not resolved (src/main.rs, lines 177 to 193).

The printed spatial orders are 2.00 (four values). The printed temporal orders are 0.98, 0.97, 0.94 and 0.89. They fall below one because the measured error is the sum of the temporal error and the spatial error of the 200-cell mesh, 5.476×10^{-6}. Against the extrapolated 200-cell value as a same-mesh reference, the errors are 4.857×10^{-4} down to 3.040×10^{-5} and the orders are 0.9990, 0.9995, 0.9998 and 0.9999 (`scripts/derivation_checks/v1_0_0_p6_orders.py`, which reads `tests/golden/v1.0/convergence_study.csv`). Computing that reference inside the program is a FUTURE improvement (Table 7.4).

7.3.5 Cost guard

Eq. (6.14) is `run_cost`, src/postprocessing/comparison.rs, lines 160 to 163 for one run and `study_cost`, src/postprocessing/comparison.rs, lines 166 to 176 for the study, with the factor 1.75 on src/postprocessing/comparison.rs, line 169. The driver skips the study when the estimate exceeds `STUDY_COST_LIMIT` $= 4 \times 10^9$ cell-steps (src/main.rs, line 21 and src/main.rs, lines 141 to 150). The factor should be 7, because each spatial point costs the seven runs of Section 7.3.3 (item K1). The synthetic estimate is 6.975×10^6 cell-steps as coded and about 2.3×10^7 with the factor 7, far below the limit, so no decision changes for this input.

7.3.6 Quantities that pass between the codes

Code A gives Code B the radii R_ℓ, the diffusivities D_ℓ, the cell counts n_ℓ, c_0 and the time controls (src/main.rs, lines 37 to 39), and gives Code C only R_ℓ and D_ℓ through `layers_from_config`, plus $t_{\min}$ from the stored times of Code B. The release fraction is normalised by the kernel inventory in both codes: M^0 in Code B and the weights w_n in Code C, where c_0 cancels. Both treat the initial support as the kernel only, impose a regular centre and use perfect contact at the interfaces, so the continuum problems agree. The only intended difference is the treatment of the interface and the outer face: Code B discretises them with series resistances between centroids and a half cell to $c = 0$, Code C satisfies them exactly. The measured disagreement is the discretisation error of Code B, mostly its time error. Table 7.2 lists the verified results with their sources.

Table 7.2 Verified results of the synthetic benchmark, with source and label. All values are calculated for the synthetic input and are not literature data.

Quantity	Value	Source	Label
Kernel inventory M^0	$3.351032163829113 \times 10^{-11}$ mol, equal to $\frac{4\pi}{3} R_1^3 c_0$ in all printed digits	`tests/golden/v1.0/timeseries.csv`, first row	calculated, synthetic input
Mesh and steps	$N_c = 200$ (40 per layer), $N_t = 1000$, $\Delta t = 200$ s, 101 stored rows	input file, `timeseries.csv`	calculated, synthetic input
$F^{\mathrm{FV}}(2 \times 10^5$ s)	0.4324286068785	`tests/golden/v1.0/main_run_stdout.txt`	calculated, synthetic input
$F^{\mathrm{ex}}(2 \times 10^5$ s)	0.43255563909286 (32 modes); 0.4325556390930383 from an independent Python script, difference 1.9×10^{-13}, below ε_F	`comparison_fv_vs_analytical.csv`; test `exact_series_matches_independent_python_oracle` in `tests/finite_volume_vs_analytical.rs`	DEMONSTRATED NUMERICALLY

Quantity	Value	Source	Label
Series data	32 modes, $\rho_{\min} = 5.504 \times 10^{-5}$, $\varepsilon_F = 3.997 \times 10^{-12}$, $t_{\text{valid}} = 1777.78$ s	`main_run_stdout.txt`	calculated, synthetic input
Largest and final difference	$\max_n e = 2.537522 \times 10^{-4}$ at 22 000 s; $e(t_{\text{end}}) = 1.270322 \times 10^{-4}$; 100 of 100 times compared, the first at 2000 s	`comparison_fv_vs_analytical.csv`	calculated, synthetic input
Spatial study	errors 2.190×10^{-5}, 5.476×10^{-6}, 1.369×10^{-6}, 3.423×10^{-7}, 8.556×10^{-8} for 100 to 1600 cells; orders 2.00	`convergence_study.csv`	DEMONSTRATED NUMERICALLY
Temporal study	errors 4.912×10^{-4}, 2.485×10^{-4}, 1.270×10^{-4}, 6.626×10^{-5}, 3.587×10^{-5} for $\Delta t = 800$ to 50 s; printed orders 0.98, 0.97, 0.94, 0.89; same-mesh orders 0.9990 to 0.9999	`convergence_study.csv`, `v1_0_0_p6_orders.py`	DEMONSTRATED NUMERICALLY
Explicit step bound	$\Delta t_{\text{FE}} = 7.812107958751 \times 10^{-3}$ s; spectral limit $2/\mu_{\max} = 7.824293 \times 10^{-3}$ s	`main_run_stdout.txt`; `v1_0_0_inventory_code_audit.py`	calculated; spectral value from a Python replica
Balance residual	$9.402717119255 \times 10^{-24}$ mol, which is $2.806 \times 10^{-13} M^0$; the same value is the worst over all 101 rows	`main_run_stdout.txt`, `timeseries.csv`	DEMONSTRATED NUMERICALLY (exact identity PROVED in Section 6.1)
Weight sum	$\sum_{n \le 32} w_n = 1.000007275592$	`v1_0_0_inventory_codeC.py`	DEMONSTRATED NUMERICALLY (Eq. (4.23) CLAIMED ONLY for the layered sphere)
Early-time series value	$F^{\text{ex}}(2000 \text{ s}) = -8.55 \times 10^{-14}$	`comparison_fv_vs_analytical.csv`, second row	calculated, synthetic input (item K9)
Study cost	6.975×10^6 cell-steps as coded; $2.17 \times 10^7 + 1.55 \times 10^6$ with the factor 7	`v1_0_0_inventory_code_audit.py`, arithmetic of Eq. (6.14)	PROVED by counting

7.4 Final equations used in the implementation

Each equation below is restated under the number it has in Parts IV to VI, which the bold label gives. The display itself carries no tag, so that every tag of the document is unique. Each item is followed by the Rust function and the pinned lines that compute it. All items are IMPLEMENTED unless the item says otherwise. Eqs. (7.1) and (7.2) are displayed in Section 7.3.2. Equations of Parts IV to VI that have no Rust counterpart follow the list.

7.4.1 Code C

1. **Eq. (4.4)**, wavenumber of a layer. `shoot`, src/analytical/sturm_liouville.rs, lines 132 to 162 at src/analytical/sturm_liouville.rs, line 141, and `eigenvalue_count` at src/analytical/sturm_liouville.rs, line 175.

$$u'' + k_\ell^2\, u = 0, \qquad k_\ell = \sqrt{\frac{\Lambda}{D_\ell}}, \qquad R_{\ell-1} < r < R_\ell.$$

2. **Eq. (4.5)**, eigenfunction in a layer. `ModalRelease::new`, src/analytical/sturm_liouville.rs, lines 323 to 364 at src/analytical/sturm_liouville.rs, lines 339 to 344.

$$u(s) = a_\ell \cos(k_\ell s) + b_\ell \sin(k_\ell s), \qquad 0 \le s \le L_\ell.$$

3. **Eq. (4.6)**, layer transfer. `transfer_layer`, src/analytical/sturm_liouville.rs, lines 86 to 92.

$$\begin{pmatrix} u \\ q \end{pmatrix}_{\text{out}} = \mathbf{T}_\ell \begin{pmatrix} u \\ q \end{pmatrix}_{\text{in}}, \qquad \mathbf{T}_\ell = \begin{pmatrix} \cos(k_\ell L_\ell) & \dfrac{\sin(k_\ell L_\ell)}{D_\ell k_\ell} \\ -D_\ell k_\ell \sin(k_\ell L_\ell) & \cos(k_\ell L_\ell) \end{pmatrix}.$$

4. **Eq. (4.8)**, interface rule. `interface_state`, src/analytical/sturm_liouville.rs, lines 98 to 100, called at src/analytical/sturm_liouville.rs, line 139 and src/analytical/sturm_liouville.rs, line 173.

$$q^+ = q^- + (D_{\ell+1} - D_\ell)\, \frac{u(R_\ell)}{R_\ell}.$$

5. **Eq. (4.9)**, start at the centre. `shoot`, src/analytical/sturm_liouville.rs, lines 132 to 162 at src/analytical/sturm_liouville.rs, line 134, and `eigenvalue_count` at src/analytical/sturm_liouville.rs, line 168.

$$(u, q)\,(r = 0) = (0, 1).$$

6. **Eq. (4.10)**, characteristic function. Computed and discarded: the value is returned at src/analytical/sturm_liouville.rs, line 161 and dropped at src/analytical/sturm_liouville.rs, line 329 (item I13).

$$g(\Lambda) = u(R; \Lambda), \qquad \text{eigenvalues: } g(\Lambda_n) = 0.$$

7. **Eq. (4.11)**, phase form. `eigenvalue_count`, src/analytical/sturm_liouville.rs, lines 166 to 192 at src/analytical/sturm_liouville.rs, lines 176 to 179.

$$u(s) = H_\ell \sin(k_\ell s + \theta_0), \qquad \theta_0 = \operatorname{atan2}(a_\ell, b_\ell), \qquad a_\ell = u_{\text{in}}, \quad b_\ell = \frac{q_{\text{in}}}{D_\ell k_\ell}.$$

8. **Eq. (4.12)**, zeros in one layer. `eigenvalue_count`, src/analytical/sturm_liouville.rs, lines 166 to 192 at src/analytical/sturm_liouville.rs, lines 180 to 181.

$$\text{zeros of } u \text{ in } 0 < s \le L_\ell = \left\lfloor \frac{\theta_0 + k_\ell L_\ell}{\pi} \right\rfloor - \left\lfloor \frac{\theta_0}{\pi} \right\rfloor.$$

9. **Eq. (4.13)**, Sturm count. `eigenvalue_count`, src/analytical/sturm_liouville.rs, lines 166 to 192, sum at src/analytical/sturm_liouville.rs, line 181, return at src/analytical/sturm_liouville.rs, line 191.

$$\mathcal{N}(\Lambda)=\sum_{\ell=1}^{5}\left(\left\lfloor\frac{\theta_{0,\ell}+k_\ell L_\ell}{\pi}\right\rfloor-\left\lfloor\frac{\theta_{0,\ell}}{\pi}\right\rfloor\right)=\#\{n:\Lambda_n\le\Lambda\}.$$

10. **Eq. (4.14)**, lower bracket. `eigenvalues`, src/analytical/sturm_liouville.rs, lines 197 to 264 at src/analytical/sturm_liouville.rs, lines 220 to 238.

$$\Lambda_{\mathrm{lo}}=10^{-6}\left(\sum_{\ell=1}^{5}\frac{L_\ell}{\sqrt{D_\ell}}\right)^{-2},\quad\text{then }\Lambda_{\mathrm{lo}}\leftarrow 0.01\,\Lambda_{\mathrm{lo}}\text{ until }\mathcal{N}(\Lambda_{\mathrm{lo}})=0.$$

11. **Eq. (4.15)**, midpoint rule and stopping width. `eigenvalues`, src/analytical/sturm_liouville.rs, lines 197 to 264, stop at src/analytical/sturm_liouville.rs, lines 246 to 252 and midpoint at src/analytical/sturm_liouville.rs, lines 253 to 257.

$$m=\begin{cases}\sqrt{ab}, & b>4a,\\ (a+b)/2, & b\le 4a,\end{cases}\qquad\text{stop when } b-a\le 4\times10^{-15}\,b.$$

12. **Eq. (4.16)**, mode cap. `eigenvalues`, src/analytical/sturm_liouville.rs, lines 197 to 264 at src/analytical/sturm_liouville.rs, lines 202 to 217.

$$\Lambda_{\mathrm{cov}}=\begin{cases}\Lambda_{\max}, & \mathcal{N}(\Lambda_{\max})\le K_{\max},\\ \text{lo of the geometric search, so that } \mathcal{N}(\Lambda_{\mathrm{cov}})\le K_{\max}, & \text{otherwise.}\end{cases}$$

13. **Eq. (4.20)**, norm and moments. `ModalRelease::new`, src/analytical/sturm_liouville.rs, lines 323 to 364 at src/analytical/sturm_liouville.rs, lines 337 to 351, with `integrate`, src/analytical/sturm_liouville.rs, lines 294 to 304.

$$I_{1,n}=\int_0^{R_1} r\,u_n\,dr,\qquad I_n=\int_0^{R} r\,u_n\,dr,\qquad N_n=\int_0^{R} u_n^2\,dr.$$

14. **Eq. (4.22)**, weights and release fraction. Weights at `ModalRelease::new`, src/analytical/sturm_liouville.rs, lines 323 to 364, src/analytical/sturm_liouville.rs, lines 352 to 356; sum at `ModalRelease::remaining_fraction`, src/analytical/sturm_liouville.rs, lines 382 to 388; fraction at `ModalRelease::release_fraction`, src/analytical/sturm_liouville.rs, lines 391 to 393.

$$F(t)=1-\sum_{n=1}^{K}w_n\,e^{-\Lambda_n t},\qquad w_n=\frac{3\,I_{1,n}\,I_n}{R_1^3\,N_n},$$

15. **Eq. (4.24)**, composite 16-point Gauss-Legendre quadrature. `gauss_legendre_16`, src/analytical/sturm_liouville.rs, lines 266 to 292 and `integrate`, src/analytical/sturm_liouville.rs, lines 294 to 304; the panel count is src/analytical/sturm_liouville.rs, line 340.

$$\int_{R_{\ell-1}}^{R_\ell} f(r)\,dr\approx\sum_{i=1}^{N_{\mathrm{p},\ell}}\frac{h}{2}\sum_{m=1}^{16}\omega_m\,f\!\left(\mathrm{mid}_i+\frac{h}{2}x_m\right),\qquad N_{\mathrm{p},\ell}=\left\lceil\frac{k_\ell L_\ell}{\pi/2}\right\rceil+1,$$

16. **Eq. (4.26)**, truncation and validity time. Limit at `compare_release`, src/postprocessing/comparison.rs, lines 48 to 91, src/postprocessing/comparison.rs, line 59, and `exact_release`, src/analytical/sturm_liouville.rs, lines 399 to 418, src/analytical/sturm_liouville.rs, line 412; validity at `ModalRelease::min_valid_time`, src/analytical/sturm_liouville.rs, lines 377 to 379; gate at src/postprocessing/comparison.rs, line 64 and src/analytical/sturm_liouville.rs, line 416.

$$\Lambda_{\max} = \frac{45}{t_{\min}}, \qquad t_{\text{valid}} = \frac{40}{\Lambda_{\text{cov}}}, \qquad F^{\text{ex}}(t) \text{ is reported only for } t \geq t_{\text{valid}}.$$

17. **Eq. (4.27)**, flux ratio. `ModalRelease::new`, src/analytical/sturm_liouville.rs, lines 323 to 364 at src/analytical/sturm_liouville.rs, lines 330 to 336.

$$\rho = \frac{|q - D\,u/R_\ell|}{|q| + |D\,u/R_\ell|}, \qquad \rho_{\min} = \min_{n,\,\ell} \rho,$$

18. **Eq. (4.28)**, accuracy floor. `ModalRelease::accuracy_floor`, src/analytical/sturm_liouville.rs, lines 371 to 373.

$$\varepsilon_F = \frac{2.2 \times 10^{-16}}{\min(\rho_{\min}, 1)}.$$

19. **Eq. (4.31)**, Robin root equation. `robin_residual`, src/analytical/sturm_liouville.rs, lines 34 to 36 and `robin_roots`, src/analytical/sturm_liouville.rs, lines 40 to 79; called by a unit test only.

$$z \cot z = 1 - \mathrm{Bi}.$$

7.4.2 Code B

20. **Eq. (5.2)**, mesh. `build_mesh`, src/numerical/finite_volume.rs, lines 50 to 99 at src/numerical/finite_volume.rs, line 55 and src/numerical/finite_volume.rs, lines 57 to 62.

$$\begin{aligned} w_\ell &= \frac{R_\ell - R_{\ell-1}}{n_\ell} = \frac{L_\ell}{n_\ell}, \\ r_{j-1/2} &= R_{\ell-1} + \iota\, w_\ell, \\ r_{j+1/2} &= \begin{cases} R_{\ell-1} + (\iota+1)\, w_\ell, & \iota < n_\ell - 1, \\ R_\ell, & \iota = n_\ell - 1. \end{cases} \end{aligned}$$

21. **Eq. (5.3)**, cell volume. `shell_volume`, src/numerical/finite_volume.rs, lines 38 to 40, called at src/numerical/finite_volume.rs, line 67.

$$V_j = \frac{4\pi}{3}\left(r_{j+1/2}^3 - r_{j-1/2}^3\right).$$

22. **Eq. (5.4)**, volume centroid. `volume_centroid`, src/numerical/finite_volume.rs, lines 42 to 48, called at src/numerical/finite_volume.rs, line 66.

$$r_j = \frac{3}{4}\,\frac{r_{j+1/2}^4 - r_{j-1/2}^4}{r_{j+1/2}^3 - r_{j-1/2}^3} \quad (j \geq 2), \qquad r_1 = \frac{3}{4}\,r_{3/2}.$$

23. **Eq. (5.5)**, face area. `build_mesh`, src/numerical/finite_volume.rs, lines 50 to 99 at src/numerical/finite_volume.rs, line 83.

$$A_{j+1/2} = 4\pi r_{j+1/2}^2.$$

24. **Eq. (5.7)**, half-cell distances. `build_mesh`, src/numerical/finite_volume.rs, lines 50 to 99 at src/numerical/finite_volume.rs, lines 84 to 85.

$$\delta_{j+1/2}^{L} = r_{j+1/2} - r_j, \qquad \delta_{j+1/2}^{R} = r_{j+1} - r_{j+1/2}.$$

25. **Eq. (5.9)**, face conductance. `build_mesh`, src/numerical/finite_volume.rs, lines 50 to 99 at src/numerical/finite_volume.rs, line 86.

$$A_{j+1/2} J_{j+1/2} = G_{j+1/2}\,(C_j - C_{j+1}), \qquad G_{j+1/2} = \frac{A_{j+1/2}}{\delta_{j+1/2}^{L}/D_j + \delta_{j+1/2}^{R}/D_{j+1}}.$$

26. **Eq. (5.10)**, closed centre. `backward_euler_step`, src/numerical/finite_volume.rs, lines 162 to 182 at src/numerical/finite_volume.rs, line 172.

$$g_1^{W} = 0.$$

27. **Eq. (5.11)**, outer conductance and release rate. Conductance at `build_mesh`, src/numerical/finite_volume.rs, lines 50 to 99, src/numerical/finite_volume.rs, lines 88 to 91; release rate in `run` at src/numerical/finite_volume.rs, line 202 and src/numerical/finite_volume.rs, line 210.

$$G_R = \frac{A(R)}{(R - r_{N_c})/D_5 + 0} = \frac{4\pi R^2 D_5}{R - r_{N_c}}, \qquad \Phi_R \approx G_R\,(C_{N_c} - 0) = G_R\,C_{N_c}.$$

28. **Eq. (5.14)**, Backward Euler row. Coefficients at `backward_euler_step`, src/numerical/finite_volume.rs, lines 162 to 182, src/numerical/finite_volume.rs, lines 171 to 180; solution by `solve_diffusion_system`, src/numerical/finite_volume.rs, lines 132 to 160.

$$-g_j^{W}\,C_{j-1}^{n+1} + \left(v_j + g_j^{W} + g_j^{E}\right) C_j^{n+1} - g_j^{E}\,C_{j+1}^{n+1} = v_j\,C_j^{n} = \mathrm{rhs}_j.$$

29. **Eq. (5.16)**, step count and step size. `run`, src/numerical/finite_volume.rs, lines 184 to 236 at src/numerical/finite_volume.rs, lines 190 to 191.

$$N_t = \left\lceil \frac{t_{\mathrm{end}}}{\Delta t_{\mathrm{in}}} \right\rceil, \qquad \Delta t = \frac{t_{\mathrm{end}}}{N_t} \leq \Delta t_{\mathrm{in}}.$$

30. **Eq. (5.18)**, subtraction-free pivot. `solve_diffusion_system`, src/numerical/finite_volume.rs, lines 132 to 160 at src/numerical/finite_volume.rs, lines 139 to 146.

$$e_1 = v_1, \qquad e_j = v_j + \frac{g_j^W \, e_{j-1}}{e_{j-1} + g_j^W} \quad (j \geq 2), \qquad d_j = e_j + g_j^E.$$

31. **Eq. (5.19)**, forward sweep and back substitution. `solve_diffusion_system`, src/numerical/finite_volume.rs, lines 132 to 160 at src/numerical/finite_volume.rs, lines 150 to 151 and src/numerical/finite_volume.rs, lines 154 to 158.

$$y_1 = \frac{\mathrm{rhs}_1}{d_1}, \qquad y_j = \frac{\mathrm{rhs}_j + g_j^W \, y_{j-1}}{d_j} \quad (j = 2, \dots, N_c),$$

$$C_{N_c}^{n+1} = y_{N_c}, \quad C_j^{n+1} = y_j + \frac{g_j^E}{d_j} \, C_{j+1}^{n+1} \quad (j = N_c - 1, \dots, 1).$$

32. **Eq. (5.20)**, initial state and initial inventory. `initial_concentration`, src/numerical/finite_volume.rs, lines 101 to 112; M^0 in `run` at src/numerical/finite_volume.rs, lines 185 to 186.

$$C_j^0 = \begin{cases} c_0, & \text{cell } j \text{ in layer 1}, \\ 0, & \text{otherwise}, \end{cases} \qquad M^0 = \sum_{j=1}^{N_c} V_j \, C_j^0.$$

33. **Eq. (5.21)**, inventory. `inventory`, src/numerical/finite_volume.rs, lines 114 to 120, called in `run` at src/numerical/finite_volume.rs, line 219.

$$M^n = \sum_{j=1}^{N_c} V_j \, C_j^n.$$

34. **Eq. (5.22)**, release rate and cumulative release. The rate is computed at `run`, src/numerical/finite_volume.rs, lines 184 to 236, src/numerical/finite_volume.rs, line 210. The plain recurrence is replaced by the compensated form of Eq. (5.23).

$$\Phi_R^{n+1} = G_R \, C_{N_c}^{n+1}, \qquad M_{\mathrm{rel}}^{n+1} = M_{\mathrm{rel}}^n + \Delta t \, \Phi_R^{n+1}, \qquad M_{\mathrm{rel}}^0 = 0.$$

35. **Eq. (5.23)**, compensated summation. `run`, src/numerical/finite_volume.rs, lines 184 to 236 at src/numerical/finite_volume.rs, lines 213 to 217.

$$\begin{aligned} \upsilon &= \Delta t \, \Phi_R^{n+1} - \kappa, \\ \Sigma &= M_{\mathrm{rel}}^n + \upsilon, \\ \kappa &\leftarrow (\Sigma - M_{\mathrm{rel}}^n) - \upsilon, \\ M_{\mathrm{rel}}^{n+1} &\leftarrow \Sigma. \end{aligned}$$

36. **Eq. (5.24)**, balance residual and release fraction. `run`, src/numerical/finite_volume.rs, lines 184 to 236, residual at src/numerical/finite_volume.rs, line 220 and fraction at src/numerical/finite_volume.rs, line 227.

$$\varepsilon^n = M^n + M_{\mathrm{rel}}^n - M^0, \qquad F^n = \frac{M_{\mathrm{rel}}^n}{M^0}.$$

37. **Eq. (5.25)**, output rule. `run`, src/numerical/finite_volume.rs, lines 184 to 236 at src/numerical/finite_volume.rs, lines 221 to 233; the row at $t = 0$ is src/numerical/finite_volume.rs, lines 196 to 206.

$$n \bmod n_{\text{out}} = 0 \quad \text{or} \quad n = N_t, \qquad t_n = n\,\Delta t,$$

7.4.3 Properties, extrapolation and study

38. **Eq. (6.6)**, explicit step bound (diagnostic only). `explicit_dt_bound`, src/numerical/finite_volume.rs, lines 238 to 252, printed at src/main.rs, line 78.

$$\Delta t_{\text{FE}} = \min_j \frac{V_j}{g_j^W + g_j^E},$$

39. **Eq. (6.10)**, Richardson extrapolation. `time_extrapolated_release`, src/postprocessing/comparison.rs, lines 154 to 157, formula at src/postprocessing/comparison.rs, lines 155 to 156.

$$F_{\text{RE}} = \frac{1}{3}\left(8F(\Delta t/4) - 6F(\Delta t/2) + F(\Delta t)\right) = F_0 + \frac{1}{8}\,\chi_3 \Delta t^3 + \dots$$

40. **Eq. (6.11)**, observed order. `ConvergenceStudy::orders`, src/postprocessing/comparison.rs, lines 113 to 118, formula at src/postprocessing/comparison.rs, line 116.

$$p_k = \log_2 \frac{\text{err}_k}{\text{err}_{k+1}},$$

41. **Eq. (6.12)**, resolution test. `ConvergenceStudy::resolved`, src/postprocessing/comparison.rs, lines 122 to 126, test at src/postprocessing/comparison.rs, line 125.

$$\text{err}_k > 3\,\varepsilon_F \quad \text{for every } k,$$

42. **Eq. (6.14)**, work estimate of the study. `run_cost`, src/postprocessing/comparison.rs, lines 160 to 163 and `study_cost`, src/postprocessing/comparison.rs, lines 166 to 176 (factor at src/postprocessing/comparison.rs, line 169); guard at src/main.rs, lines 141 to 150. The coded factor 1.75 should be 7 (item K1).

$$W = \frac{t_{\text{end}}}{\Delta t_{\text{in}}}\,N_c, \qquad W_{\text{study}} = 1.75 \sum_{\lambda_x \in \{0.5,1,2,4,8\}} W(\lambda_x) + \sum_{\lambda_t \in \{4,2,1,1/2,1/4\}} W(\lambda_t),$$

43. **Eq. (7.1)** and **Eq. (7.2)**, error of the comparison and warning rule. Displayed in Section 7.3.2. Error: `compare_release`, src/postprocessing/comparison.rs, lines 48 to 91 at src/postprocessing/comparison.rs, lines 63 to 69. Warning: `main` at src/main.rs, lines 124 to 128.

7.4.4 Equations without a Rust counterpart

The following equations are not computed by any function of v1.0.0.

- DERIVED NOT IMPLEMENTED, basis of the code: Eqs. (4.1) to (4.3) and (4.7), which give the modal problem behind Code C, and Eqs. (5.1), (5.6), (5.8), (5.12), (5.13) and (5.15), which give the cell balance and the system behind Code B. Eq. (5.17) is the textbook pivot that the code replaces by Eq. (5.18).
- DERIVED NOT IMPLEMENTED, series quantities: orthogonality (4.17), the coefficients A_n of (4.18), the concentration series (4.19), the inventory series (4.21), the weight-sum limit (4.23) and the closed-form integrals (4.25). The weight-sum limit is CLAIMED ONLY as a theorem for the layered sphere and DEMONSTRATED NUMERICALLY by `modal_completeness`, examples/verify.rs, lines 228 to 246.
- DERIVED NOT IMPLEMENTED, test code only: the homogeneous closed forms (4.29) and the Crank series (4.30). They are IMPLEMENTED in the test helpers named in Section 7.2.7.
- DERIVED NOT IMPLEMENTED, properties: Eq. (6.1), checked through the residual of Eq. (5.24); Eqs. (6.2) to (6.5), properties of the sweep; Eq. (6.7), the explicit L^2 limit; Eqs. (6.8), (6.9) and (6.15), the order statements and the error expansion; Eq. (6.13), the Fourier number; Eqs. (6.16) and (6.17), the accuracy of the series.

7.5 Known inconsistencies and open issues

Table 7.3 lists every place where a delivered Phase 1 document, a code comment, a test or the code itself disagrees with another of them, as found while writing this derivation. The code is unchanged in all cases. For each item the table gives the description, the side that is wrong, the recommended correction and a status.

The Phase 1 derivation is docs/mathematics/phase1/TRISO_Finite_Volume_Derivation.md and the Phase 1 pseudocode is docs/mathematics/phase1/TRISO_Finite_Volume_Pseudocode.md. Line numbers given for them refer to the tag. The status codes are: A, handled in these documents, while editing the delivered Phase 1 file needs owner approval; B, handled in these documents and no approval is needed; C, a change to Rust code, a comment, a test or the manifest needs owner approval; D, handled in these documents, while a change to code or comments needs owner approval. In every case the code is unchanged.

Table 7.3 Known inconsistencies between the delivered Phase 1 documents, code comments, tests and code.

ID	Description	Wrong side	Recommended correction	Status
I1	The Phase 1 derivation (line 348) defines $\dot{F}^n = G_R C_N^n / M^0$. The code computes and writes $\Phi_R^n = G_R C_{N_c}^n$ in mol s^{-1} (src/numerical/finite_volume.rs, line 202 and src/numerical/finite_volume.rs, line 210, column `release_rate_mol_s` at src/main.rs, line 50).	Phase 1 document	Define Φ_R^n as implemented (Eq. (5.22)); give $\dot{F} = \Phi_R / M^0$ as DERIVED NOT IMPLEMENTED.	A

ID	Description	Wrong side	Recommended correction	Status
I2	The Phase 1 derivation (line 348) and pseudocode (line 34) show the plain sum $M_{\text{rel}}^{n+1} = M_{\text{rel}}^{n} + \Delta t\, G_R C_N^{n+1}$. The code uses Kahan compensation (src/numerical/finite_volume.rs, lines 211 to 217). The effect over the synthetic run is $4.3 \times 10^{-16} M^0$ in a Python replica.	Phase 1 document (incomplete)	State the compensated recurrence, Eq. (5.23), and why it exists.	A
I3	The Phase 1 documents use the input step. The code uses $\Delta t = t_{\text{end}}/\lceil t_{\text{end}}/\Delta t_{\text{in}} \rceil$ (src/numerical/finite_volume.rs, lines 190 to 191), which equals the input for the synthetic case (1000 steps of 200 s).	Phase 1 document (incomplete)	State the rule, Eq. (5.16); see K13.	A
I4	The Phase 1 derivation, Section 5.1, does not say that cells are uniform inside a layer or that the last face of a layer is set to R_ℓ (src/numerical/finite_volume.rs, lines 55 to 62).	Phase 1 document (incomplete)	Give $w_\ell = L_\ell/n_\ell$ and the snapping rule, Eq. (5.2).	A
I5	The Phase 1 derivation (line 331) says the textbook elimination lets the balance drift to about 10^{-9}. Code comments at src/numerical/finite_volume.rs, lines 126 to 131 and src/numerical/finite_volume.rs, line 377 say it breaks the balance and loses about 6 digits. No test reproduces this. Python replicas of two cases give a drift of 4.6×10^{-14} to 9.5×10^{-14} for the textbook pivot and 3.9×10^{-16} to 1.2×10^{-15} for the subtraction-free form.	Phase 1 document and code comments (overstated); the code is correct	Describe the subtraction-free form as a no-cancellation and floating-point positivity guarantee; quote replica numbers as replica numbers (Section 6.8).	D
I6	The Phase 1 derivation (lines 333 to 341) and pseudocode (lines 23 to 29) suggest that a forward Euler check exists. Only the bound is computed (src/numerical/finite_volume.rs, lines 238 to 252, printed at src/main.rs, line 78).	Phase 1 documents	Present Δt_{FE} as an implemented diagnostic. The scheme and Eq. (6.7) are DERIVED NOT IMPLEMENTED.	A
I7	The pseudocode (line 71) checks $C \geq 0$ and $0 \leq F \leq 1$ inside the time loop. `run` only records $\min_j C_j$ (src/numerical/finite_volume.rs, line 204 and src/numerical/finite_volume.rs, line 230). The checks are assertions in examples/verify.rs, lines 215 to 216 and src/numerical/finite_volume.rs, line 310.	Phase 1 pseudocode	Write "record $\min_j C_j^n$"; list the assertions under verification.	A

ID	Description	Wrong side	Recommended correction	Status
I8	The pseudocode (line 94) reports runtime, positivity and maximum conservation residual. The program prints only the final residual, under the key `max_final_balance_residual_mol` (src/main.rs, lines 81 to 84), and no runtime. The runtime is printed by examples/verify.rs, line 224.	Phase 1 pseudocode and the printed key name	Document it as the final residual. FUTURE: rename the key or print the maximum.	D
I9	The printed temporal orders 0.98 to 0.89 (src/main.rs, lines 189 to 193) understate first order because the 200-cell spatial error enters. Against the same-mesh reference the orders are 0.999 to 1.000 (Section 7.3.4).	Presentation without caveat	State both numbers and the reason. A same-mesh reference is FUTURE.	B
I10	The Phase 1 derivation (line 220) and pseudocode (line 84) use $t \geq 40/\Lambda_{\max}$. The code requests $\Lambda_{\max} = 45/t_{\min}$ (src/postprocessing/comparison.rs, line 59), may lower it to Λ_{cov} by the cap (src/analytical/sturm_liouville.rs, lines 202 to 217) and gates at $40/\Lambda_{\text{cov}}$ (src/analytical/sturm_liouville.rs, lines 377 to 379).	Phase 1 documents (incomplete)	Define $\Lambda_{\max}$, Λ_{cov} and $K_{\max} = 20\,000$ separately, Eq. (4.26).	A
I11	The Phase 1 derivation (line 211) writes $\sum_n w_n = 1$ beside the truncated series. Thirty-two modes give 1.000007275592.	Phase 1 document (imprecise)	Write $\lim_{K\to\infty} \sum_{n\leq K} w_n = 1$, checked numerically by `modal_completeness`, examples/verify.rs, lines 228 to 246, not proved (Eq. (4.23)).	A
I12	The Phase 1 derivation (line 198) writes $c = \sum_n A_n \phi_n e^{-\Lambda_n t}$ as part of the method. Code C never forms A_n or $c(r,t)$.	Phase 1 document	Label as DERIVED NOT IMPLEMENTED (Eqs. (4.18), (4.19)).	A
I13	The Phase 1 derivation (line 186) cites $\det \mathbf{M}(\Lambda) = 0$ and the pseudocode (lines 40 and 79) isolates roots of $g = u(R;\Lambda)$. The code evaluates g and discards it (src/analytical/sturm_liouville.rs, line 329); roots come from the Sturm count only.	Phase 1 documents	Present g and the determinant as equivalent characterisations that are DERIVED NOT IMPLEMENTED (Eq. (4.10)).	A
I14	The Phase 1 derivation (line 214) says integrals are computed layer by layer. The code uses composite 16-point Gauss-Legendre with $\lceil k_\ell L_\ell/(\pi/2)\rceil + 1$ panels (src/analytical/sturm_liouville.rs, line 340); closed forms exist and are unused.	Phase 1 document (incomplete)	Describe the rule and its exactness to degree 31 per panel (Eq. (4.24)); closed forms (4.25) are DERIVED NOT IMPLEMENTED.	A

ID	Description	Wrong side	Recommended correction	Status
I15	The Phase 1 derivation (line 221) defines ρ without saying on which side of an interface. The code uses the post-jump q and the outer layer's D (src/analytical/sturm_liouville.rs, lines 330 to 336): outer side 5.504×10^{-5} (floor 4.0×10^{-12}), inner side 1.231×10^{-4} (floor 1.8×10^{-12}).	Phase 1 document (incomplete)	State the side and call ε_F an estimate (Eqs. (4.27), (4.28)).	A
I16	The Phase 1 derivation (line 363) says the floor grows roughly in proportion to the diffusivity contrast. The code comment at src/analytical/sturm_liouville.rs, lines 369 to 370 says the true error stayed 10 to 20 times below the estimate. Neither is tested, and the second rests on one case (a factor 21).	Phase 1 document and code comment (unsupported)	Mark both as claims until a contrast sweep exists.	D
I17	The Phase 1 derivation (line 329) says the diagonal entry is larger than the sum of the other entries, "which are zero or negative". Strict dominance compares with the sum of absolute values.	Phase 1 document, wording only (the conclusion is right)	Write $a_{jj} > \sum_{k \neq j} \lvert a_{jk} \rvert$ with margin $v_j = V_j / \Delta t$ (Section 6.2).	A
I18	The Phase 1 derivation (line 29) calls D_i a "constant speed".	Phase 1 document	Write "constant diffusivity" with unit $\mathrm{m}^2\ \mathrm{s}^{-1}$.	A
I19	The Phase 1 derivation (line 188) counts the allowed fade rates "below a given Λ". The count includes equality (src/analytical/sturm_liouville.rs, lines 164 to 165).	Phase 1 document	Use $\leq$ and "zeros in $(0, R]$" (Eq. (4.13)).	A
I20	The Phase 1 derivation cites labels such as TRISO-FV-114 and TRISO-ML-474 from a working document outside STRATO (Phase 1 README, line 14).	Phase 1 document	This derivation is self-contained and has its own numbering.	B
I21	The Phase 1 derivation and pseudocode (lines 14 and 20) carry a source term $S_j V_j$. The code has none (`rhs = v * old`, src/numerical/finite_volume.rs, line 179).	Phase 1 documents (more general than the code)	Derive with S in Part II, set $S = 0$ explicitly and mark the discrete source FUTURE.	B
I22	The Phase 1 derivation proves positivity in exact arithmetic only. The sweep also guarantees $C^{n+1} \geq 0$ in floating point. The unit test at src/numerical/finite_volume.rs, line 310 uses a looser tolerance than the property.	Phase 1 document (incomplete)	State the floating-point property, Eq. (6.4).	A

ID	Description	Wrong side	Recommended correction	Status
I23	The Phase 1 derivation, Section 4.6, agrees with `robin_residual` and `robin_roots` (src/analytical/sturm_liouville.rs, lines 31 to 79) but does not say that only a unit test calls them (src/analytical/sturm_liouville.rs, lines 447 to 454).	Phase 1 document (incomplete)	Add that sentence.	A
I24	Notation clash. The Phase 1 derivation uses i for the layer and j for the cell and writes δ_P, δ_E and the subscripts P, E, W. Rust uses `i` for the cell, `mat` for the layer and `dl`, `dr`, `gw`, `ge`.	Notation	One notation table with a Rust-name column (Table 7.1 and the notation tables).	B
I25	If the cross-check fails, the program prints `analytical_cross_check=SKIPPED` and returns success, which also skips the convergence study (src/main.rs, lines 135 to 138). The pseudocode (lines 87 to 92) omits this branch.	Phase 1 pseudocode (incomplete); the exit status 0 is a code choice	Document the branch. FUTURE: a non-zero exit or a separate message for the skipped study.	D
I26	Working notes gave the explicit step bound as $7.812107958750 \times 10^{-3}$ s in some rows and ...751 in others. The golden `main_run_stdout.txt` prints `7.812107958751e-3`.	Working notes	Use the printed value, or 7.8121×10^{-3} s in prose.	B
K1	`study_cost` multiplies each spatial run cost by 1.75 (src/postprocessing/comparison.rs, line 169) while `time_extrapolated_release` costs $4 + 2 + 1 = 7$ runs. The spatial part is low by a factor 4.00 (synthetic: 5.425×10^6 against 2.17×10^7). Results are unaffected, only the skip decision.	Code	Use the factor 7 in a later release; the documents report the formula as implemented and flag it.	C
K2	The comment “each refinement halves h or dt” (src/postprocessing/comparison.rs, line 112) holds only if every n_ℓ is even (the scaling by 0.5 uses `round` at src/postprocessing/comparison.rs, line 132) and $t_{\text{end}}/\Delta t$ stays an integer. This is true for the synthetic input.	Comment (conditionally true)	State the condition. FUTURE: $p_k = \ln(\text{err}_k/\text{err}_{k+1})/\ln(h_k/h_{k+1})$.	D
K3	The Richardson comment (src/postprocessing/comparison.rs, lines 150 to 153) needs the three runs in the ratio $1 : \frac{1}{2} : \frac{1}{4}$, which holds only if $t_{\text{end}}/\Delta t_{\text{in}}$ is an integer; see K13.	Comment	As K2.	D

ID	Description	Wrong side	Recommended correction	Status
K4	The comments at src/numerical/finite_volume.rs, lines 126 to 131 and src/numerical/finite_volume.rs, line 377: the mechanism (cancellation in $b_j - a_j c'_{j-1}$) is real, the quantitative claim is not reproduced (item I5).	Comments	Soften them or add a test.	C
K5	The accuracy-floor comment at src/analytical/sturm_liouville.rs, lines 366 to 370; see I16.	Comment	Treat as a claim only.	C
K6	The comment at src/analytical/sturm_liouville.rs, lines 396 to 398 says the omitted modes retain e^{-45} at the earliest time. That holds only when the cap is not reached; with a cap, $\Lambda_{\text{cov}} < 45/t_{\text{min}}$ and early times return no value. The gate uses e^{-40} (src/analytical/sturm_liouville.rs, lines 375 to 376), conservative by e^{-5} when no cap acts.	Comment (incomplete)	State both constants and the cap case.	C
K7	src/lib.rs, line 13 points to `docs/mathematics/STRATO_Mathematical_Derivations.md`, which does not exist at the tag. The Phase 1 README (line 16) says it is being written for v2.0, while these documents live in `docs/mathematics/v1.0.0/`.	Code comment and path plan	Publish the derivation at the cited path or change the reference in a later release.	C
K8	The comment at src/analytical/sturm_liouville.rs, lines 38 to 39 says poles of cot z are skipped by scanning finite sign changes. The poles at $n\pi$ are excluded by the 10^{-8} offsets (src/analytical/sturm_liouville.rs, lines 44 to 45).	Comment (imprecise)	Rewrite it.	C
K9	`release_fraction` (src/analytical/sturm_liouville.rs, lines 391 to 393) can return a slightly negative F at early time (-8.55×10^{-14} at 2×10^3 s, inside ε_F). No comment claims otherwise.	Code behaviour	FUTURE: flag or clamp.	C
K10	`homogeneous_uniform` (examples/verify.rs, lines 62 to 85) takes $\lfloor t_{\text{end}}/\Delta t \rfloor$ steps and an uncompensated sum, so it does not exercise `run`.	None (the documents must not cite it as a test of `run`)	State this in the documents.	B
K11	`temperature_k` is read and validated (src/config.rs, lines 86 to 88) but no formula uses it.	Scope statement	The documents must not suggest a temperature dependence.	B

ID	Description	Wrong side	Recommended correction	Status
K12	The comment at src/postprocessing/comparison.rs, line 95 documents `ConvergencePoint.cells` as total cells or a time step in seconds. The field always holds the total cell count (src/postprocessing/comparison.rs, line 197 and src/postprocessing/comparison.rs, line 209); the time step is in `dt_s`.	Code comment	Fix the comment; no effect on results.	C
K13	The step rule can change N_t by one for decimal inputs, because the quotient of two doubles may land just above an integer. With $\Delta t_{\text{in}} = 0.01$ and $t_{\text{end}} = 0.07$ the quotient is 7.000000000000001 and $N_t = 8$. The three Richardson runs then take 8, 15 and 29 steps instead of 7, 14 and 28. This was found with Python double arithmetic; the Rust program was not run on these inputs. It is absent from the synthetic input, where $2 \times 10^5/200 = 1000$ exactly.	Code	FUTURE: round the quotient to the nearest integer when it is within a tolerance of one, or reject non-integer ratios.	C
K14	src/analytical/sturm_liouville.rs, line 67 uses `Option::is_none_or`, stable since Rust 1.82. `Cargo.toml` declares `edition = "2021"` but no `rust-version`, and there is no toolchain file, so an older toolchain fails to compile.	Manifest	Add `rust-version = "1.82"` in a later release; the documents state the toolchain used.	C
K15	The comment at examples/verify.rs, lines 32 to 35 says the errors at Δt and $\Delta t/2$ differ by about a factor 2. The measured ratio is 1.811, because the 200-cell spatial error is included. The assertion window $[1.8, 2.2]$ is passed with a margin of 0.6 percent.	Comment and window	State the cause; widen or change the check in a later release.	C
K16	The comment at src/analytical/sturm_liouville.rs, lines 130 to 131 documents the `rescale` parameter of `shoot` as meant for counting. Counting has its own inline rescale (src/analytical/sturm_liouville.rs, lines 183 to 187) and `shoot` is called only with `false` (src/analytical/sturm_liouville.rs, line 329), so the branch at src/analytical/sturm_liouville.rs, lines 151 to 157 is dead.	Dead code with a misleading comment	Document it as unused or remove it later.	C

ID	Description	Wrong side	Recommended correction	Status
K17	The comments at src/analytical/sturm_liouville.rs, line 7 and src/analytical/sturm_liouville.rs, line 306 describe a five-layer series, while `validate_layers` accepts any $N \geq 2$ (src/analytical/sturm_liouville.rs, lines 112 to 127).	Comment (narrower than the code)	Say "multilayer, $N \geq 2$"; the configuration still fixes five regions.	C
K18	The test `analytical_series_matches_v1_golden` (tests/phase1_regression.rs, lines 71 to 88) compares the series with the golden file at 10^{-12}, below $\varepsilon_F = 4.0 \times 10^{-12}$. It passes on the machine that produced the golden file; behaviour on another platform is untested.	Test tolerance	Document it as platform dependent; consider 10^{-10}.	C

Part IV cites this section for the future improvements named in Table 7.4. Items marked FUTURE in Table 7.3 are repeated there with the properties that stay unproved in v1.0.0.

Table 7.4 Future improvements and the properties that stay open in v1.0.0.

Improvement	Related equations and items	Present state in v1.0.0
Time-converged same-mesh reference for the temporal order, and relative errors	Eq. (6.11); I9	The orders are computed against the exact series only, which mixes in the spatial error
Completeness proof for the layered sphere and a bound on the tail $\sum_{n>K}\|w_n\|$	Eqs. (4.23), (6.16)	CLAIMED ONLY; the weight sum is checked numerically
Closed-form layer integrals in place of quadrature	Eq. (4.25); I14	DERIVED NOT IMPLEMENTED; the quadrature is exact to degree 31 per panel
Flag or clamp a slightly negative F at early time	Eq. (4.22); K9	No clamp
Source term and decay	Phase 1 source S; I21	Not in v1.0.0; `rhs = v * old`
Temperature-dependent diffusivity $D(T)$	K11	Temperature validated and unused
Partition coefficients at the interfaces	Eq. (3.2); Gelbard (2003)	Perfect contact with equal concentrations in both codes
Robin outer face for the layered particle	Eqs. (3.10), (4.31); I23	Robin roots exist for the homogeneous sphere and are called only by a unit test
Study-cost factor 7	Eq. (6.14); K1	Factor 1.75
Step-rule rounding for decimal inputs	Eq. (5.16); K13	Plain `ceil` of the floating-point quotient
Non-zero exit or separate message for a skipped cross-check	I25	Exit status 0
Contrast sweep for the accuracy floor	Eq. (4.28); I16	CLAIMED ONLY; one case
Rust toolchain pin	K14	No `rust-version`
Convergence proof for the non-uniform interface cells and the discontinuous initial step	Eq. (6.15)	PROVED for smooth data only; CLAIMED ONLY for the v1.0.0 initial step

7.5.1 Open provenance and verification items

Table 7.3 lists disagreements between documents and code. Table 7.5 lists the statements of Parts I to VI whose evidence is thinner than the label of the surrounding section might suggest. Each row names the section that makes the statement and says what is missing; the last column gives the status that the statement carries in this document. Nothing in the table changes a result; the rows tell the reader how far to trust a number or a sentence.

Table 7.5 Open provenance and verification items carried from Parts I to VI.

ID	Where	Statement and what is missing	Status
O1	Section 3.2.1, notation table (γ_ℓ)	The weight of the orthogonality integral in Gelbard (2003, Eq. (IV.7), p. 29) is $\gamma_\ell r^2$. Reproducing Fig. V.2 of Gelbard requires $c_{\mathrm{SiC}} = c_{\mathrm{PyC}}/2$, and the differences found were 0.0011 and 0.333. This rests on the STRATO source audit of Gelbard (audit note 02, a working note outside the repository). Check P3.14 confirms only the algebra of the printed orientation. v1.0.0 sets $\gamma_\ell = 1$ (FUTURE IMPROVEMENT, Table 7.4).	CLAIMED ONLY
O2	Sections 3.3 and 3.6	The statement that the perfect-sink condition of Eq. (3.4) gives the fastest release for five layers rests on the physical argument that a finite surface resistance can only hold material back. The Robin-to-Dirichlet limit is shown numerically for the homogeneous sphere (Section 3.6). No proof exists for five layers.	CLAIMED ONLY (five layers); DEMONSTRATED NUMERICALLY (homogeneous sphere)
O3	Sections 4.10 and 4.12	The Rust value of the two-mode reference $F(2 \times 10^5\ \mathrm{s}) = 0.4325556390928056$ is known from the float64 transcription of the algorithm, not from a run of the Rust code.	DEMONSTRATED NUMERICALLY (Python replica)
O4	Sections 4.12 and 6.11.3	Rust gives $F(2000\ \mathrm{s}) = -8.55 \times 10^{-14}$ (golden file), the transcription gives -8.57×10^{-14}. The difference of 2×10^{-16} is unexplained and far below the accuracy floor ε_F. All other transcription values agree with Rust to every quoted digit.	DEMONSTRATED NUMERICALLY; difference not significant

ID	Where	Statement and what is missing	Status
O5	Script `v1_0_0_inventory_codeC.py`	The script prints an INFO line, “F(2e3 s) from 32-mode replica = 3.250e-13”, beside the Rust value -8.55×10^{-14} and the oracle 1.6×10^{-15}. It is not a PASS criterion and Part IV does not quote it. All three values lie inside $\varepsilon_F = 4.0 \times 10^{-12}$, the regime where $1 - \sum_n w_n e^{-\Lambda_n t}$ is a difference of nearly equal numbers (Section 6.11.3). The replica should be reviewed in a later release.	INFO only
O6	Audit note 01 (outside the document)	Audit note 01 says that the repository test constant agrees with the 50-digit oracle “to 3e-16”. The difference between 0.4325556390930383 and 0.43255563909304174 is 3.4×10^{-15}, the value quoted in Part IV. The audit note itself is unchanged.	Discrepancy in a working note
O7	Section 4.16	The printed errata of Gelbard (2003, Eqs. (IV.11) and (IV.13), pp. 31 to 32), the partition-value inconsistency in the example of Section V and the statement that 5 to 10 modes are accurate after about one hour are attributed to audit note 02 and were not re-checked against the PDF in the final pass. None of them affects v1.0.0.	CLAIMED ONLY
O8	Section 6.8.3	Balance drift of one very long run: 9.45×10^{-14} for the textbook elimination and 1.93×10^{-16} for the subtraction-free form. An earlier draft gave 4.6×10^{-14} and 1.2×10^{-15}, and an independent replica gives 3.57×10^{-14} and 3.86×10^{-16}. All are round-off values whose digits depend on the operation order, and the Rust code does not implement the textbook form. Only the order of magnitude and the ranking are claimed.	DEMONSTRATED NUMERICALLY (replica)
O9	Sections 6.1, 6.2, 6.4, 6.10, 6.11 and Appendix A	An earlier check log found that some Part VI checks (the Gershgorin ordering, the centre faces and the study floor, besides the independent series oracle) existed only in a working directory. In the final run the four Part VI scripts of Table 6.1 were executed from the repository: 84 PASS lines, no FAIL, and every PASS line quoted in the document was found in the script output (Table A.1).	RESOLVED for the scripts in Table A.1

ID	Where	Statement and what is missing	Status
O10	Section 6.8.3	The Rust unit test `conservation_holds_at_very_large_fourier_number` was not run while this document was written, because no `target/` build existed on the machine used. It was run in the acceptance check of 2026-11-06 (`cargo test --release` at tag `v1.0.0`, rustc 1.96.0): it passed, together with the other 19 tests. The test asserts only the bound 10^{-12}; the drift values quoted in Section 6.8.3 rest on the Python replicas.	DEMONSTRATED NUMERICALLY (the bound); drift values from replicas only
O11	Section 6.10	The energy argument assumes a uniform remainder in the face truncation error. The assumption is checked for three centre faces and is not proved for a general face index.	CLAIMED ONLY
O12	Sections 6.7 and 6.10	The observed spatial order of F is 2.00, above the proved order $3/2$. No sharper proof is given. It marks a limit of the proof; the code and the proof agree.	PROVED (order $3/2$); DEMONSTRATED NUMERICALLY (observed order 2.00)
O13	Tables 4.3 and 4.7, Section 6.11.3	The Rust eigenvalues, weights and the error of $F(2 \times 10^5\ \mathrm{s})$ against a 50-digit oracle come from an audit-only Rust example that printed the 32 modes (audit note 01, item 1.10). The example and the oracle script are not in the repository, so these Rust numbers cannot be regenerated from it. The float64 transcription reproduces them to all quoted digits.	DEMONSTRATED NUMERICALLY (one-off)
O14	Sections 5.10 and 6.3	The comparison of the subtraction-free sweep with a general dense LU solver and with a 60-digit solve (relative differences 3×10^{-14} to 4×10^{-14}) was made with a replica that is not stored in the repository. The checks `P5R-12` and `P5-23` in Appendix A cover the dense double-precision comparison and the exact symbolic case.	DEMONSTRATED NUMERICALLY (replica not stored)
O15	Whole document	The Markdown was checked with the lint script of this document and with the scripts of Appendix A. The pandoc to XeLaTeX build and the workflow `.github/workflows/docs-pdf.yml` were not run, because neither pandoc nor XeLaTeX is installed on the machine used.	Not run

The citation depth is uneven, and the reader should know where. The reference list records bibliographic data that were checked (Section References), but the content of several books was not read page by page. Crank (1975) is cited for Fick’s law and for

the conservation and diffusion equations without chapter or equation numbers. The chapter of Crank (1975) for the uniform-sphere result of Eq. (4.30) and for the substitution $u = rc$ is not verified, and Ozisik (1980, as cited in Gelbard, 2003) is cited only through Gelbard. Patankar (1980), Kahan (1965), Hairer and Wanner (1996), Higham (2002), Strikwerda (2004) and Varga (2000) are cited without section or chapter, and some listings give pages 40 to 41 for Kahan (1965). Gelbard (2003) has an OSTI record (identifier 809605), which the reference list gives. The spherical divergence formula and the divergence theorem are cited as established vector calculus without a textbook.

References

The list follows the APA 7 style. Every work cited in the text appears here, and every entry is cited at least once. The bibliographic data were checked in the following ways. The Collin, Morris and Hales articles were checked against the local PDFs, which print the volume, the pages or article number and the DOI, and against the Crossref record of each DOI. The Gelbard report was checked against the title page of the local PDF (report number SAND2002-3966, March 2003); its DOI was taken from the OSTI record (identifier 809605, found by web search) and the Crossref record of that DOI gives the same title and author. The two IAEA reports were checked against the title pages of the local PDFs (publisher, year and report number, and for the 2012 report the CD-ROM format and ISBN 978-92-0-186810-7). For Higham (2002), Strikwerda (2004), Hairer and Wanner (1996), Varga (2000) and Kahan (1965) the Crossref record of the DOI gives the authors, title, publisher and year; the edition statements of Hairer and Wanner and of Varga come from library catalogue records found by web search. Crank (1975) and Patankar (1980) have no DOI, and their edition, year and publisher come from library catalogue records found by web search. The content of the books was not read page by page for this document (Section 7.5.1).

Collin, B. P., Petti, D. A., Demkowicz, P. A., & Maki, J. T. (2016). Comparison of fission product release predictions using PARFUME with results from the AGR-1 safety tests. *Nuclear Engineering and Design, 301*, 378-390. https://doi.org/10.1016/j.nucengdes.2016.03.023

Crank, J. (1975). *The mathematics of diffusion* (2nd ed.). Clarendon Press.

Gelbard, F. (2003). *Analytical modeling of fission product releases by diffusion from multicoated fuel particles* (Report No. SAND2002-3966). Sandia National Laboratories. https://doi.org/10.2172/809605

Hairer, E., & Wanner, G. (1996). *Solving ordinary differential equations II: Stiff and differential-algebraic problems* (2nd rev. ed.). Springer. https://doi.org/10.1007/978-3-642-05221-7

Hales, J. D., Jiang, W., Toptan, A., & Gamble, K. A. (2021). Modeling fission product diffusion in TRISO fuel particles with BISON. *Journal of Nuclear Materials, 548*, Article 152840. https://doi.org/10.1016/j.jnucmat.2021.152840

Higham, N. J. (2002). *Accuracy and stability of numerical algorithms* (2nd ed.). SIAM. https://doi.org/10.1137/1.9780898718027

International Atomic Energy Agency. (1997). *Fuel performance and fission product behaviour in gas cooled reactors* (IAEA-TECDOC-978).

International Atomic Energy Agency. (2012). *Advances in high temperature gas cooled reactor fuel technology* (IAEA-TECDOC-CD-1674) [CD-ROM].

International Atomic Energy Agency, Nuclear Data Section. (n.d.). *LiveChart of Nuclides: ENSDF adopted data* [Data set]. Retrieved November 5, 2026, from https://www-nds.iaea.org/relnsd/v1/data

Kahan, W. (1965). Pracniques: Further remarks on reducing truncation errors. *Communications of the ACM, 8*(1), 40. https://doi.org/10.1145/363707.363723

Morris, R. N., Baldwin, C. A., Demkowicz, P. A., Hunn, J. D., & Reber, E. L. (2016). Performance of AGR-1 high-temperature reactor fuel during post-irradiation heating tests. *Nuclear Engineering and Design, 306*, 24-35. https://doi.org/10.1016/j.nucengdes.2016.04.031

Patankar, S. V. (1980). *Numerical heat transfer and fluid flow*. Hemisphere Publishing Corporation.

Strikwerda, J. C. (2004). *Finite difference schemes and partial differential equations* (2nd ed.). SIAM. https://doi.org/10.1137/1.9780898717938

Varga, R. S. (2000). *Matrix iterative analysis* (2nd rev. and expanded ed.). Springer. https://doi.org/10.1007/978-3-642-05156-2

Ozisik (1980) is cited only as Gelbard (2003, p. 29) cites it, so the text writes "Ozisik (1980, as cited in Gelbard, 2003)" and the book has no entry of its own. It was not consulted. The code, the benchmark and the tests of STRATO are cited by file and line range at tag `v1.0.0` of https://github.com/weewzr/STRATO and not through this list.

Table R.1 states how each source is used in the text, with the source label of the Reader's guide.

Table R.1 Use of each source and its label.

Source	Used for	Label
Crank (1975)	Fick's law (Sections 1.3 and 2.0.3), the conservation law and the diffusion equation in a sphere (Part II), the substitution $u = rc$ (Section 4.2) and the uniform-sphere series of Section 4.14	Established textbook result
Gelbard (2003)	Layered-sphere eigenvalue problem, orthogonality and release series of Part IV, the symbols of Section 4.16 and the partition-coefficient discussion of Section 3.2	Adapted from published literature (STRATO sets $\gamma_\ell = 1$; the derivation in Parts III and IV is written out for STRATO)
Patankar (1980)	Finite-volume principle (Section 5.1), harmonic-mean face conductance (Section 5.5) and the Thomas algorithm (Section 5.10.2)	Established textbook result; the version with centroid distances in spherical shells is derived for STRATO
Kahan (1965)	Compensated summation of the released amount (Section 5.11.4)	Established textbook result
Higham (2002)	Rounding of non-negative results (Part VI), stability of elimination without pivoting for diagonally dominant matrices (Section 6.3.1) and the bound on the error of a sum (Section 6.8.4)	Established textbook result
Hairer and Wanner (1996)	Stability function and A-stability of Backward Euler	Established textbook result
Strikwerda (2004)	Lax equivalence theorem and the link between consistency and stability	Established textbook result

Source	Used for	Label
Varga (2000)	M-matrix property of a strictly diagonally dominant Z-matrix	Established textbook result; the elementary proof in Section 6.2 is derived for STRATO
Morris et al. (2016), Collin et al. (2016), IAEA (2012)	Layer sequence and AGR-1 dimensions of Part I, the form of Fick's law in the literature	Published data, adapted from published literature; no value enters Code A, B or C
IAEA (1997)	Particle functions, the partition-coefficient statement, the layer descriptions in Parts I and III	Published description, adapted from published literature
Hales et al. (2021)	Statement on one-dimensional meshes and the sign of Fick's law	Published description, adapted from published literature
IAEA Nuclear Data Section (n.d.)	Half-lives quoted for orientation in Section 1.1	Published nuclear data; no value enters the code

Appendix A: Verification scripts

This appendix lists every script that supports a statement in the document, with what it checks and the result of the final run. The scripts live in `scripts/derivation_checks/` and `scripts/figures/` of the working tree. They are not part of tag `v1.0.0` (they were added after the tag, together with this document), so the links elsewhere in the document point to the tagged Rust source only. The Rust code was not edited.

Each script was run from the repository root with

```
.venv/Scripts/python.exe -I -X utf8 scripts/derivation_checks/<script>.py
```

on Windows with the project virtual environment (SymPy, NumPy, SciPy, Matplotlib and mpmath). Every script exited with status 0, printed no FAIL line and wrote nothing to the repository, except the three figure scripts, which rewrite their PNG files in `docs/mathematics/v1.0.0/figures/` (the files came out with the same size as before). The counts below are the number of output lines that start with `PASS`. Most scripts also print a line that starts with `INFO`, which reports a number without testing it. The four scripts that need the longest run times took 205 s (`v1_0_0_p5_fv_scheme.py`), 60 s (`v1_0_0_p6_truncation.py`), 30 s (`v1_0_0_p4_sturm_weights.py`) and 25 s (`v1_0_0_p4_separation.py` and `v1_0_0_inventory_codeC.py`). The others finished in 2 s or less.

Table A.1 Check scripts of `scripts/derivation_checks/` and the result of the final run (345 PASS lines in total, no FAIL).

Script	What it checks	Result line
`v1_0_0_p1_geometry.py`	Part I: sphere area and shell volume, Fick's law units, thin-shell sum for the inventory, kernel inventory against the code value of M^0	17 PASS; `ALL PASS`
`v1_0_0_p2_conservation.py`	Part II: units of D, thin-shell algebra and limit, divergence, the spherical diffusion equation, worked numbers and diffusion times	23 PASS; `SUMMARY 23/23 PASS ALL PASS`
`v1_0_0_p3_conditions.py`	Part III: condition count, centre regularity and point-source tests, interface conditions and the Gelbard orientation, inventory and global balance, units, the Robin reference and Table 3.2 (independent finite-volume solve), the energy bound on the slowest decay rate	25 PASS; `SUMMARY 25/25 PASS ALL PASS`

Script	What it checks	Result line
v1_0_0_p4_separation.py	Part IV: separation of variables, radial transformation, transfer matrix, interface matching, Sturm count, orthogonality, release weights, homogeneous and Robin checks (SymPy)	41 PASS; SUMMARY 41/41 PASS and ALL PASS
v1_0_0_p4_sturm_weights.py	Part IV: independent re-implementation of the five-layer eigenproblem (absolute r, 2 by 2 matching, bisection in mpmath), eigenvalues, weights, validity time	24 PASS; SUMMARY 24/24 PASS and ALL PASS
v1_0_0_p5_fv_scheme.py	Part V: symbolic and exact-rational checks of the cell balance, face conductance, Backward Euler system, subtraction-free elimination, step rule and Kahan summation	33 PASS; ALL PASS
v1_0_0_p5_fv_replica.py	Part V: replica of Code B that follows the Rust arithmetic, checked against the numbers quoted in Part V	22 PASS; ALL PASS
v1_0_0_p6_properties.py	Part VI: telescoping, symmetry, positive definiteness, M-matrix, positivity, maximum principle, stability of Backward Euler, forward Euler bound, Fourier numbers, pivot forms	32 PASS; ALL PASS
v1_0_0_p6_truncation.py	Part VI: face truncation errors (interior, interface, outer face), cell residuals, Backward Euler local error (SymPy)	16 PASS; ALL PASS
v1_0_0_p6_orders.py	Part VI: Richardson weights, observed orders, contamination model, energy estimate, study cost	20 PASS; ALL PASS
v1_0_0_p6_series.py	Part VI: truncation and validity time of the series, weight and eigenvalue accuracy against a 40-digit oracle, early-time cancellation (replica of Code C)	16 PASS; ALL PASS
v1_0_0_inventory_codeB.py	Part VII: formulas of Code B and the Code A helpers against the Rust arithmetic (SymPy and NumPy replica)	18 PASS; ALL PASS
v1_0_0_inventory_codeC.py	Part VII: layer transfer, interface jump, Sturm phase count, weights and Robin roots of Code C (SymPy), and a replica that reproduces the main-run numbers	26 PASS; ALL PASS
v1_0_0_inventory_code_audit.py	Part VII: every formula of the implementation inventory against an independent derivation or float64 re-implementation	32 PASS; ALL PASS

The figure scripts check the dimensions they draw. They print the radii and the caption text, and they write the PNG files that the figures of the document use.

Table A.2 Figure scripts of `scripts/figures/` and the result of the final run.

Script	What it checks	Result line
`scripts/figures/v1_0_0_triso_cross_section.py`	Radii R_1 to R_5 computed from the AGR-1 baseline dimensions of Morris et al. (2016), Table 2	1 PASS; `PASS radii R_1..R_5 (um) = 175.0, 278.5, 317.9, 353.2, 394.2`; wrote `triso_cross_section.png` and `radial_domains.png`
`scripts/figures/v1_0_0_eigenfunctions.py`	First three eigenfunctions of the synthetic case, computed independently of the Rust code	No test lines; printed $\Lambda_1 = 3.273255671176018 \times 10^{-6}$ s^{-1}, $\Lambda_2 = 7.791893309400192 \times 10^{-5}$ s^{-1}, $\Lambda_3 = 2.583469531655617 \times 10^{-4}$ s^{-1}; wrote `eigenfunctions.png`
`scripts/figures/v1_0_0_control_volume.py`	Layout of the control-volume figure with centroid positions from the formula of Section 5.3	No test lines; wrote `control_volume.png`

The following points limit what the scripts show.

- Passing a script confirms the algebra, the replica or the number that the script contains. It does not prove the Rust code correct, because the replicas re-implement the arithmetic in Python. The Rust code is checked by its own tests and by `examples/verify.rs`. The 20 Rust tests passed under `cargo test --release` in the acceptance check of 2026-11-06 (Table 7.5, item O10); `examples/verify.rs` was not re-run for this appendix.
- The INFO line of `v1_0_0_inventory_codeC.py` for $F(2 \times 10^3 \text{ s})$ is discussed in Table 7.5, item O5. The INFO line of `v1_0_0_inventory_codeB.py` reports a drift of 4.55×10^{-14} for the textbook elimination, against the "about 10^{-9}" of the Phase 1 derivation (Table 7.3, item I5).
- The scripts `sec*_*.py`, `b4_*.py` and `D5_parameter_register.py` in the same directory belong to the Phase 2 documents and are not used here.